Become a

NINJA

with



ANGULAR

ninja squad

Become a ninja with Angular

Ninja Squad

Table of Contents

1. Introduction . . . 1
2. A gentle introduction to ECMAScript 2015+ . . . 4
    2.1. Transpilers . . . 4
    2.2. let . . . 5
    2.3. Constants . . . 6
    2.4. Shorthands in object creation . . . 7
    2.5. Destructuring assignment . . . 8
    2.6. Default parameters and values . . . 9
    2.7. Rest operator . . . 11
    2.8. Classes . . . 12
    2.9. Promises . . . 14
    2.10. Arrow functions . . . 17
    2.11. Async/await . . . 21
    2.12. Sets and Maps . . . 22
    2.13. Template literals . . . 23
    2.14. Modules . . . 24
    2.15. Conclusion . . . 26
3. Going further than ES2015+ . . . 27
    3.1. Dynamic, static and optional types . . . 27
    3.2. Enters TypeScript . . . 28
    3.3. A practical example with DI . . . 28
4. Diving into TypeScript . . . 31
    4.1. Types as in TypeScript . . . 31
    4.2. Enums . . . 32
    4.3. Return types . . . 33
    4.4. Interfaces . . . 34
    4.5. Optional arguments . . . 34
    4.6. Functions as property . . . 35
    4.7. Classes . . . 35
    4.8. Working with other libraries . . . 37
    4.9. Decorators . . . 38
5. Advanced TypeScript . . . 41
    5.1. readonly . . . 41
    5.2. keyof . . . 41
    5.3. Mapped type . . . 42
    5.4. Union types and type guards . . . 44
6. The wonderful land of Web Components . . . 49
    6.1. A brave new world . . . 49

6.2. Custom elements . . . 50
6.3. Shadow DOM . . . 51
6.4. Template . . . 51
6.5. Frameworks on top of Web Components . . . 52
7. Grasping Angular's philosophy . . . 54
8. From zero to something . . . 58
8.1. Node.js and NPM . . . 58
8.2. Angular CLI . . . 58
8.3. Application structure . . . 59
8.4. Our first component . . . 60
8.5. Our first Angular Module . . . 62
8.6. Bootstrapping the app . . . 64
9. The templating syntax . . . 66
9.1. Interpolation . . . 66
9.2. Using other components in our templates . . . 69
9.3. Property binding . . . 71
9.4. Events . . . 74
9.5. Expressions vs statements . . . 77
9.6. Local variables . . . 78
9.7. Structural directives . . . 79
9.8. Other template directives . . . 83
9.9. Canonical syntax . . . 83
9.10. Summary . . . 84
10. Dependency injection . . . 88
10.1. DI yourself . . . 88
10.2. Easy to develop . . . 88
10.3. Easy to configure . . . 92
10.4. Other types of provider . . . 95
10.5. Hierarchical injectors . . . 97
10.6. DI without types . . . 99
11. Services . . . 102
11.1. Title service . . . 102
11.2. Meta service . . . 102
11.3. Making your own service . . . 103
12. Pipes . . . 105
12.1. Pied piper . . . 105
12.2. json . . . 105
12.3. slice . . . 106
12.4. keyvalue . . . 108
12.5. uppercase . . . 109
12.6. lowercase . . . 109

12.7. titlecase . . . 110
12.8. number . . . 110
12.9. percent . . . 111
12.10. currency . . . 111
12.11. date . . . 112
12.12. async . . . 112
12.13. A pipe in your code . . . 114
12.14. Creating your own pipes . . . 115
13. Reactive Programming . . . 117
13.1. Call me maybe . . . 117
13.2. General principles . . . 117
13.3. RxJS . . . 118
13.4. Reactive programming in Angular . . . 120
14. Building components and directives . . . 122
14.1. Introduction . . . 122
14.2. Directives . . . 122
14.3. Components . . . 133
15. Styling components and encapsulation . . . 136
15.1. Shadow DOM strategy . . . 137
15.2. Emulated strategy . . . 138
15.3. None strategy . . . 138
15.4. Styling the host . . . 139
16. Testing your app . . . 140
16.1. The problem with troubleshooting is that trouble shoots back . . . 140
16.2. Unit test . . . 140
16.3. Fake dependencies . . . 146
16.4. Testing components . . . 148
16.5. Testing with fake templates, providers... . . . 152
16.6. Simpler, cleaner unit tests with `ngx-speculoos` . . . 154
16.7. End-to-end tests (e2e) . . . 157
17. Send and receive data through HTTP . . . 159
17.1. Getting data . . . 160
17.2. Transforming data . . . 162
17.3. Advanced options . . . 162
17.4. Jsonp . . . 163
17.5. Interceptors . . . 164
17.6. Tests . . . 165
18. Router . . . 167
18.1. En route . . . 167
18.2. Navigation . . . 169
18.3. Redirects . . . 173

18.4. Matching strategy . . . 173
18.5. Hierarchical and empty-path routes . . . 173
18.6. Guards . . . 175
18.7. Resolvers . . . 177
18.8. Router events . . . 180
18.9. Parameters and data . . . 180
18.10. Lazy loading . . . 182
19. Forms . . . 185
19.1. Forms, dear forms . . . 185
19.2. Template-driven . . . 187
19.3. Code-driven . . . 192
19.4. Adding some validation . . . 195
19.5. Errors and submission . . . 197
19.6. Add some style . . . 200
19.7. Creating a custom validator . . . 201
19.8. Grouping fields . . . 206
19.9. Reacting to changes . . . 207
19.10. Updating on blur or on submit only . . . 209
19.11. Super simple validation error messages with `ngx-valdemort` . . . 210
19.12. Summary . . . 211
20. Zones and the Angular magic . . . 213
20.1. AngularJS 1.x and the digest cycle . . . 213
20.2. Angular and zones . . . 216
21. Angular compilation: Just in Time vs Ahead of Time . . . 222
21.1. Code generation . . . 222
21.2. Ahead of Time compilation . . . 224
22. Advanced observables . . . 226
22.1. Subscribe, unsubscribe and async pipe . . . 226
22.2. Leveraging operators . . . 232
22.3. Building your own Observable . . . 235
23. Advanced components and directives . . . 238
23.1. View queries . . . 238
23.2. Content . . . 241
23.3. Content queries . . . 243
23.4. Host listener . . . 246
23.5. Host binding . . . 247
24. Internationalization . . . 250
24.1. The locale . . . 250
24.2. Default currency . . . 253
24.3. Translating text . . . 253
24.4. Process and tooling . . . 254

24.5. Translating messages in the code . . . 260
24.6. Pluralization . . . 261
24.7. Best practices . . . 263
25. Performances . . . 265
25.1. First load . . . 265
25.2. Reload . . . 269
25.3. Profiling . . . 270
25.4. Runtime performances . . . 271
26. Going to production . . . 291
26.1. Polyfills and differential loading . . . 291
26.2. Environments and configurations . . . 291
26.3. fullTemplateTypeCheck and strictTemplates . . . 293
26.4. enableProdMode . . . 293
26.5. Package your application . . . 294
26.6. Server configuration . . . 294
26.7. Conclusion . . . 295
27. This is the end . . . 296
Appendix A: Changelog . . . 299
A.1. v11.2.0 - 2021-02-12 . . . 299
A.2. v11.1.0 - 2021-01-21 . . . 299
A.3. v11.0.0 - 2020-11-12 . . . 299
A.4. v10.2.0 - 2020-10-22 . . . 299
A.5. v10.1.0 - 2020-09-03 . . . 299
A.6. v10.0.0 - 2020-06-25 . . . 299
A.7. v9.1.0 - 2020-03-26 . . . 300
A.8. v9.0.0 - 2020-02-07 . . . 300
A.9. v8.2.0 - 2019-08-01 . . . 301
A.10. v8.1.0 - 2019-07-02 . . . 301
A.11. v8.0.0 - 2019-05-29 . . . 301
A.12. v7.2.0 - 2019-01-09 . . . 302
A.13. v7.1.0 - 2018-11-27 . . . 302
A.14. v7.0.0 - 2018-10-25 . . . 303
A.15. v6.1.0 - 2018-07-26 . . . 303
A.16. v6.0.0 - 2018-05-04 . . . 304
A.17. v5.2.0 - 2018-01-10 . . . 305
A.18. v5.0.0 - 2017-11-02 . . . 306
A.19. v4.3.0 - 2017-07-16 . . . 306
A.20. v4.2.0 - 2017-06-09 . . . 307
A.21. v4.0.0 - 2017-03-24 . . . 307
A.22. v2.4.4 - 2017-01-25 . . . 309
A.23. v2.2.0 - 2016-11-18 . . . 309

A.24. v2.0.0 - 2016-09-15 310
A.25. v2.0.0-rc.5 - 2016-08-25 310
A.26. v2.0.0-rc.0 - 2016-05-06 311
A.27. v2.0.0-alpha.47 - 2016-01-15 313

Chapter 1. Introduction

So you want to be a ninja, huh? Well, you're in good hands!

But we have a long road, you and me, with lots of things to learn :).

We're living exciting times in Web development. There is a new Angular. A complete rewrite of the good old AngularJS. Why a complete rewrite? Was AngularJS 1.x not enough?

I like the old AngularJS very much. In our small company, we have built several projects with it, contributed code to the core framework, trained hundreds of developers (yes, really), and even written a book about it (in French, but that still counts).

AngularJS is incredibly productive once you have mastered it. Despite all of this, it doesn't prevent us from seeing its weaknesses. AngularJS is not perfect, with some very difficult concepts to grasp, and traps hard to avoid.

Most of all, the Web has changed since AngularJS was conceived. JavaScript has changed. New frameworks have emerged, with great ideas, or better implementation. We are not the kind of developers to tell you that you should use this tool instead of that one. We just happen to know some tools very well, and know what fits the project. AngularJS was one of those tools, allowing us to build well-tested web applications, and to build them fast. We also tried to bend it where it didn't fit. Don't blame us, it happens to the best of us.

Will Angular be the tool we will use without hesitation in our future projects? It's hard to say right now, because the framework is really young and the ecosystem only just blooming.

But Angular has a lot of interesting points, and a vision that few other frameworks have. It has been designed for the Web of tomorrow, with ECMAScript 6, Web Components and Mobile in mind. When it was first announced, I was, like many, sad at first that the 2.0 version would not be a simple update (I'm sorry if you're just learning about it).

But I was also eager to see what solution the talented Google team would come up with.

So I started to write this ebook, pretty much after the first commits, reading the design docs, watching the conference videos, reviewing every commit since the beginning. When I wrote my first ebook, about AngularJS 1.x, it was already a stable and known beast. This one is very different. It started when Angular was not even clear in the minds of its designers. Because I knew I would learn a lot, not only about Angular but also about the concepts that would shape the future of Web development, some of which have nothing to do with Angular. And I did. I had to dig deep about some of these concepts, and I hope that you will enjoy the journey of learning about them, and how they relate to Angular, as much as I did.

The ambition of this ebook is to evolve with Angular. If it turns out that Angular is the great framework we hope, you will receive updates with the best practices and some new features as they emerge (and with fewer typos, because, despite our countless reviews, there are probably some left…). And I would love to hear back from you - if some chapters aren't clear enough, if you spot a mistake or if you have a better way for some parts.

I'm fairly confident about the code samples, though, as they are all in a real project, with several

hundred unit tests. It was the only way to write an ebook with a newborn framework, and to be able to catch all the problems that inevitably arose with each release.

Even if you are not convinced by Angular in the end, I'm pretty sure you will have learnt a thing or two along your read.

If you have bought the "Pro package" (thank you!), you'll build a small application piece by piece along the book. This application is called **PonyRacer**, and it is a website where you can bet on pony races. You can even test the application here! Go on, I'll wait for you.

Fun, isn't it?

But it's not just a fun application, it's a complete one. You'll have to write components, forms, tests, use the router, call an HTTP API (that we have already built) and even do Web Sockets. It has all the pieces you'll need for writing a real app. Each exercise will come with a skeleton, a set of instructions and a few tests. Once you have all the tests in success, you have completed the exercise!

The first 6 exercises of the Pro Pack are free. The other ones are only accessible if you buy our online training. At the end of every chapter, we will link to the exercises of the Pro Pack that are related to the features explained in the chapter, mark the free ones with the following label: 🐴 , and mark the other ones with the following label: 🦄.

If you did not buy the "Pro package" (but really you should), don't worry: you'll learn everything that's needed. But you will not build this awesome application with beautiful ponies in pixel art. Your loss :)!

You will quickly see that, beyond Angular itself, we have tried to explain the core concepts the framework uses. The first chapters don't even talk about Angular: they are what I call the "Concept Chapters", here to help you level up with the new and exciting things happening in our field.

Then we will slowly build our knowledge of the framework, with components, templates, pipes, forms, http, routing, tests…

And finally we will learn about the advanced topics. But that's another story.

Enough with the introduction, let's start with one of the things that will definitely change the way we code: ECMAScript 6.



The ebook is using Angular version 11.2.3 for the examples.



Angular and versioning

This book used to be named "Become a Ninja with Angular 2". Because, originally, Google named its framework Angular 2. But in October 2016, they reviewed their versioning and release policy.

We now have a major release every six months, according to the plan. And the framework should be called just “Angular”.

Don’t worry, these releases are not a complete rewrite with no backward compatibility like Angular 2 was to AngularJS 1.x.

As this ebook will be updated (for free) with all the future major releases, it is now named "Become a Ninja with Angular" (without any number).

Chapter 2. A gentle introduction to ECMAScript 2015+

If you're reading this, we can be pretty sure you have heard of JavaScript. What we call JS is one implementation of a standard specification, called ECMAScript. The spec version you know the most about is version 5, that has been used these last years.

But, in 2015, a new version of the spec was released, called ECMAScript 2015, ES2015, or sometimes ES6, as it was the sixth version of the specification. And since then, we have had a yearly release of the specification (ES2016, ES2017, etc.), with a few new features every year. From now on, I'll mainly say ES2015, as it is the most popular way to reference it, or ES2015+ to reference ES2015, ES2016, ES2017, etc. It adds A LOT of things to JavaScript, like classes, constants, arrow functions, generators… It has so much stuff that we can't go through all of it, as it would take the whole book. But Angular has been designed to take advantage of the brand new version of JavaScript. And, even if you can still use your old JavaScript, things will be more awesome if you use ES2015+. So we're going to spend some time in this chapter to get a grip on what ES2015+ is, and what will be useful to us when building an Angular app.

That means we're going to leave a lot of stuff aside, and we won't be exhaustive on the rest, but it will be a great starting point. If you already know ES2015+, you can skip these pages. And if you don't, you will learn some pretty amazing things that will be useful to you even if you end up not using Angular in the future!

2.1. Transpilers

The sixth version of the specification reached its final state in 2015. So it's now supported by modern browsers, but there are still browsers in the wild that don't support it yet, or only support it partially. And of course, now that we have a new specification every year (ES2016, ES2017, etc.), some browsers will always be late. You might be thinking: what's the point of all this, if I need to be careful on what I can use? And you'd be right, because there aren't that many apps that can afford to ignore older browsers. But, since virtually every JS developer who has tried ES2015+ wants to write ES2015+ apps, the community has found a solution: a transpiler.

A transpiler takes ES2015+ source code and generates ES5 code that can run in every browser. It even generates the source map files, which allows you to debug directly the ES2015+ source code from the browser. Back in 2015, there were two main alternatives to transpile ES2015+ code:

- Traceur, a Google project, historically the first one but now unmaintained.
- Babeljs, a project started by a young developer, Sebastian McKenzie (17 years old at the time, yeah, that hurts me too), with a lot of diverse contributions.

The source code of Angular itself was at first transpiled with Traceur, before switching to TypeScript. TypeScript is an open source language developed by Microsoft. It's a typed superset of JavaScript that compiles to plain JavaScript, but we'll dive into it very soon.

Let's be honest: Babel has waaaay more steam than Traceur nowadays, so I would advise you to use it. It is now the de-facto standard in this area.

So if you want to play with ES2015+, or set it up in one of your projects, take a look at these transpilers, and add a build step to your process. It will take your ES2015+ source files and generate the equivalent ES5 code. It works very well but, of course, some of the new features are quite hard or impossible to transform in ES5, as they just did not exist. However, the current state is largely good enough for us to use without worrying, so let's have a look at all these shiny new things we can do in JavaScript!

2.2. let

If you have been writing JS for some time, you know that the `var` declaration is tricky. In pretty much any other language, a variable is declared where the declaration is done. But in JS, there is a concept, called "hoisting", which actually declares a variable at the top of the function, even if you declared it later.

So declaring a variable like `name` in the `if` block:

```
function getPonyFullName(pony) {
  if (pony.isChampion) {
    var name = 'Champion ' + pony.name;
    return name;
  }
  return pony.name;
}
```

is equivalent to declaring it at the top of the function:

```
function getPonyFullName(pony) {
  var name;
  if (pony.isChampion) {
    name = 'Champion ' + pony.name;
    return name;
  }
  // name is still accessible here
  return pony.name;
}
```

ES2015 introduces a new keyword for variable declaration, `let`, behaving much more like what you would expect:

```
function getPonyFullName(pony) {
  if (pony.isChampion) {
    let name = 'Champion ' + pony.name;
    return name;
  }
  // name is not accessible here
  return pony.name;
}
```

The variable `name` is now restricted to its block. `let` has been introduced to replace `var` in the long run, so you can pretty much drop the good old `var` keyword and start using `let` instead. The cool thing is, it should be painless to use `let`, and if you can't, you have probably spotted something wrong with your code!

2.3. Constants

Since we are on the topic of new keywords and variables, there is another one that can be of interest. ES2015 introduces `const` to declare... constants! When you declare a variable with `const`, it has to be initialized and you can't assign another value later.

```
const poniesInRace = 6;
```

```
poniesInRace = 7; // SyntaxError
```

As for variables declared with `let`, constants are not hoisted and are only declared at the block level.

One small thing might surprise you: you can initialize a constant with an object and later modify the object content.

```
const PONY = {};
PONY.color = 'blue'; // works
```

But you can't assign another object:

```
const PONY = {};
```

```
PONY = {color: 'blue'}; // SyntaxError
```

Same thing with arrays:

```
const PONIES = [];
PONIES.push({ color: 'blue' }); // works
```

```
PONIES = []; // SyntaxError
```

2.4. Shorthands in object creation

Not a new keyword, but it can also catch your attention when reading ES2015 code. There is now a shortcut for creating objects, when the object property you want to create has the same name as the variable used as the value.

Example:

```
function createPony() {
  const name = 'Rainbow Dash';
  const color = 'blue';
  return { name: name, color: color };
}
```

can be simplified to:

```
function createPony() {
  const name = 'Rainbow Dash';
  const color = 'blue';
  return { name, color };
}
```

Similarly, when you want to define a method in the object:

```
function createPony() {
  return {
    run: () => {
      console.log('Run!');
    }
  };
}
```

you can simplify it to:

```
function createPony() {
  return {
    run() {
      console.log('Run!');
    }
  };
}
```

2.5. Destructuring assignment

This new feature can also catch your attention when reading ES2015 code. There is now a shortcut for assigning variables from objects or arrays.

In ES5:

```
var httpOptions = { timeout: 2000, isCache: true };
// later
var httpTimeout = httpOptions.timeout;
var httpCache = httpOptions.isCache;
```

Now, in ES2015, you can do:

```
const httpOptions = { timeout: 2000, isCache: true };
// later
const { timeout: httpTimeout, isCache: httpCache } = httpOptions;
```

And you will have the same result. It can be a little disturbing, as the key is the property to look for in the object and the value is the variable to assign. But it works great! Even better: if the variable you want to assign has the same name as the property, you can simply write:

```
const httpOptions = { timeout: 2000, isCache: true };
// later
const { timeout, isCache } = httpOptions;
// you now have a variable named 'timeout'
// and one named 'isCache' with correct values
```

The cool thing is that it also works with nested objects:

```
const httpOptions = { timeout: 2000, cache: { age: 2 } };
// later
const {
  cache: { age }
} = httpOptions;
// you now have a variable named 'age' with value 2
```

And the same is possible with arrays:

```
const timeouts = [1000, 2000, 3000];
// later
const [shortTimeout, mediumTimeout] = timeouts;
// you now have a variable named 'shortTimeout' with value 1000
// and a variable named 'mediumTimeout' with value 2000
```

Of course it also works for arrays in arrays, or arrays in objects, etc.

One interesting use of this can be for multiple return values. Imagine a function `randomPonyInRace` that returns a pony and its position in a race.

```
function randomPonyInRace() {
  const pony = { name: 'Rainbow Dash' };
  const position = 2;
  // ...
  return { pony, position };
}

const { position, pony } = randomPonyInRace();
```

The new destructuring feature assigns the position returned by the method to the position variable, and the pony to the pony variable! And if you don't care about the position, you can write:

```
function randomPonyInRace() {
  const pony = { name: 'Rainbow Dash' };
  const position = 2;
  // ...
  return { pony, position };
}

const { pony } = randomPonyInRace();
```

And you will only have the pony!

2.6. Default parameters and values

One of the characteristics of JavaScript is that it allows developers to call a function with any number of arguments:

- if you pass more arguments than the number of the parameters, the extra arguments are ignored (well, you can still use them with the special `arguments` variable, to be accurate).
- if you pass fewer arguments than the number of the parameters, the missing parameter will be set to `undefined`.

The last case is the one that is the most relevant to us. Usually, we pass fewer arguments when the parameters are optional, like in the following example:

```
function getPonies(size, page) {
  size = size || 10;
  page = page || 1;
  // ...
  server.get(size, page);
}
```

The optional parameters usually have a default value. The OR operator will return the right operand if the left one is `undefined`, as will be the case if the parameter was not provided (to be completely accurate, if it is *falsy*, i.e `0`, `false`, `""`, etc.). Using this trick, the function `getPonies` can then be called:

```
getPonies(20, 2);
getPonies(); // same as getPonies(10, 1);
getPonies(15); // same as getPonies(15, 1);
```

This worked alright, but it was not really obvious that the parameters were optional ones with default values, without reading the function body. ES2015 introduces a more precise way to have default parameters, directly in the function definition:

```
function getPonies(size = 10, page = 1) {
  // ...
  server.get(size, page);
}
```

Now it is perfectly clear that the `size` parameter will be `10` and the `page` parameter will be `1` if not provided.



There is a small difference though, as now `0` or `""` are valid values and will not be replaced by the default one, as `size = size || 10` would have done. It will be more like `size = size === undefined ? 10: size;`.

The default value can also be a function call:

```
function getPonies(size = defaultSize(), page = 1) {
  // the defaultSize method will be called if size is not provided
  // ...
  server.get(size, page);
}
```

or even other variables, either global variables, or other parameters of the function:

```
function getPonies(size = defaultSize(), page = size - 1) {
  // if page is not provided, it will be set to the value
  // of the size parameter minus one.
  // ...
  server.get(size, page);
}
```

This mechanism for parameters can also be applied to values, for example when using a destructuring assignment:

```
const { timeout = 1000 } = httpOptions;
// you now have a variable named 'timeout',
// with the value of 'httpOptions.timeout' if it exists
// or 1000 if not
```

2.7. Rest operator

ES2015 introduces a new syntax to define variable parameters in functions. As said in the previous part, you could always pass extra arguments to a function and get them with the special `arguments` variable. So you could have done something like this:

```
function addPonies(ponies) {
  for (var i = 0; i < arguments.length; i++) {
    poniesInRace.push(arguments[i]);
  }
}

addPonies('Rainbow Dash', 'Pinkie Pie');
```

But I think we can agree that it's neither pretty nor obvious: since the `ponies` parameter is never used, how do we know that we can pass several ponies?

ES2015 gives us a way better syntax, using the rest operator `...`:

```
function addPonies(...ponies) {
  for (let pony of ponies) {
    poniesInRace.push(pony);
  }
}
```

`ponies` is now a true array on which we can iterate. The `for ... of` loop used for iteration is also a new feature in ES2015. It makes sure that you iterate over the collection values, and not also over its properties as `for ... in` would do. Don't you think our code is prettier and more obvious now?

The rest operator can also work when destructuring data:

```
const [winner, ...losers] = poniesInRace;
// assuming 'poniesInRace' is an array containing several ponies
// 'winner' will have the first pony,
// and 'losers' will be an array of the other ones
```

The rest operator is not to be confused with the spread operator which, I'll give you that, looks awfully similar! But the spread operator is the opposite: it takes an array and spreads it in variable arguments. The only examples I have in mind are functions like min or max, that receive variable arguments, and that you might want to call on an array:

```
const ponyPrices = [12, 3, 4];
const minPrice = Math.min(...ponyPrices);
```

2.8. Classes

One of the most emblematic new features, and one that we will vastly use when writing an Angular app: ES2015 introduces classes to JavaScript! You can now easily use classes and inheritance in JavaScript. You always could, using prototypal inheritance, but that was not an easy task, especially for beginners.

Now it's very easy, take a look:

```
class Pony {
  constructor(color) {
    this.color = color;
  }

  toString() {
    return `${this.color} pony`;
    // see that? It is another cool feature of ES2015, called template literals
    // we'll talk about these quickly!
  }
}
const bluePony = new Pony('blue');
console.log(bluePony.toString()); // blue pony
```

Class declarations, unlike function declarations, are not hoisted, so you need to declare a class before using it. You may have noticed the special function `constructor`. It is the function being called when we create a new pony, with the `new` operator. Here it needs a color, and we create a new Pony instance with the color set to "blue". A class can also have methods, callable on an instance, as the method `toString()` here.

It can also have static attributes and methods:

```
class Pony {
  static defaultSpeed() {
    return 10;
  }
}
```

Static methods can be called only on the class directly:

```
const speed = Pony.defaultSpeed();
```

A class can have getters and setters, if you want to hook onto these operations:

```
class Pony {
  get color() {
    console.log('get color');
    return this._color;
  }

  set color(newColor) {
    console.log(`set color ${newColor}`);
    this._color = newColor;
  }
}
const pony = new Pony();
pony.color = 'red';
// 'set color red'
console.log(pony.color);
// 'get color'
// 'red'
```

And, of course, if you have classes, you also have inheritance out of the box in ES2015.

```
class Animal {
  speed() {
    return 10;
  }
}
class Pony extends Animal {}
const pony = new Pony();
console.log(pony.speed()); // 10, as Pony inherits the parent method
```

Animal is called the base class, and Pony the derived class. As you can see, the derived class has the methods of the base class. It can also override them:

```
class Animal {
  speed() {
    return 10;
  }
}
class Pony extends Animal {
  speed() {
    return super.speed() + 10;
  }
}
const pony = new Pony();
console.log(pony.speed()); // 20, as Pony overrides the parent method
```

As you can see, the keyword `super` allows calling the method of the base class, with `super.speed()` for example.

The `super` keyword can also be used in constructors, to call the base class constructor:

```
class Animal {
  constructor(speed) {
    this.speed = speed;
  }
}
class Pony extends Animal {
  constructor(speed, color) {
    super(speed);
    this.color = color;
  }
}
const pony = new Pony(20, 'blue');
console.log(pony.speed); // 20
```

2.9. Promises

Promises are not so new, and you might know them or use them already, as they were a big part of AngularJS 1.x. But since you will use them a lot in Angular, and even if you're just using JS, I think it's important to make a stop.

Promises aim to simplify asynchronous programming. Our JS code is full of async stuff, like AJAX requests, and usually we use callbacks to handle the result and the error. But it can get messy, with callbacks inside callbacks, and it makes the code hard to read and to maintain. Promises are much nicer than callbacks, as they flatten the code, and thus make it easier to understand. Let's consider a simple use case, where we need to fetch a user, then their rights, then update a menu when we have these.

With callbacks:

```
getUser(login, function (user) {
  getRights(user, function (rights) {
    updateMenu(rights);
  });
});
```

Now, let's compare it with promises:

```
getUser(login)
  .then(function (user) {
    return getRights(user);
  })
  .then(function (rights) {
    updateMenu(rights);
  })
```

I like this version, because it executes as you read it: I want to fetch a user, then get their rights, then update the menu.

As you can see, a promise is a 'thenable' object, which simply means it has a `then` method. This method takes two arguments: one success callback and one reject callback. The promise has three states:

- pending: while the promise is not done, for example, our server call is not completed yet.
- fulfilled: when the promise is completed with success, for example, the server call returns an OK HTTP status.
- rejected: when the promise has failed, for example, the server returns a 404 status.

When the promise is fulfilled, then the success callback is called, with the result as an argument. If the promise is rejected, then the reject callback is called, with a rejected value or an error as the argument.

So, how do you create a promise? Pretty simple, there is a new class called `Promise`, whose constructor expects a function with two parameters, `resolve` and `reject`.

```
const getUser = function (login) {
  return new Promise(function (resolve, reject) {
    // async stuff, like fetching users from server, returning a response
    if (response.status === 200) {
      resolve(response.data);
    } else {
      reject('No user');
    }
  });
};
```

Once you have created the promise, you can register callbacks, using the `then` method. This method can receive two parameters, the two callbacks you want to call in case of success or in case of failure. Here we only pass a success callback, ignoring the potential error:

```
getUser(login)
  .then(function (user) {
    console.log(user);
  })
```

Once the promise is resolved, the success callback (here simply logging the user on the console) will be called.

The cool part is that it flattens the code. For example, if your resolve callback is also returning a promise, you can write:

```
getUser(login)
  .then(function (user) {
    return getRights(user) // getRights is returning a promise
      .then(function (rights) {
        return updateMenu(rights);
      });
  })
```

but more beautifully:

```
getUser(login)
  .then(function (user) {
    return getRights(user); // getRights is returning a promise
  })
  .then(function (rights) {
    return updateMenu(rights);
  })
```

Another interesting thing is the error handling, as you can use one handler per promise, or one for all the chain.

One per promise:

```
getUser(login)
  .then(
    function (user) {
      return getRights(user);
    },
    function (error) {
      console.log(error); // will be called if getUser fails
      return Promise.reject(error);
    }
  )
  .then(
    function (rights) {
      return updateMenu(rights);
    },
    function (error) {
      console.log(error); // will be called if getRights fails
      return Promise.reject(error);
    }
  )
```

One for the chain:

```
getUser(login)
  .then(function (user) {
    return getRights(user);
  })
  .then(function (rights) {
    return updateMenu(rights);
  })
  .catch(function (error) {
    console.log(error); // will be called if getUser or getRights fails
  })
```

You should seriously look into Promises, because they are going to be the new way to write APIs, and every library will use them. Even the standard ones: the new Fetch API does for example.

2.10. Arrow functions

One thing I like a lot in ES2015 is the new arrow function syntax, using the 'fat arrow' operator (⇒). It is SO useful for callbacks and anonymous functions!

Let's take our previous example with promises:

```
getUser(login)
  .then(function (user) {
    return getRights(user); // getRights is returning a promise
  })
  .then(function (rights) {
    return updateMenu(rights);
  })
```

can be written with arrow functions like this:

```
getUser(login)
  .then(user => getRights(user))
  .then(rights => updateMenu(rights))
```

How cool is it? THAT cool!

Note that the return is also implicit if there is no block: no need to write `user ⇒ return getRights(user)`. But if we did have a block, we would need the explicit return:

```
getUser(login)
  .then(user => {
    console.log(user);
    return getRights(user);
  })
  .then(rights => updateMenu(rights))
```

And it has a special trick, a great power over normal functions: the `this` stays lexically bounded, which means that these functions don't have a new `this` as other functions do. Let's take an example, where you are iterating over an array with the `map` function to find the max.

In ES5:

```
var maxFinder = {
  max: 0,
  find: function (numbers) {
    // let's iterate
    numbers.forEach(function (element) {
      // if the element is greater, set it as the max
      if (element > this.max) {
        this.max = element;
      }
    });
  }
};

maxFinder.find([2, 3, 4]);
// log the result
console.log(maxFinder.max);
```

You would expect this to work, but it doesn't. If you have a good eye, you may have noticed that the `forEach` in the `find` function uses `this`, but the `this` is not bound to an object. So `this.max` is not the `max` of the `maxFinder` object... Of course you can fix it easily, using an alias:

```
var maxFinder = {
  max: 0,
  find: function (numbers) {
    var self = this;
    numbers.forEach(function (element) {
      if (element > self.max) {
        self.max = element;
      }
    });
  }
};

maxFinder.find([2, 3, 4]);
// log the result
console.log(maxFinder.max);
```

or binding the `this`:

```
var maxFinder = {
  max: 0,
  find: function (numbers) {
    numbers.forEach(
      function (element) {
        if (element > this.max) {
          this.max = element;
        }
      }.bind(this)
    );
  }
};

maxFinder.find([2, 3, 4]);
// log the result
console.log(maxFinder.max);
```

or even passing it as a second parameter of the `forEach` function (as it was designed for):

```
var maxFinder = {
  max: 0,
  find: function (numbers) {
    numbers.forEach(function (element) {
      if (element > this.max) {
        this.max = element;
      }
    }, this);
  }
};

maxFinder.find([2, 3, 4]);
// log the result
console.log(maxFinder.max);
```

But there is now an even more elegant solution with the arrow function syntax:

```
const maxFinder = {
  max: 0,
  find: function (numbers) {
    numbers.forEach(element => {
      if (element > this.max) {
        this.max = element;
      }
    });
  }
};

maxFinder.find([2, 3, 4]);
// log the result
console.log(maxFinder.max);
```

That makes the arrow functions the perfect candidates for anonymous functions in callbacks!

2.11. Async/await

We were talking about promises earlier, and it's worth knowing that another keyword was introduced to handle them more synchronously: `await`.

This is not a feature introduced in ECMAScript 2015 but in ECMAScript 2017, and to use `await`, your function must be marked as `async`. When you use the `await` keyword in front of a Promise, you pause the execution of your `async` function, wait for the Promise to resolve, and then resume the execution of the `async` function. The returned value will be the resolved value.

So we can write our previous example using `async/await` like this:

```
async function getUserRightsAndUpdateMenu() {
  // getUser is a promise
  const user = await getUser(login);
  // getRights is a promise
  const rights = await getRights(user);
  updateMenu(rights);
}
await getUserRightsAndUpdateMenu();
```

And your code now looks like it is synchronous! Another cool feature of `async/await` is that you can use a simple `try/catch` to handle errors:

```
async function getUserRightsAndUpdateMenu() {
  try {
    // getUser is a promise
    const user = await getUser(login);
    // getRights is a promise
    const rights = await getRights(user);
    updateMenu(rights);
  } catch (e) {
    // will be called if getUser, getRights or updateMenu fails
    console.log(e);
  }
}
await getUserRightsAndUpdateMenu();
```

Note that async/await is still asynchronous, although it looks like synchronous. The function execution is paused and resumed, but just like with callbacks, this doesn't block the thread: other JavaScript events can be handled while the execution is paused.

2.12. Sets and Maps

This is a short one: you now have proper collections in ES2015. Yay \o/! We used to have dictionaries filling the role of a map, but we can now use the class `Map`:

```
const cedric = { id: 1, name: 'Cedric' };
const users = new Map();
users.set(cedric.id, cedric); // adds a user
console.log(users.has(cedric.id)); // true
console.log(users.size); // 1
users.delete(cedric.id); // removes the user
```

We also have a class `Set`:

```
const cedric = { id: 1, name: 'Cedric' };
const users = new Set();
users.add(cedric); // adds a user
console.log(users.has(cedric)); // true
console.log(users.size); // 1
users.delete(cedric); // removes the user
```

You can iterate over a collection, with the new syntax `for ⋯ of`:

```
for (let user of users) {
  console.log(user.name);
}
```

You'll see that the `for ... of` syntax is the one the Angular team chose in order to iterate over a collection in a template.

2.13. Template literals

Composing strings has always been painful in JavaScript, as we usually have to use concatenation:

```
const fullname = 'Miss ' + firstname + ' ' + lastname;
```

Template literals are a new small feature, where you have to use backticks (`) instead of quotes or simple quotes, and you have a basic templating system, with multiline support:

```
const fullname = `Miss ${firstname} ${lastname}`;
```

The multiline support is especially great when your are writing HTML strings, as we will do for our Angular components:

```
const template = `<div>
  <h1>Hello</h1>
</div>`;
```

One last feature is the ability to tag them. You can define a function, and apply it to a template string. Here `askQuestion` adds an interrogation point at the end of the string:

```
const askQuestion = strings => strings + '?';
const template = askQuestion`Hello there`;
```

So what's the difference with a simple function? The tag function in fact receives several arguments:

- an array of the static parts of the string
- the values resulting from the evaluation of the expressions

For example if we have a template string containing expressions:

```
const person1 = 'Cedric';
const person2 = 'Agnes';
const template = `Hello ${person1}! Where is ${person2}?`;
```

then the tag function will receive the various static and dynamic parts. Here we have a tag function to uppercase the names of the protagonists:

```
const uppercaseNames = (strings, ...values) => {
  // `strings` is an array with the static parts ['Hello ', '! Where is ', '?']
  // `values` is an array with the evaluated expressions ['Cedric', 'Agnes']
  const names = values.map(name => name.toUpperCase());
  // `names` now has ['CEDRIC', 'AGNES']
  // let's merge the `strings` and `names` arrays
  return strings.map((string, i) => `${string}${names[i] ? names[i] : ''}`).join('');
};
const result = uppercaseNames`Hello ${person1}! Where is ${person2}?`;
// returns 'Hello CEDRIC! Where is AGNES?'
```

Let's now talk about one of the big changes introduced: modules.

2.14. Modules

A standard way to organize functions in namespaces and to dynamically load code in JS has always been lacking. Node.js has been one of the leaders in this, with a thriving ecosystem of modules using the CommonJS convention. On the browser side, there is also the AMD (Asynchronous Module Definition) API, used by RequireJS. But none of these were a real standard, thus leading to endless discussions on what's best.

ES2015 aims to create a syntax using the best from both worlds, without caring about the actual implementation. The Ecma TC39 committee (which is responsible for evolving ES2015 and authoring the specification of the language) wanted to have a nice and easy syntax (that's arguably CommonJS's strong suit), but to support asynchronous loading (like AMD), and a few goodies like the possibility to statically analyze the code by tools and support cyclic dependencies nicely. The new syntax handles how you export and import things to and from modules.

This module thing is really important in Angular, as pretty much everything is defined in modules, which you have to import when you want to use them. Let's say I want to expose a function to bet on a specific pony in a race and a function to start the race.

In races.service.js:

```
export function bet(race, pony) {
  // ...
}
export function start(race) {
  // ...
}
```

As you can see, this is fairly easy: the new keyword `export` does a straightforward job and exports the two functions.

Now, let's say one of our application components needs to call these functions.

In another file:

```
import { bet, start } from './races.service';
```

```
// later
bet(race, pony1);
start(race);
```

That's what is called a *named export*. Here we are importing the two functions, and we have to specify the filename containing these functions - here 'races.service'. Of course, you can import only one method if you need, and you can even give it an alias:

```
import { start as startRace } from './races.service';
```

```
// later
startRace(race);
```

And if you want to use all the exported symbols (functions, constants, classes etc.) from the module, you can use a wildcard '*'.

As you would do with other languages, use the wildcard with care, only when you really want all the functions, or most of them. As this will be analyzed by our IDEs, we will see auto-import soon and that will free us from the bother of importing the right things.

With a wildcard, you have to use an alias, and I kind of like it, because it makes the rest of the code clearer:

```
import * as racesService from './races.service';
```

```
// later
racesService.bet(race, pony1);
racesService.start(race);
```

If your module exposes only one function or value or class, you don't have to use named export, and you can leverage the default keyword. It works great for classes for example:

```
// pony.js
export default class Pony {}
// races.service.js
import Pony from './pony';
```

Notice the lack of curly braces to import a default. You can import it with the alias you want, but to be consistent, it's better to call the import with the module name (except if you have multiple modules with the same name of course, then you can choose an alias that allows you to distinguish

them). And of course, you can mix default export with named ones, but obviously with only one default per module.

In Angular, you're going to use a lot of these imports in your app. Each component and service will be a class, generally isolated in their own file and exported, and then imported when needed in other components.

2.15. Conclusion

That ends our gentle introduction to ES2015+. We skipped some other parts, but if you're comfortable with this chapter, you will have no problem writing your apps in ES2015+. If you want to have a deeper understanding of this, I highly recommend Exploring JS by Axel Rauschmayer or Understanding ES6 from Nicholas C. Zakas... Both ebooks can be read online for free, but don't forget to buy it to support their authors. They have done great work! Actually I've re-read Speaking JS, Axel's previous book, and I again learned a few things, so if you want to refresh your JS skills, I definitely recommend it!

Chapter 3. Going further than ES2015+

3.1. Dynamic, static and optional types

You may have heard that Angular apps can be written in TypeScript. And you may be wondering what TypeScript is, or what it brings to the table.

JavaScript is dynamically typed. That means you can do things like:

```
let pony = 'Rainbow Dash';
pony = 2;
```

And it works. That's great for all sort of things, as you can pass pretty much any object to a function and it works, as long as the object has the properties the function needs:

```
const pony = { name: 'Rainbow Dash', color: 'blue' };
const horse = { speed: 40, color: 'black' };
const printColor = animal => console.log(animal.color);
// works as long as the object has a `color` property
```

This dynamic nature allows wonderful things but it is also a pain for a few other reasons compared to more statically-typed languages. The most obvious might be when you call an unknown function in JS from another API, you pretty much have to read the doc (or, worse, the function code) to know what the parameter should look like. Take a look at our previous example: the method `printColor` needs a parameter with a `color` property. That can be hard to guess, and of course it is much worse in day-to-day work, where we use various libraries and services developed by fellow developers. One of Ninja Squad's co-founders is often complaining about the lack of types in JS, and finds it regrettable he can't be as productive and write as good code as he would in a more statically-typed environment. And he is not entirely wrong, even if he is sometimes ranting for the sake of it too! Without type information, IDEs have no real clue if you're doing something wrong, and tools can't help you find bugs in your code. Of course, we have tests in our apps, and Angular has always been keen on making testing easy, but it's nearly impossible to have a perfect test coverage.

That leads to the maintainability topic. JS code can become hard to maintain, despite tests and documentation. Refactoring a huge JS app is no easy task, compared to what could be done in other statically-typed languages. Maintainability is a very important topic, and types help humans and tools to avoid mistakes when writing and maintaining code. Google has always been keen to push new solutions in that direction: it's easy to understand as they have some of the biggest web apps of the world, with GMail, Google apps, Maps… So they have tried several approaches to front-end maintainability: GWT, Google Closure, Dart… All trying to help writing big webapps.

For Angular, the Google team wanted to help us to write better JS, by adding some type information to our code. It's not a very new concept in JS. It was even the subject of the ECMAScript 4 specification, which was later abandoned. At first they announced AtScript, as a superset of ES2015+ with annotations (types annotations and another kind I'll discuss later). They also announced the support of TypeScript, the Microsoft language, with additional type annotations.

And then, a few months later, the TypeScript team announced that they had worked closely with the Google team, and the new version of the language (1.5) would have all the shiny new things AtScript had. And the Google team announced that AtScript was officially dropped, and that TypeScript was the new top-notch way to write Angular apps!

3.2. Enters TypeScript

I think this was a smart move for several reasons. For one, no one really wants to learn another language extension. And TypeScript was already there, with an active community and ecosystem. I never really used it before Angular, but I heard good things about it, from various people. TypeScript is a Microsoft project. But it's not the Microsoft you have in mind, from the Ballmer and Gates years. It's the Microsoft of the Nadella era, the one opening up to its community, and, well, open-source. Google knows this, and it's far better for them to contribute to an existing project, rather than to have to bear the burden of maintaining their own. And the TypeScript framework will gain a huge popularity boost: win-win, as your manager would say.

But the main reason to bet on TypeScript is the type system it offers. It's an optional type system that helps without getting in the way. In fact, after coding some time with it, you'll probably want to code every application with it. I do like what they have done, and we will have a look at what TypeScript offers in the next section. At the end, you'll have enough understanding to read any Angular code, and you'll be able to choose whether you want to use it or not, in your apps.

You may be wondering: why use typed code in Angular apps? Let's take an example. Angular 1 and 2 have been built around a powerful concept named "dependency injection". You might already be familiar with it, as it is a common design pattern used in several frameworks for different languages and, as I said, already used in AngularJS 1.x.

3.3. A practical example with DI

To sum up what dependency injection is, think about a component of the app, let's say `RaceList`, needing to access the races list that the service `RaceService` can give. You would write `RaceList` like this:

```
class RaceList {
  constructor() {
    this.raceService = new RaceService();
    // let's say that list() returns a promise
    this.raceService
      .list()
      // we store the races returned into a member of `RaceList`
      .then(races => (this.races = races));
    // arrow functions, FTW!
  }
}
```

But it has several flaws. One of them is the testability: it is now very hard to replace the `raceService` by a fake (mock) one, to test our component.

If we use the Dependency Injection (DI) pattern, we delegate the creation of the `RaceService` to the framework, and we simply ask for an instance. The framework is now in charge of the creation of the dependency, and, well, injects it:

```
class RaceList {
  constructor(raceService) {
    this.raceService = raceService;
    this.raceService.list().then(races => (this.races = races));
  }
}
```

Now, when we test this class, we can easily pass a fake service to the constructor:

```
// in a test
const fakeService = {
  list: () => {
    // returns a fake promise
  }
};
const raceList = new RaceList(fakeService);
// now we are sure that the race list
// is the one we want for the test
```

But how does the framework know what to inject in the constructor? Good question! AngularJS 1.x relied on the parameter's names, but it had a severe limitation, because minification of your code would have changed the param name… You could use the array syntax to fix this, or add a metadata to the class:

```
RaceList.$inject = ['RaceService'];
```

We had to add some metadata for the framework to understand what classes needed to be injected with. And that's exactly what type annotations give: a metadata giving the framework a hint it needs to do the right injection. In Angular, using TypeScript, we can write our `RaceList` component like:

```
class RaceList {
  raceService: RaceService;
  races: Array<string>;

  constructor(raceService: RaceService) {
    // the interesting part is `: RaceService`
    this.raceService = raceService;
    this.raceService.list().then(races => (this.races = races));
  }
}
```

Now the injection can be done!

That's why we're going to spend some time learning TypeScript (TS). Angular is clearly built to leverage this language, so we will have the easiest time writing our apps using it. And the Angular team really hopes to submit the type system to the standard committee, so maybe one day we'll have types in JS, and all this will be usual.

Let's dive in!

Chapter 4. Diving into TypeScript

TypeScript has been around since 2012. It's a superset of JavaScript, adding a few things to ES5. The most important one is the type system, giving TypeScript its name. From version 1.5, released in 2015, the library is trying to be a superset of ES2015+, including all the shiny features we saw in the previous chapter, and a few new things as well, like decorators. Writing TypeScript feels very much like writing JavaScript. By convention, TypeScript files are named with a `.ts` extension, and they will need to be compiled to standard JavaScript, usually at build time, using the TypeScript compiler. The generated code is very readable.

```
npm install -g typescript
tsc test.ts
```

But let's start with the beginning.



4.1. Types as in TypeScript

The general syntax to add type info in TypeScript is rather straightforward:

```
let variable: type;
```

The types are easy to remember:

```
const ponyNumber: number = 0;
const ponyName: string = 'Rainbow Dash';
```

In such cases, the types are optional because the TS compiler can guess them (it's called "type inference") from the values.

The type can also come from your app, as with the following class `Pony`:

```
const pony: Pony = new Pony();
```

TypeScript also supports what some languages call "generics", for example for an array:

```
const ponies: Array<Pony> = [new Pony()];
```

The array can only contain ponies, and the generic notation, using `<>`, indicates this. You may be wondering what the point of doing this is. Adding types information will help the compiler catch possible mistakes:

```
ponies.push('hello'); // error TS2345
// Argument of type 'string' is not assignable to parameter of type 'Pony'.
```

So, if you need a variable to have multiple types, does it mean you're screwed? No, because TS has a special type, called `any`.

```
let changing: any = 2;
changing = true; // no problem
```

It's really useful when you don't know the type of a value, either because it's from a dynamic content or from a library you're using.

If your variable can only be of type `number` or `boolean`, you can use a union type:

```
let changing: number | boolean = 2;
changing = true; // no problem
```

4.2. Enums

TypeScript also offers `enum`. For example, a race in our app can be either `ready`, `started` or `done`.

```
enum RaceStatus {
  Ready,
  Started,
  Done
}
```

```
const race = new Race();
race.status = RaceStatus.Ready;
```

The enum is in fact a numeric value, starting at 0. You can set the value you want, though:

```
enum Medal {
  Gold = 1,
  Silver,
  Bronze
}
```

Since TypeScript 2.4, you can even specify a string value:

```
enum Position {
  First = 'First',
  Second = 'Second',
  Other = 'Other'
}
```

To be honest though, we don't use enums a lot in our projects: we use union types. They are simpler and cover roughly the same use-cases:

```
let color: 'blue' | 'red' | 'green';
// we can only give one of these values to `color`
color = 'blue';
```

TypeScript even allows you to create your own types, so you could do something like:

```
type Color = 'blue' | 'red' | 'green';
const ponyColor: Color = 'blue';
```

4.3. Return types

You can also set the return type of a function:

```
function startRace(race: Race): Race {
  race.status = RaceStatus.Started;
  return race;
}
```

If the function returns nothing, you can show it using `void`:

```
function startRace(race: Race): void {
  race.status = RaceStatus.Started;
}
```

4.4. Interfaces

That's a good first step. But as I said earlier, JavaScript is great for its dynamic nature. A function will work if it receives an object with the correct property:

```
function addPointsToScore(player, points): void {
  player.score += points;
}
```

This function can be applied to any object with a `score` property. How do you translate this in TypeScript? It's easy: you define an interface, which is like the "shape" of the object.

```
function addPointsToScore(player: { score: number }, points: number): void {
  player.score += points;
}
```

It means that the parameter must have a property called `score` of the type `number`. You can name these interfaces, of course:

```
interface HasScore {
  score: number;
}
```

```
function addPointsToScore(player: HasScore, points: number): void {
  player.score += points;
}
```

You'll see that we often use interfaces throughout the book to represent our entities. We use interfaces for our models in our other projects as well. We usually append a `Model` suffix to make it clear. It's then very easy to create a new entity:

```
interface PonyModel {
  name: string;
  speed: number;
}
const pony: PonyModel = { name: 'Light Shoe', speed: 56 };
```

4.5. Optional arguments

Another treat of JavaScript is that arguments are optional. You can omit them, and they will become `undefined`. But if you define a function with typed parameter in TypeScript, the compiler will shout at you if you forget them:

```
addPointsToScore(player); // error TS2346
// Supplied parameters do not match any signature of call target.
```

To show that a parameter is optional in a function (or a property in an interface), you can add `?` after the parameter. Here, the `points` parameter could be optional:

```
function addPointsToScore(player: HasScore, points?: number): void {
  points = points || 0;
  player.score += points;
}
```

4.6. Functions as property

You may also be interested in describing a parameter that must have a specific function instead of a property:

```
function startRunning(pony): void {
  pony.run(10);
}
```

The interface definition will be:

```
interface CanRun {
  run(meters: number): void;
}
```

```
function startRunning(pony: CanRun): void {
  pony.run(10);
}

const ponyOne = {
  run: (meters: number) => logger.log(`pony runs ${meters}m`)
};
startRunning(ponyOne);
```

4.7. Classes

A class can implement an interface. For us, the `Pony` class should be able to run, so we can write:

```
class Pony implements CanRun {
  run(meters: number): void {
    logger.log(`pony runs ${meters}m`);
  }
}
```

The compiler will force us to implement a `run` method in the class. If we implement it badly, by expecting a `string` instead of a `number` for example, the compiler will yell:

```
class IllegalPony implements CanRun {
  run(meters: string) {
    console.log(`pony runs ${meters}m`);
  }
}
// error TS2420: Class 'IllegalPony' incorrectly implements interface 'CanRun'.
// Types of property 'run' are incompatible.
```

You can also implement several interfaces if you want:

```
class HungryPony implements CanRun, CanEat {
  run(meters: number): void {
    logger.log(`pony runs ${meters}m`);
  }

  eat(): void {
    logger.log(`pony eats`);
  }
}
```

And an interface can extend one or several others:

```
interface Animal extends CanRun, CanEat {}

class Pony implements Animal {
  // ...
}
```

When you're defining a class in TypeScript, you can have properties and methods in your class. You may realize that properties in classes are not a standard ES2015+ feature. It is only possible in TypeScript.

```
class SpeedyPony {
  speed = 10;

  run(): void {
    logger.log(`pony runs at ${this.speed}m/s`);
  }
}
```

Everything is public by default, but you can use the `private` keyword to hide a property or a method. If you add `private` or `public` to a constructor parameter, it is a shortcut to create and initialize a private or public member:

```
class NamedPony {
  constructor(public name: string, private speed: number) {}

  run(): void {
    logger.log(`pony runs at ${this.speed}m/s`);
  }
}
```

```
const pony = new NamedPony('Rainbow Dash', 10);
// defines a public property name with 'Rainbow Dash'
// and a private one speed with 10
```

Which is the same as the more verbose:

```
class NamedPonyWithoutShortcut {
  public name: string;
  private speed: number;

  constructor(name: string, speed: number) {
    this.name = name;
    this.speed = speed;
  }

  run(): void {
    logger.log(`pony runs at ${this.speed}m/s`);
  }
}
```

These shortcuts are really useful and we'll rely on them a lot in Angular!

4.8. Working with other libraries

When working with external libraries written in JS, you may think we are doomed because we

don't know what types of parameter the function in that library will expect. That's one of the cool things with the TypeScript community: its members have defined interfaces for the types and functions exposed by the popular JavaScript libraries!

The files containing these interfaces have a special `.d.ts` extension. They contain a list of the library's public functions. A good place to look for these files is DefinitelyTyped. For example, if you want to use TS in your AngularJS 1.x apps, you can download the proper file from the repo directly with NPM:

```
npm install --save-dev @types/angular
```

or download it manually. Then include the file at the top of your code, and enjoy the compilation checks:

```
/// <reference path="angular.d.ts" />
angular.module(10, []); // the module name should be a string
// so when I compile, I get:
// Argument of type 'number' is not assignable to parameter of type 'string'.
```

`/// <reference path="angular.d.ts" />` is a special comment recognized by TS, telling the compiler to look for the interface `angular.d.ts`. Now, if you misuse an AngularJS method, the compiler will complain, and you can fix it on the spot, without having to manually run your app!

Even cooler, since TypeScript 1.6, the compiler will auto-discover the type definitions of an NPM library if they are packaged with the library itself. More and more projects are adopting this approach, and so is Angular. So you don't even have to worry about including the interfaces in your Angular project: the TS compiler will figure it out by itself if you are using NPM to manage your dependencies!

4.9. Decorators

This feature was added in TypeScript 1.5, notably to help support Angular. Indeed, as we will shortly see, Angular components can be described using decorators. You may not have heard about decorators, as not every language has them. A decorator is a way to do some meta-programming. They are fairly similar to annotations which are mainly used in Java, C# and Python, and maybe other languages I don't know. Depending on the language, you add an annotation to a method, an attribute, or a class. Generally, annotations are not really used by the language itself, but mainly by frameworks and libraries.

Decorators are really powerful: they can modify their target (method, classes, etc.), and for example alter the parameters of the call, tamper with the result, call other methods when the target is called or add metadata for a framework (which is what Angular decorators do). Until now, it was not something that was possible in JavaScript. But the language is evolving and there is now an official proposal for `decorators`, which may be standardized one day in the future (possibly in ES7/ES2016). Note that the TypeScript implementation goes slightly further than the proposed standard.

In Angular, we will use the decorators provided by the framework. Their role is fairly basic: they

add some metadata to our classes, attributes or parameters to say things like "this class is a component", "this is an optional dependency", "this is a custom property", etc. You are not required to use them, as you can add the metadata manually (if you want to stick to ES5 for example), but the code will definitely be more elegant using decorators, as provided by TypeScript.

In TypeScript, decorators start with an `@`, and can be applied to a class, a class property, a function or a function parameter. They can't be applied to a constructor, but can be applied to its parameters.

To have a better grasp on this, let's try to build a simple decorator, `@Log()`, that will log something every time a method is called.

It will be used like this:

```
class RaceService {
  @Log()
  getRaces() {
    // call API
  }

  @Log()
  getRace(raceId: number) {
    // call API
  }
}
```

To define it, we have to write a method returning a function like this:

```
const Log = () => {
  return (target: any, name: string, descriptor: any) => {
    logger.log(`call to ${name}`);
    return descriptor;
  };
};
```

Depending on what you want to apply your decorator to, the function will not have exactly the same arguments. Here we have a method decorator that takes 3 parameters:

- `target`: the method targeted by our decorator
- `name`: the name of the targeted method
- `descriptor`: a descriptor of the targeted method (is the method enumerable, writable, etc.)

Here we simply log the method name, but you could do pretty much whatever you want: interfere with the parameters, the result, calling another function, etc.

So, in our simple example, every time the `getRace()` or `getRaces()` methods are called, we'll see a trace in the browser logs:

```
raceService.getRaces();
// logs: call to getRaces
raceService.getRace(1);
// logs: call to getRace
```

As a user, let's look at what a decorator in Angular looks like:

```
@Component({ selector: 'ns-home', template: 'home' })
class HomeComponent {
  constructor(@Optional() hello: HelloService) {
    logger.log(hello);
  }
}
```

The `@Component` decorator is added to the class `Home`. When Angular loads our app, it will find the class `Home` and will understand that it is a component, based on the metadata the decorator will add. Cool, huh? As you can see, a decorator can also receive parameters, here a configuration object.

I just wanted to introduce the raw concept of decorators; we'll look into every decorator available in Angular throughout the book.

So my advice would be to give TypeScript a try! All my examples from here will be in TypeScript, as Angular and all the tooling around are really designed for it.

Chapter 5. Advanced TypeScript

If you're just starting to learn TypeScript, you can safely skip this chapter for now and come back later. This chapter is here to showcase some more advanced usages of TypeScript. They'll only make sense if you already have some familiarity with the language

5.1. readonly

You can use the `readonly` keyword to mark the property of a class or interface as... read only! That way, the compiler will refuse to compile any code trying to assign a new value to the property:

```
interface Config {
  readonly timeout: number;
}

const config: Config = { timeout: 2000 };
// `config.timeout` is now readonly and can't be reassigned
```

5.2. keyof

The `keyof` keyword can be used to get a type representing the union of the names of the properties of another type. For example, you have a `PonyModel` interface:

```
interface PonyModel {
  name: string;
  color: string;
  speed: number;
}
```

You want to build a function that returns the value of a property. You could implement a naive version:

```
function getProperty(obj: any, key: string): any {
  return obj[key];
}

const pony: PonyModel = {
  name: 'Rainbow Dash',
  color: 'blue',
  speed: 45
};
const nameValue = getProperty(pony, 'name');
```

Two problems here:

- you can give any value to the `key` parameter, even keys that don't exist on `PonyModel`.
- the return type being `any`, you are losing a lot of type information.

This is where `keyof` can shine. `keyof` allows you to list all the keys of a type:

```
type PonyModelKey = keyof PonyModel;
// this is the same as `'name'|'speed'|'color'`
let property: PonyModelKey = 'name'; // works
property = 'speed'; // works
// key = 'other' would not compile
```

So we can use this type to make `getProperty` safer, by declaring that:

- the first parameter is of type `T`
- the second parameter is of type `K`, which is a key of `T`

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}

const pony: PonyModel = {
  name: 'Rainbow Dash',
  color: 'blue',
  speed: 45
};
// TypeScript infers that `nameValue` is of type `string`!
const nameValue = getProperty(pony, 'name');
```

We killed two birds with one stone here:

- `key` can now only be an existing property of `PonyModel`
- the return value will be inferred by TypeScript (which is pretty awesome!)

Now let's see how we can leverage `keyof` to do even more.

5.3. Mapped type

Let's say you want to create a type that has exactly the same properties as `PonyModel`, but you want every property to be optional. You can of course define it manually:

```
interface PartialPonyModel {
  name?: string;
  color?: string;
  speed?: number;
}

const pony: PartialPonyModel = {
  name: 'Rainbow Dash'
};
```

But you can do something more generic with a mapped type:

```
type Partial<T> = {
  [P in keyof T]?: T[P];
};

const pony: Partial<PonyModel> = {
  name: 'Rainbow Dash'
};
```

The `Partial` type is a transformation that applies the `?` modifier to every property of a type! In fact, you don't have to define the type `Partial` yourself, because since version 2.1, it's part of the language itself, and it's declared exactly like in the above example.

TypeScript offers other mapped types out of the box.

5.3.1. Readonly

`Readonly` makes all the properties of an object `readonly`:

```
const pony: Readonly<PonyModel> = {
  name: 'Rainbow Dash',
  color: 'blue',
  speed: 45
};
// all properties are `readonly`
```

5.3.2. Pick

`Pick` helps you build a type with only some of the original properties:

```
const pony: Pick<PonyModel, 'name' | 'color'> = {
  name: 'Rainbow Dash',
  color: 'blue'
};
// `pony` can't have a `speed` property
```

5.3.3. Record

`Record` helps you build a type with the same properties as another type, but with a different type:

```
interface FormValue {
  value: string;
  valid: boolean;
}

const pony: Record<keyof PonyModel, FormValue> = {
  name: { value: 'Rainbow Dash', valid: true },
  color: { value: 'blue', valid: true },
  speed: { value: '45', valid: true }
};
```

There are even more than that, but these are the most useful.

5.4. Union types and type guards

Union types are really handy. Let's say your application has authenticated users and anonymous users, and sometimes you need to do a different action depending on that. You can model this as:

```ts
interface User {
  type: 'authenticated' | 'anonymous';
  name: string;
  // other fields
}

interface AuthenticatedUser extends User {
  type: 'authenticated';
  loggedSince: number;
}

interface AnonymousUser extends User {
  type: 'anonymous';
  visitingSince: number;
}

function onWebsiteSince(user: User): number {
  if (user.type === 'authenticated') {
    // this is a LoggedUser
    return (user as AuthenticatedUser).loggedSince;
  } else if (user.type === 'anonymous') {
    // this is an AnonymousUser
    return (user as AnonymousUser).visitingSince;
  }
  // TS doesn't know every possibility was covered
  // so we have to return something here
  return 0;
}
```

I don't know about you, but I don't like these `as ⋯` explicit casts. Maybe we can do better?

One possibility is to use a type guard, a special function whose sole purpose is to help the TypeScript compiler.

```
function isAuthenticated(user: User): user is AuthenticatedUser {
  return user.type === 'authenticated';
}

function isAnonymous(user: User): user is AnonymousUser {
  return user.type === 'anonymous';
}

function onWebsiteSince(user: User): number {
  if (isAuthenticated(user)) {
    // this is inferred as a LoggedUser
    return user.loggedSince;
  } else if (isAnonymous(user)) {
    // this is inferred as an AnonymousUser
    return user.visitingSince;
  }
  // TS still doesn't know every possibility was covered
  // so we have to return something here
  return 0;
}
```

This is better! But we still need to return a default value, even if we covered all the possibilities.

We can slightly improve the situation if we drop the type guards and use a union type instead.

```ts
interface BaseUser {
  name: string;
  // other fields
}

interface AuthenticatedUser extends BaseUser {
  type: 'authenticated';
  loggedSince: number;
}

interface AnonymousUser extends BaseUser {
  type: 'anonymous';
  visitingSince: number;
}

type User = AuthenticatedUser | AnonymousUser;

function onWebsiteSince(user: User): number {
  if (user.type === 'authenticated') {
    // this is inferred as a LoggedUser
    return user.loggedSince;
  } else {
    // this is narrowed as an AnonymousUser
    // without even testing the type!
    return user.visitingSince;
  }
  // no need to return a default value
  // as TS knows that we covered every possibility!
}
```

This is even better, as TypeScript automatically narrows the type in the `else` branch.

Sometimes you know that the model will grow in the future, and that more cases will need to be handled. For example if you introduce an `AdminUser`. In that case, you can use a `switch`. A `switch` statement will break if one of the cases is not handled. So introducing our `AdminUser`, or another type of user later, would automatically add compilation errors in every place you need to handle it!

```ts
interface AdminUser extends BaseUser {
  type: 'admin';
  adminSince: number;
}

type User = AuthenticatedUser | AnonymousUser | AdminUser;

function onWebsiteSince(user: User): number {
  switch (user.type) {
    case 'authenticated':
      return user.loggedSince;
    case 'anonymous':
      return user.visitingSince;
    case 'admin':
      // without this case, we could not even compile the code
      // as TS would complain that all possible paths are not returning a value
      return user.adminSince;
  }
}
```

I hope these patterns will help you. Now let's focus on Web Components.

Chapter 6. The wonderful land of Web Components

Before going further, I'd like to make a brief stop to talk about Web Components. You don't have to know about Web Components to write Angular code. But I think it's a good thing to have an overview of what they are, because some choices in Angular have been made to facilitate the integration with Web Components, or to make the components we will build similar to Web Components. Feel free to skip this part if you have no interest in this topic; however, I do believe you'll learn a thing or two that will be useful for the rest of the road.

6.1. A brave new world

Components are an old fantasy in development. Something you can grab off the shelves and drop into your app, something that would work right away and bring a needed functionality to your users.

My friends, this time has come.

Well, maybe. At least, there is the start of something.

That's not completely new. We have had components in web development for quite some time, but they usually require some kind of dependency, like jQuery, Dojo, Prototype, AngularJS, etc. Not necessarily libraries you wanted to add to your app.

Web Components attempt to solve this problem: let's have reusable and encapsulated components.



They rely on a set of emerging standards that browsers don't perfectly support yet. But, still, it's an interesting topic, even if there's a chance we'll have to wait a few years to use them fully, or even if the concept never takes off.

This emerging standard is defined in 3 specifications:

- Custom elements
- Shadow DOM
- Template

Note that the samples are most likely to work in a recent Chrome or Firefox browser.

6.2. Custom elements

Custom elements are a new standard allowing developers to create their own DOM elements, making something like `<ns-pony></ns-pony>` a perfectly valid HTML element. The specification defines how to declare such elements, how to make them extend existing elements, how to define your API, etc.

Declaring a custom element is done using `customElements.define`:

```
class PonyComponent extends HTMLElement {
  constructor() {
    super();
    console.log("I'm a pony!");
  }

}

customElements.define('ns-pony', PonyComponent);
```

And you can then use it:

```
<ns-pony></ns-pony>
```

Note that the name must contain a dash, so that the browser knows it is a custom element. Of course, your custom element can have properties and methods, and it also has lifecycle callbacks, to be able to execute code when the component is inserted or removed, or when one of its attributes changes. It can also have a template of its own. Maybe the `ns-pony` displays an image of the pony or just its name:

```
class PonyComponent extends HTMLElement {
  constructor() {
    super();
    console.log("I'm a pony!");
  }


  /**
   * This is called when the component is inserted
   */
  connectedCallback() {
    this.innerHTML = '<h1>General Soda</h1>';
  }

}
```

If you try to look at the DOM, you'll see `<ns-pony><h1>General Soda</h1></ns-pony>`. But that means

the CSS and JavaScript logic of your app can have undesired effects on your component. So, usually, the template is hidden and encapsulated in something called Shadow DOM, and you'll only see `<ns-pony></ns-pony>` if you inspect the DOM, despite the fact that the browser displays the pony's name.

6.3. Shadow DOM

With a mysterious name like this, you expect something with great powers. And surely it is. The Shadow DOM is a way to encapsulate the DOM of our component. This encapsulation means that the stylesheet and JavaScript logic of your app will not apply on the component and ruin it inadvertently. It gives us the perfect tool to hide the internals of a component, and be sure nothing leaks from the component to the app, or vice-versa.

Going back to our previous example:

```
class PonyComponent extends HTMLElement {
  constructor() {
    super();
    const shadow = this.attachShadow({ mode: 'open' });
    const title = document.createElement('h1');
    title.textContent = 'General Soda';
    shadow.appendChild(title);
  }
}
```

If you try to inspect it now you should see:

```
<ns-pony>
  #shadow-root (open)
    <h1>General Soda</h1>
</ns-pony>
```

Now, even if you try to add some style to the `h1` elements, the visual aspect of the component won't change at all: that's because the Shadow DOM acts like a barrier.

Until now, we just used a string as a template of our web component. But that's usually not the way you do that. Instead, the best practice is to use the <template> element.

6.4. Template

A template specified in a `<template>` element is not displayed in your browser. Its main goal is to be cloned in an element at some point. What you declare inside will be inert: scripts don't run, images don't load, etc. Its content can't be queried by the rest of the page using usual methods like `getElementById()` and it can be safely placed anywhere in your page.

To use a template, it needs to be cloned:

```
<template id="pony-template">
  <style>
    h1 {
      color: orange;
    }
  </style>
  <h1>General Soda</h1>
</template>
```

```
class PonyComponent extends HTMLElement {
  constructor() {
    super();
    const template = document.querySelector('#pony-template');
    const clonedTemplate = document.importNode(template.content, true);
    const shadow = this.attachShadow({ mode: 'open' });
    shadow.appendChild(clonedTemplate);
  }
}
```

6.5. Frameworks on top of Web Components

All these things put together make the Web Components. I'm far from being an expert on this topic, and there are all sorts of twisted traps on this road.

As Web Components are not fully supported by every browser, there is a polyfill you can include in your app to make sure it will work. The polyfill is called web-component.js, and it's worth noting that it is a joint effort from Google, Mozilla and Microsoft among others.

On top of this polyfill, a few libraries have seen the light. All aim to facilitate working with Web Components, and often come with some ready-to-use Web Components.

Among the most notable initiatives, you find:

- Polymer, the first attempt from Google
- LitElement, a more recent project from the Polymer team ;
- X-tag from Mozilla and Microsoft
- Stencil.

I won't go into the details, but you can easily use an already existing component. Let's say you want a Google Map in your app:

```
<!-- Polyfill Web Components support for older browsers -->
<script src="webcomponents.js"></script>

<!-- Import element -->
<script src="google-map.js"></script>

<!-- Use element -->
<body>
  <google-map latitude="45.780" longitude="4.842"></google-map>
</body>
```

There are a LOT of components out there. You can have an overview on https://www.webcomponents.org/.

You can do a lot of cool things with LitElement and other similar frameworks, like two-way data binding, default values for attributes, emit custom events, react to attribute changes, repeat elements if we give a collection to a component, etc.

That's obviously far too short a chapter to tell you everything there is to say on Web Components, but you'll see that some of the concepts are going to pop out along your read. And you'll definitely see that the Google team designed Angular to make it easy to use Web Components with our Angular components. It is even possible to export our own Angular components as Web Components, with the help of Angular Elements.

Chapter 7. Grasping Angular's philosophy

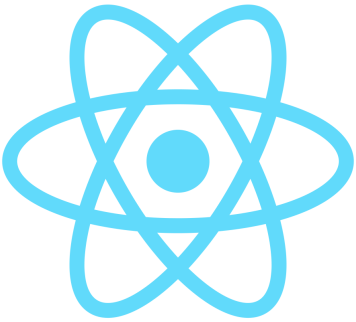
To write an Angular application, you have to grasp a few things on the framework's philosophy.



First and foremost, Angular is component-oriented. You will write tiny components and, together, they will constitute a whole application. A component is a group of HTML elements in a template, dedicated to a particular task. For this, you will usually also need to have some logic linked to that template, to populate data, and react to events for example. For the veterans of AngularJS 1.x, it's a bit like a 'template/controller' duo, or a directive.

This component orientation is something that is becoming widely shared across front-end frameworks: React, the cool kid from Facebook, has been doing it that way from the beginning; Ember and AngularJS have their way of doing something similar; and others like Svelte or Vue.js are betting on building small components too.



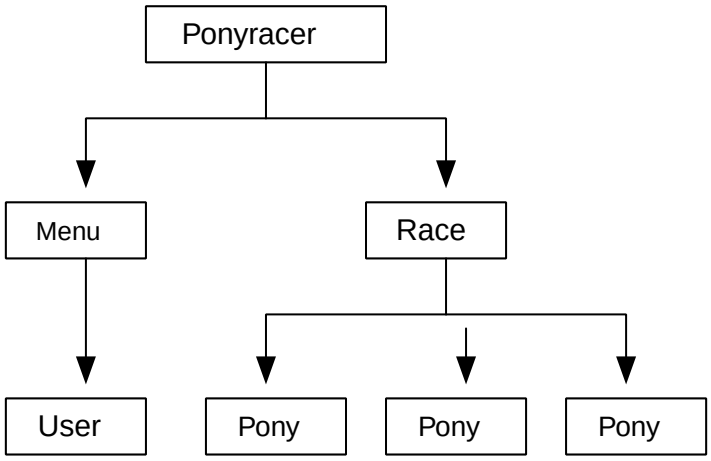






Angular is not alone in this, but it is among the first (it might actually be the first?) to really care about the integration of Web Components (the standard ones). But let's forget about this for now, as it is a more advanced topic.

Your components will be arranged in a hierarchical way, like the DOM is. A root component will have child components, each of them will also have children, etc. If you want to display a pony race (who wouldn't?), you'll have something like an app (`Ponyracer`), displaying a menu (`Menu`) with the logged in user (`User`) and a child view (`Race`), displaying, of course, the ponies (`Pony`) in the races:



Writing components will be your everyday work, so let's see what it looks like. The Angular team wanted to harness another goodness of today's web development: ES2015+. So you can write your

components in ES5 (but that's not very cool) or in ES2015+ (way cooler!). But that was not enough for them. They wanted to use a feature that is not a standard (yet): decorators. So they worked closely with the transpiler teams (Traceur and Babel) and the TypeScript team at Microsoft, to enable us to use decorators in our Angular apps. A few decorators are available, allowing us to easily declare a component for example. I hope you already know all of that, as I just spent two chapters on these things!

For example, if we simplify, the Race component could look like this:

```
import { Component } from '@angular/core';
import { RacesService } from './services';

@Component({
  selector: 'ns-race',
  templateUrl: './race.component.html'
})
export class RaceComponent {
  race: any;

  constructor(racesService: RacesService) {
    racesService.get().then(race => (this.race = race));
  }
}
```

And the template looks like this:

```
<div>
  <h2>{{ race.name }}</h2>
  <div>{{ race.status }}</div>
  <div *ngFor="let pony of race.ponies">
    <ns-pony [pony]="pony"></ns-pony>
  </div>
</div>
```

If you already know AngularJS 1.x, the template should look familiar, with the same expression in curly braces `{{ }}`, which will be evaluated and replaced by the corresponding value. Some things have changed though: no more `ng-repeat` for example. I don't want to go too deep for now, merely just give you a feel of what the code looks like.

A component is a very isolated piece of your app. Your app *is* a component like the others.

You will group components in one or several coherent entities, called modules (Angular Modules, not ES2015 Modules).

You can also take available modules from the community and just put them in your app, and be able to enjoy their features.

Such modules can offer UI components, or drag and drop capability, or validation for your forms,

or whatever you can think of.

In the next chapters, we are going to explore how to get started, how to build a small component, your first module and the templating syntax.

There is another concept that is at the core, and that is Dependency injection (often called by its little name, DI). It is a very powerful pattern, and you will quickly get used to it after reading the dedicated chapter. It is especially useful to test your application, and I love doing tests, watching the progress bar go all green in my IDE. It makes me feel I'm doing a good job. So there will be an entire chapter on testing everything: your components, your services, your UI…

Angular still has the magic feeling it had in v1, where changes were automatically detected by the framework and applied to the model and the views. But it is done in a very different way than it was then: the change detection now uses a concept called `zones`. We will look into this, of course.

Angular is also a complete framework which provides a lot of help for performing common tasks in web development. Writing forms, calling a HTTP backend, routing, interacting with other libraries, animations, you name it: you're covered.

Well, that's a lot of things to learn! We should start with the beginning: bootstrap an app and write our first component.

Chapter 8. From zero to something

Let's start by creating our first Angular app and our first component, with a minimum of tooling.

8.1. Node.js and NPM

Pretty much all the modern JavaScript tools are built for Node.js and NPM these days. You'll have to install Node.js and NPM on your system. The best way to do that depends on your operating system - you can find more information on the official website. Make sure you have a recent enough version of Node.js (by executing `node --version`).

8.2. Angular CLI

You *could* setup everything by yourself, starting with a TypeScript project, then install every dependency needed, etc.

But in a real project, you'll probably have to set up several other things too, like:

- some tests to check if we're not breaking things
- maybe a linter to check your code
- maybe a CSS preprocessor
- a build tool, to orchestrate the various tasks (compile, test, package, etc.)

But it's a bit cumbersome to setup everything yourself, especially when there are sooooo many tools to learn first.

These past few years, a lot of small project generators have seen the light, pretty much all using the great Yeoman. It used to be the case for AngularJS 1.x, and there were a few attempts for Angular from the community.

But this time, the Google team has been working on this issue, and they have come up with something: `Angular CLI`.



`Angular CLI` is a command line utility to easily quick start a project, already configured with Webpack as a build tool (the popular kid these years), tests, packaging, etc.

The idea is not new, and is in fact borrowed from another popular framework: EmberJS and its

popularly acclaimed `ember-cli`.

The tool is under continuous development, with a dedicated Google team working on it and making it better and better. It is now the recommended and *de facto* standard way of creating and building Angular apps. So let's give it a try, and discover the ton of cool stuff packed into it!



If you want, you can follow our online exercise Getting Started 🏇! It's free and part of our Pro Pack, where you'll learn how to build a complete application step by step. The first exercise is about getting everything up and running with Angular CLI, and goes further than what we see in the chapter.

First let's install Angular CLI, and generate a new application with the `ng new` command. If you want to use exactly the same CLI version than we are (`11.2.1`), you can use `npm install -g @angular/cli@11.2.1` instead.

```
npm install -g @angular/cli
```

```
ng new ponyracer --prefix ns --defaults --strict
```

This will create a project skeleton in a new directory called `ponyracer`. From this directory, you can start your app with:

```
ng serve
```

This will start a small HTTP server locally, with a hot reload configuration. It means that every time you modify and save a file, the server will rebuild the app, and the browser will reload it immediately.

Tada! You have your first application up and running! 🎉

8.3. Application structure

Let's dive for a few seconds into the generated code.

Open the project in your preferred IDE. You can use pretty much anything you want, but you should activate the TypeScript support for maximum comfort. Pick your favorite: Webstorm, Visual Studio Code, Atom… All of them have great support for TypeScript.



If your IDE supports it, code completion should work as the Angular dependencies have their own `d.ts` files in the `node_modules` directory, and TypeScript is able to detect them. You can even navigate to the type definitions if you want to. TypeScript will bring its type-checking to the table, so you'll see what mistakes you make as you type. As we are using source maps, you can see the TS code directly from your browser, and even debug your app by setting breakpoints in the TypeScript code.

You should see a bunch of configuration files in the root directory: welcome to Modern JavaScript!

The first one you may recognize is the `package.json` file: that's where the dependencies of the application are defined. You can have a look inside, it should now contain the following dependencies:

- the different `@angular` packages.
- `rxjs`, a really cool library for reactive programming. We have a dedicated chapter on this topic and about RxJS in particular.
- `zone.js`, doing the heavy lifting for detecting the changes (we'll dive into this later also).
- some dependencies for developing the application, like the CLI, TypeScript, some test librairies, some typings…

TypeScript itself has a configuration file `tsconfig.json` (and another one in `src` called `tsconfig.app.json`), which stores the compilation options. As we saw in the previous chapters, we are using TypeScript with decorators (hence the two options about decorators), and we want our code to transpile to ECMAScript 5, allowing it to run in every browser. The `sourceMap` option allows you to generate source maps, i.e. files that contain a mapping between the generated ES5 code and the original TypeScript code. Those source maps are used by the browser to let you debug the ES5 code it executes by stepping through the original TypeScript code that you have written.

TypeScript projects often also use TSLint, a linter used to check your code against the best practices. TSLint has its own options, stored in `tslint.json`, where you add/remove some of its rules.

Angular CLI itself has a configuration file `angular.json` if you want to override some of its defaults.

The last one, `karma.conf.js` is used to configure the testing tool (more about that when we'll talk about testing).



This ebook is using Angular version `11.2.3` for the examples. Angular CLI will probably install the most recent version, which might not be exactly the same. If you want to use the same version as we are, replace the version in the `package.json` by `11.2.3` for each Angular package. That might save you a few headaches! Or, even better, follow our free online exercise Getting Started 🐴! which is always up-to-date and battle-tested!

Now that we have been over the configuration, let's see the application code.

8.4. Our first component

As we saw in the previous section, a component is a combination of a view (the template) and some logic (our TS class). The CLI has already created one for us: `src/app/app.component.ts`. Let's check it out:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'ponyracer';
}
```

Our application itself is a simple component. To tell Angular that it is a component, we use the `@Component` decorator. And to be able to use this decorator, we have to import it as you can see at the top of the file.

When you write new components, don't forget to import the `Component` decorator. You may forget to do so at the beginning, but it won't last, as the compiler will yell at you! ;)

You'll see that most of the things we need are in the `@angular/core` module, but that's not always the case. For example, when dealing with HTTP, we'll use imports from `@angular/http`; or, if we use the router, we'll import from `@angular/router`, etc.

```
import { Component } from '@angular/core';

@Component({
})
export class AppComponent {
  title = 'ponyracer';
}
```

The `@Component` decorator is expecting a configuration object. We'll see later in detail what you can configure here, but for now only one property is expected: the `selector` one. It will tell Angular what to look for in our HTML pages. Every time Angular finds an element in our HTML which matches the selector of the component, it will create an instance of the component, and replace the content of the element by the template of the component.

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-root',
})
export class AppComponent {
  title = 'ponyracer';
}
```

So, here, every time our HTML contains an element like `<ns-root></ns-root>`, Angular will instantiate a new instance of our `AppComponent` class.



There is a clear naming convention established, and applied by Angular CLI. Component classes end with `Component`, and they are defined in a file ending with `.component.ts`. Angular also recommends using a prefix in component selectors, to avoid name clashes with external components. For example, since our company is named **N**inja **S**quad, we chose to use the prefix `ns`. Our pony component selector is thus named `ns-pony`. You can configure Angular CLI so that it prepends this prefix to every generated component.

A component must also have a template. We can have an inline template or externalize it in another file, like the CLI does:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'ponyracer';
}
```

The corresponding HTML is defined in `app.component.html`, with a bunch of static elements, except the first `h1`:

```
<span>{{ title }} app is running!</span>
```

8.5. Our first Angular Module

Like we briefly said in the previous chapter, we are going to group our components and other pieces we'll see later in coherent entities: Angular Modules.

An Angular Module is different from the ES2015 Modules we came across earlier: here we are talking about **application** modules.

Your application will always have at least one module, the **root module**. Maybe, later, when your application grows, you'll add other modules, by feature. For example, you could add a module dedicated to the Admin part of your application, containing all the components and the logic for this part. But we'll come back to this later. We'll also see that third party libraries and Angular itself expose modules, which we can use in our app.

Of course, the CLI has generated a module too, called `app.module.ts`.

The class is decorated with `@NgModule`.

```
import { NgModule } from '@angular/core';

@NgModule({
})
export class AppModule { }
```

Like the `@Component` decorator, it takes a configuration object.

As we are building an app for the browser, the root module imports `BrowserModule`. This is not the only target possible for Angular, you could choose to render the app on the server for example, and therefore import another module. `BrowserModule` contains all kinds of useful stuff we will use later. A module can choose to *export* components, directives and pipes. When you import a module, you will make all the directives, components and pipes exported by the imported module usable in your module. Our root module won't be imported into another module, so we don't have `exports`, but we will have several `imports` in the end.

The terminology is not beginner friendly here. We were talking about ES2015+ and TS modules in the first chapters, which define imports and exports. And now we are talking about Angular modules, which also have imports and exports… I'm not a fan of having the same terms for different things, so let me explain a little bit more.

You can see an ES2015+ or TS import purely as a language feature, like an import statement in Java: it allows you to use the imported classes/functions in your source code. It also declares a dependency for the bundler or module loader (Webpack or SystemJS, for example), which knows that if `a.ts` is loaded, then `b.ts` must also be loaded since `a` imports `b`. You have to use imports and exports with ES2015+ and TypeScript, whether or not you're using Angular or any other framework.

On the other hand, importing an Angular module (for example `BrowserModule`) in your own Angular module (`AppModule`), has a functional meaning. It tells Angular: all the components, directives and pipes that are exported by `BrowserModule` should be made available to my Angular components/templates. It has no special meaning for the TypeScript compiler.

Back to `NgModule`: in its configuration object, we must declare the components that belong to our root module, with the `declarations` field. You can see it contains the previous component we talked about: `AppComponent`.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
})
export class AppModule { }
```

Since this is the root module, we need to tell Angular which component is the root component, i.e. the component that will be started when we bootstrap the app. That's what the `bootstrap` field of the configuration object is for:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Module ready, let's bootstrap the app!

8.6. Bootstrapping the app

Finally, we need to start our app, using the `bootstrapModule` method. This method is exposed on an object returned by a method called `platformBrowserDynamic`. You have to import it too, from `@angular/platform-browser-dynamic`. Now, that's a strange module! Why is it not `@angular/core`? Good question: it's because you might want to run your app somewhere else than in a browser, as Angular supports server-side rendering or running in a Web Worker for example. And in these cases, the bootstrap logic would be a bit different. But we'll see this later, as we are just focusing on the browser right now.

Angular CLI will by default generate a separate file containing this bootstrap logic: `main.ts`:

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Yay! But wait a second. We need an HTML file to serve to our users, right?

The CLI created an `index.html` file for us, which is the single page of our application. You might wonder how it could possibly work, since it doesn't contain any `script` element.

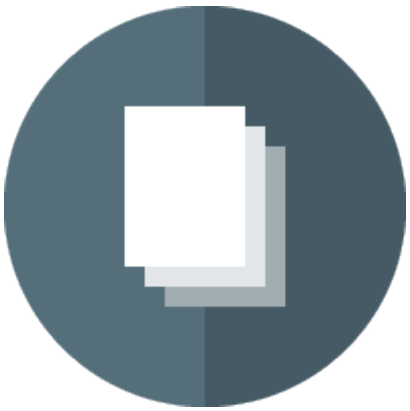
When you run `ng serve`, the CLI calls the TypeScript compiler. The compiler outputs JavaScript files. The CLI then bundles them and adds the necessary `script` elements to the `index.html` file (using Webpack behind the scenes).

Hopefully, you now have a better understanding of the various parts of this first Angular application. It's not really a dynamic app yet, and we could have done the same in one second in a static HTML page, I'll give you that. So let's jump to the next sections, and learn all about dependency injection and templating.

Chapter 9. The templating syntax

We've seen that a component needs to have a view. To define a view, you can define a template inline or in a separate file. You're probably familiar with a templating syntax, maybe even the one from AngularJS 1.x. To simplify things, a template helps us to render HTML with some dynamic parts depending on our data.

Angular has its own templating syntax that we need to learn before going further.



Let's take a simple example, by modifying our first component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-root',
  template: '<h1>PonyRacer</h1>'
})
export class AppComponent {}
```

Now we want to display some dynamic data on this first page, maybe the number of users registered into our app. Later we'll see how to get data from a server, but for now we'll say that this number of users is directly hard-coded in our class:

```
@Component({
  selector: 'ns-root',
  template: '<h1>PonyRacer</h1>'
})
export class AppComponent {
  numberOfUsers = 146;
}
```

Now, how do we change our template to display this variable? The answer is interpolation.

9.1. Interpolation

Interpolation is a big word for a simple concept.

Quick example:

```
@Component({
  selector: 'ns-root',
  template: `
    <h1>PonyRacer</h1>
    <h2>{{ numberOfUsers }} users</h2>
  `
})
export class AppComponent {
  numberOfUsers = 146;
}
```

We have an `AppComponent` component that will be activated every time Angular finds a `<ns-root>` tag. The `AppComponent` class has a property, `numberOfUsers`. And the template has been augmented with an `<h2>` tag, using the famous double curly braces (a.k.a. "mustaches") to indicate that an expression has to be evaluated. This kind of templating is called interpolation.

We should now see in the browser:

```
<ns-root>
  <h1>PonyRacer</h1>
  <h2>146 users</h2>
</ns-root>
```

as `{{ numberOfUsers }}` will be replaced by its value. When Angular detects a `<ns-root>` element in the page, it creates an instance of the `AppComponent` class, and this instance is the evaluation context of the template's expressions. Here the `AppComponent` instance sets the `numberOfUsers` property to '146', so we have '146' displayed on screen.

The magic is that, whenever the value of `numberOfUsers` changes in our object, the template will be automatically updated! That's called 'change detection', and it's one of the great features of Angular.

One important fact to remember, though: if we try to display a variable that does not exist, then, instead of displaying `undefined`, Angular is going to display an empty string. The same will happen for a `null` variable.

Let's say that, instead of a simple value, our first component has a more complex `user` object, reflecting the current user.

```
@Component({
  selector: 'ns-root',
  template: `
    <h1>PonyRacer</h1>
    <h2>Welcome {{ user.name }}</h2>
  `
})
export class AppComponent {
  user: any = { name: 'Cédric' };
}
```

As you can see, we can interpolate more complex expressions, like accessing the property of an object.

```
<ns-root>
  <h1>PonyRacer</h1>
  <h2>Welcome Cédric</h2>
</ns-root>
```

What happens if we have a typo in our template, with a property that does not exist in the class?

```
@Component({
  selector: 'ns-root',
  // typo: users is not user!
  template: `
    <h1>PonyRacer</h1>
    <h2>Welcome {{ users.name }}</h2>
  `
})
export class AppComponent {
  user = { name: 'Cédric' };
}
```

When loading the app, you will have an error, telling you that this property does not exist:

```
Cannot read property 'name' of undefined in [{{ users.name }} in AppComponent]
```

That's great, because now you are quite sure that your templates are correct. One of the most often encountered problem in AngularJS 1.x was that this type of error could not be detected, and you could lose quite some time trying to figure out what was going on (usually a typo, like `{{ users.name }}` instead of `{{ user.name }}`). We have given quite a few training sessions, and I can assure you that 30% of beginners were having this problem on the first day. I got a bit tired of it, and I even submitted a pull-request to display a warning when the parser would find an unknown variable, which was refused for valid reasons and with a comment from the core team saying they had an idea on how to solve this in Angular. And they did!

One last little but handy feature. What happens if my `user` object is in fact fetched from the server, and thus initialized to `undefined` before being valued with the result of the server call? Is there a way to avoid the errors when the template is compiled?

Yes, there is: instead of writing `user.name`, you write `user?.name`:

```
@Component({
  selector: 'ns-root',
  // user is undefined
  // but the ?. will avoid the error
  template: `
    <h1>PonyRacer</h1>
    <h2>Welcome {{ user?.name }}</h2>
  `
})
export class AppComponent {
  user: any;
}
```

And you don't have errors anymore! The `?.` is sometimes called the "Safe Navigation Operator".

So we can write our templates more safely, and be assured that they will behave properly.

Let's go back to our example. We are now displaying a greeting message. Maybe we can go a step further and display the upcoming pony races.

That should lead us to write our second component. For now, we'll just make it simple:

```
// in another file, races.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'ns-races',
  template: `<h2>Races</h2>`
})
export class RacesComponent {}
```

Nothing fancy: a simple class, decorated with `@Component` to give it a `selector` to match and an inline template.

Now we want to include this component in our `AppComponent` template. What do we need to do?

9.2. Using other components in our templates

We have our app component, `AppComponent`, where we want to display the pony races component, `RacesComponent`.

```
// in app.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'ns-root',
  // added the RacesComponent component
  template: `
    <h1>PonyRacer</h1>
    <ns-races></ns-races>
  `
})
export class AppComponent {}
```

As you can see, we added the `RacesComponent` component in the template, by including a tag whose name matches the selector we defined for the component.

Buuuuuut, that will not work: your browser will not display the races component.

Why is that? The reason is simple: Angular doesn't know about this `RacesComponent` yet.

But the fix is simple. Do you remember that we had to add `AppComponent` in the `declarations` of the `@NgModule` decorator? Now, since we have a second component, it must be declared, too.

`RacesComponent` is not the root component of our application, so it must be in the declarations, but not in the `bootstrap`.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';

import { AppComponent } from './app.component';
// do not forget to import the component
import { RacesComponent } from './races.component';

@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent, RacesComponent],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Note that you will pass the class directly, so you'll have to import it. And in order to be able import it, you need to export the class `RacesComponent` in its source file `races.component.ts` (read the section about ES2015 modules again if that's not clear for you). So `RacesComponent` will look like:

```
// in another file, races.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'ns-races',
  template: `<h2>Races</h2>`
})
export class RacesComponent {}
```

Now, our races component will proudly be displayed in our browser:

```
<ns-root>
  <h1>PonyRacer</h1>
  <ns-races>
    <h2>Races</h2>
  </ns-races>
</ns-root>
```

9.3. Property binding

Interpolation is only one of the ways to have dynamic parts in your template.

In fact, the interpolation we just saw is just an easy way to use what is the core of Angular templating system: property binding.

In Angular, every DOM property can be written to via special attributes on HTML elements surrounded with square brackets `[]`. It looks weird at first, but in fact it is valid HTML (it surprised me too). An HTML attribute can start with pretty much anything you want except a few characters like quotes, apostrophes, slashes, equals, spaces…

I'm talking about DOM properties, but maybe this is not clear for you. We usually write to HTML attributes, right? Right, usually we do. Let's take this simple HTML:

```
<input type="text" value="hello">
```

The `input` tag above has two *attributes*: a `type` attribute and a `value` attribute. When the browser parses this tag, it creates a corresponding DOM node (an HTMLInputElement if we want to be accurate), which has the matching *properties* `type` and `value`. Each standard HTML attribute has a corresponding property in the DOM node. But the DOM node also has additional properties, which don't have a corresponding attribute. For example: `childElementCount`, `innerHTML` or `textContent`.

The interpolation we had above to display the user's name:

```
<p>{{ user.name }}</p>
```

is just sugar syntax for the following:

```
<p [textContent]="user.name"></p>
```

The square bracket syntax allows you to modify the DOM property `textContent`, and we give it the value `user.name` which will be evaluated in the context of the current component instance, as it was for the interpolation.

Note that the parser is case-sensitive, so you have to write the property name with the correct case: `textcontent` or `TEXTCONTENT` will not work. It has to be `textContent`.

DOM properties have a great advantage over HTML attributes: they have up-to-date values. In my input example, the `value` *attribute* will always contain 'hello', whereas the `value` *property* of the DOM node is dynamically modified by the browser, and thus contains whatever the user has entered in the text field.

Finally, properties can have boolean values, whereas some attributes can only reflect it by being present or absent on the start tag. For example, you have the `selected` attribute on the `<option>` tag: no matter what value you give it, it will select the option, as long as it is present.

```
<option selected>Rainbow Dash</option>
<option selected="false">Rainbow Dash</option> <!-- still selected -->
```

With properties access like Angular gives us, you can write:

```
<option [selected]="isPonySelected" value="Rainbow Dash">Rainbow Dash</option>
```

And the pony will be selected if `isPonySelected` is `true`, and will not be selected if it is `false`. And whenever the value of `isPoneySelected` changes, the selected property will be updated.

You can do a lot of cool things with this, things that were cumbersome in AngularJS 1.x. For example, having a dynamic source URL for an image.

```
<img src="{{ pony.avatar.url }}">
```

This syntax has a major problem: the browser will try to fetch the image as soon as it reads the `src` attribute. You can see that it will fail: it will make an HTTP request to `{{ pony.avatar.url }}` which is not a valid URL...

In AngularJS 1.x, there was a special directive to take care of that: `ng-src`.

```
<img ng-src="{{ pony.avatar.url }}">
```

Having `ng-src` instead of `src` did solve the problem, as it tricked the browser into ignoring it. Once AngularJS had compiled the app, it added the `src` attribute with a correct URL, hence triggering the

image download. Cool! But it had two downsides:

- first, you had to know, as a developer, what value to give to `ng-src`. Was it 'https://gravatar.com'? '"https://gravatar.com"'? 'pony.avatar.url'? '{{ pony.avatar.url }}'? No way to know, except by reading the documentation.
- second, the Angular team had to create a directive for each standard attribute. They did, and we had to learn them. But we are now in a world where your HTML can also contain external Web Components, looking like:

```
<ns-pony name="Rainbow Dash"></ns-pony>
```

If this is a Web Component that you want to use, you have no easy way to pass a dynamic value with most JS frameworks, except if the developer of the Web Component had taken extra care to make it possible. Read the chapter on Web Components for more information.

A Web component should act like a browser element. They have a DOM API based on properties, events and methods. With Angular, you can do:

```
<ns-pony [name]="pony.name"></ns-pony>
```

And it works!

Angular will maintain the properties and attributes in sync.

No more directives to learn! If you wish to hide an element, you can use the standard `hidden` property:

```
<div [hidden]="isHidden">Hidden or not</div>
```

And the `div` will be hidden only when `isHidden` is `true`, as Angular will work directly with the `hidden` property. No more `ng-hide`, and this is just one of the dozens of directives that were used in Angular 1.

You can also access nested properties like the `color` attribute of the `style` property.

```
<p [style.color]="foreground">Friendship is Magic</p>
```

If the `foreground` attribute is changing to 'green', then the text will update its color to 'green' too!

So Angular is using properties. Which values can we pass? We already saw the interpolation `property="{{ expression }}"`:

```
<ns-pony name="{{ pony.name }}"></ns-pony>
```

is the same as `[property]="expression"` (which you will usually prefer):

```
<ns-pony [name]="pony.name"></ns-pony>
```

If you want to write 'Pony' followed by the pony's name, you have two options:

```
<ns-pony name="Pony {{ pony.name }}"></ns-pony>
<ns-pony [name]="'Pony ' + pony.name"></ns-pony>
```

If your value is not a dynamic one, you can simply write `property="value"`:

```
<ns-pony name="Rainbow Dash"></ns-pony>
```

All of these are equivalent, and the syntax doesn't depend on how the developer chose to design their component, as it was the case in AngularJS 1.x where you had to know if the component was expecting a value or a reference for example.

Of course, the expression can also contain function calls:

```
<ns-pony name="{{ pony.fullName() }}"></ns-pony>
<ns-pony [name]="pony.fullName()"></ns-pony>
```

9.4. Events

If you're developing a webapp, you know that displaying things is just one part of the job: you also have to deal with user interactions. To allow this, the browser fires events, which you can listen to: `click`, `keyup`, `mousemove`, etc. AngularJS 1.x had one directive per event: `ng-click`, `ng-keyup`, `ng-mousemove`, etc. In Angular, this is simpler - no more directives to remember.

Going back to our `RacesComponent`, we now want to have a button that will display the races when clicked.

Reacting to an event can be done as follow:

```
<button (click)="onButtonClick()">Click me!</button>
```

A click on the button of the example above will trigger a call to the component method `onButtonClick()`.

Let's add this to our component:

```
@Component({
  selector: 'ns-races',
  template: `
    <h2>Races</h2>
    <button (click)="refreshRaces()">Refresh the races list</button>
    <p>{{ races.length }} races</p>
  `
})
export class RacesComponent {
  races: any = [];

  refreshRaces(): void {
    this.races = [{ name: 'London' }, { name: 'Lyon' }];
  }
}
```

If you try this in your browser, you should initially see:

```
<ns-root>
  <h1>PonyRacer</h1>
  <ns-races>
    <h2>Races</h2>
    <button (click)="refreshRaces()">Refresh the races list</button>
    <p>0 races</p>
  </ns-races>
</ns-root>
```

And after your click, '0 races' should become '2 races'. Yay \o/

The statement can be a function call, but it can be any executable statement, or even a sequence of executable statements, like:

```
<button (click)="firstName = 'Cédric'; lastName = 'Exbrayat'">
  Click to change name to Cédric Exbrayat
</button>
```

However I would not advise you to do this. Using methods is a better way of encapsulating the behavior: it makes your code easier to maintain and test, and it makes the view simpler.

The cool thing is that it works with standard DOM events, but also with custom events that might fire from your Angular components or from web components. We'll see later how to fire custom events.

For the moment, let's say the `RacesComponent` component emits a custom event to notify the app that a new race is available.

Our template in the `AppComponent` component would then look like:

```
@Component({
  selector: 'ns-root',
  template: `
    <h1>PonyRacer</h1>
    <ns-races (newRaceAvailable)="onNewRace()"></ns-races>
  `
})
export class AppComponent {
  onNewRace(): void {
    // add a flashy message for the user.
  }
}
```

We can easily figure out that the `<ns-races>` component has a custom event `newRaceAvailable` and that, when this event is fired, the method `onNewRace()` of our `AppComponent` is called.

Angular will listen for the event on the element and on its children, so it will react to events that bubble. Consider the template:

```
<div (click)="onButtonClick()">
  <button>Click me!</button>
</div>
```

Even though the user clicks on the button embedded inside the div, the `onButtonClick()` method will be called, because the event bubbles up.

Oh, and you can access the event in the method called! You just have to pass `$event` to your method:

```
<div (click)="onButtonClick($event)">
  <button>Click me!</button>
</div>
```

Then you can handle the event in your component class:

```
onButtonClick(event) {
  console.log(event);
}
```

By default, the event will continue to bubble up, eventually triggering other event listeners up in the hierarchy.

You can use the event to prevent the default behavior and/or cancel propagation if you want:

```
onButtonClick(event) {
  event.preventDefault();
  event.stopPropagation();
}
```

One cool feature is that you can also easily handle keyboard events with:

```
<textarea (keydown.space)="onSpacePress()">Press space!</textarea>
```

Every time you will press the *space* key, the `onSpacePress()` method will be called. And you can do a crazy combo, like `(keydown.alt.space)`, etc.

To conclude this part, I want to point out that there is a big difference between something like:

```
<component [property]="doSomething()"></component>
```

and

```
<component (event)="doSomething()"></component>
```

In the first case, with property binding, the `doSomething()` value is called an expression, and will be evaluated at each change detection cycle to see if the property needs to be updated.

In the second case, however, with event binding, the `doSomething()` value is called a statement, and will be evaluated **only when the event is triggered**.

By definition they have completely different goals and, as you might suspect, they have different restrictions.

9.5. Expressions vs statements

Expressions and statements differ in several ways.

An expression will be executed many times, as part of the change detection. It should thus be as fast as possible. Basically, an Angular expression is a simplified version of an expression you could write in JavaScript.

If you are using `user.name` as an expression, `user` should be defined, otherwise Angular will throw an error.

An expression must be single: you can't chain several ones separated with a semi-colon.

An expression should not have any side effect. That means it can't be an assignment, for example.

```
<!-- forbidden, as the expression is an assignment -->
<!-- this will throw an error -->
<component [property]="user = 'Cédric'"></component>
```

It cannot contains keywords, like `if`, `var`, etc.

A statement, on the other hand, is triggered on the matching event. If you try to use a statement like `race.show()` where `race` is `undefined`, you will have an error. You can chain several statements, separated with a semi-colon. A statement can, and generally should, have side effects. That's the point of reacting to an event: to make something happen. A statement can be a variable assignment, and can contain keywords.

9.6. Local variables

When I say that Angular will look in the component instance to find a variable, it is not technically correct. In fact, it will check the component instance and the local variables. Local variables are variables that you can dynamically declare in your template using the `#` syntax.

Let's say you want to display the value of an input:

```
<input type="text" #name>
{{ name.value }}
```

Using the `#` syntax, we are creating a local variable `name` referencing the DOM object `HTMLInputElement`. This local variable can be used anywhere in the template. As it has a `value` property, we can display this property in an interpolated expression. I'll come back to this example later.

Another useful usage of local variables is when you want to execute some kind of action on another element.

For example, you may want to give the focus on an element when you click on a button. This was a bit cumbersome in AngularJS 1.x, as you had to create a custom directive and so on.

The `focus()` method is a standard part of the DOM API, and we can leverage this. Using a local variable, it's a no-brainer in Angular:

```
<input type="text" #name>
<button (click)="name.focus()">Focus the input</button>
```

It can also be used with a custom component - one we created in our app, imported from another project, or even with a real Web Component:

```
<google-youtube #player></google-youtube>

<button (click)="player.play()">Play!</button>
```

Here, the button can start playing the video of the `<google-youtube>` component. This is actually a real Web Component written with Polymer! This component has a `play()` method that Angular will call when you click on the button, which is pretty cool!

Local variables have a few use cases, and we will gradually see them. One of them is described in the very next section.

9.7. Structural directives

Now, our `RacesComponent` is still not displaying the races :) The "proper way" in Angular would mean creating another component `RaceComponent` to display each race. We are going to do something slightly simpler, and just write a simple `<ul><li>` list.

Property and event binding is great, but it does not let us change the DOM structure, like iterating over a collection and adding an element per item. To do so, we need to use **structural directives**. A directive in Angular is really close to a component, but does not have a template. It is used to add behavior to an element.

The structural directives provided by Angular rely on using a `ng-template` element, inspired by the `template` standard tag of the HTML specification. It even used to be called `template` before version 4.0, but it's now deprecated, and you should use `ng-template`:

```
<ng-template>
  <div>Races list</div>
</ng-template>
```

Here we have defined a template, displaying a simple `div`. Alone, it does not have much use, as the browser will not display it. But if we add one 'template' element in a view, then Angular can use its content. The structural directives have the ability to do simple actions with this content, like displaying it or not, repeating it, etc.

Let's see which directives are available!

9.7.1. NgIf

We might want this template instantiated only if a condition is matched. For this, we will use the directive `ngIf`:

```
<ng-template [ngIf]="races.length > 0">
  <div><h2>Races</h2></div>
</ng-template>
```

The framework provides a few directives, like `ngIf`. They come from the module we imported earlier: `BrowserModule`. You can also define your own directives if needed: we'll come back to custom directives later.

Here, the template will be instantiated only if `races` has at least one element, that is to say if there are races. As this syntax is a bit long, there is a shorter version:

```
<div *ngIf="races.length > 0"><h2>Races</h2></div>
```

And you will virtually always use this shorter version.

The syntax uses `*` to show it is a structural directive. The `ngIf` will or will not display the `div` whenever the value of `races` changes: if there are no more races, the `div` will disappear.

The directives provided by the framework are already pre-loaded for us so we don't need to import and declare `NgIf` in the `directives` attribute of the `@Component` decorator.

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-races',
  template: `<div *ngIf="races.length > 0"><h2>Races</h2></div>`
})
export class RacesComponent {
  races: Array<any> = [];
}
```

It's also possible to use an `else` syntax since the version 4.0:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-races',
  template: `
    <div *ngIf="races.length > 0; else empty"><h2>Races</h2></div>
    <ng-template #empty><h2>No races.</h2></ng-template>
  `
})
export class RacesComponent {
  races: Array<any> = [];
}
```

9.7.2. NgFor

Working with real data will inevitably lead you to display a list of something. That's when `NgFor` proves very useful: it allows you to instantiate one template per item in a collection. Our `RacesComponent` component contains a field `races` which, as you can probably guess, is an array of

races to display.

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-races',
  template: `
    <div *ngIf="races.length > 0">
      <h2>Races</h2>
      <ul>
        <li *ngFor="let race of races">{{ race.name }}</li>
      </ul>
    </div>
  `
})
export class RacesComponent {
  races: Array<any> = [{ name: 'London' }, { name: 'Lyon' }];
}
```

And now we have a beautiful list, with one `li` tag per item in our collection!

```
<ul>
  <li>London</li>
  <li>Lyon</li>
</ul>
```

Note that `NgFor` is using a particular syntax, called a microsyntax.

```
<ul>
  <li *ngFor="let race of races">{{ race.name }}</li>
</ul>
```

It is the equivalent of the more wordy (that we'll never use):

```
<ul>
  <ng-template ngFor let-race [ngForOf]="races">
    <li>{{ race.name }}</li>
  </ng-template>
</ul>
```

Here you can recognize:

- the `template` element to declare an inline template
- the `NgFor` directive applied to it
- the `NgForOf` property where we feed the collection to display

- the variable `race` allowing us to use it in the interpolation expression, and reflecting the current element

Instead of remembering all these parts, it is easier to use the shorter form:

```
<ul>
  <li *ngFor="let race of races">{{ race.name }}</li>
</ul>
```

It is possible to declare another local variable bound to the index of the current element:

```
<ul>
  <li *ngFor="let race of races; index as i">{{ i }} - {{ race.name }}</li>
</ul>
```

The local variable `i` will receive the index of the current element, starting at zero. `index` is an exported variable. Some directives export variables that you can then assign to a local variable to be able to use them in your template:

```
<ul>
  <li>0 - London</li>
  <li>1 - Lyon</li>
</ul>
```

There are also other exported variables that can be useful:

- `even`, a boolean that is true if the element has an even index
- `odd`, a boolean that is true if the element has an odd index
- `first`, a boolean that is true if the element is the first of the collection
- `last`, a boolean that is true if the element is the last of the collection

9.7.3. NgSwitch

As you can guess from its name, this directive allows to switch between different templates based on a condition.

```
<div [ngSwitch]="messageCount">
  <p *ngSwitchCase="0">You have no message</p>
  <p *ngSwitchCase="1">You have a message</p>
  <p *ngSwitchDefault>You have some messages</p>
</div>
```

As you can see, `ngSwitch` takes a condition and the `*ngSwitchCase` take the possible values. You can also have `*ngSwitchDefault` that will be displayed if none of the values matched.

9.8. Other template directives

Two other directives can be useful when writing a template, but they are not structural directives like the ones we just saw. These directives are standard directives.

9.8.1. NgStyle

The first one is `ngStyle`. We already saw that we can act on the style of an element using:

```
<p [style.color]="foreground">Friendship is Magic</p>
```

If you need to set several styles at the same time, you can use the `ngStyle` directive:

```
<div [ngStyle]="{fontWeight: fontWeight, color: color}">I've got style</div>
```

Note that the directive expects an object whose keys are the styles to set. The keys can either be in camelCase (`fontWeight`) or in dash-case (`'font-weight'`).

9.8.2. NgClass

In the same spirit, the `ngClass` directive allows you to add or remove classes dynamically on an element.

As for the style, you can either set one class using property binding:

```
<div [class.awesome-div]="isAnAwesomeDiv()">I've got style</div>
```

Or, if you want to set several at the same time, you can use `ngClass`:

```
<div [ngClass]="{'awesome-div': isAnAwesomeDiv(), 'colored-div': isAColoredDiv()}"
>I've got style</div>
```

9.9. Canonical syntax

Every syntax we have seen has a longer equivalent called the canonical syntax. This is mainly useful if your server side templating system is having trouble with the `[]` or `()` syntax, or if you really can't bear to use `[]`, `()`, `*`...

If you want to declare a property binding, you can do:

```
<ns-pony [name]="pony.name"></ns-pony>
```

or, using the canonical syntax:

```
<ns-pony bind-name="pony.name"></ns-pony>
```

For event binding, you can do:

```
<button (click)="onButtonClick()">Click me!</button>
```

or, using the canonical syntax:

```
<button on-click="onButtonClick()">Click me!</button>
```

And for local variables, you can use `ref-`:

```
<input type="text" ref-name>
<button on-click="name.focus()">Focus the input</button>
```

instead of the shorter form:

```
<input type="text" #name>
<button (click)="name.focus()">Focus the input</button>
```

9.10. Summary

The Angular templating system gives us a powerful syntax to express the dynamic part of our HTML. It allows to express data and property binding, event binding and templating concerns, in a clear way, each with their own symbols:

- `{{}}` for interpolation
- `[]` for property binding
- `()` for event binding
- `#` for variable declaration
- `*` for structural directives

It provides a way to interact with standard Web Components like no other framework does. As there is no ambiguity between the various meanings, we will see our tools and IDEs gradually improve to give us meaningful warnings on what we are writing in our templates.

All these symbols are shorter versions of their canonical counterparts, which you can also use if you wish.

It takes some time to be fluent in this syntax, but you will soon be up to speed, and then it's easy to read and write.

Let's go through a complete example before moving on.

I want to write a `PoniesComponent` component, displaying a list of ponies. Each pony should be represented by a `PonyComponent` component, but we haven't seen yet how to pass parameters to a component. So, for now, we are going to display a simple list. The list should be displayed only if it's not empty, and I'd like to have some color for the even lines of my list. Finally, I want to be able to refresh the list with a button click.

Ready?

We start to write our component, in its own file:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: ``
})
export class PoniesComponent {}
```

You can add it to the `AppComponent` component we wrote in the previous chapter to test it. You will have to import it, add it to the directives and insert the tag `<ns-ponies></ns-ponies>` in the template.

Our new component has a list of ponies:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: ``
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }];
}
```

We are going to display this list, using `NgFor`:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `
    <ul>
      <li *ngFor="let pony of ponies">{{ pony.name }}</li>
    </ul>
  `
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }];
}
```

One thing is missing, the refresh button:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `
    <button (click)="refreshPonies()">Refresh</button>
    <ul>
      <li *ngFor="let pony of ponies">{{ pony.name }}</li>
    </ul>
  `
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }];

  refreshPonies(): void {
    this.ponies = [{ name: 'Fluttershy' }, { name: 'Rarity' }];
  }
}
```

And of course, a touch of color to finish, with the use of `[style.color]` and the `isEven` exported variable:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `
    <button (click)="refreshPonies()">Refresh</button>
    <ul>
      <li *ngFor="let pony of ponies; even as isEven" [style.color]="isEven ? 'green'
: 'black'">
        {{ pony.name }}
      </li>
    </ul>
  `
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }];

  refreshPonies(): void {
    this.ponies = [{ name: 'Fluttershy' }, { name: 'Rarity' }];
  }
}
```

As you can see, we have used all the range of the templating syntax, and we have a perfectly working component. Our data is still hard-coded though: soon, we are going to see how to use a service to fetch them! This means learning about dependency injection first, so that we can use the HTTP service!



Try our two exercises Templates 🐴 and List of races 🐴! They are free and part of our Pro Pack, where you'll learn how to build a complete application step by step. The first one is all about building a small component, a responsive menu, and play with its template. The second guides you in building another component: the list of races.

Chapter 10. Dependency injection

10.1. DI yourself

Dependency injection is a well-known design pattern. Let's take a component of our application. This component may need some features offered by other parts of our app (let's say a service). That's what we call a dependency. Instead of letting the component create its dependencies, the idea is to let the framework create them, and provide them to the component. That is known as "inversion of control".



It has several interesting features:

- it allows easy development, by just saying what we want and where we want it.
- it allows easy testing, by replacing dependencies with mock ones.
- it allows easy configuration, by swapping implementation.

It's a concept vastly used on the server side, but AngularJS 1.x was one of the first to use it on the frontend side.

10.2. Easy to develop

To be able to use dependency injection, we need a few things:

- a way to register a dependency, to make it available for injection into another component/service.
- a way to declare what dependencies are needed in the current component/service.

The framework does the rest of the job. When we declare a dependency in a component, it will look into the registry if it can find it, will get the instance of the dependency or create one, and actually inject it in our component.

A dependency can be a service provided by Angular, or a service we have written ourselves.

Let's take an example with an `ApiService` service, already written by one of your colleagues. Since he's the lazy guy of the team, he just wrote a class with a method named `get` returning an empty array, but you can already guess that the service will be used to communicate with a backend API.

```
export class ApiService {
  get(path: string): Array<any> {
    // todo: call the backend API
  }
}
```

Using TypeScript, it's easy to declare a dependency for our component or service - we just have to use the type system.

Let's say we want to write a `RaceService` that would use the `ApiService`:

```
import { ApiService } from './api.service';

export class RaceService {
  constructor(private apiService: ApiService) {}

}
```

Angular will fetch the `ApiService` service for us and inject it into our constructor. When `RaceService` is needed, the constructor will be called, and we will have an `apiService` field referencing the `ApiService` service.

Now, we can add a method `list()` to our service, which will call our backend using the `ApiService` service:

```
import { ApiService } from './api.service';

export class RaceService {
  constructor(private apiService: ApiService) {}

  list(): Array<RaceModel> {
    return this.apiService.get('/races');
  }

}
```

To inform Angular that this service has some dependencies itself, we need to add a class decorator: `@Injectable()`:

```
import { Injectable } from '@angular/core';
import { RaceModel } from './race.model';
import { ApiService } from './api.service';

@Injectable()
export class RaceService {
  constructor(private apiService: ApiService) {}

  list(): Array<RaceModel> {
    return this.apiService.get('/races');
  }

}
```

As we are using the `ApiService`, we need to "register" it, so as to make it available for injection.

Starting with Angular 6.0, the easiest (and recommended) way to do this is to add the `@Injectable` decorator on `ApiService` and use `providedIn` directly inside:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ApiService {
  get(path: string): Array<any> {
    // todo: call the backend API
  }
}
```

An other way to do this is to use the `providers` attribute of the `@NgModule` decorator we saw earlier (that's what we used to do before Angular 6.0):

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { ApiService } from '../services/api.service';

@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent],
  providers: [
    ApiService
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Now, if we want to make our `RaceService` available for injection in other services or components, we have to register it too:

```
import { Injectable } from '@angular/core';
import { ApiService } from './api.service';
import { RaceModel } from './race.model';

@Injectable({
  providedIn: 'root'
})
export class RaceService {
  constructor(private apiService: ApiService) {}

  list(): Array<RaceModel> {
    return this.apiService.get('/races');
  }
}
```

And we're done!

We can use our new service wherever we want. Let's test it in the `AppComponent` component:

```
import { Component } from '@angular/core';
import { RaceService } from '../services/race.service';
import { RaceModel } from '../services/race.model';

@Component({
  selector: 'ns-root',
  template: `
    <h1>PonyRacer</h1>
    <p>{{ list() }}</p>
  `
})
export class AppComponent {
  // add a constructor with RaceService
  constructor(private raceService: RaceService) {}

  list(): Array<RaceModel> {
    return this.raceService.list();
  }
}
```

As our lazy colleague returned an empty array in the `get` method of `ApiService`, you should get nothing if you try to call the `list()` method.

Maybe we can do something about it...

10.3. Easy to configure

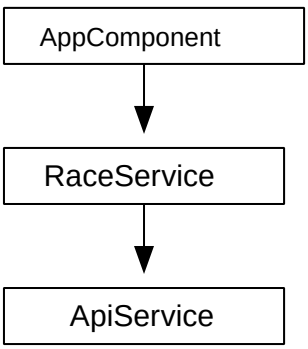
I'll come back to the testability advantages brought by dependency injection in a following chapter, but we can have a look at the configuration problem. Here we are calling a backend that does not exist. Maybe the backend team is not ready yet, or you want to do it later. In any case, we would like to use some fake data.

DI provides a nice way to do this. Let's go back to the registration part, if you chose to declare it in the module:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { RaceService } from '../services/race.service';
import { ApiService } from '../services/api.service';

@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent],
  providers: [
    RaceService,
    ApiService
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

We can represent the relations between component and services like this, where the arrows mean *depends on*:



In fact, what we wrote was the short form of:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { RaceService } from '../services/race.service';
import { ApiService } from '../services/api.service';

@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent],
  bootstrap: [AppComponent],
  providers: [
    { provide: RaceService, useClass: RaceService },
    { provide: ApiService, useClass: ApiService }
  ]
})
export class AppModule {}
```

We are telling the `Injector` that we want to create a link between a token (the type `RaceService`) and the class `RaceService`. The `Injector` is a service which keeps track of the injectable components by maintaining a registry and is actually injecting them when needed. The registry is a map that associates keys, called tokens, with classes. The tokens are not necessarily strings, unlike in many dependency injection frameworks. They can be anything, like Type references, for example. And that will usually be the case.

Since, in our example, the token and the class to inject are the same, you can write the same thing in the shorter form:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { AppComponent } from './app.component';
import { RaceService } from '../services/race.service';
import { ApiService } from '../services/api.service';

@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent],
  providers: [
    RaceService,
    ApiService
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

The token has to uniquely identify the dependency.

This injector is returned by the `bootstrapModule` promise, so we can play with it:

```
  // in our module
  providers: [
    ApiService,
    { provide: RaceService, useClass: RaceService },
    // let's add another provider to the same class
    // with another token
    // A token can be declared like this:
    // const token = new InjectionToken<RaceService>('RaceServiceToken');
    { provide: token, useClass: RaceService }
  ]

// let's bootstrap the module
platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .then(
    // and play with the returned injector
    appRef => playWithInjector(appRef.injector)
  );
```

The interesting part is in the `playWithInjector` function.

```
function playWithInjector(inj: Injector): void {
  console.log(inj.get(RaceService));
  // logs "RaceService {apiService: ApiService}"
  console.log(inj.get(token));
  // logs "RaceService {apiService: ApiService}" again
  console.log(inj.get(RaceService) === inj.get(RaceService));
  // logs "true", as the same instance is returned every time for a token
  console.log(inj.get(RaceService) === inj.get(token));
  // logs "false", as the providers are different,
  // so there are two distinct instances
}
```

As you can see, we can ask the injector for a dependency with the `get` method and a token. As I have declared the `RaceService` twice, with two different tokens, we have two providers. The injector will create an instance of `RaceService` the first time it is asked to for a specific token, and then returns the same instance for this token every time. It will do the same for each provider, so here we will actually have two instances of `RaceService` in our app, one for each token.

However, you will not use the token very often, or even at all. In TypeScript, you rely on the types to get the job done, so the token is a Type reference, usually bound to the corresponding class. If you want to use another token, you have to use the decorator `@Inject()`: see the last part of this chapter for more information about this.

This whole example was just to point out a few things:

- a provider links a token to a service.
- the injector returns the same instance every time it is asked for the same token.

- we can have a token name different from the class name

The fact that the instance returned is created on the first call and then is always the same is also a well-known design pattern: it's called a *singleton*. This is really useful, as you can share information between components using a service, and you will be sure they share the same service instance.

Now, back to our fake `RaceService` problem. I can write a new class, doing the same job as `RaceService` but returning hardcoded data:

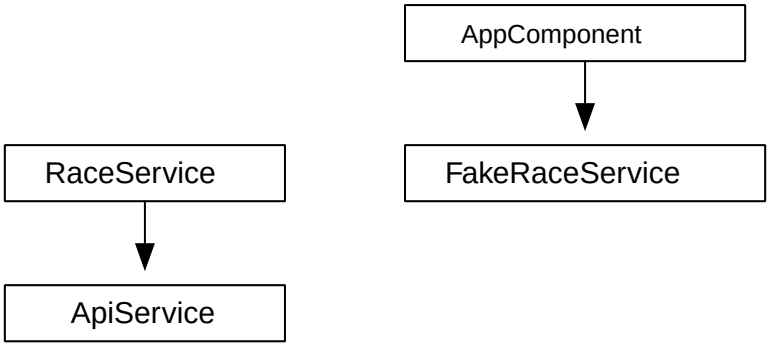
```
@Injectable()
export class FakeRaceService {
  list(): Array<RaceModel> {
    return [{ name: 'London' }];
  }
}
```

We can use the provider declaration to replace `RaceService` with our `FakeRaceService`:

```
// in our module
providers: [
  // we provide a fake service
  { provide: RaceService, useClass: FakeRaceService }
]
```

If you restart your app, you will see that there we have one race this time, because our app is using the fake service instead of the first one!

Now we have a relation like this:



That can be really useful when you test your app manually or, as we will soon see, when you are writing automated tests.

10.4. Other types of provider

In our example, we might want to use `FakeRaceService` when we are developing our app, and use

the real `RaceService` when we are in production. You can change it manually of course, but you can also use another type of provider: `useFactory`.

```
// we just have to change this constant when going to prod
const IS_PROD = false;

  // in our module
  providers: [
    // we provide a factory
    {
      provide: RaceService,
      useFactory: () => (IS_PROD ? new RaceService(null) : new FakeRaceService())
    }
  ]
```

In this example, we are using `useFactory` instead of `useClass`. A factory is a function with one job, creating an instance. Our example tests a constant and returns the fake service or the real service.

But wait, if we switch back to the real service, as we are using `new` to create the `RaceService`, it will not have its `ApiService` dependency instantiated! Right, if we want to make this example work, we have to pass an `ApiService` instance to the constructor call. Good news: `useFactory` can be used with another property named `deps`, where you can specify an array of dependencies:

```
// we just have to change this constant when going to prod
const IS_PROD = true;

  // in our module
  providers: [
    ApiService,
    // we provide a factory
    {
      provide: RaceService,
      // the apiService instance will be injected in the factory
      // so we can pass it to RaceService
      useFactory: (apiService: ApiService) => (IS_PROD ? new RaceService(apiService) :
new FakeRaceService()),
      deps: [ApiService]
    }
  ]
```

Hooray!



Be careful, the order of the parameters should be the same as the order in the array if you have several dependencies!

Of course, this example is just to demonstrate the use of `useFactory` and its dependencies. You could, and should, write:

```
// in our module
providers: [ApiService, { provide: RaceService, useClass: IS_PROD ? RaceService :
FakeRaceService }]
```

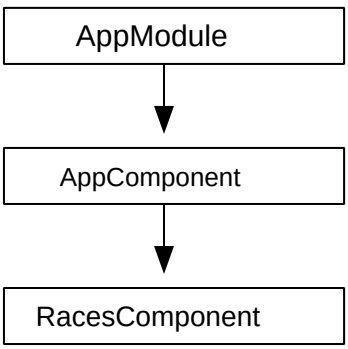
Declaring a constant for `IS_PROD` is really bothering: maybe we can use dependency injection too? I'm pushing things a bit as you can see :) You don't necessarily need to force all things in DI, but this is just to show you another provider type: `useValue`.

```
// in our module
providers: [
  ApiService,
  // we provide a factory
  { provide: 'IS_PROD', useValue: true },
  {
    provide: RaceService,
    useFactory: (IS_PROD: boolean, apiService: ApiService) =>
      IS_PROD ? new RaceService(apiService) : new FakeRaceService(),
    deps: ['IS_PROD', ApiService]
  }
]
```

10.5. Hierarchical injectors

One last crucial thing to understand in Angular: there are several injectors in your app. In fact, there is one injector per component, and this injector inherits from the injector of its parent.

Let's say we have an app looking like:

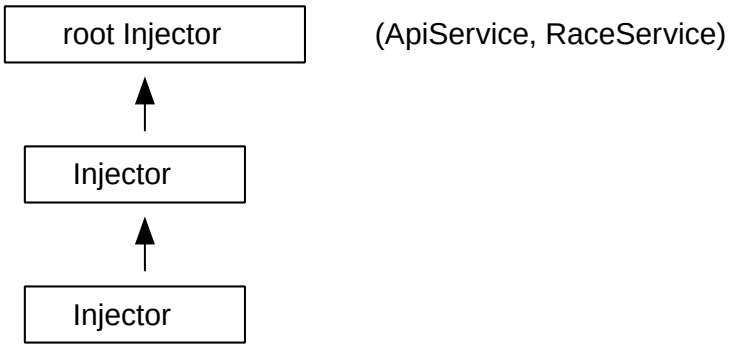


We have a module `AppModule` with a root component `AppComponent`, with a child component `RacesComponent`.

When we bootstrap the app, we create the root injector for the module. Then, every component will create its own injector, inheriting its parent one.

When you register a service using the recommended `providedIn: 'root'`, or by using the providers

of the module, you add these services to the root injector. Note that `'root'` is not the only value allowed in `providedIn`: in fact, if you have several modules in your application, you can choose to add it only to one module, like `providedIn: RacesModule` for example.



It means that when we are declaring a dependency in a component, Angular will begin its search in the current injector. If it finds the dependency, perfect, it returns it. If not, it will do the same in the parent injector, and again, until it finds the dependency. If it doesn't, it will throw an exception.

From this, we can deduce two things:

- the dependencies declared in the root injector are available for every component in the app. For example, `ApiService` and `RaceService` can be used everywhere.
- we can declare dependencies at another level than the module. How do we do this?

The `@Component` decorator can take another configuration option, called `providers`. This `providers` attribute can take an array with a list of dependencies, as we did for the `providers` attribute of `@NgModule`.

We can imagine a `RacesComponent` that would declare its own `RaceService` provider:

```
@Component({
  selector: 'ns-races',
  providers: [{ provide: RaceService, useClass: FakeRaceService }],
  template: `<strong>Races list: {{ list() }}</strong>`
})
export class RacesComponent {
  constructor(private raceService: RaceService) {}

  list(): Array<RaceModel> {
    return this.raceService.list();
  }
}
```

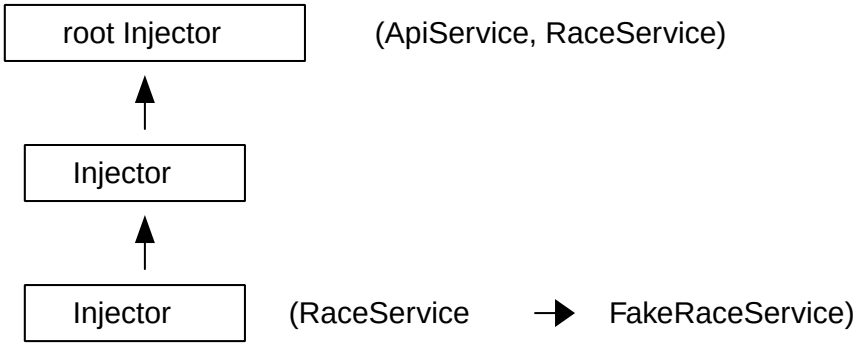
In this component, the provider with the token `RaceService` will always give an instance of `FakeRaceService`, whatever was defined in the root injector. It's really useful if you want to have a different instance of a service for a given component, or if you want to have perfectly encapsulated

components that declare everything they need.

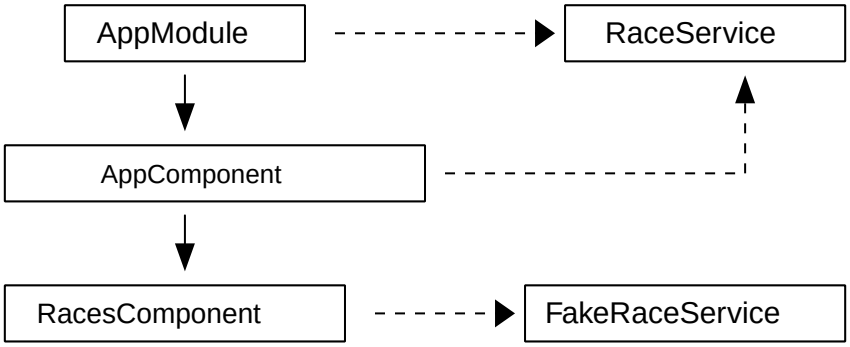


If you declare a dependency in the module of your app and in the `providers` attribute of your component, there will be two distinct instances of this dependency created and used!

Here we have:



The injection will then be resolved as:



As a rule of thumb, if only one component needs to have access to a service, it's a good idea to only provide this service in the component's injector, using the `providers` attribute. If the dependency can be used by the whole app, declare it in the root module.

10.6. DI without types

It's also possible to not use a type for dependency injection, by using the `@Inject()` decorator. With this decorator, you can then inject services but also simple values:

```
import { Inject, Injectable } from '@angular/core';

import { BACKEND_URL } from './tokens';

@Injectable({
  providedIn: 'root'
})
export class RaceService {
  constructor(@Inject(BACKEND_URL) private url: string) {}

}
```

To use `@Inject()`, you have to give it a token. This token is defined like this:

```
import { InjectionToken } from '@angular/core';

export const BACKEND_URL = new InjectionToken<string>('API URL');
```

We then use this token in the providers of the module to define its value:

```
{ provide: BACKEND_URL, useValue: 'http://localhost:8080' }
```

Or you can register it directly with `providedIn`:

```
export const BACKEND_URL_PROVIDED = new InjectionToken<string>('API URL', {
  providedIn: 'root',
  factory: () => 'http://localhost:8080'
});
```

Note that you can use the environment variables feature of Angular CLI for this use case.

Angular itself uses this token mechanism, and allows us to define the locale of the application for example, by using the token `LOCALE_ID`:

```
@NgModule({
  imports: [BrowserModule],
  declarations: [
    CustomLocaleComponent
    // and other components
  ],
  providers: [
    { provide: LOCALE_ID, useValue: 'fr-FR' }
  ]
  // ...
})
export class AppModule {}

@Component({
  selector: 'ns-locale',
  template: `
    <p>The locale is {{ locale }}</p>
    <!-- will display 'fr-FR' -->

    <p>{{ 1234.56 | number }}</p>
    <!-- will display '1 234,56' -->
  `
})
export class CustomLocaleComponent {
  constructor(@Inject(LOCALE_ID) public locale: string) {}
}
```

But we'll talk about it later when we see how to internationalize your application!

Chapter 11. Services

Angular contains the concept of services: classes you can inject into an other.

A few services are provided by the framework, some by the common modules, and others can be built by you. We will see the ones provided by the common modules in dedicated chapters; right now, let's have a look at the core ones, and discover how we can build ours.

11.1. Title service

The core framework provides very few services, and those you will use in your apps are even scarcer: actually there are just two for now :).

One question that pops up frequently is how can I change the title of my page? Easy! There is a `Title` service you can inject and it offers a getter and a setter method:

```
import { Component } from '@angular/core';
import { Title } from '@angular/platform-browser';

@Component({
  selector: 'ns-root',
  template: `<h1>PonyRacer</h1>`
})
export class AppComponent {
  constructor(title: Title) {
    title.setTitle('PonyRacer - Bet on ponies');
  }
}
```

The service will automatically create the `title` element in the `head` if needed and correctly set the value for you!

11.2. Meta service

The other service is somewhat similar: it allows you to get or update the "meta" values of the page.

```
import { Component } from '@angular/core';
import { Meta } from '@angular/platform-browser';

@Component({
  selector: 'ns-root',
  template: `<h1>PonyRacer</h1>`
})
export class AppComponent {
  constructor(meta: Meta) {
    meta.addTag({ name: 'author', content: 'Ninja Squad' });
  }
}
```

11.3. Making your own service

That's really simple. Just build a class, and you're done!

```
import { RaceModel } from './race.model';

export class RacesService {
  list(): Array<RaceModel> {
    return [{ name: 'London' }];
  }
}
```

Just like in AngularJS 1.x, a service is a singleton, so the same, unique instance of the class will be injected everywhere. It thus makes a service a great candidate to share state between several unrelated components!

If your service has some dependencies itself, then you need to add the `@Injectable()` decorator on it. Without this decorator, the framework won't do the dependency injection.

Our `RacesService` probably fetches the races from a REST API instead of returning the same list every time. To perform an HTTP request, the framework provides the `HttpClient` service. Don't worry, we'll soon see how it works.

Our service has a dependency on `HttpClient` to fetch the races, so we need to add a constructor with the `HttpClient` service as an argument, and add the `@Injectable()` decorator on the class.

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class RacesServiceWithHttp {
  constructor(private http: HttpClient) {}

  list(): Observable<Array<RaceModel>> {
    return this.http.get<Array<RaceModel>>('/api/races');
  }
}
```

Then you have to register it, by adding it to the `providers` attribute of a component, or to the `providers` of your root module, or starting with Angular 6.0, by using `providedIn` in `@Injectable()` as we did above.



Try our exercise Race service 🏇! It's free and part of our Pro Pack, where you'll learn how to build a complete application step by step. This exercise lets you build your first service!

Chapter 12. Pipes

12.1. Pied piper

Sometimes the raw data is not what we want to display in the view. We often want to transform it, filter it, limit its number, etc. AngularJS 1.x had a very handy feature to do this, very badly named 'filters'. Lessons have been learned and now these data transformers have a meaningful name! Nah, I'm just kidding, they are called 'pipes' :).

A pipe can be used either in HTML or in your applicative code. Let's take an example and see how we can use it.

12.2. json

A pipe that is not really useful in a production app, but very handy when you are debugging your app, is `JsonPipe`. Basically, this pipe applies `JSON.stringify()` to your data. If you have some data in your component, an array of ponies called `ponies`, for example, and you want to quickly see what's inside, you may want to try something like:

```
<p>{{ ponies }}</p>
```

Tough luck, it's going to display `[object Object]`...

But `JsonPipe` is here to rescue us. You can use it in your HTML, in any expression:

```
<p>{{ ponies | json }}</p>
```

And it will display the JSON representation of your object:

```
<p>[ { "name": "Rainbow Dash" }, { "name": "Pinkie Pie" } ]</p>
```

You can see where the name 'pipe' is coming from. To use a pipe, you have to add a pipe (`|`) character after your data, and then the name of the pipe you want to use. The expression is evaluated and the result goes through the pipe. It's possible to chain several pipes, one after another, like:

```
<p>{{ ponies | slice:0:2 | json }}</p>
```

We'll come back to the `slice` pipe, but you can see that we are chaining the `slice` pipe and then the `json` one.

You can use it in an interpolation expression or in a property expression, but **not** in an event statement.

```
<p [textContent]="ponies | json"></p>
```

12.3. slice

If you want to display just a part of a list, `slice` is your friend. It works like the `slice` method in JavaScript, and takes two arguments: a start index and, optionally, an end index.

To pass an argument to a pipe, you have to add a colon `:`, then the first argument, then possibly, another colon and the second argument etc.

```
<p>{{ ponies | slice:0:2 | json }}</p>
```

This example will display the first two elements of my list of ponies.

`slice` works with arrays and strings, so you can also truncate a string:

```
<p>{{ 'Ninja Squad' | slice:0:5 }}</p>
```

and that will display only 'Ninja'.

You can give the `slice` pipe only one index n, and it will take the elements from n to the end.

```
<p>{{ 'Ninja Squad' | slice:3 }}</p>
<!-- will display 'ja Squad' -->
```

If you give it a negative integer, it will take the n **last** elements.

```
<p>{{ 'Ninja Squad' | slice:-5 }}</p>
<!-- will display 'Squad' -->
```

As we saw, you can also give the pipe an end index: it will take the elements until this index. If this index is negative, it will take the elements until the index, but starting from the end.

```
<p>{{ 'Ninja Squad' | slice:2:-2 }}</p>
<!-- will display 'nja Squ' -->
```

As you can use `slice` in any expression, you can use it even with `NgFor`:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `<div *ngFor="let pony of ponies | slice: 0:2">{{ pony.name }}</div>`
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }, { name:
'Fluttershy' }];
}
```

The component will create only two `div` elements here, for the first two ponies, as we have applied the `slice` pipe to the collection.

The pipe argument can of course be a dynamic value:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `<div *ngFor="let pony of ponies | slice: 0:size">{{ pony.name }}</div>`
})
export class PoniesComponent {
  size = 2;
  ponies = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }, { name: 'Fluttershy' }];
}
```

You can use this to create a dynamic display where your user chooses how many elements she/he wants to see.

Note that it's also possible to store the result of the `slice` in a variable, using the `as` syntax introduced in 4.0:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-ponies',
  template: `
    <div *ngFor="let pony of ponies | slice: 0:2 as total; index as i">
      {{ i + 1 }}/{{ total.length }}: {{ pony.name }}
    </div>
  `
})
export class PoniesComponent {
  ponies: Array<any> = [{ name: 'Rainbow Dash' }, { name: 'Pinkie Pie' }, { name:
'Fluttershy' }];
}
```

12.4. keyvalue

This pipe, introduced in Angular 6.1, allows you to iterate over a Map or an object, and to display the keys/values in our templates.

Note that it orders the keys:

- first lexicographically if they are both strings
- then by their value if they are both numbers
- then by their boolean value if they are both booleans (`false` before `true`).

And if the keys have different types, they will be cast to strings and then compared.

```
@Component({
  selector: 'ns-ponies',
  template: `
    <ul>
      <!-- entry contains { key: number, value: PonyModel } -->
      <li *ngFor="let entry of ponies | keyvalue">{{ entry.key }} - {{
entry.value.name }}</li>
    </ul>
  `
})
export class PoniesComponent {
  ponies = new Map<number, PonyModel>();

  constructor() {
    this.ponies.set(103, { name: 'Rainbow Dash' });
    this.ponies.set(56, { name: 'Pinkie Pie' });
  }
}
```

If you have `null` or `undefined` keys, they will be displayed at the end.

It's also possible to define your own comparator function:

```
@Component({
  selector: 'ns-ponies',
  template: `
    <ul>
      <!-- entry contains { key: PonyModel, value: number } -->
      <li *ngFor="let entry of poniesWithScore | keyvalue: ponyComparator">{{
entry.key.name }} - {{ entry.value }}</li>
    </ul>
  `
})
export class PoniesComponent {
  poniesWithScore = new Map<PonyModel, number>();

  constructor() {
    this.poniesWithScore.set({ name: 'Rainbow Dash' }, 430);
    this.poniesWithScore.set({ name: 'Rainbow Dash' }, 680);
    this.poniesWithScore.set({ name: 'Pinkie Pie' }, 125);
  }

  /*
   * Defines a custom comparator to order the elements by the name of the PonyModel
(the key)
   */
  ponyComparator(a: KeyValue<PonyModel, number>, b: KeyValue<PonyModel, number>): -1 |
0 | 1 {
    if (a.key.name === b.key.name) {
      return 0;
    }
    return a.key.name < b.key.name ? -1 : 1;
  }
}
```

12.5. uppercase

As its name makes it clear enough, this pipe transforms a string into its uppercase version:

```
<p>{{ 'Ninja Squad' | uppercase }}</p>
<!-- will display 'NINJA SQUAD' -->
```

12.6. lowercase

The counterpart of the previous one, this pipe transforms a string into its lowercase version:

```
<p>{{ 'Ninja Squad' | lowercase }}</p>
<!-- will display 'ninja squad' -->
```

12.7. titlecase

Angular 4 introduced a new `titlecase` pipe. It capitalizes the first letter of all words:

```
<p>{{ 'ninja squad' | titlecase }}</p>
<!-- will display 'Ninja Squad' -->
```

12.8. number



The following pipes (`number`, `percent`, `currency`, `date`) can help with internationalization. They have been completely overhauled in Angular 5.0, and don't use the Intl API of the browsers anymore (it caused numerous bugs). The Angular team has now implemented the internationalization logic themselves. The following examples use the new implementation of the pipes, which comes with Angular 5, without diving into the internationalization details, as they are covered in a chapter at the end of the book. The examples also use the default locale of Angular, `en-US`.

This pipe lets you format a number.

It takes one parameter, a string, formatted as `{integerDigits}.{minFractionDigits}-{maxFractionDigits}`, but every part is optional. Each part indicates:

- how many numbers you want in the integer part
- how many numbers you want at least in the decimal part
- how many numbers you want at most in the decimal part

A few examples, starting with what we have with no pipe:

```
<p>{{ 12345 }}</p>
<!-- will display '12345' -->
<p>{{ 12345 }}</p>
<!-- will display '12345' -->
```

Using the `number` pipe will group the integer part, even with no digits required:

```
<p>{{ 12345 | number }}</p>
<!-- will display '12,345' -->
```

The `integerDigits` parameter will left-pad the integer part with zeros if needed:

```
<p>{{ 12345 | number:'6.' }}</p>
<!-- will display '012,345' -->
```

The `minFractionDigits` is the minimum size of the decimal part, so it will pad zeros on the right until reached:

```
<p>{{ 12345 | number:'.2' }}</p>
<!-- will display '12,345.00' -->
```

The `maxFractionDigits` is the maximum size of the decimal part. You have to specify a `minFractionDigits`, even at 0, if you want to use it. If the number has more decimals than that, then it is rounded:

```
<p>{{ 12345.13 | number:'.1-1' }}</p>
<!-- will display '12,345.1' -->

<p>{{ 12345.16 | number:'.1-1' }}</p>
<!-- will display '12,345.2' -->
```

12.9. percent

Based on the same principle as `number`, `percent` lets you display... a percentage!

```
<p>{{ 0.8 | percent }}</p>
<!-- will display '80%' -->

<p>{{ 0.8 | percent:'.3' }}</p>
<!-- will display '80.000%' -->
```

12.10. currency

As you can imagine, this pipe lets you format an amount of money in the currency you want. You have to give it at least one parameter:

- the ISO string representing the currency ('EUR', 'USD'...)
- optionally, an option to say if you want to use the symbol ('€', '$', CA$') with `'symbol'` or the ISO code with `'code'`, or even the narrow symbol with `'symbol-narrow'`. The narrow symbol is for example `$`, when the symbol is `CA$` for Canadian dollars. The default value of this option is `'symbol'`.
- optionally also, a string to format the amount, using the same syntax as `number`.

```
<p>{{ 10.6 | currency:'CAD' }}</p>
<!-- will display 'CA$10.60' -->

<p>{{ 10.6 | currency:'CAD':'symbol-narrow' }}</p>
<!-- will display '$10.60' -->

<p>{{ 10.6 | currency:'EUR':'code':'.3' }}</p>
<!-- will display 'EUR10.600' -->
```

If you don't provide the ISO string representing the currency, then `USD` is used, unless you configure it globally (since Angular 9). Check the Internationalization chapter if you want to learn how.

This pipe is way more powerful than what you might expect: it relies on the ISO 4217 specification to determine the number of digits in the decimal part. For example, formatting an amount in Chilean pesos results in an amount with no digits (as pesos don't have cents), whereas formatting an amount in Tunisian dinars results in an amount with 3 digits (as it has millimes).

12.11. date

The `date` pipe formats a date value to a string of the desired format. The date can be a `Date` object or a number of milliseconds. The format specified can be either a pattern like 'dd/MM/yyyy', 'MM-yy' or one of the predefined symbolic names available like 'short', 'longDate', etc.:

```
<p>{{ birthday | date:'dd/MM/yyyy' }}</p>
<!-- will display '16/07/1986' -->

<p>{{ birthday | date:'longDate' }}</p>
<!-- will display 'July 16, 1986' -->
```

Of course, you can also display the time portion of the date:

```
<p>{{ birthday | date:'HH:mm' }}</p>
<!-- will display '15:30' -->

<p>{{ birthday | date:'shortTime' }}</p>
<!-- will display '3:30 PM' -->
```



To learn more about internationalization in general, and, in particular, about the way you can set the language used to format numbers and dates, you can refer to the Internationalization chapter .

12.12. async

The `async` pipe allows data obtained asynchronously to be displayed. Under the hood, it uses `PromisePipe` or `ObservablePipe` depending on whether your async data comes from a Promise or an

Observable. I hope you now know what a Promise is (otherwise go back to the ES2015+ chapter), and we'll come back to Observables quickly.

The async pipe returns an empty string until the data is finally available (i.e. until the promise is resolved, in case of a promise). Once resolved, the resolved value is returned. More importantly, it triggers a change detection check once the data is available.

The following example uses a Promise:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-greeting',
  template: `<div>{{ asyncGreeting | async }}</div>`
})
export class GreetingComponent {
  asyncGreeting = new Promise(resolve => {
    // after 1 second, the promise will resolve
    window.setTimeout(() => resolve('hello'), 1000);
  });
}
```

You can see the `async` pipe is applied to the variable `asyncGreeting`. This one is a promise, resolved after 1 second. Once the promise is resolved, our browser will display:

```
<div>hello</div>
```

Even more interesting, if the source is an Observable, then the pipe will do the unsubscribe part itself when the component is destroyed (for example when the user navigates to another component).

And to avoid multiple subscriptions to your Observable or calling your promise multiple times, you can store the result of the call with `as` (since 4.0):

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-user',
  template: `<div *ngIf="asyncUser | async as user">{{ user.name }}</div>`
})
export class UserComponent {
  asyncUser = new Promise<UserModel>(resolve => {
    // after 1 second, the promise will resolve
    window.setTimeout(() => resolve({ name: 'Cédric' }), 1000);
  });
}
```

If you want to learn more about this pipe, check the Performances chapter near the end of this ebook.

12.13. A pipe in your code

You can also use it in your code, via dependency injection:

```
import { Component, Inject, LOCALE_ID } from '@angular/core';
// you need to import the pipe you want to use
import { DecimalPipe } from '@angular/common';

@Component({
  selector: 'ns-pony',
  template: `<p>{{ formattedSpeed }}</p>`
})
export class PonyComponent {
  pony = { name: 'Rainbow Dash', speed: 15 };
  formattedSpeed: string;

  // inject the Pipe you want
  constructor(decimalPipe: DecimalPipe, @Inject(LOCALE_ID) locale: string) {
    // and then call the transform method on it
    this.formattedSpeed = decimalPipe.transform(this.pony.speed, '.2', locale);
  }
}
```

But beware: the pipe must be added to the `providers` of your `@NgModule` (or `@Component`) in order to use it that way.

This is the same for every pipe.

But since Angular 6, it is now possible to use the formatting functions offered by the framework directly. For example, to format a number, we can use the `formatNumber` function:

```
import { Component, Inject, LOCALE_ID } from '@angular/core';
// you need to import the function you want to use
import { formatNumber } from '@angular/common';

@Component({
  selector: 'ns-pony',
  template: `<p>{{ formattedSpeed }}</p>`
})
export class PonyComponent {
  pony = { name: 'Rainbow Dash', speed: 15 };
  formattedSpeed: string;

  constructor(@Inject(LOCALE_ID) locale: string) {
    // use the format function
    this.formattedSpeed = formatNumber(this.pony.speed, locale, '.2');
  }
}
```

12.14. Creating your own pipes

Of course, you can also create your own pipes. That's sometimes very useful. In AngularJS 1.x, we often used custom filters. For example, we built one to display how much time elapsed since an action the user did (like `12 seconds ago` or `3 days ago`) in several of our apps. Let's see how we would do this in Angular!

First we need to create a new class. It should implement the `PipeTransform` interface, which forces us to have a `transform()` method, the one doing the heavy lifting.

Does not sound too hard. Let's give it a try!

```
import { Pipe, PipeTransform } from '@angular/core';

export class FromNowPipe implements PipeTransform {
  transform(value: string, args: Array<unknown>): string {
    // do something here
  }
}
```

We are going to use the `parseISO` function from date-fns to parse the date, and the `formatDistanceToNow` function to display how much time has elapsed since the date.

You can install date-fns using `NPM` if you want:

```
npm install date-fns
```

The types for date-fns are already included in the NPM dependency, so the TypeScript compiler

should be happy without us doing anything.

```
import { Pipe, PipeTransform } from '@angular/core';
import { formatDistanceToNowStrict, parseISO } from 'date-fns';

export class FromNowPipe implements PipeTransform {
  transform(value: string, args: Array<unknown>): string {
    const date = parseISO(value);
    return formatDistanceToNowStrict(date, { addSuffix: true });
  }
}
```

Now, we need to register the pipe in our app. For this, there is a special decorator we can use: `@Pipe`.

```
import { Pipe, PipeTransform } from '@angular/core';
import { formatDistanceToNowStrict, parseISO } from 'date-fns';

@Pipe({ name: 'fromNow' })
export class FromNowPipe implements PipeTransform {
  transform(value: string, args: Array<unknown>): string {
    const date = parseISO(value);
    return formatDistanceToNowStrict(date, { addSuffix: true });
  }
}
```

The chosen name will be the one allowing to use the pipe in the template.

To use the pipe in a template, the last thing you need to do is to add the pipe to the `declarations` of your `@NgModule`.

```
@NgModule({
  imports: [BrowserModule],
  declarations: [AppComponent, RacesComponent, FromNowPipe],
  bootstrap: [AppComponent]
})
export class AppModule {}
```



Try our exercise Pipes 🐴! It's free and part of our Pro Pack, where you'll learn how to build a complete application step by step. This exercise lets you use your first pipe. Later, the Custom pipe with date-fns 🦄 exercise will make you build an awesome custom pipe!

Chapter 13. Reactive Programming

13.1. Call me maybe

You may have heard of reactive or functional reactive programming lately. It has become quite popular in several languages platforms, like in .Net with the *Reactive Extensions* library, which is now available in pretty much every language (RxJava, RxJS, etc.).

Reactive programming is not really new. It is a way to build an app using events and reacting to them (hence the name). The events can be composed, filtered, grouped, etc. using functions like `map`, `filter`, etc. That's why you sometimes find the terms "functional reactive programming". But, to be accurate, reactive programming is not really functional programming, as it does not necessarily include the concepts of immutability, the lack of side-effects etc. Reacting on events is something you may have done:

- in the browser, when setting listeners to user events;
- on the backend side, reacting to events coming from a message bus.

In reactive programming, all data coming in will be in a stream. These streams can be listened to, modified of course (filtered, merged…), and can even become a new stream that can be listened to. This technique allows for fairly decoupled programs: you don't have to worry much about the consequences of your method call, you just raise an event, and every part of your app interested in this business will react accordingly. And maybe one of these parts will raise an event also, etc.

Now, why am I telling you about that? What does it have to do with Angular?

Well, Angular is built using reactive programming, and we will use this technique for some parts as well. Reacting on a HTTP request? Reactive programming. Spawning a custom event for our component? Reactive programming. Dealing with value changes in our forms? Reactive programming.

So let's focus on this topic for a few minutes. Nothing hard to handle, but it's better to have a clear mind on this.

13.2. General principles

In reactive programming, everything is a stream. A stream is an ordered sequence of events. These events represent values (look, another value!), errors (that went bad) or completion events (ok, I'm done). All these are pushed from the data producer to the consumer. As a developer, your job will be to *subscribe* to these streams, i.e. defining a listener capable of handling the three possibilities. Such a listener is called an *observer*, and the stream, an *observable*. These terms were coined a long time ago, as it is a well-known design pattern: the *observer* pattern.

They are different from promises, even if they look a bit similar, as they both handle asynchronous values. But an observer is not a one-time thing like a promise: it will continue to listen until it receives a 'completion' event.

For now, observables aren't part of the official ECMAScript specification, but they might be part of a

future version, as there is an effort being made to standardize it.

Observables are very close to arrays. An array is a collection of values, like an observable. An observable only adds the concept of values over time: in an array, you have all the values at once, while the values will come over time in an observable, maybe every few minutes.

The most popular reactive programming library in the JavaScript ecosystem is RxJS. And that's the one that Angular relies on and lets us use.

So let's have a look.

13.3. RxJS

Every observable, just like every array, can be transformed using functions you may have already encountered:

- `take(n)` will pick the first n events (e.g. the first five).
- `map(fn)` will apply `fn` to each event and return the result.
- `filter(predicate)` will only let through the events that fulfill the predicate.
- `reduce(fn)` will apply `fn` to every event to reduce the stream to a single value.
- `merge(s1, s2)` will merge the streams.
- `subscribe(fn)` will apply `fn` to each event it receives.
- and much more…

So, if you take an array of numbers and want to multiply each by 2, filter those under 5, and print them, you can do:

```
[1, 2, 3, 4, 5]
  .map(x => x * 2)
  .filter(x => x > 5)
  .forEach(x => console.log(x)); // 6, 8, 10
```

RxJS let us build an observable from an array. And, as you can see, we can do the exact same thing:

```
import { from } from 'rxjs';
import { filter, map } from 'rxjs/operators';
```

```
from([1, 2, 3, 4, 5])
  .pipe(
    map(x => x * 2),
    filter(x => x > 5)
  )
  .subscribe(x => console.log(x)); // 6, 8, 10
```

But an observable is more than a collection. It is an asynchronous collection, where the events arrive over time. A good example is browser events. They will happen over time, so they are a good use-case for an observable. Here is an example using jQuery:

```
import { fromEvent } from 'rxjs';
```

```
const input = $('input');

fromEvent(input, 'keyup').subscribe(() => console.log('keyup!'));

input.trigger('keyup'); // logs "keyup!"
input.trigger('keyup'); // logs "keyup!"
```

You can build observables from AJAX requests, browser events, Web sockets responses, a promise, whatever you can think of. And from a function of course:

```
const observable = new Observable(observer => observer.next('hello'));

observable.subscribe(value => console.log(value));
// logs "hello"
```

`new Observable()` takes a function that will emit events on the `observer` given as parameter. Here it simply emits one event for the demonstration.

You can also handle errors, because your observable may go wrong. The `subscribe` method also accepts an object as an argument instead of a simple callback function. This object can define a callback to handle events (named `next`), and a callback to handle errors (named `error`).

Here the `map` method throws an exception, so the error handler will log it.

```
range(1, 5)
  .pipe(
    map(x => {
      if (x % 2 === 1) {
        throw new Error('something went wrong');
      } else {
        return x;
      }
    }),
    filter(x => x > 5)
  )
  .subscribe({
    next: x => console.log(x),
    error: error => console.log(error)
  });
// something went wrong
```

Once the observable is done, it will emit a completion event that you can catch with a third handler. Here, the `range` method we are using to create the events will iterate from 1 to 5 and then emit the 'completed' signal:

```
range(1, 5)
  .pipe(
    map(x => x * 2),
    filter(x => x > 5)
  )
  .subscribe({
    next: x => console.log(x),
    error: error => console.log(error),
    complete: () => console.log('done')
  });
// 6, 8, 10, done
```



It's also possible to pass two or three callback functions as arguments to `subscribe()` but this is less readable and will be deprecated in RxJS 7. We thus advise you to pass an object as argument every time you need to handle the error or the completion.

You can do many, many things with an observable:

- transformation (delaying, debouncing...)
- combination (merge, zip, combineLatest...)
- filtering (distinct, filter, last...)
- maths (min, max, average, reduce...)
- conditions (amb, includes...)

We would need a whole book to go through it all! If you want to have a good visual representation of what each function does, go to rxmarbles.com.

Now let's have a look at how we will use observables in Angular.

13.4. Reactive programming in Angular

Angular uses RxJS, and it allows us to use it too. The framework provides an adapter around the `Observable` object: `EventEmitter`. The `EventEmitter` has a method `subscribe()` to react to events and this method can receive three parameters:

- a method to react to events.
- a method to react to errors.
- a method to react to completion

The `EventEmitter` can emit an event by calling the `emit()` method.

```
const emitter = new EventEmitter();

emitter.subscribe({
  next: (value: string) => console.log(value),
  error: (error: any) => console.log(error),
  complete: () => console.log('done')
});

emitter.emit('hello');
emitter.emit('there');
emitter.complete();

// logs "hello", then "there", then "done"
```

Note that the `subscribe` method returns a subscription object, with a method `unsubscribe` to... unsubscribe.

```
const emitter = new EventEmitter();

const subscription = emitter.subscribe({
  next: (value: string) => console.log(value),
  error: (error: any) => console.log(error),
  complete: () => console.log('done')
});

emitter.emit('hello');
subscription.unsubscribe(); // unsubscribe
emitter.emit('there');

// logs "hello" only
```

Now that we know a little bit more about reactive programming, and the `EventEmitter`, let's see how Angular uses it.



Try our exercise Observables 🐴! It's free and part of our Pro Pack, where you'll learn how to build a complete application step by step. In this exercise, you will transform the `RaceService` to make it reactive!

Chapter 14. Building components and directives

14.1. Introduction

So far, we have seen some small components. And of course, you can sense that, as they are the backbone of our apps, they can be more complex than what we have seen. How do we pass data? How do we manage the lifecycle of our component? What is best practice building these things?

Directives: What do they do? Do they do things? Let's find out!

14.2. Directives

A directive is very much like a component, except it does not have a template. In fact, the `Component` class inherits from the `Directive` class in the framework.

So it makes sense to start by studying directives, as everything we will see regarding directives also applies to components. We will look into the configuration options you are most likely to use. The more advanced ones are in a later chapter, ready for you when you master the basics.

As for a component, your directive will be annotated with a decorator, but instead of `@Component`, it will be `@Directive`.

Directives are very small pieces. You can think of them as decorators for your HTML: they will attach a behavior to elements in the DOM. You can have multiple directives on the same element.

A directive must have a CSS selector, which indicates to the framework where to activate it in our template.

14.2.1. Selectors

Selectors can be of various types:

- an element, as it's usually the case for components: `footer`.
- a class, not so frequent: `.alert`.
- an attribute, the most frequent for directives: `[color]`.
- an attribute with a specific value: `[color=red]`.
- a combination of the above: `footer[color=red]` matches an element named `footer` having an attribute `color` whose value is `red`. `[color],footer.alert` matches any element having an attribute `color` or (`,`) any element named `footer` with the CSS class `alert`. `footer:not(.alert)` matches any element named `footer` that does not (`:not()`) have the CSS class `alert`.

For example, this is a very simple directive that does nothing but gets activated if the attribute `doNothing` is on an element:

```
@Directive({
  selector: '[doNothing]'
})
export class DoNothingDirective {
  constructor() {
    console.log('Do nothing directive');
  }
}
```

Such a directive will be activated in a component like this `TestComponent`:

```
@Component({
  selector: 'ns-test',
  template: '<div doNothing>Click me</div>'
})
export class TestComponent {
}
```

A more complex selector could be:

```
@Directive({
  selector: 'div.loggable[logText]:not([notLoggable=true])'
})
export class ComplexSelectorDirective {
  constructor() {
    console.log('Complex selector directive');
  }
}
```

Here it will match all `div` elements with a `loggable` class and a `logText` attribute that don't have an attribute `notLoggable` with a `true` value.

So this template will trigger the directive:

```
<div class="loggable" logText="text">Hello</div>
```

But this one will not:

```
<div class="loggable" logText="text" notLoggable="true">Hello</div>
```

Let's be honest, though: if you are writing something like this, there is something wrong! :)



CSS selector like descendants, siblings, ids, wildcards and pseudos (other than `:not`) are not supported.



Don't start your attribute selectors with `bind-`, `on-`, `let-` or `ref-`: they have other meanings for the parser, as they are part of the canonical templating syntax.

Great, we know how to declare a directive. Let's make one that actually does something.

14.2.2. Inputs

Data binding is usually a big part of the job of creating a component or a directive. Every time you want a top component to pass data to one of its children, you will use a property binding.

To do this, we will define all the properties that accept data binding, using the `inputs` attribute of the `@Directive` decorator. This attribute accepts an array of strings, each one of the form `property: binding`. `property` represents the directive instance property and `binding` is the DOM property that will contain the expression.

For example, this directive is binding the DOM property `logText` to the directive instance property `text`:

```
@Directive({
  selector: '[loggable]',
  inputs: ['text: logText']
})
export class SimpleTextDirective {}
```

If the property does not exist in your directive, it is created for you. Then, every time the input changes, the property is updated automatically.

```
<div loggable logText="Some text">Hello</div>
```

If you want to be notified when the property changes, you can add a setter to your directive. The setter will be called every time the `logText` property changes.

```
@Directive({
  selector: '[loggable]',
  inputs: ['text: logText']
})
export class SimpleTextWithSetterDirective {
  set text(value: string) {
    console.log(value);
  }
}
```

So if we use it:

```
<div loggable logText="Some text">Hello</div>
// our directive will log "Some text"
```

There is also another way which we'll see in a few minutes.

Here the text is static, but of course it could easily be a dynamic value, with an interpolation:

```
<div loggable logText="{{ expression }}">Hello</div>
// our directive will log the value of 'expression' in the component
```

or the square bracket syntax:

```
<div loggable [logText]="expression">Hello</div>
// our directive will log the value of 'expression' in the component
```

That's one of the greatest features of the new template syntax: as a component developer, you don't care how your component is used. You just define which properties are bound (if you wrote some AngularJS 1.x, you know it was slightly different with all the '@' and '=' syntax).

You can also use pipes in your bindings:

```
<div loggable [logText]="expression | uppercase">Hello</div>
// our directive will log the value of 'expression' in the component in uppercase
```

If you want to bind a DOM property to an attribute of your directive that has the same name, you can simply write `property` instead of `property: binding`:

```
@Directive({
  selector: '[loggable]',
  inputs: ['logText']
})
export class SameNameInputDirective {
  set logText(value: string) {
    console.log(value);
  }
}
```

```
<div loggable logText="Hello">Hello</div>
// our directive will log "Hello"
```

There is another way to declare an input in your directive: with the `@Input` decorator. I like it very much, and the official guide style also recommends using it, so a lot of examples will use it from now on.

The examples above can be rewritten as:

```
@Directive({
  selector: '[loggable]'
})
export class InputDecoratorDirective {
  @Input('logText') text: string;
}
```

or, if the property and the binding have the same name:

```
@Directive({
  selector: '[loggable]'
})
export class SameNameInputDecoratorDirective {
  @Input() logText: string;
}
```

This will work but having a field and a setter with the same name will make the TypeScript compiler unhappy. One way to fix it if you need a setter (you don't always need it) is to add the `@Input` decorator directly on the setter.

```
@Directive({
  selector: '[loggable]'
})
export class InputDecoratorOnSetterDirective {
  @Input('logText')
  set text(value: string) {
    console.log(value);
  }
}
```

or, if the setter and the binding have the same name:

```
@Directive({
  selector: '[loggable]'
})
export class SameNameInputDecoratorOnSetterDirective {
  @Input()
  set logText(value: string) {
    console.log(value);
  }
}
```

Inputs are great to pass data from a top element to a bottom element. For example, if you want to have a component displaying a list of ponies, it is very likely that you will have a top component

containing the list, and another component to display a pony:

```
@Component({
  selector: 'ns-pony',
  template: `<div>{{ pony.name }}</div>`
})
export class PonyComponent {
  @Input() pony: PonyModel;
}

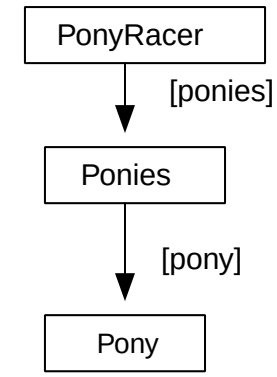
@Component({
  selector: 'ns-ponies',
  template: `
    <div>
      <h2>Ponies</h2>
      // the pony is handed to PonyComponent via [pony]="currentPony"
      <ns-pony *ngFor="let currentPony of ponies" [pony]="currentPony"></ns-pony>
    </div>
  `
})
export class PoniesComponent {
  ponies: Array<PonyModel> = [
    { id: 1, name: 'Rainbow Dash' },
    { id: 2, name: 'Pinkie Pie' }
  ];
}
```

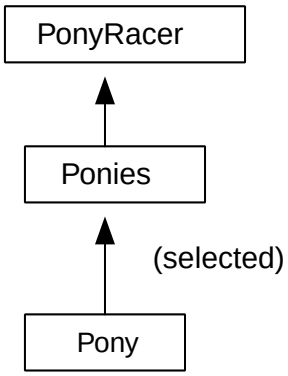
Ok, now what about passing data up? We can't use properties to pass data from `PonyComponent` to `PoniesComponent`. But we can use events!

14.2.3. Outputs

Let's go back to our latest example, and say we want to be able to select a pony by clicking on it, and inform the parent component. For this, we will use a custom event.

This is important. In Angular, data flows into a component via properties, and flows out of a component via events.





You remember the previous chapter on reactive programming? Cool, that's going to be useful! Custom events are emitted using an `EventEmitter`, and must be declared in the decorator, using the `outputs` attribute. Like the `inputs` attribute, it accepts an array with the list of events you want your directive/component to emit. And, like the inputs, it's better to use the `@Output()` decorator.

Let's say we want to emit an event called `ponySelected`. We have three things to do:

- declare the output in the decorator
- create an `EventEmitter` (the Angular style guide recommends marking it `readonly`)
- emit an event when the pony is selected

```
@Component({
  selector: 'ns-pony',
  // the method `selectPony()` will be called on click
  template: ` <div (click)="selectPony()">{{ pony.name }}</div> `
})
export class SelectablePonyComponent {
  @Input() pony: PonyModel;

  // we declare the custom event as an output,
  // the EventEmitter is used to emit the event
  @Output() readonly ponySelected = new EventEmitter<PonyModel>();

  /**
   * Selects a pony when the component is clicked.
   * Emits a custom event.
   */
  selectPony(): void {
    this.ponySelected.emit(this.pony);
  }
}
```

To use it in the template:

```
<ns-pony [pony]="pony" (ponySelected)="betOnPony($event)"></ns-pony>
```

In the above example, every time the user clicks on the pony name, it emits an event `ponySelected`, with the pony as a value (the parameter of the `emit()` method). The parent component is listening to this event, as you can see in the template, and will call its `betOnPony` method with the value of the event `$event`. `$event` is the syntax you have to use to access the event emitted: here, it will be the emitted pony.

The parent component must then have a method `betOnPony()`, which will be called with the selected pony:

```
betOnPony(pony: string): void {
  // do something with the pony
}
```

If you wish, you can specify an event name different from the event emitter name, with the syntax `emitter: event`:

```
@Component({
  selector: 'ns-pony',
  template: ` <div (click)="selectPony()">{{ pony.name }}</div> `
})
export class OtherSelectablePonyComponent {
  @Input() pony: PonyModel;
  // the emitter is called `emitter`
  // and the event `ponySelected`
  @Output('ponySelected') readonly emitter = new EventEmitter<PonyModel>();

  selectPony(): void {
    this.emitter.emit(this.pony);
  }
}
```

14.2.4. Lifecycle

You may want your directive to react on a specific moment of its life.

This is quite advanced stuff, and you won't need it every day, so I'll go fast.

One thing is really important to understand though, and you'll save quite some time if you do: **the inputs of a component are not evaluated yet in its constructor**.

That means that the following component will not work:

```
@Directive({
  selector: '[undefinedInputs]'
})
export class UndefinedInputsDirective {
  @Input() pony: string;

  constructor() {
    console.log(`inputs are ${this.pony}`);
    // will log "inputs are undefined", always
  }
}
```

If you want to access the value of an input, to load additional data from the server for example, you have to use a lifecycle phase.

Several phases are available, and have their own specificity:

- `ngOnChanges` will be the first to be called when the value of a bound property changes. It will receive a `changes` map, containing the current and previous values of the binding, wrapped in a `SimpleChange`. It will not be called if there is no change.
- `ngOnInit` will be called only once after the first change (whereas `ngOnChanges` is called on every change). It makes this phase perfect for initialization work, as the name suggests.

- `ngOnDestroy` is called when the component is removed. Really useful to do some cleanup.

Other phases are available, but are for more advanced use cases:

- `ngDoCheck` is slightly different. If present it will be called at each change detection cycle, overriding the default change detection algorithm, which looks for difference between every bound property value. That means that if at least one input has changed, by default the component is considered changed by the framework, and its children will be checked and rendered. But you can override it if you know that some inputs have no effect even if they have changed. That can be useful if you want to accelerate the change detection by just checking the bare minimum and not using the default algorithm, but usually you will not use this.
- `ngAfterContentInit` is called when all the projected contents of the component have been checked for the first time.
- `ngAfterContentChecked` is called when all the projected contents of the component have been checked, even if they haven't changed.
- `ngAfterViewInit` is called when all the bindings of the template have been checked for the first time.
- `ngAfterViewChecked` is called when all the bindings of the template have been checked, even if they haven't changed. It can be useful if your component or directive is waiting for an element to be available to do something with it, like focusing it for example.

If this sounds a bit hard to grasp, don't worry: keep on reading, and we'll explain all this better in the Advanced components and directives chapter a little further.

Our previous sample will work better using `ngOnInit`. Angular invokes the method `ngOnInit()` if it's present, so you just have to implement it in your directive. If you are using TypeScript for your app, you can leverage the available interface `OnInit` that forces you to implement the method:

```
@Directive({
  selector: '[initDirective]'
})
export class OnInitDirective implements OnInit {
  @Input() pony: string;

  ngOnInit(): void {
    console.log(`inputs are ${this.pony}`);
    // inputs are not undefined \o/
  }
}
```

Now we have access to our inputs!

If you want to do something every time a property changes, use `ngOnChanges`:

```
@Directive({
  selector: '[changeDirective]'
})
export class OnChangesDirective implements OnChanges {
  @Input() pony: string;

  ngOnChanges(changes: SimpleChanges): void {
    const ponyValue = changes.pony;
    console.log(`changed from ${ponyValue.previousValue} to ${ponyValue.currentValue}
`);
    console.log(`is it the first change? ${ponyValue.isFirstChange()}`);
  }
}
```

The `changes` parameter is a map, with the binding names as keys, and a `SimpleChange` object with two attributes (the previous and the current value) as value, as well as a method `isFirstChange()` to know if it is... the first change!

You can also use a setter if you want to react only on the change of one of your bindings. The following example will produce the same output as the previous one.

```
@Directive({
  selector: '[setterDirective]'
})
export class SetterDirective {
  private ponyModel: string;

  @Input()
  set pony(newPony: string) {
    console.log(`changed from ${this.ponyModel} to ${newPony}`);
    this.ponyModel = newPony;
  }
}
```

`ngOnChanges` is more useful if you need to watch several bindings at the same time. It will only be invoked if at least one binding has changed and will contain only the properties that have changed.

The `ngOnDestroy` phase is perfect to clean the component - for example, to cancel background tasks. Here, the `OnDestroyDirective` is logging "hello" every second when it is created. When the component is removed from the page, you want to stop the `setInterval` to avoid a memory leak:

```
@Directive({
  selector: '[destroyDirective]'
})
export class OnDestroyDirective implements OnDestroy {
  sayHello: number;

  constructor() {
    this.sayHello = window.setInterval(() => console.log('hello'), 1000);
  }

  ngOnDestroy(): void {
    window.clearInterval(this.sayHello);
  }
}
```

If you don't do this, you will have the thread logging "hello" until the end or the crash...

14.2.5. Providers

We already talked about providers in the Dependency Injection chapter . This attribute allows you to declare services that will be injectable in the current directive and its children.

```
@Directive({
  selector: '[providersDirective]',
  providers: [PoniesService]
})
export class ProvidersDirective {
  constructor(poniesService: PoniesService) {
    const ponies = poniesService.list();
    console.log(`ponies are: ${ponies}`);
  }
}
```

14.3. Components

A component is not really different from a directive: it just has two more optional attributes and **must** have an associated view. It does not bring a lot of new attributes compared to the directive.

14.3.1. View providers

We saw that you can specify injectables using `providers`. `viewProviders` is slightly similar but the providers will only be available for the current component, not for its children.

14.3.2. Template / Template URL

The main feature of a `@Component` is to have a template, whereas a directive does not have one. You can either declare your template inline, using `template` or use a URL to put it in a separate file with

`templateURL` (but you can't do both at the same time).

As a rule of thumb, if your template is small (1-2 lines), it's perfectly fine to keep it inline. When it starts to grow, move it to its own file to avoid cluttering your component.

You can use an absolute path for your URL, a relative one or even a complete HTTP URL.

When the component is loaded, Angular resolves the URL and tries fetching the template. If it succeeds, the template is the Shadow Root of the component, and its expressions are evaluated.

If I have a big component, I usually put the template in a separate file of the same folder, and use a relative URL to load it.

```
@Component({
  selector: 'ns-templated-pony',
  templateUrl: './components/pony/templated-pony.html'
})
export class TemplatedPonyComponent {
  @Input() pony: any;
}
```

If you use a relative URL, the URL will be resolved using the base URL of your app. The URL can be cumbersome, because if your component is in a directory `components/pony`, your template URL will be `components/pony/pony.html`.

But you can do slightly better if you package your application using CommonJS modules, by using the property `moduleId`. Its value must be `module.id`, a value that CommonJS will set at runtime. Angular can then use this value and build the correct relative URL. Your template URL can now look like:

```
@Component({
  selector: 'ns-templated-pony',
  templateUrl: './templated-pony.html',
  moduleId: module.id
})
export class ModuleIdPonyComponent {
  @Input() pony: any;
}
```

And you can locate your template in the same directory as your component!

Even better: if you are using Webpack, with a bit of configuration (already done for you if you are using Angular CLI), you can even remove the `module.id` and use a relative path directly. Webpack will be able to figure out the complete URL for you!

14.3.3. Styles / Styles URL

You can also specify the styles of your component. It is particularly useful if you plan to have really

isolated components. You can specify this using `styles` or `styleUrls`.

As you can see below, the `styles` attribute takes an array of CSS rules as a string. You can imagine it grows pretty quickly, so using a separate file and `styleUrls` is a good idea. As the name of the latter suggests, you can specify an array of URLs.

```
@Component({
  selector: 'ns-styled-pony',
  template: '<div class="pony">{{ pony.name }}</div>',
  styles: ['.pony{ color: red; }']
})
export class StyledPonyComponent {
  @Input() pony: any;
}
```

14.3.4. Declarations

Remember that you have to declare every directive and component you are using in the `declarations` of your `@NgModule`. If you don't, your component will not be picked up in the template, and you will waste a lot of time figuring out why.

The two most common mistakes are **forgetting to declare the directive** and **using the wrong selector**. If you don't understand why nothing happens, look out for these!

We have left a few things out for now, like the queries, change detection, exports, encapsulation options, etc. As they are more advanced options, you won't need them immediately; but don't worry, we'll see them soon in an advanced chapter!



Try our exercises Race detail 🦄 and Pony component 🦄! These exercises will guide you in building two more advanced components, with inputs and outputs.

Chapter 15. Styling components and encapsulation

Let's stop to talk about styles and CSS for a minute. I know right? Why talk about freaking CSS?

Well because Angular is doing a lot of things for us behind the scenes.

As a Web developer, you often add CSS classes to elements. And the essence of CSS is that it will *cascade*. That's sometimes what you want (to change the font everywhere in your app for example), or sometimes not. Imagine you want to add a style on a selected element in a list: you will usually use a very narrow CSS selector in your CSS, like `li.selected`. Or an even narrower one, using conventions like BEM, because you just want to style the selected element in a specific part of your app.

That's where Angular can be useful. The styles you define in a component (either with the `styles` attribute, or in a dedicated CSS file for the component with `styleUrls`), are scoped by Angular to this component and only this one. That's called style encapsulation. How does it achieve this?

It starts with you writing some styles. Then it depends on the strategy you select for the attribute `encapsulation` of the component decorator. This attribute can have three different values:

- `ViewEncapsulation.Emulated`, which is the default one
- `ViewEncapsulation.Native`, which relies on Shadow DOM v0 (first version of the specification, now deprecated)
- `ViewEncapsulation.ShadowDom`, which relies on Shadow DOM v1 (new option introduced in Angular 6.1, to support the new Shadow DOM specification)
- `ViewEncapsulation.None`, which means you don't want encapsulation

Each value will induce a different behavior of course, so let's have a look. We'll take a component you're starting to know well, i.e. our `PonyComponent`. This is a really simple version of the component, only displaying the pony's name in a `div`. For the purpose of the example, we add a CSS class `red` to this `div`:

```
import { Component, ViewEncapsulation } from '@angular/core';

@Component({
  selector: 'ns-pony',
  template: `<div class="red">{{ name }}</div>`,
  styles: [
    `
      .red {
        color: red;
      }
    `
  ],
  // that's the same as the default mode
  encapsulation: ViewEncapsulation.Emulated
})
export class PonyComponent {
  name = 'Rainbow Dash';
}
```

This class is then used in the styles of the component:

```
.red {
  color: red;
}
```

As you can see, we want to display the pony's name in a red font.

15.1. Shadow DOM strategy

If you use the `ShadowDom` option, you're telling Angular to use the Shadow DOM of your browser to take care of the encapsulation. The Shadow DOM is a part of the rather new Web Component specification. This specification allows you to create elements in a special DOM, which is perfectly encapsulated. With this strategy, if we look at the generated DOM with our browser's inspector, we'll see:

```
<ns-pony>
  #shadow-root (open)
    <style>.red {color: red}</style>
    <div class="red">Rainbow Dash</div>
</ns-pony>
```

You can spot the `#shadow-root (open)` that Chrome will display in the inspector: that's because our component has been included in a Shadow DOM element! And we can also see that the style was added at the top of our component's content.

With the `ShadowDom` strategy, you are sure that your component's styles are not "bleeding" into your

child components. If we have another component inside the `PonyComponent`, it can also define its own `red` CSS class with a different style: you are sure that the correct one will be used, with no interaction between each others!

But remember, Shadow DOM is a rather new specification, so it's not available in every browser. You can check the availability on the awesome website caniuse.com. So be careful when you use it in your apps!

15.2. Emulated strategy

As said earlier, this is the default strategy. And the reason is really simple: it emulates (hence the name) the `ShadowDom` strategy, but without using the Shadow DOM. So it's safe to use everywhere, and will have the same behavior.

To achieve that, Angular will take the CSS defined for the component, and inline it inside the `<head>` element of the page (and not in each component as we saw for the `ShadowDom` strategy). But before inlining it, it's going to rewrite the CSS selector, to append a unique attribute identifier. This unique attribute is then added to all the elements of our component's template! That way the style will only apply to our component. The same example, would now give:

```
<html>
  <head>
    <style>.red[_ngcontent-dvb-3] {color: red}</style>
  </head>
  <body>
    ...
    <ns-pony _ngcontent-dvb-2="" _nghost-dvb-3="">
      <div _ngcontent-dvb-3="" class="red">Rainbow Dash</div>
    </ns-pony>
  </body>
</html>
```

The `red` class selector has been rewritten to `.red[_ngcontent-dvb-3]`, so it will only apply on elements that have both the class `red` and the attribute `_ngcontent-dvb-3`. You can see that this attribute has also been added to our `div` automatically, so that works perfectly. The `<ns-pony>` element also has a few attributes: `_ngcontent-dvb-2` which is the unique identifier generated for its parent, and `_nghost-dvb-3` which is a unique identifier for the host element itself. Yes, we can also add styles that apply on the host element, as we'll see shortly.

15.3. None strategy

This strategy is not doing any encapsulation. The styles will be inlined at the top of the page (as for the `Emulated` strategy), but not rewritten. They then behave like "normal" styles, cascading into children.

15.4. Styling the host

A special CSS selector exists to style only the host element. It is called `:host`, and it comes from the Web Component specification:

```
:host {
  display: block;
}
```

It will be kept as is for the `ShadowDom` strategy and rewritten into `[_nghost-xxx]` if you use `Emulated`.

To conclude, you don't have to do much to have perfectly encapsulated styles, because the `Emulated` strategy takes care of this business for us. You can switch the strategy to use the `ShadowDom` one if you target only specific browsers, or `None` if you don't want to encapsulate styles. This strategy can be tweaked per component, or globally for your whole app in the root module.

Chapter 16. Testing your app

16.1. The problem with troubleshooting is that trouble shoots back

I love automated testing. My professional life revolves around the test progress bar going green in my IDE, patting me in the back for doing my job properly. And I hope you do care about tests too, as they are the only safety net we have when we write code. Nothing is more tedious than manually testing code.

Angular does a great job to let us easily write tests. So did AngularJS 1.x, and that's partly why I loved using it. As in AngularJS 1.x, we can write two types of tests:

- unit tests
- end-to-end tests

The first ones are there to verify that a small unit of code (a component, a service, a pipe…) works correctly in isolation, i.e. without considering its dependencies. Writing such a unit test requires you to execute each of the component/service/pipe methods, and check that the outputs are what we expected regarding the inputs we fed it. We can also check that the dependencies used by this unit are correctly called: for example we can check that a service will do the correct HTTP request.

We can also write end-to-end tests. Their purpose is to emulate a real user interacting with your app, by starting a real instance and then driving the browser to enter values in inputs, click on buttons, etc. We'll then check that the rendered page is in the state we expect, that the URL is correct - whatever you can think of.

We're going to cover all this, but let's begin with the unit test part.

16.2. Unit test

As we saw earlier, unit tests are there to check a small unit of code in isolation. These tests can only verify a small part of your app works as intended, but they have several advantages:

- they are really fast - you can run several hundreds in a few seconds.
- they are a very efficient way to test (nearly) all your code, especially the tricky cases, which can be hard to manually test in the real app.

One of the core concepts of unit testing is **isolation**: we don't want our test to be biased by its dependencies. So we usually use "mock" objects as dependencies. These are fake objects that we create just for testing purposes.

To do this, we are going to rely on a few tools. First we need a library to write tests. One of the most popular (if not the most popular) is Jasmine, so we are going to use it!

16.2.1. Jasmine and Karma

Jasmine gives us a few methods to declare our tests:

- `describe()` declares a test suite (a group of tests)
- `it()` declares a test
- `expect()` declares an assertion

A basic JavaScript test using Jasmine looks like:

```
class Pony {
  constructor(public name: string, public speed: number) {}

  isFasterThan(speed: number): boolean {
    return this.speed > speed;
  }
}

describe('My first test suite', () => {
  it('should construct a Pony', () => {
    const pony = new Pony('Rainbow Dash', 10);
    expect(pony.name).toBe('Rainbow Dash');
    expect(pony.speed).not.toBe(1);
    expect(pony.isFasterThan(8)).toBe(true);
  });
});
```

The `expect()` call can be chained with a lot of methods like `toBe()`, `toBeLessThan()`, `toBeUndefined()`, etc. Every method can be negated with the `not` attribute of the object returned by `expect()`.

The test file is a separate file from the code you want to test, usually with an extension like `.spec.ts`. The test for a Pony class written in a `pony.ts` file will likely be in a file named `pony.spec.ts`. You can either put your test right next to the file you're testing, or in a dedicated directory with all your tests. I tend to put the code and test in the same directory, but both approaches are perfectly valid: pick your team.



One cool trick is that if you use `fdescribe()` instead of `describe()` then only this test suite will run (f stands for focus). Same thing if you want to run only one test: use `fit()` instead of `it()`. If you want to exclude a test, use `xit()`, or `xdescribe()` for a suite.

You can also use the `beforeEach()` method to set up a context before each test: the **fixture**. If I have several tests on the same pony, it makes sense to use `beforeEach()` to initialize the pony, instead of copy/pasting the same thing in every test.

```
describe('Pony', () => {
  let pony: Pony;

  beforeEach(() => {
    pony = new Pony('Rainbow Dash', 10);
  });

  it('should have a name', () => {
    expect(pony.name).toBe('Rainbow Dash');
  });

  it('should have a speed', () => {
    expect(pony.speed).not.toBe(1);
    expect(pony.speed).toBeGreaterThan(9);
  });
});
```

There is also an `afterEach` method, but I basically never use it…

One last trick: Jasmine lets us create fake objects (mocks or spies, as you want), or even spy on a method of a real object. We can then do some assertions on these methods, like with `toHaveBeenCalled()` that checks if the method has been called, or with `toHaveBeenCalledWith()` that checks the exact parameters of the call to the spied method. You can also check how many times the method has been called, or check if it has ever been called, etc.

For example, let's say we have a `Race` class with a `start()` method, that calls `run()` on every pony in the race, and filters the ponies that did not started running (`run()` returns a boolean):

Race.ts

```
class Race {
  constructor(private ponies: Array<Pony>) {}

  start(): Array<Pony> {
    return (
      this.ponies
        // start every pony
        // and only keeps the ones that started running
        .filter(pony => pony.run(10))
    );
  }
}
```

We want to test the `start()` method, and see if it properly calls `run()`. So we spy on the `run()` method of all the ponies in the race:

Race.spec.ts

```
describe('Race', () => {
  let rainbowDash: Pony;
  let pinkiePie: Pony;
  let race: Race;

  beforeEach(() => {
    rainbowDash = new Pony('Rainbow Dash');
    // first pony agrees to run
    spyOn(rainbowDash, 'run').and.returnValue(true);

    pinkiePie = new Pony('Pinkie Pie');
    // second pony refuses to run
    spyOn(pinkiePie, 'run').and.returnValue(false);

    // create a race with these two ponies
    race = new Race([rainbowDash, pinkiePie]);
  });

});
```

and test if the methods are called:

Race.spec.ts

```
it('should make the ponies run when it starts', () => {
  // start the race
  const runningPonies: Array<Pony> = race.start();
  // should have called `run()` on the ponies
  expect(pinkiePie.run).toHaveBeenCalled();
  // with a speed of 10
  expect(rainbowDash.run).toHaveBeenCalledWith(10);
  // as one pony refused to start, the result should be an array of one pony
  expect(runningPonies).toEqual([rainbowDash]);
});
```

When you write unit tests, keep in mind that they should be small and readable. And don't forget to make them fail at first, to be sure you're testing the right thing.

The next step is to run our tests. For this, the Angular team has developed Karma, whose sole purpose is to run the tests in one or several browsers. It can also watch your files to re-run the tests on every save. As running the tests is really fast, it's actually really nice to do this and have (almost) instant feedback on your code.



I won't dive into the details on how to setup Karma, but it's a very interesting project with a lot of plugins you can use, to make it work with your favorite tools, to have a coverage report, etc. If you're writing your code in TypeScript like me, the strategy you can adopt is to let the TypeScript compiler watch your code and tests, produce the compiled files in a separate output directory, and have Karma watch this directory.

So we now know how to write a unit test in JavaScript. Let's add Angular to the mix.

16.2.2. Using dependency injection

Let's say I have an Angular application with a simple service like `RaceService`, containing a method returning a hard-coded races list.

```
@Injectable({
  providedIn: 'root'
})
export class RaceService {
  list(): Array<RaceModel> {
    const race1: RaceModel = { name: 'London' };
    const race2: RaceModel = { name: 'Lyon' };
    return [race1, race2];
  }
}
```

Let's write a test for this.

```
describe('RaceService', () => {
  it('should return races when list() is called', () => {
    const raceService = new RaceService();
    expect(raceService.list().length).toBe(2);
  });
});
```

That works great. But we can also rely on the dependency injection offered by Angular to grab the RaceService and inject it in our test. It's especially useful if our `RaceService` has some dependencies itself: instead of having to instantiate these dependencies ourselves, we could just rely on the injector to do it for us by saying: "hey, we want the RaceService, go figure out what you need to

create it and give it to me".

To use the dependency injection system in our test, the framework has a utility method in `TestBed` called `inject`.



`TestBed.inject` was introduced in Angular 9.0 to replace `TestBed.get`, which was not typesafe and is now deprecated.

This method allows you to get a specific dependency from the injector inside a test function.

Let's go back to our example, using `TestBed.inject` this time:

```
import { TestBed } from '@angular/core/testing';

describe('RaceService', () => {
  it('should return races when list() is called', () => {
    const raceService = TestBed.inject(RaceService);
    expect(raceService.list().length).toBe(2);
  });
});
```

That works, because the service is declared with `providedIn: 'root'`, which make it available in the test. It will be instantiated and injected lazily when needed in the test.

As we did in the simple Jasmine example, we can maybe move the `RaceService` initialization into a `beforeEach` method. We can also use `TestBed.inject` in a `beforeEach`, so let's do it:

```
import { TestBed } from '@angular/core/testing';

describe('RaceService', () => {
  let service: RaceService;

  beforeEach(() => (service = TestBed.inject(RaceService)));

  it('should return races when list() is called', () => {
    expect(service.list().length).toBe(2);
  });
});
```

We moved the `TestBed.inject` logic in a `beforeEach` and now our test is pretty clean.

Of course, a real `RaceService` will not have a hard-coded list of races, and there is a big chance that the response will be an asynchronous one. Let's say that the `list` returns an observable. What does that change in our test? Well, now we have to set up the `expect` in the `subscribe` callback:

```
import { TestBed, waitForAsync } from '@angular/core/testing';

describe('RaceService', () => {
  let service: RaceService;

  beforeEach(() => (service = TestBed.inject(RaceService)));

  it(
    'should return an observable of 2 races',
    waitForAsync(() => {
      service.list().subscribe(races => {
        expect(races.length).toBe(2);
      });
    })
  );
});
```

You may be thinking that this will not work, as the test will end before the subscribe is called, and our expectation will never run.

But here we wrap the test with the `waitForAsync()` function. And this method is really smart: it keeps track of the asynchronous calls made in the test and waits for them to resolve.

Angular uses a new concept called **zones**. These **zones** are execution contexts, and, to simplify, they keep track of all the stuff going on within them (timeouts, event listeners, callbacks...). They also provide hooks that can be called when we enter or leave the zone. An Angular application runs in a zone, and that's how the framework knows it has to refresh the DOM when an asynchronous action is done.

This concept is also used in the tests if your test uses `waitForAsync()`: the test runs in a zone, so the framework knows when all the asynchronous actions are done, and won't complete until then.

So our asynchronous expectation will be executed. Great!

There is another way to deal with async tests in Angular, using `fakeAsync()` and `tick()` but that's for a more advanced chapter.

16.3. Fake dependencies

The `TestBed` class will help us to declare fake dependencies. Its `configureTestingModule` method allows you to declare what can be injected in the test, by creating a test module containing only what we need. Try to inject only what's necessary in your test, to make them as loosely coupled to the rest of the app as possible. The method is called in the `beforeEach` Jasmine method, and takes a module configuration, really close to what you can pass to the `@NgModule` decorator. The `providers` attribute takes an array of dependencies that will become available to injection.

Being able to declare the dependencies with the test module has its use. We can without too much trouble declare a fake service as a dependency instead of a real one.

For the sake of the example, let's say that my `RaceService` uses the local storage to store the races, with a key `'races'`. Your colleagues have developed a service called `LocalStorageService` that deals with the JSON serialization, etc. that our `RaceService` uses. The `list()` method looks like:

```
@Injectable({
  providedIn: 'root'
})
export class RaceService {
  constructor(private localStorage: LocalStorageService) {}

  list(): Array<RaceModel> {
    return this.localStorage.get('races');
  }
}
```

Now, we don't really want to test the `LocalStorageService` service: we just want to test our `RaceService`. That can easily be done by leveraging the dependency injection system to give a fake `LocalStorageService`:

```
export class FakeLocalStorage {
  get(key: string): any {
    return [{ name: 'Lyon' }, { name: 'London' }];
  }
}
```

to `RaceService` in our test, using `provide`:

```
import { TestBed } from '@angular/core/testing';

describe('RaceService', () => {
  beforeEach(() =>
    TestBed.configureTestingModule({
      providers: [{ provide: LocalStorageService, useClass: FakeLocalStorage }]
    })
  );

  it('should return 2 races from localStorage', () => {
    const service = TestBed.inject(RaceService);
    const races = service.list();
    expect(races.length).toBe(2);
  });
});
```

Great! But I'm not completely satisfied with this test. Creating a fake service by hand is tedious, and Jasmine can help us spy on the service and replace its implementation with a fake one. It also allows you to verify that the `get()` method has been called with the proper key 'races'.

```
import { TestBed } from '@angular/core/testing';

describe('RaceService', () => {
  const localStorage = jasmine.createSpyObj<LocalStorageService>('LocalStorageService
', ['get']);

  beforeEach(() =>
    TestBed.configureTestingModule({
      providers: [{ provide: LocalStorageService, useValue: localStorage }]
    })
  );

  it('should return 2 races from localStorage', () => {
    localStorage.get.and.returnValue([{ name: 'Lyon' }, { name: 'London' }]);

    const service = TestBed.inject(RaceService);
    const races = service.list();

    expect(races.length).toBe(2);
    expect(localStorage.get).toHaveBeenCalledWith('races');
  });
});
```

16.4. Testing components

The next step after testing a simple service is to test a component. A component test is slightly different because we have to create the component. We can't rely on the dependency injection system to give us an instance of the component to test (you may have noticed by now that components are not injectable in other components :)).

Let's start by writing a component to test. Why not our `PonyComponent` component? It takes a pony as an input and emits an event `ponyClicked` when the component is clicked.

```
@Component({
  selector: 'ns-pony',
  template: `
    <img [src]="'./images/pony-' + pony.color.toLowerCase() + '.png'"
[alt]="pony.name" (click)="clickOnPony()" />
  `
})
export class PonyComponent {
  @Input() pony: PonyModel;
  @Output() readonly ponyClicked = new EventEmitter<PonyModel>();

  clickOnPony(): void {
    this.ponyClicked.emit(this.pony);
  }
}
```

It comes with a fairly simple template: an image with a dynamic source depending on the pony color, and a click handler.

To test such a component, you first need to create an instance. To do this, we can also use `TestBed`. This class comes with a utility method, named `createComponent`, to create a component. The method returns a `ComponentFixture`, a representation of our component. Note that to create a component, it must be known from the test module, so we need to add it to the `declarations` attribute:

```
import { TestBed } from '@angular/core/testing';

import { PonyComponent } from './pony.component';

describe('PonyComponent', () => {
  it('should have an image', () => {
    TestBed.configureTestingModule({
      declarations: [PonyComponent]
    });
    const fixture = TestBed.createComponent(PonyComponent);
    // given a component instance with a pony input initialized
    const ponyComponent = fixture.componentInstance;
    ponyComponent.pony = { name: 'Rainbow Dash', color: 'BLUE' };

    // when we trigger the change detection
    fixture.detectChanges();

    // then we should have an image with the correct source attribute
    // depending of the pony color
    const element = fixture.nativeElement;
    const imageElement = element.querySelector('img');
    expect(imageElement.getAttribute('src')).toBe('./images/pony-blue.png');
    expect(imageElement.getAttribute('alt')).toBe('Rainbow Dash');
  });
});
```

Here, we follow the "Given/When/Then" pattern to write the unit test. You'll find a whole literature on the subject, but it boils down to:

- a "Given" phase, where we set up the test context. We get the component instance created and provide a pony. It emulates an input that would come from a parent component in the real app.
- a "When" phase, where we manually trigger the change detection, using the `detectChanges()` method. In a test, the change detection is our responsibility: it's not automatic as it is in an app.
- and a "Then" phase, containing the expectations. We can get the native element and query the DOM as you would do with the browser (using `querySelector()` for example). Here we test if the image source is the correct one.

We can also test if the component really emits an event:

```
it('should emit an event on click', () => {
  TestBed.configureTestingModule({
    declarations: [PonyComponent]
  });
  const fixture = TestBed.createComponent(PonyComponent);

  // given a pony
  const ponyComponent = fixture.componentInstance;
  ponyComponent.pony = { name: 'Rainbow Dash', color: 'BLUE' };

  // we fake the event emitter with a spy
  spyOn(ponyComponent.ponyClicked, 'emit');

  // when we click on the pony
  const element = fixture.nativeElement;
  const image = element.querySelector('img');
  image.dispatchEvent(new Event('click'));

  // and we trigger the change detection
  fixture.detectChanges();

  // then the event emitter should have fired an event
  expect(ponyComponent.ponyClicked.emit).toHaveBeenCalled();
});
```

Let's have a look at another component:

```
@Component({
  selector: 'ns-race',
  template: `
    <div>
      <h1>{{ race.name }}</h1>
      <ns-pony *ngFor="let currentPony of race.ponies" [pony]="currentPony"></ns-pony>
    </div>
  `
})
export class RaceComponent {
  @Input() race: any;
}
```

and its test:

```
describe('RaceComponent', () => {
  let fixture: ComponentFixture<RaceComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [RaceComponent, PonyComponent]
    });
    fixture = TestBed.createComponent(RaceComponent);
  });

  it('should have a name and a list of ponies', () => {
    // given a component instance with a race input initialized
    const raceComponent = fixture.componentInstance;
    raceComponent.race = { name: 'London', ponies: [{ name: 'Rainbow Dash', color:
'BLUE' }] };

    // when we trigger the change detection
    fixture.detectChanges();

    // then we should have a title with the race name
    const element = fixture.nativeElement;
    expect(element.querySelector('h1').textContent).toBe('London');

    // and a list of ponies
    const ponies = fixture.debugElement.queryAll(By.directive(PonyComponent));
    expect(ponies.length).toBe(1);
    // we can check if the pony is correctly initialized
    const rainbowDash = ponies[0].componentInstance.pony;
    expect(rainbowDash.name).toBe('Rainbow Dash');
  });
});
```

Here we query all the directives of type `PonyComponent` and test if the first pony is correctly initialized.

You can get the components inside your component with `children` or query them with `query()` and `queryAll()`. These methods take a predicate as argument that can be either `By.css` or `By.directive`. That's what we do to get the ponies displayed, as they are instances of `PonyComponent`. Keep in mind that this is different from a DOM query using `querySelector()`: it will only find the elements handled by Angular, and will return a `ComponentFixture`, not a DOM element (so you'll have access to the `componentInstance` of the result, for example).

16.5. Testing with fake templates, providers...

When testing a component, we sometimes want to create a parent component that uses it. And if there are several use cases, we'll have to create several parent components just to try different inputs for example.

Hopefully, when we are in a test, we can modify any component to reuse it in different tests, by

overriding its template.

To do this, the `TestBed` gives an `overrideComponent()` method, to call before the `createComponent()` one:

```
describe('RaceComponent', () => {
  let fixture: ComponentFixture<RaceComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [RaceComponent, PonyComponent]
    });
    TestBed.overrideComponent(RaceComponent, { set: { template: '<h2>{{ race.name
}}</h2>' } });
    fixture = TestBed.createComponent(RaceComponent);
  });

  it('should have a name', () => {
    // given a component instance with a race input initialized
    const raceComponent = fixture.componentInstance;
    raceComponent.race = { name: 'London' };

    // when we trigger the change detection
    fixture.detectChanges();

    // then we should have a name
    const element = fixture.nativeElement;
    expect(element.querySelector('h2').textContent).toBe('London');
  });
});
```

As you can see, the method takes two arguments:

- the component you want to override
- the metadata you want to set, add or remove (for example here we set the template)

That means you can modify the template of the component you are testing, or one of its children (to replace a component with a big template with a dumb one).

`template` is not the only metadata available, you can also use:

- `providers` to replace the dependencies of a component
- `styles` to replace the styles used in the template of a component
- or any property you can set in the `@Component` decorator…

As replacing the template is the most common use-case, Angular 4 introduced an `overrideTemplate()` method:

```
describe('RaceComponent', () => {
  let fixture: ComponentFixture<RaceComponent>;

  beforeEach(() => {
    TestBed.configureTestingModule({
      declarations: [RaceComponent, PonyComponent]
    });
    TestBed.overrideTemplate(RaceComponent, '<h2>{{ race.name }}</h2>');
    fixture = TestBed.createComponent(RaceComponent);
  });

  it('should have a name', () => {
    // given a component instance with a race input initialized
    const raceComponent = fixture.componentInstance;
    raceComponent.race = { name: 'London' };

    // when we trigger the change detection
    fixture.detectChanges();

    // then we should have a name
    const element = fixture.nativeElement;
    expect(element.querySelector('h2').textContent).toBe('London');
  });
});
```

Now you're ready to test your app!

16.6. Simpler, cleaner unit tests with `ngx-speculoos`

Angular tests can quickly be very verbose. As we don't really like that, we wrote a tiny open-source library called ngx-speculoos.

Instead of writing a test looking like this:

```
let fixture: ComponentFixture<UserComponent>;

beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [TestModule]
  });
  fixture = TestBed.createComponent(UserComponent);
  fixture.detectChanges();
});

it('should display French cities when selecting the country France', () => {
  const countrySelect = fixture.nativeElement.querySelector('#country'); //
countrySelect is of type any
  expect(countrySelect.selectedIndex).toBe(0);
  countrySelect.selectedIndex = 2; // what is at index 2?
  countrySelect.dispatchEvent(new Event('change')); // why do I need to do that?
  fixture.detectChanges();

  const city = fixture.nativeElement.querySelector('#city'); // city is of type any
  expect(city).toBeTruthy();
  expect(city.options.length).toBe(3);
  expect(city.options[0].value).toBe('');
  expect(city.options[0].label).toBe('');
  expect(city.options[1].value).toBe('PARIS');
  expect(city.options[1].label).toBe('Paris');
  expect(city.options[2].value).toBe('LYON');
  expect(city.options[2].label).toBe('Lyon');
});

it('should hide cities when selecting the empty country option', () => {
  const countrySelect = fixture.nativeElement.querySelector('#country'); // I did that
previously. What about DRY?
  countrySelect.selectedIndex = 0;
  countrySelect.dispatchEvent(new Event('change')); // why do I need to do that?
  fixture.detectChanges(); // why do I need to do that?

  expect(fixture.nativeElement.querySelector('#city')).toBeFalsy(); // I did that
previously. What about DRY?
});
```

you can write a cleaner and simpler test with `ngx-speculoos`:

```
class UserComponentTester extends ComponentTester<UserComponent> {
  constructor() {
    super(UserComponent);
  }

  get country(): TestSelect {
    return this.select('#country'); // returns a TestSelect object, not any. Similar
methods exist for inputs, buttons, etc.
  }

  get city(): TestSelect {
    return this.select('#city'); // returns a TestSelect object, not any
  }
}

let tester: UserComponentTester;

beforeEach(() => {
  TestBed.configureTestingModule({
    imports: [TestModule]
  });
  tester = new UserComponentTester();
  tester.detectChanges();
});

it('should display French cities when selecting the country France', () => {
  tester.country.selectLabel('France'); // no dispatchEvent, no detectChanges needed

  expect(tester.city.optionValues).toEqual(['', 'PARIS', 'LYON']);
  expect(tester.city.optionLabels).toEqual(['', 'Paris', 'Lyon']);
});

it('should hide cities when selecting empty country option', () => {
  tester.country.selectIndex(0); // no repetition of the selector, no dispatchEvent,
no detectChanges needed

  expect(tester.city).toBeFalsy(); // no repetition of the selector
});
```

You can go one step further with the custom matcher for Jasmine we wrote:

```
beforeEach(() => jasmine.addMatchers(speculoosMatchers));

it('should contain a pre-populated form', () => {
  expect(tester.informationMessage).toContainText('Please check that everything is
correct');
  expect(tester.country).toHaveSelectedValue('');
  expect(tester.name).toHaveValue('Doe');
  expect(tester.newsletter).toBeChecked();
});
```

Give it a try!

16.7. End-to-end tests (e2e)

End-to-end tests are the other type of tests we can run. An end-to-end test consists in really launching your app in a browser and emulating a user interacting with it (clicking on buttons, filling forms, etc.). They have the advantage of really testing the application as a whole, but:

- they are slower (several seconds per test)
- it's hard to test the edge cases.

As you may guess, you don't have to choose between unit tests and e2e tests: you will combine both to have great coverage and some guarantees that your complete application runs as intended.

E2e tests rely on a tool called Protractor. It's identical to the tool we used in AngularJS 1.x for the same purpose. And the great news is that it works both with AngularJS 1.x and Angular!



You will write your test suite using Jasmine like in the unit tests, but you will use the Protractor API to interact with your app.

A simple test would look like this:

```
describe('Home', () => {

  it('should display title, tagline and logo', async () => {
    await browser.get('/');
    expect(await element.all(by.css('img')).count()).toEqual(1);
    expect(await $('h1').getText()).toContain('PonyRacer');
    expect(await $('small').getText()).toBe('Always a pleasure to bet on ponies');
  });

});
```

Protractor gives us a `browser` object, with a few utility methods like `get()` to go to a page. Then you have `element.all()` to select all the elements matching a predicate. This predicate often relies on `by` and its various methods (`by.css()` to do a CSS query, `by.id()` to retrieve an element by id, etc.). `element.all()` will return a promise, with a special method `count()` used in the test above.

`$('h1')` is a shortcut, equivalent of writing `element(by.css('h1'))`. It will fetch the first element matching the CSS query. You can use several methods on the promise returned by `$()`, like `getText()` and `getAttribute()` to retrieve information, or methods like `click()` and `sendKeys()` to act on this element.

These tests can be quite long to write and debug (much more than unit tests), but they are really useful. You can do all sorts of great things with them, like testing several browsers, do a screenshot every time a test fails, etc.

With unit tests and e2e tests, you have the keys to build a robust and maintainable application!



All our Pro Pack exercises come with unit tests! If you want to learn more, we strongly encourage you to take a look at them: we tested every possible part of the application (100% code coverage)! In the end you'll have dozens of test examples, which you can use in your own projects.

Chapter 17. Send and receive data through HTTP



This chapter uses the new `HttpClientModule` introduced in Angular 4.3 in the `@angular/common/http` package, which is a complete rewrite of the `HttpModule` that existed before. This chapter does *not* talk about the old `HttpModule` that was used previously from the `@angular/http` package.

It won't come as a surprise, but a lot of our job consists in asking a backend server to send data to our webapp, and then sending data back.

Usually this is done over HTTP, even though you have other alternatives nowadays, like WebSockets. Angular provides an http module, but doesn't force you to use it. If you prefer, you can use your favorite library to send HTTP requests.



One of the possibilities is the fetch API, which is currently available as a polyfill, but should become a standard in browsers. You can perfectly build your app using `fetch` or another library. In fact, that's what I used before the Http part was done in Angular. It works great, with no need of special calls to make the framework aware that we have received data and that it needs to run the change detection (unlike in AngularJS 1.x, where you would have to call `$scope.apply()` if you were using an external library: that's the magic of Angular and its zones!).

But if you feel comfortable with the framework, you will use a small module called `HttpClientModule`, provided by the core team. It is an independent module, so, really, do as you like. Note that it mirrors closely enough the Fetch API proposal.

If you want to use it, you have to use the classes from the `@angular/common/http` package.

Why prefer this module over, say, `fetch`? The answer is simple: testing. As we will show, the Http module allows you to mock your backend server and return fake responses. That's really, really useful.

A last thing before we dive into the API: the Http module heavily uses the reactive programming paradigm. So if you skipped the Reactive Programming chapter, now might be a good time to go back and read it ;).

17.1. Getting data

The Http module offers a service called `HttpClient` that you can inject in any constructor. As the service is coming from another module, you have to manually make it available to your component or service. To do so, import the `HttpClientModule` into the root module:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [BrowserModule, HttpClientModule],
  declarations: [AppComponent],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

Once this is done, you can inject the `HttpClient` service wherever you need it:

```
@Component({
  selector: 'ns-races',
  template: `<h1>Races</h1>`
})
export class RacesComponent {
  constructor(private http: HttpClient) {}

}
```

By default, the `HttpClient` service will do AJAX requests using `XMLHttpRequest`.

It offers several methods, matching the most common HTTP verbs:

- `get`
- `post`
- `put`
- `delete`
- `patch`
- `head`
- `jsonp`

If you used the `$http` service in AngularJS 1.x, you might remember that it heavily relied on Promises. In Angular, however, all these methods return an `Observable` object.

A few advantages come with the use of Observables for `HttpClient` like the ability to cancel requests, to retry, to easily compose them, etc.

Let's start by fetching the races available in PonyRacer. We'll assume that a backend is already up and running, providing a RESTful API. To fetch the races, we'll send a GET request to a URL like 'http://backend.url/api/races'.

Usually, the base URL of your HTTP calls will be stored in a variable or a service, that you can easily configure depending on your environment. Or, if the REST API is served by the same server as the Angular application, you can simply use a relative URL: '/api/races'.

Using the `HttpClient` service, such a request is straightforward:

```
http
  .get<Array<RaceModel>>(`${baseUrl}/api/races`)
```

Note that you don't need to deserialize the response body from a string to a JavaScript array or object. That is done automatically by Angular. However, Angular won't make any check to verify that the JSON in the response indeed conforms to the generic type you specified. It's up to you to make sure that you're using the correct generic type, and that the `RaceModel` interface indeed matches with the JSON that the server sends back.

This returns an Observable, to which you can subscribe to receive the response.

The response body is the most interesting part, and it is directly emitted by the Observable:

```
http.get<Array<RaceModel>>(`${baseUrl}/api/races`).subscribe((response: Array
<RaceModel>) => {
  console.log(response);
  // logs the array of races
});
```

Of course, you can also have access to the full HTTP response. The object returned is then an `HttpResponse` object, with a few fields like the `status` code, `headers`, etc.

```
http
  .get<Array<RaceModel>>(`${baseUrl}/api/races`, { observe: 'response' })
  .subscribe((response: HttpResponse<Array<RaceModel>>) => {
    console.log(response.status); // logs 200
    console.log(response.headers.keys()); // logs []
  });
```

The observable will throw an error if the response status is different from 2xx or 3xx, and the error is then of type `HttpErrorResponse`.

Sending data is fairly easy too. Just call the `post()` or `put()` method, with the URL and the object to post:

```
http
  .post<RaceModel>(`${baseUrl}/api/races`, newRace)
```

Once again, no need to serialize the race object being sent to JSON. Angular does that for you. The generic type `RaceModel` here is, just as with the `get()` method, the type of the response body. So this example endpoint takes a `RaceModel` as input and returns the created `RaceModel`.

I won't show you the other methods - I'm sure you get the idea.

17.2. Transforming data

This kind of work will usually be done in a dedicated service. I tend to create a service, like `RaceService`, where all the job is done. Then, my component just needs to subscribe to my service method, without knowing what's going on under the hood.

```
raceService.list().subscribe(races => {
  // store the array of the races in the component
  this.races = races;
});
```

You can also leverage the power of RxJS to retry a failed request a few times, for example.

```
raceService
  .list()
  .pipe(
    // if the request fails, retry 3 times
    retry(3)
  )
  .subscribe(races => {
    // store the array of the races in the component
    this.races = races;
  });
```

17.3. Advanced options

Of course, you can tune your requests more finely. Every method takes an `options` object as an optional parameter, where you can configure your request. A few options are really useful and you can override everything in the request.

`params` represents the URL search parameters (also known as the query string) to add to the URL.

```
const params = {
  sort: 'ascending',
  page: '1'
};

http
  .get<Array<RaceModel>>(`${baseUrl}/api/races`, { params })
  // will call the URL ${baseUrl}/api/races?sort=ascending&page=1
  .subscribe(response => {
    // will return the races sorted
    this.races = response;
  });
```

The `headers` option is often useful to add a few custom headers to your request. It happens to be necessary for some authentication techniques like JSON Web Token for example:

```
const headers = { Authorization: `Bearer ${token}` };

http
  .get<Array<RaceModel>>(`${baseUrl}/api/races`, { headers })
  .subscribe(response => {
    // will return the races visible for the authenticated user
    this.races = response;
  });
```

17.4. Jsonp

To let you access their API without being blocked by the Same Origin Policy enforced by web browsers, some web services don't use CORS, but use JSONP (JSON with Padding).

The server will not return the JSON data directly, but wrap them in the function passed as a callback. The response comes back as a script, and scripts are not subject to the Same Origin Policy. Once loaded, you can access the JSON value contained in the response.

In addition to the classic HTTP methods, the `HttpClient` offers a `jsonp` method. All you have to do is specify the URL of the service you want to call, and add the name of the callback you want.

In the following example, we are fetching all the public repos from our Github organization using JSONP.

```
http
  .jsonp<{ data: Array<GithupRepo> }>('https://api.github.com/orgs/Ninja-Squad/repos',
'callback')
  .pipe(
    // extract data
    map((res: { data: Array<GithupRepo> }) => res.data)
  )
  .subscribe(response => {
    // will return the public repos of Ninja-Squad
    this.repos = response;
  });
```

17.5. Interceptors

One of the reasons of the Http module rewrite was the introduction of interceptors. Interceptors are interesting when you want to... intercept requests or responses in your application.

For example, if you want to intercept every request to add a specific header to some of them, you can now write an interceptor like this one:

```
@Injectable()
export class GithubAPIInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    // if it is a Github API request
    if (req.url.includes('api.github.com')) {
      // we need to add an OAUTH token as a header to access the Github API
      const clone = req.clone({ setHeaders: { Authorization: `token ${OAUTH_TOKEN}` }
});
      return next.handle(clone);
    }
    // if it's not a Github API request, we just hand it to the next handler
    return next.handle(req);
  }
}
```

Note that you have to clone the request to update it (requests are immutable).

Then add your interceptor to the `HTTP_INTERCEPTORS` array via dependency injection:

```
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: GithubAPIInterceptor, multi: true }
]
```

Now every request will go through the interceptor, and receive the custom header if needed (here the requests to the Github API).

You can also intercept the response, which can be handy to handle errors in a generic way:

```
@Injectable()
export class ErrorHandlerInterceptor implements HttpInterceptor {
  constructor(private router: Router, private errorHandler: ErrorHandler) {}

  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>> {
    return next.handle(req).pipe(
      // we catch the error
      catchError((errorResponse: HttpErrorResponse) => {
        // if the status is Unauthorized
        if (errorResponse.status === 401) {
          // we redirect to login
          this.router.navigateByUrl('/login');
        } else {
          // else we notify the user
          this.errorHandler.handle(errorResponse);
        }
        return throwError(errorResponse);
      })
    );
  }
}
```

17.6. Tests

We now have a service calling an HTTP endpoint to fetch the races. How do we test it?

```
@Injectable({
  providedIn: 'root'
})
export class RaceService {
  constructor(private http: HttpClient) {}

  list(): Observable<Array<RaceModel>> {
    return this.http.get<Array<RaceModel>>('/api/races');
  }
}
```

In a unit test, you don't want to really call the HTTP server: that's not what we are testing. We want to "fake" the HTTP call to return fake data. To do this, we can replace the dependency to the `HttpClient` service with a fake implementation by importing the `HttpClientTestingModule`. We can then use a class provided by the framework called `HttpTestingController` to fake the HTTP responses.

And you can also add a few assertions on the underlying HTTP request:

```
import { TestBed } from '@angular/core/testing';
import { HttpClientTestingModule, HttpTestingController } from
'@angular/common/http/testing';
describe('RaceService', () => {
  let raceService: RaceService;
  let http: HttpTestingController;

  beforeEach(() =>
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule]
    })
  );

  beforeEach(() => {
    raceService = TestBed.inject(RaceService);
    http = TestBed.inject(HttpTestingController);
  });

  afterEach(() => {
    http.verify();
  });

  it('should return an Observable of 2 races', () => {
    // fake response
    const hardcodedRaces = [{ name: 'London' }, { name: 'Lyon' }];

    // call the service
    let actualRaces = [];
    raceService.list().subscribe(races => (actualRaces = races));

    // check that the underlying HTTP request was correct
    http
      .expectOne('/api/races')
      // return the fake response when we receive a request
      .flush(hardcodedRaces);

    // check that the returned array is deserialized as expected
    expect(actualRaces.length).toBe(2);
  });
});
```

And we're done!



Try our exercise HTTP 🦄! We prepared a full REST API, ready for you to use. Let's fetch some races using the `HttpClient` service. Later you'll learn how to call a secured API with an authentication mechanism and interceptors in exercises HTTP with authentication 🦄, Bet on a pony 🦄 and Cancel a bet 🦄. Slightly related, we'll also use WebSockets 🦄.

Chapter 18. Router

It is fairly common to want to map a URL to a state of the application. That makes sense: you want your user to be able to bookmark a page and come back, and it provides a better experience overall.

The piece in charge of doing this job is called a router, and every framework has its own (or several ones).



The router in Angular has a simple goal: the creation of meaningful URLs reflecting the state of our app, and each URL knowing what component should be initialized and inserted in the page. It will execute all this without refreshing the page and without triggering a new request to our backend server: this is the whole point of having a Single Page Application.

You probably know there was already a router in AngularJS 1.x, maintained by the core team, in a module called `ngRoute`. You may also know that it was a very simplistic one: OK for simple applications, but it was only allowing a single view per URL and no nesting was possible. It was a bit limited when working on bigger apps, where you often have views inside views. There was a very popular community module, called `ui-router`, that a lot of people were using and which was doing a really great job.

The team behind Angular decided to bridge the gap and wrote a new module called `RouterModule`. This module will hopefully fulfill all our needs!

Some new features are really interesting. So let's go!

18.1. En route

Let's start using the router. It is an optional module, that is thus not included in the core framework.

As we saw for the other modules, you have to include it in your root module if you want to use it. But for that, we need a configuration to define the mapping between URLs and components. We can do this with a dedicated file, generally named like `app.routes.ts`, and containing an array representing the configuration:

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { RacesComponent } from './races/races.component';

export const ROUTES: Routes = [
  { path: '', component: HomeComponent },
  { path: 'races', component: RacesComponent }
];
```

Then we need to import the router module in our root module, initialized with the proper configuration:

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { RouterModule } from '@angular/router';
import { ROUTES } from './app.routes';
import { HomeComponent } from './home/home.component';
import { RacesComponent } from './races/races.component';

@NgModule({
  imports: [BrowserModule, RouterModule.forRoot(ROUTES)],
  declarations: [AppComponent, HomeComponent, RacesComponent],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

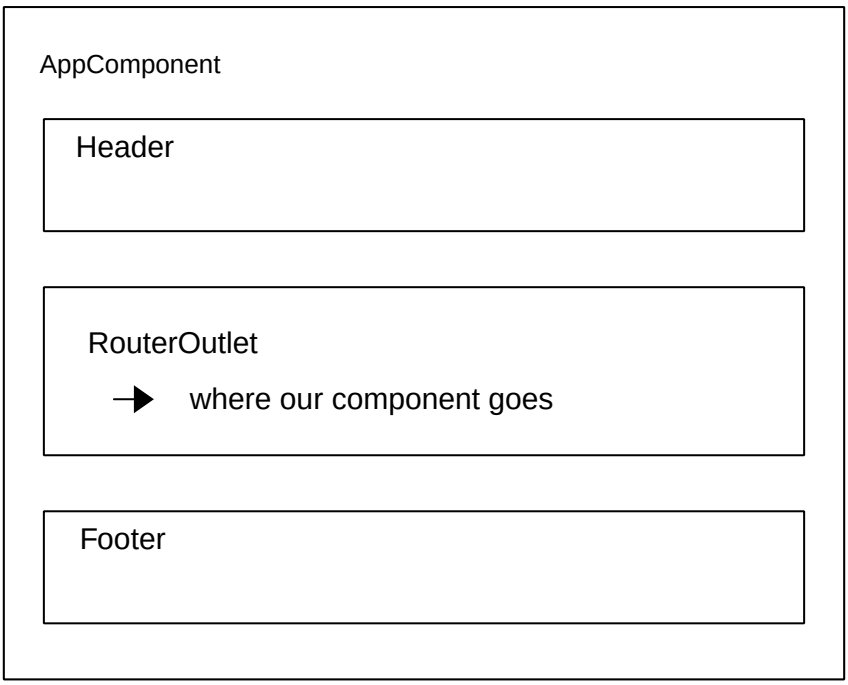


You also need to declare all the components used by the router module in the `declarations` attribute of your root module.

As you can see, the `Routes` is an array of objects, each one being a... route. A route configuration is usually a pair of properties:

- `path`: what URL will trigger the navigation
- `component`: which component will be initialized and inserted

You may be wondering where the component will be inserted in the page, and that's a good question. For a component to be included in our app, like the `RacesComponent` in the example above, we must use a special tag in the template of the primary component: `<router-outlet>`.



This is, of course, an Angular directive, whose only job is to act as a placeholder for the template of the component of the current route. Our app template would look like:

```
<header>
  <nav>...</nav>
</header>
<main>
  <router-outlet></router-outlet>
  <!-- the component's template will be inserted here-->
</main>
<footer>made with &lt;3 by Ninja Squad</footer>
```

When we navigate, everything will stay (the header, main and footer here) and the component matching the current route will be inserted just after the `RouterOutlet` directive.

18.2. Navigation

How can we navigate between the different components? Well, you can manually type the URL and reload the page, but that's not very convenient. And we don't want to use "classic" links, with `<a href="···"></a>`. Indeed, clicking on that link makes the browser load the page at that URL, and restart the whole Angular application. But the goal of Angular is to avoid such page reloads: we want to create a Single Page Application. Of course, there is a built-in solution.

In a template, you can insert a link with the directive `RouterLink` pointing to the path you want to go to. We can use this directive because our root module imports the `RouterModule`, making all the exported directives of `RouterModule` available to the root module. The `RouterLink` directive can receive a constant representing the path you want to go to or an array of strings, representing the

path and its params. For example in our `RacesComponent` template, if we want to navigate to the `HomeComponent`, we can imagine something like:

```
<a href="" routerLink="/">Home</a>
<!-- same as -->
<a href="" [routerLink]="['/']">Home</a>
```

At runtime, the link `href` will be computed by the router and will point to `/`.



The leading slash in the path is necessary. If not included, `RouterLink` builds the URL relatively to the current path (which can be useful with nested components, as we'll see later). Adding a slash indicates that the URL must be computed from the application base URL.

The `RouterLink` directive can be used with the `RouterLinkActive` directive which can set a CSS class automatically if the link points to the current route. This allows you, for example, to style a menu item as selected when it points to the current page.

```
<a href="" routerLink="/" routerLinkActive="selected-menu">Home</a>
```

We can even put a reference on this directive, to know if the route is active, and use it in the template:

```
<a href="" routerLink="/" routerLinkActive #route="routerLinkActive">Home {{
route.isActive ? '(here)' : '' }}</a>
```

It's also possible to navigate from the code, by using the `Router` service and its method `navigate()`. It's often handy when you want to redirect your user after an action:

```
export class RacesComponent {
  constructor(private router: Router) {}

  saveAndMoveBackToHome(): void {
    // ... save logic ...
    this.router.navigate(['']);
  }
}
```

The method takes an array of parameters, with the path you want to navigate to as the first element.

It is also possible to have parameters in the URL, and it's really useful to define dynamic URLs. For example, we want to display a detail page for a pony, with a meaningful URL for this page, like `ponies/id-of-the-pony-/name-of-the-pony`.

To do so, let's define a route in the configuration with one (or several) dynamic parameters.

```
export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'races', component: RacesComponent },
  { path: 'races/:raceId/ponies/:ponyId', component: PonyComponent }
];
```

We can then define dynamic links with `routerLink`:

```
<a href="" [routerLink]="['/races', race.id, 'ponies', pony.id]">See pony</a>
```

Of course, the target component needs to access those parameters to be able to load and display the pony with the given identifier. To get the value of the parameters, the router provides a service, that you can of course inject in the component, named `ActivatedRoute`. This object can be used inside `ngOnInit`, and has a very useful field: `snapshot`. This field has all the parameters of the URL in `paramMap`!

```
export class PonyComponent implements OnInit {
  pony: PonyModel;

  constructor(private ponyService: PonyService, private route: ActivatedRoute) {}

  ngOnInit(): void {
    const id = this.route.snapshot.paramMap.get('ponyId');
    this.ponyService.get(id).subscribe(pony => (this.pony = pony));
  }
}
```

This hook is also a good place to do the initialization work of the component as you can see.

As you may have spotted, we are using `snapshot`. Is there a non snapshot version? Yes there is. And it provides a way to subscribe to parameter changes, with, you guessed it, an observable. This observable is called `paramMap`.



This is very important: the router will reuse your component if it can! Let's say our app has a "Next" button to see the next pony. The URL will change from `/ponies/1` to `/ponies/2` for example when the user clicks. The router will then reuse our component instance: that means the `ngOnInit` will not be called again! If you want your component to update for this kind of navigation, you have no other way than using the `paramMap` observable!

```
export class PonyReusableComponent implements OnInit {
  pony: PonyModel;

  constructor(private ponyService: PonyService, private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.route.paramMap.subscribe((params: ParamMap) => {
      const id = params.get('ponyId');
      this.ponyService.get(id).subscribe(pony => (this.pony = pony));
    });
  }
}
```

Here we subscribe to the observable offered by `ActivatedRoute`. Now, every time the URL changes from `/ponies/1` to `/ponies/2` for example, the `paramMap` observable will emit an event, and we'll fetch the correct pony to display on screen.

Note that instead of subscribing to the result of the `PonyService` inside the subscribe of the params update, we can use the more elegant `switchMap` operator:

```
export class PonySwitchMapComponent implements OnInit {
  pony: PonyModel;

  constructor(private ponyService: PonyService, private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.route.paramMap
      .pipe(
        map((params: ParamMap) => params.get('ponyId')),
        switchMap(id => this.ponyService.get(id))
      )
      .subscribe(pony => (this.pony = pony));
  }
}
```



Try our exercise Router 🦄 to learn how to configure the router, navigate between components, and test all this.

What you have just learnt should cover your basic routing needs. But the router goes well beyond this and offers many additional features. Covering them all in detail is quite a big task, and you can feel overwhelmed when trying to learn them all.

This section will try to present most of the additional features as concisely as possible, by explaining what they're useful for. But it will not try covering every detail. If you really need an extensive coverage of the router features, there's a whole book available for that, written by the main contributor to the Angular router, Victor Savkin.

18.3. Redirects

A common use-case is to have a URL simply redirect to another URL in the application. This can happen because you want, for example, the root URL of your news app to redirect to the `/breaking` news category, or an old URL to redirect to a new one after a refactoring. This is possible using

```
{ path: '', pathMatch: 'full', redirectTo: '/breaking' },
```

18.4. Matching strategy

In the above example illustrating a redirect, I applied a strategy for matching the route: `'full'`. The default strategy is `'prefix'`, which matches a route with a URL when the URL starts with the path of the route. If we used this default strategy here, all URLs would redirect to `/breaking`, since all URLs start with an empty string.

The matching strategy consists in finding the first route that matches the complete URL. So, for example, if you define routes like

```
{ path: 'races/:id', component: RaceComponent },
{ path: 'races/new', component: RaceCreationComponent }
```

and the URL is `races/new`, the component that the router will activate is in fact the `RaceComponent`. Indeed, `races/:id` matches with `races/new` and comes first in the list of routes. To solve this problem, change the order of the routes:

```
{ path: 'races/new', component: RaceCreationComponent },
{ path: 'races/:id', component: RaceComponent }
```

18.5. Hierarchical and empty-path routes

Routes can have children. This can be useful for several reasons:

- applying guards to several routes at once (see later);
- applying resolvers to several routes at once (see later);
- sharing a a common template between several routes.

As we have seen before, when the router activates a route, the component of the route is inserted in the page at the location marked by the `router-outlet` directive.

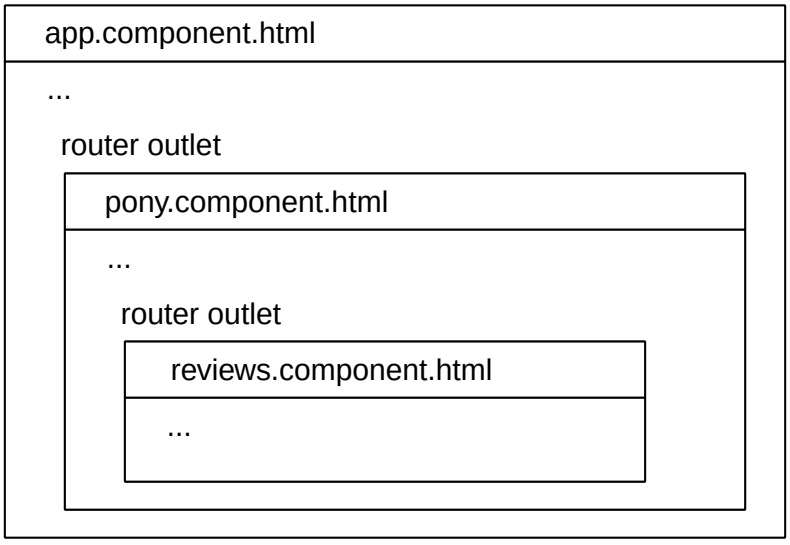
This mechanism can in fact be used in nested components, too. Suppose you have a complex page to display the profile of a pony. This page would display its name and portrait at the top, and would have several tabs at the bottom: one to display its birth certificate, one to display its track record, and one to display journalist reviews about this pony. You want to have a URL for each tab, in order to be able to directly link to them. But you don't want to reload the pony and repeat its name and

portrait on every on these three tab components.

The solution is to use a nested `router-outlet` in the template of the `PonyComponent`, and to define a parent pony route, this way:

```
{
  path: 'ponies/:ponyId',
  component: PonyComponent,
  children: [
    { path: 'birth-certificate', component: BirthCertificateComponent },
    { path: 'track-record', component: TrackRecordComponent },
    { path: 'reviews', component: ReviewsComponent }
  ]
}
```

When going to the URL `ponies/42/reviews`, for example, the router will insert the PonyComponent at the location indicated by the main router-outlet, in the root component. The template of PonyComponent, besides the name and the portrait of the pony, contains a second router-outlet. This is where the child `ReviewsComponent` will be inserted.



When going to the URL `ponies/42`, the pony component will be displayed, but none of the three children components will. You might want to display the birth certificate tab by default. That can be achieved using an empty-path route, redirecting to the `birth-certificate` route:

```
{
  path: 'ponies/:ponyId',
  component: PonyComponent,
  children: [
    { path: '', pathMatch: 'full', redirectTo: 'birth-certificate' },
    { path: 'birth-certificate', component: BirthCertificateComponent },
    { path: 'track-record', component: TrackRecordComponent },
    { path: 'reviews', component: ReviewsComponent }
  ]
}
```

Note that, in the above example, the redirect is relative to the `ponies/:ponyId` route, because it doesn't start with a `/`.

Instead of redirecting, you might want to display the birth certificate at the URL `ponies/42`. This can also be achieved using a child empty-path route:

```
{
  path: 'ponies/:ponyId',
  component: PonyComponent,
  children: [
    { path: '', component: BirthCertificateComponent },
    { path: 'track-record', component: TrackRecordComponent },
    { path: 'reviews', component: ReviewsComponent }
  ]
}
```

18.6. Guards

Some routes of the application should not be accessible to all users, depending on their permissions. Of course, you should hide or disable links pointing to these routes if the user may not access them. You should also make sure that the backend doesn't allow accessing or modifying resources that the user isn't authorized to access or modify. But that still won't prevent users from accessing routes that they're not allowed to access, who can simply enter their URL in the address bar.

That's where guards come into play. There are 4 kinds of guards:

- `CanActivate`: when set on a route, the guard can disable the activation of this route. Note that the guard can also return a URL to navigate elsewhere (to be more accurate, it can return an Angular type called `UrlTree` - keep reading for an example). This can be useful for showing an error page, or to navigate to the login page when an unauthenticated user tries accessing a route that requires authentication;
- `CanActivateChild`: when set on a route, the guard can disable the activation of children of that route. This can be useful to disable access to many child routes at once, based on their URL;
- `CanLoad`: this guard is used on a route with a `loadChildren` attribute. This attribute allows lazy

loading a whole feature bundle, containing child routes (we'll talk about lazy loading a bit later). It goes further than the `CanActivate` guard by preventing even loading the feature bundle;

- `CanDeactivate`: this guard is different from the three other ones. It's used to prevent navigation from outside of the currently activated route. This can be useful for asking for confirmation before leaving a route containing a large form, for example.

Here's how you would add a `CanActivate` guard on a route. The three other guards are added in a similar way:

```
{ path: 'races', component: RacesComponent, canActivate: [LoggedInGuard] }
```

In the above example, `LoggedInGuard` is an Angular service. As any other service, it must be provided, typically by adding it to the providers of the Angular module.

This service must implement the `CanActivate` interface. It consists of deciding whether the route can be activated or not (by checking if the user is logged in or not), and in returning a `boolean`, a `Promise<boolean|UrlTree>`, an `Observable<boolean|UrlTree>` or a `UrlTree`.

The router will navigate to the route if the returned value is `true`, or if the returned promise is resolved as `true`, or if the returned observable emits `true`. If the returned value is a `UrlTree`, it cancels the current navigation and triggers a navigation to the returned `UrlTree`.

Here's what the `LoggedInGuard` might look like:

```
@Injectable({
  providedIn: 'root'
})
export class LoggedInGuard implements CanActivate {
  constructor(private router: Router, private userService: UserService) {}

  canActivate(route: ActivatedRouteSnapshot, state: RouterStateSnapshot): boolean |
UrlTree {
    // returns `true` if the user is logged in or redirects to the login page
    // note that you can also use `router.createUrlTree()` to build a `UrlTree` with
parameters
    return this.userService.isLoggedIn() || this.router.parseUrl('/login');
  }
}
```



Instead of defining a service and passing its class in the route configuration, a function with the same signature as the service method can also be passed. I wouldn't advise it in general cases though, as it is less type-safe and doesn't allow injecting a service in the guard.

Hierarchical routes combined with empty-path routes can be very handy when you want to apply a guard on several routes at once. For example, if you want both the races and the ponies routes to be accessible only to logged in users, instead of

```
{ path: 'ponies/:ponyId', component: PonyComponent, canActivate: [LoggedInGuard] },
{ path: 'races', component: RacesComponent, canActivate: [LoggedInGuard] }
```

you can introduce an empty-path, componentless route as a parent. This route won't consume any URL segment, and won't activate any component, but its guards will be called when navigating to any of its children:

```
{
  path: '',
  canActivate: [LoggedInGuard],
  children: [
    { path: 'ponies/:ponyId', component: PonyComponent },
    { path: 'races', component: RacesComponent }
  ]
}
```

18.7. Resolvers

In a good old multi-page application where the pages are generated server-side, when clicking on a link, here's what happens: a request is sent to the server, the browser typically shows a spinning icon on the tab, and when the response finally comes back from the server, the URL in the address bar changes and the content of the new page is displayed.

In an Angular single-page application, it doesn't exactly work that way. The user clicks on a link to display a pony race (for example). The router creates an instance of the `RaceComponent`, and the component sends an AJAX request to load the race. The router immediately inserts the component template at the router-outlet location and changes the URL in the address bar. At this time, immediately after the click, the user sees the new page, but without any race. When the response to the AJAX request comes back, the race is stored in the component and the DOM is updated.

This has advantages and drawbacks:

- the navigation to the new page feels faster;
- the user can be confused if loading the race is too long, because the page appears blank, which looks like a bug;
- the template must be coded carefully, in order to work fine during the small period of time when the race is `null` or `undefined`;
- the template can however provide an immediate feedback by displaying a message or a spinning animation indicating that the race is being loaded;
- if loading the race fails (because the connectivity is lost, for example), then the navigation has been made and the URL has changed, although the page can't display any race, instead of staying on the previous page.

A resolver allows making the application behave almost like a traditional multi-page application. Instead of letting the race component load the race, you apply a resolver on the route, and the

resolver loads the race on behalf of the component.

Like a guard, a resolver can return data synchronously (by returning a race) or asynchronously (by returning a promise or observable of a race). The router only navigates to the route once the promise has been resolved, or once the observable has completed with at least an emitted race. Here's what a resolver for a race would look like:

```
@Injectable({
  providedIn: 'root'
})
export class RaceResolver implements Resolve<RaceModel> {
  constructor(private raceService: RaceService) {}

  resolve(
    route: ActivatedRouteSnapshot,
    state: RouterStateSnapshot
  ): Observable<RaceModel> | Promise<RaceModel> | RaceModel {
    return this.raceService.get(+route.paramMap.get('raceId'));
  }
}
```

As you can see, it's a simple service, which uses the activated route snapshot passed by the router to get the value of the `raceId` parameter, and returns an `Observable<RaceModel>`.



You might wonder why we use `+route.paramMap.get('raceId')` and not simply `route.paramMap.get('raceId')`. That's because the parameter is of type string (it's a segment of a URL), and the service expects the race ID to be a number. `+` is the simplest way to convert a string into a number.

Here's how the resolver would be applied to the route:

```
{
  path: 'races/:raceId',
  component: RaceComponent,
  resolve: {
    race: RaceResolver
  }
}
```

As you can see, the `RaceResolver` is associated with an object key that I chose to name `race`. This is the key that the router will use to store the loaded race into the `data` of the activated route snapshot. So the race component can simply obtain the race the following way:

```
export class RaceComponent implements OnInit {
  race: RaceModel;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.race = this.route.snapshot.data.race;
  }
}
```

Note that, if you navigate from a route to the same route, but with different parameters (for example, if you have a *Next race* link on the page), then the guards and the resolvers applied to the route are called again. The component, in that case, will still be reused, and should still subscribe to an observable to get the race (or just store the observable in the component and use the `async` pipe in the template):

```
export class RaceComponent implements OnInit {
  race: RaceModel;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    this.route.data.subscribe(data => (this.race = data.race));
  }
}
```

Resolvers have many advantages over loading data from the activated component:

- they make the navigation more traditional;
- they can be shared and reused by several routes;
- they make the code of the component and its template simpler: no need to load data, no need to care about the temporary undefined or null model, no need to apply somewhat complex RxJS operators to get the data from the parameters;
- if the navigation fails, the current page is preserved and the user can just click on the link again to retry it.

The only drawback I can find is that, when you know that loading the data is slow (because it requires substantial computations or external service calls by the server), then the application can feel a bit unresponsive: you click on a link, and nothing happens until the data has been loaded. This is where loading the data from the activated component and displaying a loading message or animation can be more user-friendly. Another workaround would be to rely on router events to display this loading message.

18.8. Router events

The router emits several events when navigating to a route. You can be notified of these events by injecting the `Router` service and subscribing to its `events` observable. The emitted events have several types that you can filter using `event instanceof NavigationStart` (for example). Here are the various types of router events:

- `NavigationStart`: emitted when a navigation is requested (when clicking on a link, for example). It can be used, for example, to start displaying a spinner;
- `NavigationEnd`: emitted when a navigation ends successfully. It can be used to stop displaying the spinner. Another use-case is to send a *hit* to an analytics service (like Google Analytics for example), which allows analyzing the browsing habits and popular pages in your application;
- `NavigationError`: emitted when a navigation fails due to an unexpected error (like a resolver returning an empty or error observable). It can be used to stop displaying a spinner, or to try sending an error log to the server;
- `NavigationCancel`: emitted when a navigation is cancelled, because a guard prevented the navigation for example. If a spinner has been shown when the navigation started, it should be hidden when this event is emitted.

There are other kinds of events for the route configuration loading (`RouteConfigLoadStart`, `RouteConfigLoadEnd`, `RoutesRecognized`) and, since version 4.3, for the resolvers (`ResolveStart`, `ResolveEnd`) and guards (`GuardsCheckStart`, `GuardsCheckEnd`). Version 5.0 added more fine-grained navigation events (`ChildActivationStart`, `ChildActivationEnd`). Version 6.1 added a `Scroll` event, along with the `scrollPositionRestoration` configuration option that allows you to restore the scroll position when navigating back to a component.

18.9. Parameters and data

We've seen before that routes can have parameters. For example, the route `races/:raceId` has one parameter named `raceId`, and the value of this parameter, when navigating to `/races/42` is the string `'42'`. But this route can actually have additional parameters named *matrix parameters*.



Matrix parameters are not an Angular-specific feature. Although rarely used and thus lesser-known than query parameters, they're a standard part of URIs, which are supported by many server-side frameworks, too.

If you navigate to the URL

```
/races/42;foo=bar;baz=wiz
```

then the `params` and `paramMap` properties of the activated route will contain two additional parameters 'foo' and 'baz' having the values 'bar' and 'wiz'.

Those matrix parameters are specific to the route. So, for example, if the URL is

```
/races/42;foo=bar;baz=wiz/ponies
```

then the component associated with the `ponies` segment won't have `foo` nor `bar` in the parameters of its activated route. Only the component associated with the `races/42` segment will.

To navigate to such a URL, you would use the following code:

```
router.navigate(['/races', 42, { foo: 'bar', baz: 'wiz' }, 'ponies']);
```

or an equivalent router link:

```
<a [routerLink]="['/races', 42, { foo: 'bar', baz: 'wiz' }, 'ponies']">Link</a>
```

Query parameters, on the other hand, are shared by all the route segments. They look like this in the URL:

```
/races/42/ponies?foo=bar&baz=wiz
```

These query parameters are accessible from any route, using the `queryParams` or `queryParamMap` property.

To navigate to such a URL, you would use the following code

```
router.navigate(['/races', 42, 'ponies'], { queryParams: { foo: 'bar', baz: 'wiz' }
});
```

or the equivalent router link:

```
<a [routerLink]="['/races', 42, 'ponies']" [queryParams]="{ foo: 'bar', baz: 'wiz' }"
>Link</a>
```

Finally, we've seen that resolvers allowed adding properties to the `data` property of the activated route, before the route is activated. It's also possible to add additional data to a route directly from its configuration. This can be useful when the same component can be used in two different contexts for example:

```
{
  path: 'races',
  component: RacesComponent,
  data: {
    allowDeletion: false
  }
}
```

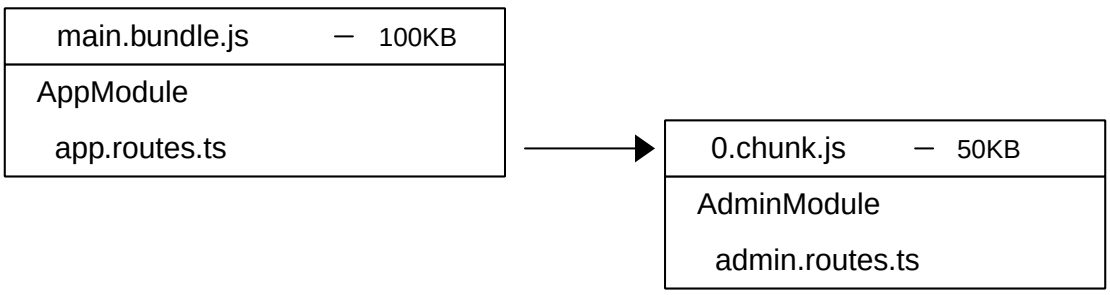
18.10. Lazy loading

This section will conclude this long chapter about the Angular router.

When the application grows in size and features, loading the whole application at once can become a problem: the application bundle is too large and takes too much time to load and parse. Moreover, some parts of the application are only used by some users of the application, or are used rarely, and loading them eagerly is a waste of time and bandwidth. This is where lazy-loading is useful.



Lazy loading consists in splitting the application into multiple Angular modules, and into several JavaScript bundles. A lazy-loaded module defines its own routes, components and services, bundled into a separate JavaScript file. The main module only defines a route allowing to access the lazy-loaded module. As the name implies, the Angular router waits for the user to navigate to this route before loading the JavaScript module, and adding the child routes, module, component and services to the application.





it's actually possible to load the lazy-loading modules in the background, after the root module has been loaded and the application has started, without waiting for the user to navigate to the module, thanks to an alternative `preloadingStrategy`.

To illustrate how we can configure lazy loading, we will assume that you want to define an *admin* section in your application, that should be lazy loaded. We will also assume that you're using Angular CLI to build your application.

The first step is to define a child Angular module. The `admin.module.ts` file would look like this:

```
@NgModule({
  imports: [CommonModule],
  declarations: []
})
export class AdminModule {}
```

There is an important difference from the root module: this child module doesn't import BrowserModule. Instead, it imports CommonModule.

The second step is to define an admin component, and at least one route for this component in a file named `admin.routes.ts`:

```
export const ADMIN_ROUTES: Routes = [{ path: '', component: AdminComponent }];
```

This routing configuration must be imported in the admin module, but once again, there is a subtle but important difference in the way it's done in the root module:

```
@NgModule({
  imports: [CommonModule, RouterModule.forChild(ADMIN_ROUTES)],
  declarations: [AdminComponent]
})
export class AdminModule {}
```

Instead of using `RouterModule.forRoot()` we must use `RouterModule.forChild()`, because this is a child module. Since the router is a stateful singleton service, we must not provide it a second time from this child module.



These differences between a child and a root module are not specific to lazy-loaded modules. We will explain the rationale for these differences and the subtleties of root and child modules in a future chapter.

The `AdminComponent` is declared in this child admin module, and not in the root module. Declaring it in the root module would ruin the goal we have: only loading this component when needed, lazily.

The final step is to add a route in the main `app.routes.ts` file, and tell the router to lazy-load the admin module when navigating to that route (or any child route it might have):

```
{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(m => m
.AdminModule) }
```

As you can see, this is achieved by using the `loadChildren` property of the route definition and the dynamic import function from TypeScript.



The value of this property has been for a long time a string containing the relative path of the module to load lazily, followed by the name of the class decorated with `NgModule` (for example `./admin/admin.module#AdminModule`). Angular 8.0 introduced the more standard, typed and less magical `() ⇒ import().then()` syntax, and it is now the recommended way.

When building this application, Angular CLI parses the route configurations and detects that the admin child module is lazy-loaded. Without any more work on your part, it generates an additional JavaScript bundle for the admin module (named `0.chunk.js`), and generates the necessary JavaScript to load this bundle when the router requires `'./admin/admin.module'`.



Try our exercises Protected routes with guards 🦄 , Resolvers, child routes and redirections 🦄 and Lazy loading 🦄 to learn how to use the advanced features of the router.

Chapter 19. Forms

19.1. Forms, dear forms

Forms have always been extra polished in Angular. That's one of the features that was the most demoed in 1.x, and, as pretty much every app has forms, it won the hearts of a lot of developers.



Forms are hard: you have to validate the inputs of your user, display errors, you can have fields required or not, or depending on another field, you want to react to some field changes, etc. We also need to test these forms, and that was impossible to achieve with a unit-test in AngularJS 1.x. It was only feasible with an end-to-end test, which can be slow.

In Angular, the same care has been applied to forms, and the framework gives us a nice way to write our forms. In fact, it gives us several ways!

You can either write your form using only directives in your template: that's the "template-driven" way. From our experience, it shines when you have a simple form, with not much validation.

The other way is the "code-driven" way, where you will write a description of the form in your component, then use directives to bind this form to the inputs/textareas/selects in your template. It's more verbose, but also more powerful, especially if you want to do add custom validation, or to generate dynamic forms.

Let's go through the same use case twice, using each way, and see the differences.

We are going to write a simple form, to be able to register new users in our awesome PonyRacer app. We need a base component for each use case, so let's begin with this:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-register',
  template: `
    <h2>Sign up</h2>
    <form></form>
  `
})
export class RegisterFormComponent {}
```

Nothing fancy: a component with a simple template containing a form. In the next few minutes, we will build a form allowing you to register a user with a username and a password.

For both methods, Angular will create a representation of our form.

In the "template-driven" way, it's pretty much automatic: we just need to add the proper directives in the template and the framework takes care of the form representation creation.

In the "code-driven" way, we create this form representation manually, and then bind the form representation to the inputs using directives.

Behind the scenes, a form field, like an `input` or a `select`, is represented by a `FormControl` in Angular. It is the smallest part of a form, and it encapsulates the state of the field and its value.

A `FormControl` has several attributes:

- `valid`: if the field is valid, regarding the requirements and validations applied on it.
- `invalid`: if the field is invalid, regarding the requirements and validations applied on it.
- `errors`: an object containing the field errors
- `dirty`: false until the user has modified its value.
- `pristine`: the opposite of dirty.
- `touched`: false until the user has entered it.
- `untouched`: the opposite of `touched`.
- `value`: the value of the field.
- `valueChanges`: an Observable emitting every time there is a change on the field

It also offers some methods like `hasError()` to check if the control has a specific error.

So you can do something like this:

```
const password = new FormControl();
console.log(password.dirty); // false until the user enters a value
console.log(password.value); // null until the user enters a value
console.log(password.hasError('required')); // false
```

Note that you can pass an argument to the constructor, and that this argument will be the value.

```
const password = new FormControl('Cédric');
console.log(password.value); // logs "Cédric"
```

These controls can be grouped in a `FormGroup` to represent a part of the form and have dedicated validation rules. The form itself is a group.

A `FormGroup` has the same properties as a `FormControl`, with a few differences:

- `valid`: if all fields are valid, then the group is valid.
- `invalid`: if one of the fields is invalid, then the group is invalid.
- `errors`: an object containing the group errors or `null` if the group is valid. Each error is a key, whose value is an array containing every control affected by this error.
- `dirty`: false until one of the controls gets dirty.
- `pristine`: the opposite of dirty.
- `touched`: false until one of the controls gets touched.
- `untouched`: the opposite of `touched`.
- `value`: the value of the group. To be more accurate, it's an object with key/values representing the controls and their values.
- `valueChanges`: an Observable emitting every time there is a change on the group

It offers the same methods as `FormControl` like `hasError()`. It also has a method `get()` to retrieve a control in the group.

You can create one like this:

```
const form = new FormGroup({
  username: new FormControl('Cédric'),
  password: new FormControl()
});
console.log(form.dirty); // logs false until the user enters a value
console.log(form.value); // logs Object {username: "Cédric", password: null}
console.log(form.get('username')); // logs the Control
```

Let's begin with a "template-driven" form!

19.2. Template-driven

With this method, we are going to use a bunch of directives in our form, and let the framework build the necessary `FormControl` and `FormGroup` instances. For example, the `NgForm` directive transforms the `form` element into its powerful Angular version - think of it as the difference between Bruce Wayne and Batman.

All the directives we need are included in the `FormsModule` module, so we need to import it into our root module.

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';

@NgModule({
  imports: [BrowserModule, FormsModule],
  declarations: [AppComponent],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

`FormsModule` contains the directives for the "template-driven" way. We'll see later that there exists another module, `ReactiveFormsModule`, in the same package `@angular/forms`, which is needed for the "code-driven" way.

Let's add the submit button:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-register',
  template: `
    <h2>Sign up</h2>
    <form (ngSubmit)="register()">
      <button type="submit">Register</button>
    </form>
  `
})
export class RegisterFormComponent {
  register(): void {
    // we will have to handle the submission
  }
}
```

I added a button, and defined an event handler for `ngSubmit` on the `form` tag. The `ngSubmit` event is emitted by the `NgForm` directive when `submit` is triggered. It calls the `register()` method of our controller, which will be implemented later.

Last thing: our template will quickly grow, so let's extract it to a dedicated file, using `templateUrl`:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  register(): void {
    // we will have to handle the submission
  }
}
```

In the "template-driven" way, you write your forms pretty much like in AngularJS 1.x, with a lot of things in your template and not many in your component.

In its simplest form, you just add `ngModel` directives to your form template and that's all. The `NgModel` directive creates the `FormControl` for you, and the form automatically creates the `FormGroup`. Note that you have to give a name to the input, which will be used by the framework to create the `FormGroup`.

```
<h2>Sign up</h2>
<form (ngSubmit)="register()">
  <div>
    <label>Username</label><input name="username" ngModel>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel>
  </div>
  <button type="submit">Register</button>
</form>
```

Now of course we need to do something for the submission, and to get hold of the user name and password. To achieve that, we can define a local variable and assign it to the `NgForm` object created by Angular for the form. Remember these from the Template chapter? Here, we are going to define a variable, `userForm`, referencing the form. We can do that because the `form` directive exports the `NgForm` directive instance, which has the same methods as the `FormGroup` class. We'll see the exporting part in more detail when we study how to build advanced directives.

```
<h2>Sign up</h2>
<!-- we use a local variable #userForm -->
<!-- and give its value to the register method -->
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel>
  </div>
  <button type="submit">Register</button>
</form>
```

Our `register` method is now called with the form value as the argument:

```
import { Component } from '@angular/core';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  register(user): void {
    console.log(user);
  }
}
```

This is only one-way data-binding though. If you update the field, the model will be updated, but updating the model will not update your field value. But `ngModel` is more powerful than you think!

19.2.1. Two-way data-binding

If you have been using AngularJS 1.x, or even just read an article about it, you must have seen the famous example with an `input` and an expression displaying the `input` value, updated every time the user modified the `input`, and the field automatically updated when the model changed. The famous "Two-Way Data-Binding", something like:

```
<!-- AngularJS 1.x code example -->
<input type="text" ng-model="username">
<p>{{ username }}</p>
```

We can do a similar thing with Angular.

You start by defining a model of what will be filled in the form. We'll do this in a `UserModel` interface:

```
export interface UserModel {
  username: string;
  password: string;
}
```

Our `RegisterFormComponent` should have a field `user` of type `User`:

```
@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  user: UserModel = {
    username: '',
    password: ''
  };

  register(): void {
    console.log(this.user);
  }
}
```

As you can see this time, the `register()` method is now directly logging the `user` object.

We are ready to add the inputs of our form. We need to bind our `input`s to the model we have defined. For this, we'll use the `ngModel` directive:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()">
  <div>
    <label>Username</label><input name="username" [(ngModel)]="user.username">
  </div>
  <div>
    <label>Password</label><input type="password" name="password"
[(ngModel)]="user.password">
  </div>
  <button type="submit">Register</button>
</form>
```

Wow! `[(ngModel)]`? What is this syntax? It's a syntactic sugar that has been introduced to express the same thing as:

```
<input name="username" [ngModel]="user.username" (ngModelChange)="user.username =
$event">
```

The `NgModel` directive updates the `input` value every time the related model `user.username` changes,

hence the `[ngModel]="user.username"` part. And it emits an event from an output named `ngModelChange` every time the `input` is updated by the user, where the event is the new value, hence the `(ngModelChange)="user.username = $event"` part, which will update the model `user.username` with this new value.

Instead of writing the long form, we can use the new syntax `[()]`. If, like me, you have trouble remembering if it is `[()]` or `([])`, there is a cool mnemonic tip: it's a banana-box! Yes, look: the `[]` is a box, and, inside, there are two bananas facing each other `()`!

Now, every time we type something in our `input`, the model is updated. And if the model is updated in our component, our field will automatically display the correct value:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()">
  <div>
    <label>Username</label><input name="username" [(ngModel)]="user.username">
    <small>{{ user.username }} is an awesome username!</small>
  </div>
  <div>
    <label>Password</label><input type="password" name="password"
[(ngModel)]="user.password">
  </div>
  <button type="submit">Register</button>
</form>
```

If you try the example above, you will see that the two-way data-binding works. And so does our form: we can submit it, and the component will log our `user` object!

19.3. Code-driven

In AngularJS 1.x you had to build your forms mostly in your templates. Angular introduces an imperative way, which allows you to construct the form programmatically rather than through a template.

Now we can handle forms directly in our code. It's more verbose but more powerful.

To build a form in our component code, we'll use the abstractions we talked about: `FormControl` and `FormGroup`.

With these basic elements we can build a form in our component. But instead of writing `new FormControl()` or `new FormGroup()`, we will use a helper class, `FormBuilder`, that we can inject:

```
import { Component } from '@angular/core';
import { FormBuilder } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  constructor(fb: FormBuilder) {
    // we will have to build the form
  }

  register(): void {
    // we will have to handle the submission
  }
}
```

The `FormBuilder` is a helper class, with a handful of methods to create controls and groups. Let's start simple, and create a small form with two controls, a username and a password.

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  userForm: FormGroup;

  constructor(fb: FormBuilder) {
    this.userForm = fb.group({
      username: '',
      password: ''
    });
  }

  register(): void {
    // we will have to handle the submission
  }
}
```

We created a `form` with two controls. You can see that each control is created with the value `''`. That's the same as using the helper method `control()` of the `FormBuilder` with this string as parameter, and the same as calling the `new FormControl('')` constructor: the string represents the initial value you want to display in your form. Here it is empty, so the inputs will be empty. But you can have a value here, of course, if you want to edit an existing entity for example. The helper method can also have other specific attributes, as we will see later.

We need to implement the `register` method. As we saw, the `FormGroup` object has a `value` attribute, so we can simply log its content with:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  userForm: FormGroup;

  constructor(fb: FormBuilder) {
    this.userForm = fb.group({
      username: fb.control(''),
      password: fb.control('')
    });
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

We now need to do some work in the template. We are going to use other directives from those we saw for the "template-driven" forms. These directives are in the `ReactiveFormsModule` that you have to import it into your root module. Their names begin with `form` instead of `ng` as was the case for the "template-driven" forms.

The form needs to be bound to our `userForm` object, thanks to the `formGroup` directive. Each `input` field is bound to a control, thanks to the `formControlName` directive:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
  </div>
  <button type="submit">Register</button>
</form>
```

We want to bind our component's attribute `userForm` object to `formGroup`, so we use the bracket notation `[formGroup]="userForm"`. Each `input` receives the `formControlName` directive with a string literal representing the control it is bound to. If you specify a name that does not exist, you will have an error. As we pass a value (and not an expression), we don't put the `[]` around

`formControlName`.

And we're done: clicking on the submit button will log an object containing the username and the chosen password!

If you need to, you can update the value of a `FormControl` from your component, using `setValue()`:

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, FormControl } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  usernameCtrl: FormControl;
  passwordCtrl: FormControl;
  userForm: FormGroup;

  constructor(fb: FormBuilder) {
    this.usernameCtrl = fb.control('');
    this.passwordCtrl = fb.control('');
    this.userForm = fb.group({
      username: this.usernameCtrl,
      password: this.passwordCtrl
    });
  }

  reset(): void {
    this.usernameCtrl.setValue('');
    this.passwordCtrl.setValue('');
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

19.4. Adding some validation

Validation is usually a big part of form-building. Some fields are required, some depend on one another, some should be in a specific format, some should not have a value greater or lower than X, for example.

Let's start by adding basic validation rules: all our fields are required.

19.4.1. In a code-driven form

To specify that every field is required, we will use a `Validator`. A validator returns a map of errors

or `null` if it detects no error.

A few validators are provided by the framework:

- `Validators.required` to ensure that a control is not empty
- `Validators.minLength(n)` to ensure that the value entered has at least *n* characters
- `Validators.maxLength(n)` to ensure that the value entered has at most *n* characters
- `Validators.email()` (available since version 4.0) to ensure that the value entered is a valid email address (good luck finding the correct regular expression by yourself for this one…)
- `Validators.min(n)` (available since version 4.2) to ensure that the value entered is at least *n*
- `Validators.max(n)` (available since version 4.2) to ensure that the value entered is at most *n*
- `Validators.pattern(p)` to ensure that the value matches the regular expression *p*

You can apply several validators at once, by using an array, on a `FormControl` or on a `FormGroup`. Here we want every field to be mandatory, so we can add the required validator to each control, and make sure that the username is 3 characters at least.

```
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  userForm: FormGroup;

  constructor(fb: FormBuilder) {
    this.userForm = fb.group({
      username: fb.control('', [Validators.required, Validators.minLength(3)]),
      password: fb.control('', Validators.required)
    });
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

19.4.2. In a template-driven form

Adding a required field in a template-driven form is also really straightforward: you just have to add the `required` attribute to the `input`s. `required` is a provided directive, and will automatically add the validator to this field. Same thing with `minlength`, `maxlength`, and `email` (`min` and `max` are not yet available as directives).

Starting from the two-way data-binding example:

```
<h2>Sign up</h2>
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel required minlength="3">
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel required>
  </div>
  <button type="submit">Register</button>
</form>
```

Note that this can be done in a "code-driven" form too.

19.5. Errors and submission

Of course, our user should not be able to submit the form while there are still errors left, and these errors should be perfectly displayed.

If you try the examples, you will see that even if the fields are required, we can still submit our form. Maybe we can do something about that?

We know that we can easily disable a button using the `disabled` property, but we need to give it an expression reflecting the state of the current form.

19.5.1. Errors and submission in a code-driven form

We added a field `userForm`, of type `FormGroup`, to our component. This field gives us a complete view of the form and field states and errors.

For example, we can disable the form submission if the form is not valid:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

As you can see on the last line, we just need to link `disabled` to the `invalid` property of `userForm`.

Now we can only submit when all controls are valid. To help our user understand why the form can't be submitted, we should display error messages.

Still using the `userForm`, we can do:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="userForm.get('username').hasError('required')">Username is
required</div>
    <div *ngIf="userForm.get('username').hasError('minlength')">Username should be 3
characters min</div>
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
    <div *ngIf="userForm.get('password').hasError('required')">Password is
required</div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Cool! The errors are now displayed if the fields are empty, and they disappear when there is a value. But they are displayed right away when the form is shown. Maybe we can hide them until the user changes the value?

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="userForm.get('username').dirty &&
userForm.get('username').hasError('required')">
      Username is required
    </div>
    <div *ngIf="userForm.get('username').dirty &&
userForm.get('username').hasError('minlength')">
      Username should be 3 characters min
    </div>
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
    <div *ngIf="userForm.get('password').dirty &&
userForm.get('password').hasError('required')">
      Password is required
    </div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

It's a bit verbose, so you can create a reference for each control in your component:

```
@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  userForm: FormGroup;
  usernameCtrl: FormControl;
  passwordCtrl: FormControl;

  constructor(fb: FormBuilder) {
    this.usernameCtrl = fb.control('', Validators.required);
    this.passwordCtrl = fb.control('', Validators.required);

    this.userForm = fb.group({
      username: this.usernameCtrl,
      password: this.passwordCtrl
    });
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

And then use the references in your template:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('minlength')">Username
should be 3 characters min</div>
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
    <div *ngIf="passwordCtrl.dirty && passwordCtrl.hasError('required')">Password is
required</div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

19.5.2. Errors and submission in a template-driven form

In a template-driven form, we don't have any field in our component referring to the `FormGroup`, but we already declared a local variable in the template, referring to the `NgForm` object exported by the form directive. Once again, this variable allows you to know the state of the form and accessing its

controls.

```
<h2>Sign up</h2>
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel required>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel required>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Now we need to display the errors of each field.

Like the form directive, each control exports its `FormControl` object, so we can create a local variable to access the errors:

```
<h2>Sign up</h2>
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel required #username="ngModel
">
    <div *ngIf="username.dirty && username.hasError('required')">Username is
required</div>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel required
#password="ngModel">
    <div *ngIf="password.dirty && password.hasError('required')">Password is
required</div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Yay!

19.6. Add some style

Whatever way you choose to create your forms, Angular does another awesome job for us: it automatically adds and removes CSS classes on each field (and on the form) to allow us to add some visual style.

For example, a field will have the class `ng-invalid` if one of its validators fails, or `ng-valid` if all the validators succeed. That means you can easily add some style, like a nice red border around the fields failing the validation:

```
<style>
  input.ng-invalid {
    border: 3px red solid;
  }
</style>
<h2>Sign up</h2>
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel required>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel required>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Another useful CSS class is `ng-dirty` which will be present if the user has changed the value. Its opposite is `ng-pristine`, present if the user never changed the value. I usually display the red border only when the user has changed the value at least once:

```
<style>
  input.ng-invalid.ng-dirty {
    border: 3px red solid;
  }
</style>
<h2>Sign up</h2>
<form (ngSubmit)="register(userForm.value)" #userForm="ngForm">
  <div>
    <label>Username</label><input name="username" ngModel required>
  </div>
  <div>
    <label>Password</label><input type="password" name="password" ngModel required>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Finally, there is a last CSS class: `ng-touched`. It will be present if the user enters and leaves the field at least once (even if she/he did not change the value). Its opposite is `ng-untouched`.

When you display a form for the first time, a field will usually have the CSS classes `ng-pristine ng-untouched ng-invalid`. Then, when the user enters and leaves the field, it switches to `ng-pristine ng-touched ng-invalid`. When the user changes the value, still for an invalid one, we'll have `ng-dirty ng-touched ng-invalid`. And finally, when the value is valid: `ng-dirty ng-touched ng-valid`.

19.7. Creating a custom validator

Pony races are an addictive game so you are only allowed to register if you are over 18. And we

want the user to enter the password twice, to be sure she/he hasn't made a mistake.

How do we do this? We create a custom validator.

To do so, we just have to create a method that takes a `FormControl`, tests its `value` and returns an object with the errors or `null`, if the validation passes.

```
const isOldEnough = (control: FormControl) => {
  // control is a date input, so we can build the Date from the value
  const birthDatePlus18 = new Date(control.value);
  birthDatePlus18.setFullYear(birthDatePlus18.getFullYear() + 18);
  return birthDatePlus18 < new Date() ? null : { tooYoung: true };
};
```

Our validation method is pretty easy: we take the value of the control, we build the date, check if the 18th birthday is before now and return an error with the key 'tooYoung' if not.

Now we need to include this validator.

19.7.1. Using a validator in a code-driven form

We need to add a new control in our form with this validator, using the `FormBuilder`:

```
import { Component } from '@angular/core';
import { FormBuilder, FormControl, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  usernameCtrl: FormControl;
  passwordCtrl: FormControl;
  birthdateCtrl: FormControl;
  userForm: FormGroup;

  static isOldEnough(control: FormControl): { tooYoung: true } | null {
    // control is a date input, so we can build the Date from the value
    const birthDatePlus18 = new Date(control.value);
    birthDatePlus18.setFullYear(birthDatePlus18.getFullYear() + 18);
    return birthDatePlus18 < new Date() ? null : { tooYoung: true };
  }

  constructor(fb: FormBuilder) {
    this.usernameCtrl = fb.control('', Validators.required);
    this.passwordCtrl = fb.control('', Validators.required);
    this.birthdateCtrl = fb.control('', [Validators.required, RegisterFormComponent
.isOldEnough]);
    this.userForm = fb.group({
      username: this.usernameCtrl,
      password: this.passwordCtrl,
      birthdate: this.birthdateCtrl
    });
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

As you can see, we have added a new control `birthdate`, with two validators composed. The first validator is `required` and the other is a static method of our class `isOldEnough`. Of course this method could be in another class if you wanted (`required` is a static method for example).

Don’t forget to add the field and display the errors in the form:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
    <div *ngIf="passwordCtrl.dirty && passwordCtrl.hasError('required')">Password is
required</div>
  </div>
  <div>
    <label>Birth date</label><input type="date" formControlName="birthdate">
    <div *ngIf="birthdateCtrl.dirty">
      <div *ngIf="birthdateCtrl.hasError('required')">Birth date is required</div>
      <div *ngIf="birthdateCtrl.hasError('tooYoung')">You're way too young to be
betting on pony races</div>
    </div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Pretty easy, no?

Note that it's also possible to create and add asynchronous validators (for example to check with the backend if a username is available).

```
@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  usernameCtrl: FormControl;
  userForm: FormGroup;

  constructor(fb: FormBuilder, private userService: UserService) {
    this.usernameCtrl = fb.control('', Validators.required, control => this
.isUsernameAvailable(control));
    this.userForm = fb.group({
      username: this.usernameCtrl
    });
  }

  isUsernameAvailable(control: AbstractControl): Observable<{ alreadyUsed: true } |
null> {
    const username = control.value;
    return this.userService
      .isUsernameAvailable(username)
      .pipe(map(available => (available ? null : { alreadyUsed: true })));
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

This asynchronous validator is not a static method this time because it needs to access the service.

The method from the service returns an `Observable` that emits either `null` if there is no error (the username is available), or an object to represent the error (the key will be the error, as with synchronous validators).

An interesting feature: the class `ng-pending` is dynamically added to the field while the asynchronous validator is still completing its job. It allows you to display a spinner for example to show that the validation is still ongoing.

19.7.2. Using a validator in a template-driven form

To add a custom validator in a template-driven form, we need to add it in… the template!

To do this, you need to build a custom directive that we will apply on the `input`, but honestly this is way easier when using a "code-driven" form…

19.8. Grouping fields

Until now, we just had one group: the complete form. But we can declare groups inside a group. That's very useful if you want to validate a group of fields together like an address, or, like in our example, if you want to check if the password and its confirmation match.

The solution is to use a code-driven form.

First, create a new group, `passwordForm` with the two fields and add it in the group `userForm`:

```
import { Component } from '@angular/core';
import { AbstractControl, FormBuilder, FormControl, FormGroup, Validators } from
'@angular/forms';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  passwordForm: FormGroup;
  userForm: FormGroup;
  usernameCtrl: FormControl;
  passwordCtrl: FormControl;
  confirmCtrl: FormControl;

  static passwordMatch(group: AbstractControl): { matchingError: true } | null {
    const password = group.get('password').value;
    const confirm = group.get('confirm').value;
    return password === confirm ? null : { matchingError: true };
  }

  constructor(fb: FormBuilder) {
    this.usernameCtrl = fb.control('', Validators.required);
    this.passwordCtrl = fb.control('', Validators.required);
    this.confirmCtrl = fb.control('', Validators.required);

    this.passwordForm = fb.group(
      { password: this.passwordCtrl, confirm: this.confirmCtrl },
      { validators: RegisterFormComponent.passwordMatch }
    );

    this.userForm = fb.group({ username: this.usernameCtrl, passwordForm: this
.passwordForm });
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

As you can see, we have added a validator on the group, `passwordMatch`, that will be called every time one of the fields changes.

Let's update the template to reflect the new form, using the `formGroupName` directive:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
  </div>
  <div formGroupName="passwordForm">
    <div>
      <label>Password</label><input type="password" formControlName="password">
      <div *ngIf="passwordCtrl.dirty && passwordCtrl.hasError('required')">Password is
required</div>
    </div>
    <div>
      <label>Confirm password</label><input type="password" formControlName="confirm">
      <div *ngIf="confirmCtrl.dirty && confirmCtrl.hasError('required')">Confirm your
password</div>
    </div>
    <div *ngIf="passwordForm.dirty && passwordForm.hasError('matchingError')">Your
password does not match</div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

Voilà!

19.9. Reacting to changes

A last cool feature when using a code-driven form: you can easily react to value changes, using the observable `valueChanges`. Reactive programming FTW! For example, let's say we want our password field to display a strength indicator. We want to compute the strength at every change of the password value:

```
import { Component } from '@angular/core';
import { FormBuilder, FormControl, FormGroup, Validators } from '@angular/forms';
import { debounceTime, distinctUntilChanged } from 'rxjs/operators';

@Component({
  selector: 'ns-register',
  templateUrl: './register-form.component.html'
})
export class RegisterFormComponent {
  userForm: FormGroup;
  usernameCtrl: FormControl;
  passwordCtrl: FormControl;
  passwordStrength = 0;

  constructor(fb: FormBuilder) {
    this.usernameCtrl = fb.control('', Validators.required);
    this.passwordCtrl = fb.control('', Validators.required);

    this.userForm = fb.group({
      username: this.usernameCtrl,
      password: this.passwordCtrl
    });

    // we subscribe to every password change
    this.passwordCtrl.valueChanges
      .pipe(
        // only recompute when the user stops typing for 400ms
        debounceTime(400),
        // only recompute if the new value is different than the last
        distinctUntilChanged()
      )
      .subscribe(newValue => (this.passwordStrength = newValue.length));
  }

  register(): void {
    console.log(this.userForm.value);
  }
}
```

Now we have a `passwordStrength` field in our component instance, that we can display to our user:

```
<h2>Sign up</h2>
<form (ngSubmit)="register()" [formGroup]="userForm">
  <div>
    <label>Username</label><input formControlName="username">
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
  </div>
  <div>
    <label>Password</label><input type="password" formControlName="password">
    <div>Strength: {{ passwordStrength }}</div>
    <div *ngIf="passwordCtrl.dirty && passwordCtrl.hasError('required')">Password is
required</div>
  </div>
  <button type="submit" [disabled]="userForm.invalid">Register</button>
</form>
```

We are leveraging RxJS operators to add a few cool features:

- `debounceTime(400)` will only emit values if the user stops typing for 400ms. That avoids computing the password strength on every value the user enters. That's really interesting if the computing takes a long time, or launches an HTTP request.
- `distinctUntilChanged()` will only emit values if the new value entered is different than the last one. Again that's really interesting: imagine that the user enters `'password'` then stops typing. We compute the strength. Then she enters a new character and removes it quickly (before 400ms). The next event out of `debounceTime` will again be `'password'`. It makes no sense to recompute the password strength again! This operator will not even emit the value, and saves us the recomputing.

RxJS can do tons of work for you: imagine coding yourself what we just did in two lines. It can also easily combine with HTTP work, as the `HttpClient` service uses observables too.

19.10. Updating on blur or on submit only

Angular 5.0 introduced the possibility of waiting for the `blur` or the `submit` event to update the field's value and validity. To do so, the `FormControl` constructor accepts an `options` object as the second parameter, to define the synchronous and asynchronous validators, and also the `updateOn` option. Its value can be:

- `change`, it's the default: the value and validity are updated on every change;
- `blur`, the value and validity are then updated only when the field loses the focus.
- `submit`, the value and validity are then updated only when the parent form is submitted.

```
this.usernameCtrl = new FormControl('', Validators.required);
this.passwordCtrl = new FormControl('', {
  validators: Validators.required,
  updateOn: 'blur'
});
```

It's also possible to configure this option on a group of fields all at once:

```
this.userForm = new FormGroup(
  {
    username: this.usernameCtrl,
    password: this.passwordCtrl
  },
  {
    updateOn: 'blur'
  }
);
```

The same feature is available in template-driven forms, with the `ngModelOptions` input of the `NgModel` directive:

```
<label>Username</label>
<input name="username" #usernameCtrl="ngModel"
  [(ngModel)]="user.username" [ngModelOptions]="{ updateOn: 'blur' }" required>
<div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
```

or globally on a form with the `NgFormOptions` input (which appeared with Angular 5.0) of the directive `NgForm`:

```
<form (ngSubmit)="register()" [ngFormOptions]="{ updateOn: 'blur' }">
  <div>
    <label>Username</label>
    <input name="username" #usernameCtrl="ngModel"
      [(ngModel)]="user.username" required>
    <div *ngIf="usernameCtrl.dirty && usernameCtrl.hasError('required')">Username is
required</div>
```

19.11. Super simple validation error messages with `ngx-valdemort`

As you may have noticed, the templates can quickly become very verbose with all the error messages that we have to repeat for each error type on each field, in every form. You are quickly stuck with very long `ngIf` and copied/pasted boilerplate between components.

As we find it distasteful as well, we wrote a tiny open-source library to make it easier (heavily inspired by `ngMessages` in AngularJS): ngx-valdemort.

Instead of

```
<input id="email" formControlName="email" class="form-control" type="email" />
<div class="invalid-feedback" *ngIf="form.get('email').invalid && (f.submitted ||
form.get('email').touched)">
  <div *ngIf="form.get('email').hasError('required')">The email is required</div>
  <div *ngIf="form.get('email').hasError('email')">The email must be a valid email
address</div>
</div>
```

the library allows you to write:

```
<input id="email" formControlName="email" class="form-control" type="email" />
<val-errors controlName="email">
  <ng-template valError="required">The email is required</ng-template>
  <ng-template valError="email">The email must be a valid email address</ng-template>
</val-errors>
```

We can even do better by defining default messages once and for all:

```
<val-default-errors>
  <ng-template valError="required" let-label> {{ label || 'This field' }} is required
</ng-template>
  <ng-template valError="email" let-label> {{ label || 'This field' }} must be a valid
email address </ng-template>
  <ng-template valError="min" let-error="error" let-label>
    {{ label || 'This field' }} must be at least {{ error.min | number }}
  </ng-template>
  <!-- same for the other types of error -->
</val-default-errors>
```

And then simply use:

```
<input id="email" formControlName="email" class="form-control" type="email" />
<val-errors controlName="email" label="The email"></val-errors>
```

We provide an integration with Bootstrap 4 and Material, to have error messages with a coherent style if you use one of these CSS frameworks. Give it a try, you won't regret it!

19.12. Summary

Angular offers two ways to build a form:

- one by setting up everything in the template. But, as you have seen, it forces us to have custom directives for the validation and is harder to test. This way of doing things is useful for simple forms, with just one or a few fields for example, and it gives us two-way data-binding.
- one by setting up almost everything in the component. This way allows an easier setup for validation and testing, with several levels of groups if you need them. It is your weapon of choice for building complex forms. You can even react to changes on a group, or on a field.

This is maybe the most pragmatic approach: go with template-based and bidirectional binding if you like it, and as soon as you need access to form groups or form controls, for example to add custom validation or reactive behavior, then declare the ones you need in the component, and bind the inputs and divs to them using the appropriate directives.



Try our exercises Register form 🦄, Custom validators in forms 🦄 and Login form 🦄. You'll learn how to build a simple form using the code-driven way and another one with the template-driven way. You'll also learn how to write custom validators, how to test forms, and how to authenticate your users!

Chapter 20. Zones and the Angular magic

Developing with AngularJS 1.X gave a "magic" feeling, and Angular still still gives that same effect: you type some value in an input and everything is magically updating all over the place.

I love magic, but I prefer to understand what's going on with the tools I use. If you are like me, I think this part will be interesting for you too: we are going to see how Angular works under the hood!

But first, let's start with how AngularJS 1.x works, which should be interesting, even you've never used it.

All JavaScript frameworks work roughly the same way: they help the developer to react to application events, to update the application state, and to refresh the DOM accordingly. But they don't all use the same way to achieve that goal.

EmberJS, for example, asks the developers to use *setters* to change the state of the objects, in order for the framework to be able to intercept the calls to these setters. That's what allows it to know which changes have been applied to the model, and thus to update the DOM accordingly.

React, on the other hand, chose to recompute the DOM after each change. But since modifying the whole DOM is a costly operation, it starts by applying the changes to a virtual DOM, and then only applies the changes between the virtual DOM and the actual DOM.

Angular doesn't use any setter, and doesn't use any virtual DOM either. So, how does it know what to change in the DOM?

20.1. AngularJS 1.x and the digest cycle

The first step is to detect changes in the model. A change is always triggered by an event, coming either from the user directly (for example, a button click or an input in a form), or from a "system" event (an HTTP response, an asynchronous method execution after a timeout, etc.).

So, how does AngularJS 1.x know that an event has happened? That part is actually pretty simple: it forces us to use its directives, for example `ng-click` to react to a click event, or `ng-model` to observe changes to an input. It also forces us to use its services, for example `$http` for HTTP requests or `$timeout` to execute tasks asynchronously.

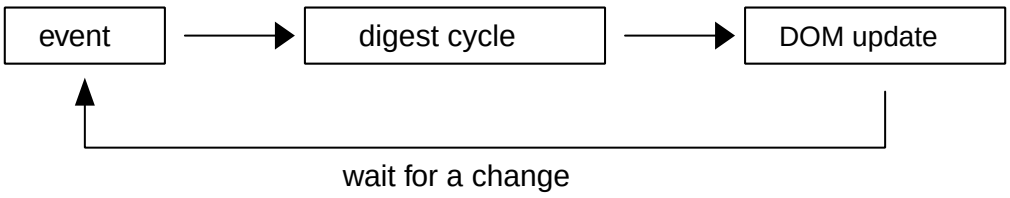
Using these directives and services allows the framework to be well informed about any event that has happened. That's the first part of the magic! And that's this first part which triggers the second one: the framework now has to analyze the changes made to the model, in order to decide which part of the DOM must be updated (and how).

To do that, in version 1.x, the framework maintains a list of *watchers*, all observing and reacting to changes in a specific part of the model. To simplify, a watcher is created for every dynamic expression used in the HTML templates. This can lead to several hundreds of watchers in a page.

These watchers are central to AngularJS 1.x: they are the memory of the framework, and are here to remember the state of the app.

Every time the framework detects an event (a user typing something in an input with an `ng-model`, an HTTP response, a timeout execution, etc.), it triggers what is called the *digest cycle*.

This *digest cycle* evaluates all the expressions stored in the watchers and compares their new value with their old value. If there is a change, then the framework knows that it has to update the DOM, so that the UI displays the new value instead of the old one. This technique is called *dirty checking*.



During this *digest cycle*, Angular is going over the whole list of watchers, and evaluates every watched expression. But there is a catch: it will execute this whole cycle again until the results are stable, i.e. until all the new values are equal to the old values.

Why does it do that? Because each time a change is detected in the value of a watched expression, a callback function is called. And this callback function can, in turn, modify the model, and thus change the value of one or several other watched expressions!

Let's take a minimalistic example: a page with two fields that the user must fill: name and password. Let's also say that the page displays a password strength indicator. A watcher is used to watch the value of the password, and recomputes its strength every time it changes.

After the first iteration of the digest cycle, once the user has entered the first letter of his/her password, we thus have this list of watchers:

```
$$watchers (expression -> value)
- "user.name" -> "Cédric"
- "user.password" -> "h"
- "passwordStrength" -> 0
```

The callback function of the watcher observing the `user.password` expression is then called, and it computes the new password strength: 3.

Angular doesn't know anything about the changes that might have been done on the model, so it starts a second digest iteration to know if the model is stable.

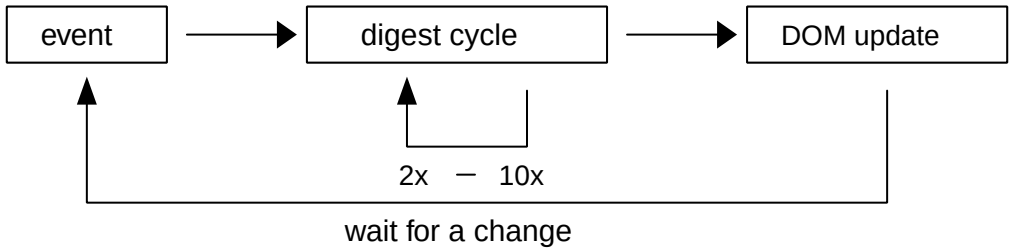
```
$$watchers
- "user.name" -> "Cédric"
- "user.password" -> "h"
- "passwordStrength" -> 3
```

The model isn't stable: the value of `passwordStrength` has changed since the first iteration. So it starts another digest iteration.

```
$$watchers
- "user.name" -> "Cédric"
- "user.password" -> "h"
- "passwordStrength" -> 3
```

This time, the model is stable. It's only at that time that AngularJS 1.x flushes the result to the DOM. So the digest cycle happens at least 2 times for every change in your application. The digest can iterate up to 10 times, but no more: after 10 iterations, if the results are not stable, the framework considers there is an infinite loop and throws an exception.

So my little drawing earlier is much more like:



And to insist: this is going on after *each event*. That means that if your user is entering 5 characters in the password field, the digest cycle will run 5 times, with 3 iterations every time, which leads to a total of 15 iterations.

In a real application, you can have several hundred watchers, and thus thousands of expression evaluations after every event!

Even if that sounds crazy, it works fine, because modern browsers are very fast, and there are ways to optimize a few things if necessary.

So let's recap the two important points about AngularJS 1.x:

- you must use the services and directives of the framework for everything that can make a change to the model;
- modifying the model after an event that is not handled by Angular is possible, but you have to trigger the detection change mechanism explicitly (by using the famous `$scope.$apply()` method, which starts the digest cycle). For example, if you want to send an HTTP request without using the `$http` service, you need to call `$scope.$apply()` in the response callback to tell the framework: "Hey, I have stored new values in the model - would you please start the digest cycle?".

The magic in the framework can be split in two parts:

- the triggering of the change detection after each event;
- the change detection itself, thanks to the watchers and the digest cycles.

Now let's see how Angular works, and how it differs from AngularJS.

20.2. Angular and zones

Angular is based on the same principles, but implements them in a different way. And we might even say, in a smarter way.

For the first part of the problem — triggering the change detection — the Angular team has built a small side-project named Zone.js. This project is not tied to Angular, because *zones* are a tool that can be useful in other projects. *Zones* are not a brand new concept either: they already exist in the Dart language (another Google project), for quite some time. They also have similarities with *Domains* in Node.js (now abandoned) or the *ThreadLocals* in Java.

But it's probably the first time that zones are being used in JavaScript. No worries, we'll discover them together.

20.2.1. Zones

A zone is an execution context. This context receives code to execute, and this code can be synchronous or asynchronous. A zone brings some benefits:

- hooks that can be executed before and after the code to execute
- a way to intercept potential errors of the code to execute
- a way to store variables bound to this context

Let's take an example. Suppose I have the following code in my application:

```
// score computation -> synchronous
const score = computeScore();
// player score update -> synchronous
updatePlayer(playerOne, score);
```

When we execute that code, we obtain:

```
computeScore: new score: 1000
udpatePlayer: player 1 has 1000 points
```

Now suppose I want to measure how much time is spent executing that code. I can do something like this:

```
startTimer();
const score = computeScore();
updatePlayer(playerOne, score);
stopTimer();
```

And that would produce:

```
start
computeScore: new score: 1000
udpatePlayer: player 1 has 1000 points
stop: 12ms
```

Easy. Now what happens if `updatePlayer` is an asynchronous function? JavaScript works in quite a special way: asynchronous operations are put at the end of the execution queue, and will thus be executed after the synchronous operations.

So, my previous code:

```
    startTimer();
    const score = computeScore();
    updatePlayer(playerOne, score); // asynchronous
    stopTimer();
```

will in fact produce :

```
start
computeScore: new score: 1000
stop: 5ms
udpatePlayer: player 1 has 1000 points
```

My execution time is not correct anymore: it only measures the time taken by the synchronous operations in my code, and not the time elapsed from the start until the end of the score update! That's where zones can be useful. We will execute the code in a dedicated execution context: a zone:

```
const scoreZone = Zone.current.fork({ name: 'scoreZone' });
scoreZone.run(() => {
  const score = computeScore();
  updatePlayer(playerOne, score); // asynchronous
});
```

Why does this help? Well, if the zone.js library is loaded by the browser, it starts by patching all the asynchronous functions in the JavaScript runtime. So, every time we use `setTimeout()` or `setInterval()`, or we use an asynchronous API like Promises, XMLHttpRequest, WebSocket, FileReader, GeoLocation, etc., we will in fact call the patched version of zone.js. Thus, Zone.js knows when asynchronous operations are done and allows us, developers, to execute some *hooks* at that time.

A zone offers several hooks:

- `onInvoke` is called before executing the code wrapped in the zone;
- `onHasTask` is called after the code wrapped in the zone has finished executing;

- `onHandleError` is called when the code wrapped in the zone throws an exception;
- `onFork` is called when the zone is created.

We can thus use a zone and its hooks to measure the whole time spent from the start to the end of my asynchronous operation:

```
const scoreZone = Zone.current.fork({
  name: 'scoreZone',
  onInvoke(
    delegate: ZoneDelegate,
    current: Zone,
    target: Zone,
    task: Function,
    applyThis: any,
    applyArgs: Array<any>,
    source: string
  ): any {
    // start the timer
    startTimer();
    return delegate.invoke(target, task, applyThis, applyArgs, source);
  },
  onHasTask(delegate: ZoneDelegate, current: Zone, target: Zone, hasTaskState:
HasTaskState): void {
    delegate.hasTask(target, hasTaskState);
    if (!hasTaskState.macroTask) {
      // if the zone run is done, stop the timer
      stopTimer();
    }
  }
});
scoreZone.run(() => {
  const score = computeScore();
  updatePlayer(playerOne, score);
});
```

And this time, the result is correct!

```
start
computeScore: new score: 1000
udpatePlayer: player 1 has 1000 points
stop: 12ms
```

Now you can guess how Angular can benefit from this mechanism. Indeed, the first problem of the framework is to know when change detection must be triggered. By using zones, and by running the code we write in a zone, the framework has a good view of what is happening. It can handle errors in a better way. But more importantly, it can trigger change detection every time an asynchronous operation is done!

To simplify, Angular does something like this :

```
const angularZone = Zone.current.fork({
  name: 'angular',
  onHasTask: triggerChangeDetection
});
angularZone.run(() => {
  // your application code
});
```

And the first problem is thus solved! That's why, in Angular, unlike AngularJS 1.x, it's not necessary to use special services to benefit from automatic change detection. You can use whatever you want, and zones will deal with it.

20.2.2. Change detection in Angular

The second part of the problem is change detection itself. It's one thing to know when and how it's started, but it's another to know how it works.

First of all, we have to remember that an Angular application is a tree of components. When change detection starts, the frameworks goes through all the components in the tree to know if their state has changed and if the new state affects their view. If that's the case, the DOM part of the component which is affected by the change is updated.

The tree traversal goes from the root to the leaves and, unlike AngularJS 1.x, is done only once. Because there is now a big difference: the change detection does not change the application model in Angular, whereas a watcher in AngularJS 1.x could change the model during that phase.

Another big difference is that a component may modify its state and the state of its children, but it must not modify the state of its ancestors. So cascading changes are now over!

Change detection is therefore only used for checking changes in the model and modify the DOM accordingly. It can't have side-effects on the model as it could in version 1.x, and thus doesn't need more than one pass through the tree, since the model can't be modified by the traversal!

To be accurate, the traversal, in development mode, is made twice precisely to verify that such undesirable side effects are non-existent (for example, a child component modifying the model of its parent). If the second pass detects such a change, an exception is thrown to warn the developer about the problem.

This strategy has many advantages:

- it's easier to reason with this in our applications, because state changes are made in one way, from the parent to the child, instead of being made in both ways;
- the change detection can't have infinite loops anymore;
- the change detection is significantly faster.

About this last point, it's relatively easy to visualize: AngularJS 1.x did (M watchers) * (N cycles)

verifications, whereas Angular only does M verifications.

But there is another parameter to take into account in the performance improvements of Angular: the time spent by the framework making these verifications. Once again, the Google team has employed all the knowledge it has about computer science and virtual machines.

To understand how the performance has been improved, we have to examine how expressions are evaluated and values compared in both versions of the framework.

In version 1.x, the mechanism is very generic. A single generic method is called for every watcher, capable of comparing the old and the new values. The problem is that virtual machines that execute the JavaScript code in our browsers (V8 if you're using Google Chrome, for example), don't really like generic code.

If you don't mind, I'll digress a little to talk about the behavior of virtual machines. You didn't expect that in a book about a JavaScript framework, did you? Virtual machines are quite extraordinary programs: you give them a piece of code, and they transform it to machine code and execute it. Very few among us (and certainly not me), are able to produce efficient machine code, but we don't really care: we use a high-level language, and let the virtual machine deal with it. Not only do they translate the code, but they optimize it. And they're quite good at that. Some even produce code that runs faster than manually optimized code, because they benefit from runtime information that is impossible to know in advance.

To improve performance, virtual machines like the ones executing dynamic JavaScript code, use a strategy named *inline caching*. It's a very old technique, invented for SmallTalk, 40 years ago (an eternity in IT), relying on a relatively simple principle: if a program calls a function frequently, with the objects having the same shape, the VM should recall how it evaluates the properties of the object. This technique thus uses a cache, hence the name *inline caching*. When receiving an object, it looks in the cache to see if it recognizes the shape of the object. And if it does, it uses the cached optimized way of accessing the properties of the object.

This kind of cache is really beneficial if the arguments of the function have the same shape. For example, `{name: 'Cédric'}` and `{name: 'Cyril'}` have the same shape. But `{name: 'JB', skills: []}` doesn't have the same shape as the two other ones.

When the arguments always have the same shape, the cache is *monomorphic*, and thus produces very fast results. If the cache only has a few entries, it is *polymorphic*. That means that the method is called with some different kinds of objects which makes the code a bit slower. Finally, if there are too many different object shapes, the VM drops the cache completely, because it is *megamorphic*. That, of course, is the worst case in terms of performance.

Now let's come back to our change detection in AngularJS 1.x. We can understand now that this unique generic function called for all the watchers is not optimizable using inline caching. We're in a *megamorphic* state, where the code is the slowest. And even if that isn't a problem in most situations, some pages with many watchers make us reach a limit where the performance is not sufficient anymore.

In order to benefit from the inline caching optimizations of the VM, Angular has adopted a different strategy. Instead of using a single method able to compare all kinds of objects, the Google team decided to dynamically generate a comparator for every type. At the startup of the application, the

framework goes through the tree of components, and generates a set of change detection functions, specific to each component.

For example, given a component `PonyComponent` with an attribute `name` displayed in the view, the framework will generate a change detection function for the whole component, pretty much like this one (this is a dumbed down version):

```
(view) => {
  var component = view.component;
  const currVal = component.name;
  if (currVal !== view.oldValue) {
    view.oldValue = currVal;
    updateView(view, 'p', currVal);
  };
});
```

This code is similar to code that you would have written by hand, and that the VM can optimize because it's monomorphic. The result is a significantly faster code, allowing for more complex pages.

To recap, Angular needs to evaluate fewer expressions and compare fewer values than AngularJS 1.x (a single pass is enough), and those evaluations and comparisons are faster!

From the beginning, the Google team has been monitoring the performance with benchmarks comparing AngularJS 1.x, Angular and even Polymer and React on various use cases, in order to check if the new version keeps getting faster.

If really needed, it's even possible to go further than those automatic optimizations provided by the framework: the *ChangeDetection* strategy can be changed from its default value, and thus be adapted and tuned to specific use-cases. But that's for another chapter.

Chapter 21. Angular compilation: Just in Time vs Ahead of Time

21.1. Code generation

In the previous chapter, we talked briefly about how the framework *generates* a change detection function for each component.

This is a very interesting and particular point in Angular, which you don't see in other frameworks: Angular, at the start of your application, will *compile* your templates and *generate* dynamic code for each component.

The HTML you write in your templates is never read by the browser directly. Instead, Angular generates a component definition for each component that represents exactly what you wrote in your template. This component definition is inlined in a static field of your component.

Let's take an example, with our well-known `PonyComponent`. The template is mainly an image with a bound source property, and a `figcaption` element with an interpolation.

```
<figure>
  <img [src]="getPonyImageUrl()">
  <figcaption>{{ ponyModel.name }}</figcaption>
</figure>
```

When Angular compiles this, it first starts by parsing the template to generate what is called an Abstract Syntax Tree (AST). An AST is a tree representing the structure of the template, commonly used by compilers to represent such things. It is the result of the *syntax analysis* step of the compilation. This AST will then be used to generate the dynamic JavaScript code, a "component definition" per component, inlined in a static field of our component class. A component definition contains several things, and among them the template, represented by a function.

```
elementStart(0, 'figure');
{
  element(1, 'img');
}
{
  elementStart(2, 'figcaption');
  text(3);
  elementEnd();
}
elementEnd();
```



This chapter describes the code generated by the compiler/renderer introduced in Angular 8.0 and called "Ivy". We wrote a detailed blog post about Ivy if you want to learn more. This renderer is the third iteration, as the initial renderer has already been rewritten in Angular 4.0. These iterations have brought either better performance or bundle size improvements or both, while keeping the same template syntax, allowing a backward compatibility with existing code, and allowing us developers to migrate very easily. Most people haven't noticed that the renderer changed in Angular 4.0 for example.

With this function, Angular is able to create the DOM corresponding to our `PonyComponent`: it basically appends the corresponding `HTMLElement` to the DOM, for every element.

But how does it handle the change detection? The template function is in fact a little bit longer:

```
template: (renderFlags: RenderFlags, component: PonyComponent) => {
  if (renderFlags & RenderFlags.Create) {
    elementStart(0, 'figure');
    {
      element(1, 'img');
    }
    {
      elementStart(2, 'figcaption');
      text(3);
      elementEnd();
    }
    elementEnd();
  }
  if (renderFlags & RenderFlags.Update) {
    advance(1);
    property('src', component.getPonyImageUrl());
    advance(2);
    textInterpolate(component.ponyModel.name);
  }
},
```

This template function has two parts:

- the creation of the component that we explained above
- the update of the component which does basically what you would write by hand (i.e. the image has a source property that reflects the result of the method `getPonyImageUrl()` of my component, and the `figcaption` has a text that reflects the pony's name).

It is called by the framework every time it needs to check if there is a change (see the previous chapter). It basically gets the values of the dynamic bits (the `src` attribute and the interpolation), and then calls a function of the framework with the index of the element to update, and another one with its property and the new value. The framework then compares the previous value stored for this element and if it changed, updates it, and replaces it with the new value.

This generated code is very fast and can be optimized by JS engines (see the part on monomorphism and inline caching in the previous chapter). On the other end, it takes some time to generate this code when the application starts. This is called the *Just in Time* (JiT) compilation.

To sum up, when you are using the JiT compilation: you write TypeScript code and HTML templates, you compile your TypeScript to JavaScript and send the JS and HTML to your users. At runtime, the HTML is then compiled to JS code too.

21.2. Ahead of Time compilation

But could we generate this code *before* starting the application? And indeed we can, using a compiler that the Angular team wrote: the *Ahead of Time* (AoT) compiler. You can call it manually in your project (`ngc`), or if you are using the CLI (as you should), it is a simple flag to add when you build or serve the app:

```
ng build --aot
```

The build will now use the Angular compiler to compile the templates to TypeScript files. To TypeScript? Yes, because it then makes sure that we didn't make a mistake in our templates! The generated TypeScript code and our application code will then be compiled by TypeScript (`ngc` will call the TypeScript compiler immediately), and if you made a mistake in a template, you'll see an error:

```
<figure>
  <!-- wrong method name -->
  <img [src]="getPonyImageUr()">
  <figcaption>{{ ponyModel.name }}</figcaption>
</figure>
```

The Angular compiler then generates this TypeScript code:

```
template: (component: PonyComponent, create: boolean) => {
  if (create) {
    // ...
  }
  advance(1);
  <!-- wrong method name in the generated code -->
  property('src', component.getPonyImageUr());
  advance(2);
  textInterpolate(component.ponyModel.name);
},
```

which raises an error in the TypeScript compilation:

```
Property 'getPonyImageUr' does not exist on type 'PonyComponent'.
```

This is great, because it means you can check all your templates before even running the application. Your future refactoring will be painless: if you rename a method or a property in a component, you'll know straight away that the template must be updated, too, because it breaks the build.

It also means that this compilation may throw an error whereas your code can be fine in the *Just in Time* mode.

For example, if you have a private `@Input()` in your component, it will be fine in JiT mode (because JavaScript has no notion of private property), but will break in AoT mode (as the generated TypeScript code of another component needs to access the property).

Compiling your application before shipping to your users will also greatly speed up the start of the application, as the compilation will already be done!

To sum up, in AoT mode: you still write TypeScript and HTML, you compile the HTML to TypeScript and then all TypeScript code to JavaScript, and send this JS code to your users.

The downside will be the size of the JavaScript bundle you will ship to your users: the generated code is larger than the uncompiled templates. This is somewhat compensated by the fact that you don't need to ship the Angular compiler to your users, as the templates are already compiled. And the compiler is a big piece of code, so this is a nice win. On a medium or large application, this generally doesn't compensate the increase in size from the generated code though. If you want to have all the perks of the AoT compilation AND a small bundle, you'll need to dig into the lazy-loading feature we explained in the Router chapter.

Chapter 22. Advanced observables

I must confess that I made a mistake: I under-estimated the value of RxJS and Observables. And that's a bit sad because I did the same with AngularJS 1.x and Promises. Promises were extremely useful in AngularJS 1.x - once you get them, you can handle any asynchronous part of your application elegantly. But it took some time to get there, and there were traps to avoid (check the blog post we wrote about Traps, anti-patterns and tips about AngularJS promises, if you want to learn more about that).

Angular relies on RxJS, and exposes Observables in a few APIs (Http, Forms, Router...). After coding for a while with RxJS, I think this ebook deserves a more "advanced" chapter about Observables, their creation, subscription, operators, possible uses with Angular, etc. I hope this will give you a few hints or spark the curiosity to dive deeper into it, because RxJS will play a big role in how you orchestrate your app, and it can do a wonderful job in simplifying your life.

22.1. Subscribe, unsubscribe and async pipe

To sum up what we learned in the chapter about Reactive Programming, an Observable represents a sequence of events to which we can subscribe.

This stream of events can happen at any time, and there can be only one event or ten thousands of them. But there is a distinction to understand between two kinds of Observables: cold ones and hot ones.

Cold observables will only emit events when they are subscribed to. You can think of it as watching a Youtube video: the video will only stream when you hit the "Play" button. For example, the observables returned by the `HttpClient` class are cold observables: they will only trigger the request when you `subscribe`.

Hot observables are slightly different: they emit events from the moment they are created. You can think of them as live television: you turn on the TV and you land in the middle of a show, that could have started minutes or hours ago. The observable representing the `valueChanges` in a `FormControl` is also a hot observable. You will not receive the values emitted before the moment you subscribed, only the value from the moment you subscribe.

When you subscribe to an observable, you can pass three possible parameters:

- a function to handle the next event
- a function to handle an error
- a function to handle the completion

The first one is pretty obvious. The observable is a stream of events and you define what to do if an event occurs.

The second one allows you to handle a potential error. It's not always necessary to pass this function. If the stream of events represents a stream of clicks, there are no possible errors that can occur, even if your user broke his finger (this joke is not mine, it's from a great presentation from André Staltz on RxJS at NgEurope 2016). But in most cases, it is very useful to define an error

handler. It allows you to define what to do if you get an error response from the HTTP backend, for example.

One thing to understand about this: an error is a terminal event - the observable will not emit new events after it. So if it is important for you to continue to listen, you probably want to handle this properly (we'll come back to this in a minute).

The third function you can pass allows you to handle the completion: that's because an observable can finish (your Youtube video is over for example). And sometimes you want to do something if the observable is over, like warning your user, or computing a value…

In our Ponyracer app, a race is represented by an observable, emitting the positions of the ponies. When the race is over, the observable stops emitting events. Then we want to compute which pony won the race, change the UI to reflect that, etc.

But maybe the user won't stay around until the end of the race. What happens then? The component will be destroyed. But, if we subscribed to an observable in that component before the destruction, then the `next` function will continue to do its job every time an event occurs. Even if the component is no longer displayed! That can lead to memory leaks and all sorts of trouble…

So the best practice is to store the subscription returned by the `subscribe` function, and on the destruction of the component, call `unsubscribe` on this subscription (typically in the `ngOnDestroy` method). Wrap the unsubscribe in a check to test if the subscription is present (maybe the subscription is not initialize yet, and then you'll have an error because the object will be undefined when you'll call `unsubscribe`).

This general rule has a few exceptions. You don't necessarily need to do this for observables that will only emit an event then complete, like an HTTP request. And you definitely don't need to do this if you subscribe to one of the router observables, like the `params` observable: the router will clean up for you, yay!

Last, but not least, the `async` pipe. Angular provides a special pipe, called `async`. You can use it in your templates to directly subscribe to an observable.

It has a few advantages:

- you can directly store the observable in your component and don't have to subscribe manually, then store the value emitted in a component's field;
- the `async` pipe will handle the unsubscription for you on the component's destruction;
- it can be used to do some performance magic (more on that later).

It also have a downside: you have to be careful when using this syntax multiple times in a template.

Let me illustrate the last point, in a `RaceComponent`, displaying the race properties:

```
@Component({
  selector: 'ns-race',
  template: `
    <div>
      <h2>{{ (race | async)?.name }}</h2>
      <small>{{ (race | async)?.date }}</small>
    </div>
  `
})
export class RaceComponent implements OnInit {
  race: Observable<RaceModel>;

  constructor(private raceService: RaceService) {}

  ngOnInit(): void {
    this.race = this.raceService.get();
  }
}
```

This code sample assumes that the method `raceService.get()` returns an Observable emitting a `RaceModel`. We store this observable in a field named `race` (you'll sometimes see the name `race$` in blog posts, because other frameworks use that naming convention for variables of type Observable). Then we use the `race` observable in the template with the `async` pipe twice: once to display the name and once to display the date. The component's code is quite elegant: you don't have to manually subscribe to the observable.

But maybe you can spot the problem. We call the `async` pipe twice, which means that we subscribe twice to the observable. If the `raceService.get()` method performs an HTTP request to fetch the race details, this request will be performed twice!

A first solution would be to modify the observable to share it between the different subscribers. This can be achieved with the `shareReplay` operator for example:

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { shareReplay } from 'rxjs/operators';
import { RaceService, RaceModel } from './race.service';

@Component({
  selector: 'ns-race',
  template: `
    <div>
      <h2>{{ (race | async)?.name }}</h2>
      <small>{{ (race | async)?.date }}</small>
    </div>
  `
})
export class RaceComponent implements OnInit {
  race: Observable<RaceModel>;

  constructor(private raceService: RaceService) {}

  ngOnInit(): void {
    this.race = this.raceService.get().pipe(
      // will share the subscription between the subscribers
      shareReplay()
    );
  }
}
```

But, even if that's now correct and does not produce two different HTTP requests, I still don't really like the template. Quite often, the component needs to access the race anyway for some presentation logic, and not only the template. And it has to handle errors, too. A subscription, or a `tap()` and/or `catchError()` operator in the component is often necessary anyway. So storing the race in a field is not a big burden, and makes the template code simpler.

Note that the 4.0 release introduced an `as` syntax that can solve the problem by subscribing only once and storing the result in a variable of the template:

```
import { Component, OnInit } from '@angular/core';
import { Observable } from 'rxjs';
import { RaceService, RaceModel } from './race.service';

@Component({
  selector: 'ns-race',
  template: `
    <div *ngIf="race | async as raceModel">
      <h2>{{ raceModel.name }}</h2>
      <small>{{ raceModel.date }}</small>
    </div>
  `
})
export class RaceComponent implements OnInit {
  race: Observable<RaceModel>;

  constructor(private raceService: RaceService) {}

  ngOnInit(): void {
    this.race = this.raceService.get();
  }
}
```

This is quite elegant.

Another possible solution is to split our component in two: a smart one (responsible for the race fetching) and a dumb one (that just displays the race received in an input).

That would look like:

```
@Component({
  selector: 'ns-racecontainer',
  template: `
    <div>
      <div *ngIf="error">An error occurred while fetching the race</div>
      <ns-race *ngIf="race | async as r" [raceModel]="r"></ns-race>
    </div>
  `
})
export class RaceContainerComponent implements OnInit {
  race: Observable<RaceModel>;
  error: boolean;

  constructor(private raceService: RaceService) {}

  ngOnInit(): void {
    this.race = this.raceService.get().pipe(
      // the `catchError` operator allows to handle a potential error
      // it must return an observable (here an empty one).
      catchError(error => {
        this.error = true;
        console.error(error);
        return EMPTY;
      })
    );
  }
}

@Component({
  selector: 'ns-race',
  template: `
    <div>
      <h2>{{ raceModel.name }}</h2>
      <small>{{ raceModel.date }}</small>
    </div>
  `
})
export class RaceComponent {
  @Input() raceModel: RaceModel;
}
```

This pattern can be useful in some parts of your application, but, like every pattern, you don't have to use it everywhere. It's sometimes not a big deal to have only one component that does both. Other times, you'll want to extract a dumb component to reuse it in another part of your application for example, so two components will make sense.

22.2. Leveraging operators

We saw a few operators up until now, but I'd like to take a moment to describe a few others in a step by step example. We are going to code a typeahead input. A typeahead allows your users to enter a text in an input, and then the application displays a few suggestions based on this text (like a Google search box).

A good typeahead has quite a few features:

- it displays the results matching the request (obviously)
- it allows you to only display results if the input text is at least a few characters long
- it won't fetch the results for every keystroke from our user, but will wait for some time to make sure the user is done typing
- it won't trigger the same request twice if the user enters the same value

All this can be done by hand, but it's far from being trivial. But we're in luck - Angular and RxJS combine nicely to solve this kind of problem!

First let's see what a component like this would look like:

```
import { Component, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';
import { PonyService, PonyModel } from './pony.service';

@Component({
  selector: 'ns-typeahead',
  template: `
    <div>
      <input [formControl]="input" />
      <ul>
        <li *ngFor="let pony of ponies">{{ pony.name }}</li>
      </ul>
    </div>
  `
})
export class PonyTypeAheadComponent implements OnInit {
  input = new FormControl();
  ponies: Array<PonyModel> = [];

  constructor(private ponyService: PonyService) {}

  ngOnInit(): void {
    // todo: do something with the input
  }
}
```

In the `ngOnInit` method, we can start by subscribing to the `valueChanges` observable exposed by the `FormControl` (check the chapter on forms if you need to refresh your memory).

```
this.input.valueChanges.subscribe(value => console.log(value));
```

Next we want to use this value to fetch the ponies matching the given input. Our `PonyService` has a method `search` that does exactly that! We can suppose that this method does a HTTP request behind the scenes to fetch the results from the server, so it returns an `Observable<Array<PonyModel>>`, an observable that emits arrays of ponies.

Let's subscribe to this method to update the `ponies` field of our component:

```
this.input.valueChanges.subscribe(value => {
  this.ponyService.search(value).subscribe(results => (this.ponies = results));
});
```

OK, that works. But when I see something like this, it reminds me of Promises and nested `then` calls, which can be flattened. And indeed you can do the same with Observables, with the `concatMap` operator for example:

```
this.input.valueChanges
  .pipe(concatMap(value => this.ponyService.search(value)))
  .subscribe(results => (this.ponies = results));
```

Wow, much more elegant! `concatMap` "flattens" our code. It replaces every event emitted by the source observable (i.e. the entered pony name) by the events emitted by the observable of ponies. But it's not the perfect operator for this situation. As our `search` method performs a HTTP request per search, we can run into some trouble if a request is too slow. Our user might query `n` then `ni`, but the result might come back really slowly for `n`, and fast for `ni`. That means our code above will display the second results only after the first displayed, even if we don't care about the first ones anymore! This could be tracked by hand, but that would be really cumbersome.

RxJS provides a super useful operator for this use-case: `switchMap`. Unlike `concatMap`, `switchMap` will only care about the last value emitted, and will discard the earlier values, so we're sure that the results corresponding to an old input won't be displayed.

```
this.input.valueChanges
  .pipe(switchMap(value => this.ponyService.search(value)))
  .subscribe(results => (this.ponies = results));
```

OK, now let's discard the queries that are less than three characters. Easy: we just have to use a `filter` operator!

```
this.input.valueChanges
  .pipe(
    filter(query => query.length >= 3),
    switchMap(value => this.ponyService.search(value))
  )
  .subscribe(results => (this.ponies = results));
```

We also don't want to search immediately after a keystroke: we'd like to search only after the user stops typing for 400ms for example. Yep, you guessed it: there's an operator for that, and it's called `debounceTime`:

```
this.input.valueChanges
  .pipe(
    filter(query => query.length >= 3),
    debounceTime(400),
    switchMap(value => this.ponyService.search(value))
  )
  .subscribe(results => (this.ponies = results));
```

So now a user can enter a value, delete some character, add others and the query will only fire when 400ms have passed since the last keystroke. But what if the user enters "Rainbow", waits for 400ms (which will thus send a request), then enters "Rainbow Dash" and immediately removed the "Dash" to get back to "Rainbow"? That would send two subsequent requests for "Rainbow"! Maybe we can only trigger a request if the query is different than the last one? Of course we can, with `distinctUntilChanged`:

```
this.input.valueChanges
  .pipe(
    filter(query => query.length >= 3),
    debounceTime(400),
    distinctUntilChanged(),
    switchMap(value => this.ponyService.search(value))
  )
  .subscribe(results => (this.ponies = results));
```

Last thing: we need to properly handle the errors. We know that the `valueChanges` will not emit any error, but our `ponyService.search()` observable might throw as it is dependent on the network. And the problem with observables is that an error will completely break the stream: so if one request blows, the whole typeahead will be down… We don't want that, so let's catch potential errors:

```
this.input.valueChanges
  .pipe(
    filter(query => query.length >= 3),
    debounceTime(400),
    distinctUntilChanged(),
    switchMap(value => this.ponyService.search(value).pipe(catchError(error =>
of([])))
  )
  .subscribe(results => (this.ponies = results));
```

Quite nice, don't you think? We now only trigger a search when the user enters a text with more than 3 characters and waits at least 400ms. We guarantee that we don't trigger the same request twice, and that the results are always in sync with the request! All that in 5 lines of code. Good luck doing the same by hand without adding any issue…

This is of course a really good use-case for RxJS, but the point is that it provides a lot of operators, with some gems hidden in it. It takes time to understand it, but it's worth the trouble as it can be tremendously useful in your application.

22.3. Building your own Observable

Sometimes, sadly, you need to use libraries that produce events but not using an Observable. All hope is not lost, because you can of course create your own Observables, using, `new Observable(observer ⇒ {})`.

The method passed as a parameter to the constructor is called the `subscribe` function: it will be responsible for emitting events and errors, and for completing when done.

For example, if you want to create an Observable which emits 1, then 2, then completes, you could do:

```
const numbers = new Observable(observer => {
  observer.next(1);
  observer.next(2);
  observer.complete();
});
```

Now we could subscribe to such an observable:

```
numbers.subscribe({
  next: number => console.log(number),
  error: error => console.log(error),
  complete: () => console.log('Complete!')
});
// Will log:
// 1
// 2
// Complete!
```

Now, let's say I want to emit 'hello' every 2 seconds, and never complete. We could easily do this with some built-in operators, but we can try by hand, as a small example:

```
import { Observable } from 'rxjs';

export class HelloService {
  get(): Observable<string> {
    return new Observable(observer => {
      const interval = setInterval(() => observer.next('hello'), 2000);
    });
  }
}
```

The function passed to `new Observable()` can also return a function that will be called on the unsubscription. That's really useful if you have some cleanup to do. Like us with our `HelloService`, because we'll need to stop the `setInterval` when the observable will be unsubscribed.

```
import { Observable } from 'rxjs';

export class HelloService {
  get(): Observable<string> {
    return new Observable(observer => {
      const interval = setInterval(() => observer.next('hello'), 2000);
      return () => clearInterval(interval);
    });
  }
}
```

The interval will not be created until the subscription, so we just created a cold observable.

I hope you enjoyed this small chapter on observables. They can also be used to sequence your HTTP requests, or to communicate between components (more on this soon). But you have now a good overview of what's possible!



We have several exercises that leverage RxJS and let you discover a lot of operators:

- Display the user 🦄
- Logged home 🦄
- Remember me 🦄
- Logout 🦄
- Observable tips and tricks 🦄
- Boost a pony 🦄

Chapter 23. Advanced components and directives

23.1. View queries

In the Template chapter, we talked about a nice feature called "local variables", allowing you to get a reference to a DOM element in the template. For example, you can give the focus to an input with a button easily:

```
<input #myInput />
<button (click)="myInput.focus()">Focus</button>
```

We also saw this same feature in the Forms chapter for example, when we wanted to grab a reference to a specific directive:

```
<input name="login" [(ngModel)]="user.login" required #loginCtrl="ngModel" />
<div *ngIf="loginCtrl.dirty && loginCtrl.hasError('required')">The login field is
required</div>
```

What if we need to have these references in our component code, and not only in the template? That is where "view queries" enter the scene and can save the day!

For example, you may want to focus an input as soon as your component is displayed. To do so, we need to grab a reference to the input, using the `ViewChild` decorator.

```
@Component({
  selector: 'ns-login',
  template: `<input #loginInput name="login" [(ngModel)]="credentials.login" required
/>`
})
export class LoginComponent implements AfterViewInit {
  credentials = { login: '' };

  @ViewChild('loginInput') loginInput: ElementRef<HTMLInputElement>;

  ngAfterViewInit(): void {
    this.loginInput.nativeElement.focus();
  }
}
```

We declare a field called `loginInput`, and we decorate it with `ViewChild`. This decorator needs a selector as parameter: here we use the local variable declared in our template. The decorator indicates to the framework that it needs to query the template to find an element with this local variable name. The field will be initialized with this element, of type `ElementRef<T>`. This type has

only one field, `nativeElement` of type `T`, which is a reference on the underlying DOM element.

The example also showcases a nice use of the lifecycle method `ngAfterViewInit`. This method is called as soon as the view is created, so you are sure that the element you are waiting for is indeed present. If you try to do the same in `ngOnInit`, it won't work. In fact it depended on the template of the component in earlier versions of Angular, so the `static` flag was introduced in Angular 8.0 to make the resolution timing explicit. If you pass `static: true` as an option to the decorator, the query is available in `ngOnInit`. But it is mostly intended to be used without this option and to interact with the query in `ngAfterViewInit`. The official documentation explains the very rare cases where you might need `static: true`. There is also another method called `ngAfterViewChecked`, which is called every time the view is checked (after each change detection).

If you wanted to have this feature in a lot of different components, you could create a directive for this, instead of duplicating the code in each component.

```
@Directive({
  selector: '[nsFocus]'
})
export class FocusDirective implements AfterViewInit {
  ngAfterViewInit(): void {
  }
}
```

This directive does nothing yet, but it would be used like this in a template:

```
<input nsFocus />
```

The directive needs to access its host element to give it the focus. That's where `ElementRef` is interesting, as it can be injected into our directive:

```
@Directive({
  selector: '[nsFocus]'
})
export class FocusDirective implements AfterViewInit {
  constructor(private element: ElementRef<HTMLElement>) {}

  ngAfterViewInit(): void {
    this.element.nativeElement.focus();
  }
}
```

And we're done: using this directive will give the focus to its host element!



Accessing directly the `nativeElement` like in these examples won't work if you decide to run your application in another platform other than the browser, like in Web Workers or on the server.

Let's go back to our `ViewChild` decorator: it can also accept a type as a selector.

For example, in the Forms chapter, we saw that to submit a form in the template-driven way, you can use two-way binding, or you can grab a reference to the form and pass its value to the submit method:

```
<form (ngSubmit)="authenticate(form.value)" #form="ngForm">
  <!-- ... -->
</form>
```

But we can also use a `ViewChild` for this:

```
@Component({
  selector: 'ns-login',
  template: `
    <form (ngSubmit)="authenticate()">
      <!-- ... -->
    </form>
  `
})
export class LoginFormComponent {
  @ViewChild(NgForm) credentialsForm: NgForm;

  authenticate(): void {
    console.log(this.credentialsForm.value);
  }
}
```

The cool thing with `ViewChild` is that it is a dynamic query: it will always be up to date with the template. If the element queried is destroyed, the field will be `undefined`.

This decorator also has a twin called `ViewChildren`. Unlike `ViewChild` which will get a reference to one field matching the selector (the first if there are several ones), `ViewChildren` will get a reference to all the fields. It returns a `QueryList`, a type with a few useful attributes:

- `first` returns the first element matching the query
- `last` returns the last element matching the query
- `length` returns how many elements are matching the query
- `changes` returns an observable that will emit the new `QueryList` every time an element matching the query is added, removed or moved.

Let's say we have a `RaceComponent` displaying a list of `PonyComponent` - we can easily be notified every time a `PonyComponent` is added or removed:

```
@Component({
  selector: 'ns-race',
  templateUrl: './race.component.html'
})
export class RaceComponent implements AfterViewInit {
  @Input() raceModel: RaceModel;

  @ViewChildren(PonyComponent) ponies: QueryList<PonyComponent>;

  ngAfterViewInit(): void {
    this.ponies.changes
      // this will log how many ponies are displayed in the component
      .subscribe(newList => console.log(newList.length));
  }
}
```

`QueryList` has also a lot of methods available like `toArray()`, `map()`, `filter()`, `find()`...

23.2. Content

Another common thing that we usually need as developers is the ability to build UI components whose content will be dynamic.

For example, let's say you want to build a "card" component using the Bootstrap 4 CSS framework. The template of such a card looks like this:

```
<div class="card">
  <div class="card-body">
    <h4 class="card-title">Card title</h4>
    <p class="card-text">Some quick example text</p>
  </div>
</div>
```

You can of course duplicate this HTML every time you need it in your application. But at this point, you are probably thinking about building a component. Two parts are dynamic in the card (the title and the content), so this is probably what you will come up with:

```
@Component({
  selector: 'ns-card',
  template: `
    <div class="card">
      <div class="card-body">
        <h4 class="card-title">{{ title }}</h4>
        <p class="card-text">{{ text }}</p>
      </div>
    </div>
  `
})
export class CardComponent {
  @Input() title: string;
  @Input() text: string;
}
```

And then use it like this:

```
<ns-card title="Card title" text="Some quick example text"></ns-card>
```

This works perfectly. But looking more closely to your need, you realize that the content of the card can also be complex HTML, and not just text, which is supported by Bootstrap!

Of course, Angular has your back, and it's easy to "pass" HTML to a child component, thanks to `<ng-content>`. That's what we used to call "transclusion" in AngularJS 1.x.

`ng-content` is a special tag you can use in your templates to include HTML provided by the parent component:

```
<div class="card">
  <div class="card-body">
    <h4 class="card-title">{{ title }}</h4>
    <p class="card-text">
      <ng-content></ng-content>
    </p>
  </div>
</div>
```

And you can now use the component like this:

```
<ns-card title="Card title"> Some quick <strong>example</strong> text </ns-card>
```

Later, you realize that the title can also be some complex HTML. Of course, there is a way to pass multiple contents to the card component, using multiple `ng-content` with a selector.

```
<div class="card">
  <div class="card-body">
    <h4 class="card-title">
      <ng-content select=".title"></ng-content>
    </h4>
    <p class="card-text">
      <ng-content select=".content"></ng-content>
    </p>
  </div>
</div>
```

and use it like this:

```
<ns-card>
  <p class="title">Card <strong>title</strong></p>
  <p class="content">Some quick <strong>example</strong> text</p>
</ns-card>
```

This will produce the following result:

```
<div class="card">
  <div class="card-body">
    <h4 class="card-title">
      <p class="title">Card <strong>title</strong></p>
    </h4>
    <p class="card-text">
      <p class="content">Some quick <strong>example</strong> text</p>
    </p>
  </div>
</div>
```

23.3. Content queries

When you are using these `ng-content` tags, the projected content will not be queried by `ViewChild` or `ViewChildren`. For these contents, you have to use two other decorators: `ContentChild` and `ContentChildren`.

Let's say you are building another UI component based on Bootstrap 4, this time a "tabs" component. The HTML must look like this according to the docs:

```
<ul class="nav nav-tabs">
  <li class="nav-item">
    <a class="nav-link">Races</a>
  </li>
  <li class="nav-item">
    <a class="nav-link">About</a>
  </li>
</ul>
```

But we would like to offer a nice component to our team, something like:

```
<ns-tabs>
  <ns-tab title="Races"></ns-tab>
  <ns-tab title="About"></ns-tab>
</ns-tabs>
```

We need an outer Tabs component, which must find out how many `ns-tab` directives are embedded inside the component template, iterate through each of them and generate the appropriate markup.

To do so, let's start by creating a directive `TabDirective`:

```
@Directive({
  selector: 'ns-tab'
})
export class TabDirective {
  @Input() title: string;
}
```

The directive doesn't do much: it only has an input to get the tab title. Note that we are using an element as the selector, `ns-tab`.

Now we need to build the `TabsComponent`:

```
@Component({
  selector: 'ns-tabs',
  template: `
    <ul class="nav nav-tabs">
      <li class="nav-item" *ngFor="let tab of tabs">
        <a class="nav-link">{{ tab.title }}</a>
      </li>
    </ul>
  `
})
export class TabsComponent {
  @ContentChildren(TabDirective) tabs: QueryList<TabDirective>;
}
```

As you can see, the template iterates through an array of tabs to generate an `li` element for each of them. But where does this `tabs` array come from? How can the component know about the two `ns-tab` directives embedded inside the ns-tabs component? That's what the `ContentChildren` directive allows you to do.

To grab the list of the tabs, we need to use `ContentChildren`, here with `TabDirective`. This gives us an iterable list of tabs, that you can use in an `NgFor`. As each element of this list is a `TabDirective`, we can then access the public property `title`, and display the tab's title!

Note that if, for whatever reason, you had a template like this one:

```
<ns-tabs>
  <div>
    <ns-tab title="Races"></ns-tab>
  </div>
  <ns-tabgroup>
    <ns-tab title="About"></ns-tab>
  </ns-tabgroup>
</ns-tabs>
```

Then the `QueryList` in `TabComponent` will only contain the first `TabDirective`. `ContentChild` and `ContentChildren` are indeed only looking for direct descendants, and will stop at the `ns-tabgroup` component.

If we want our component to still work with this, there is an option you can add to your decorator:

```
@Component({
  selector: 'ns-tabs',
  template: `
    <ul class="nav nav-tabs">
      <li class="nav-item" *ngFor="let tab of tabs">
        <a class="nav-link">{{ tab.title }}</a>
      </li>
    </ul>
  `
})
export class TabsWithDescendantsComponent {
  @ContentChildren(TabDirective, { descendants: true })
  tabs: QueryList<TabDirective>;
}
```

Now it finds all the `TabDirective` again!

The `tabs` field is a `QueryList`, so you can also subscribe to the changes, like we saw for `ViewChildren`. Once again, the `QueryList` is not accessible in the component constructor or even in `ngOnInit`. To be sure that the content can be queried (i.e. that the `QueryList` is properly populated), use the `ngAfterContentInit` lifecycle hook. You can also use the `ngAfterContentChecked` hook, which is called every time the content is checked.

23.4. Host listener

When writing a directive, it can be fairly common to interact with the host element.

Let's take a simple example: our customer wants to easily clear the content of some text inputs by double-clicking on them. This is the kind of behavior that you can encapsulate in a custom directive, let's say `InputClearDirective`. Its selector will be an attribute, let's say `nsInputClear`. When this attribute is added to an element, we want to listen for a double-click on this host element.



This example is simple, but not very realistic. More realistic (but also more complex) applications of this feature would be, for example, to display a tooltip or a popover when an element is being hovered or clicked

Let's create the directive:

```
@Directive({
  selector: '[nsInputClear]'
})
export class InputClearDirective {
  constructor(private element: ElementRef<HTMLInputElement>) {}

}
```

And use our directive like this:

```
<input nsInputClear />
```

We now need to react to a 'dblclick' event on our host element (here, the `input`) to clear the value.

That's where we can use the `HostListener` decorator! This decorator can be added to a method of a directive, to indicate to the framework that this method needs to be called if the event given as parameter to the decorator is triggered on the host element.

In our case, we can write:

```
@Directive({
  selector: '[nsInputClear]'
})
export class InputClearDirective {
  constructor(private element: ElementRef<HTMLInputElement>) {}

  @HostListener('dblclick')
  clearContent(): void {
    this.element.nativeElement.value = '';
  }

}
```

Now, every time a 'dblclick' event is triggered on the host element, the directive will clear the input value.

Note that it is also possible to listen to global events, like `window:resize` for example:

```
@Directive({
  selector: '[nsWindowResize]'
})
export class WindowResizeDirective {
  @HostListener('window:resize', ['$event'])
  resize(event: Event): void {
    const innerWidth = (event.target as Window).innerWidth;
    console.log(`The screen is being resized to ${innerWidth}`);
  }
}
```

Also note that you can access the event in the method, by specifying `['$event']` as a second parameter of the decorator.

23.5. Host binding

Another decorator available to create advanced directives and components is `HostBinding`. Whereas `HostListener` allows you to interact with the events on the host element, `HostBinding` allows you to interact with the properties of the host element.

Let's say we want to add a specific CSS class (`is-required`) to an input if this input has a specific validation error (`required`). Maybe this class adds a nice border around the input, or a small asterisk - that's not really important. This will not validate the field in any way - it will simply grab the result of the built-in Angular form validation, and use this result to style the input.

This is, again, a task that perfectly fits a directive:

```
@Directive({
  selector: '[nsAddClassIfRequired]'
})
export class AddClassIfRequiredDirective {
}
```

We will use it in a code-driven Angular form:

```
<input formControlName="firstName" nsAddClassIfRequired />
```

or in a template driven one:

```
<input [(ngModel)]="user.name" required nsAddClassIfRequired>
```

We then need to grab a reference to the status of the input in our directive. Angular does automatically validate the fields, and will add the `required` error if the field is required and not filled. That's where the powerful dependency injection system can help us! You can indeed ask Angular to inject into a directive another directive applied to the same host, or to one of its ancestors.

As we want our directive to work with `FormControlName` or `NgModel`, we could ask Angular to inject these two. But it would break as only one of the two will be available (as you generally either use one or the other on a given input), and Angular breaks if a dependency can't be provided. There is a trick that allows Angular to continue even if a dependency can't be provided: the `Optional` decorator.

So something like this could work:

```
constructor(
  @Optional() private formControl: FormControlName,
  @Optional() private ngModel: NgModel
) {}
```

But that's not the best we can do. Indeed, both directives inherit the same base class: `NgControl`. So instead of injecting one or the other, we can simply ask Angular to provide us with the common `NgControl`!

```
@Directive({
  selector: '[nsAddClassIfRequired]'
})
export class AddClassIfRequiredDirective {
  constructor(private control: NgControl) {}

}
```

Now that we have a reference to the `NgControl`, it's easy to know if the field has the `required` error, by using its `hasError()` method. The last step requires us to add the class `is-required` to our host element if that's the case. That's where the `HostBinding` decorator enters the scene! This decorator allows you to bind a property of the host element to a field of our directive. So it is generally used like this:

```
@HostBinding('value') innerValue;
```

And that would automatically update the host's value, every time the `innerValue` property of the directive changed.

In our case, we don't have a field to bind to. But we can define a getter that returns true or false, depending on the field's error, and decorate this getter with `HostBinding` to automatically add or remove the class `is-required`:

```
@Directive({
  selector: '[nsAddClassIfRequired]'
})
export class AddClassIfRequiredDirective {
  constructor(private control: NgControl) {}

  @HostBinding('class.is-required')
  get isRequired(): boolean {
    return this.control.hasError('required');
  }

}
```

These few lines of code are really powerful: every time the directive is used in a form, Angular will automatically add or remove our custom class depending on the new value entered by the user!

It can be used to bind other kinds of properties, not just CSS classes. For example, some component libraries use it to add accessibility attributes (`aria.xxx`) to a host element.

Note that the directive we built has a custom selector, but if you decide that you want to apply these directives to every input, you can change their selector to `input`, and they will automatically be applied on every input of your application.

Chapter 24. Internationalization

Alors comme ça, tu veux internationaliser ton application?

OK, don't worry if you didn't understand anything of this French introduction. Your role as a developer, fortunately, is not to translate your application into French, Spanish, or whatever other language. What you can do, though, is to allow this to happen. This chapter explains how to achieve that.



24.1. The locale

We already mentioned internationalization before, in the chapter about pipes. Four of the built-in Angular pipes deal with internationalization. Those are the `number`, `percent`, `currency` and `date` pipes. Until Angular 5, they used to rely on the standard JavaScript Internationalization API, which is supposed to be provided by the browser. But as that was not always the case, and there were numerous bugs and inconsistencies between browsers, the pipes have been completely overhauled in Angular 5.0.

What we don't know yet is how these three pipes decide how to format the numbers and dates. Should they use a dot or a comma as decimal separator? Should they use *January* or *Janvier* for the first month of the year? You might think that this is decided based on the preferred language configured in the browser, but actually, it's not. This depends on an injectable value named `LOCALE_ID`. And the default value of `LOCALE_ID` is `'en-US'`.

Here is an example showing how to get the value of `LOCALE_ID`. As you can see, it's a simple string value. To inject it into your components or services, you can't just rely on its type. You need to tell Angular which token identifies the value, using `@Inject(LOCALE_ID)`. This can be useful if your logic needs to know which locale the application is using.

```
@Component({
  selector: 'ns-locale',
  template: `
    <p>The locale is {{ locale }}</p>
    <!-- will display 'en-US' -->

    <p>{{ 1234.56 | number }}</p>
    <!-- will display '1,234.56' -->
  `
})
class DefaultLocaleComponent {
  constructor(@Inject(LOCALE_ID) public locale: string) {}
}
```

This is good. But how can we change the locale? Actually, you can't. The locale is a constant, that you can't change once the application has started. But that doesn't mean you can't set it to another value *before* the application starts. This is possible, simply by providing another value for the `LOCALE_ID` token in your root Angular module. Beware though: this changes the locale, but another step is required to bundle the locale-specific data (month translations, number formatting rules, etc.) with your application. Angular only bundles the `en-US` data by default.

```
import '@angular/common/locales/global/fr';
```

Here's an example showing its effect on our component:

```
@NgModule({
  imports: [BrowserModule],
  declarations: [
    CustomLocaleComponent
    // and other components
  ],
  providers: [
    { provide: LOCALE_ID, useValue: 'fr-FR' }
  ]
  // ...
})
export class AppModule {}

@Component({
  selector: 'ns-locale',
  template: `
    <p>The locale is {{ locale }}</p>
    <!-- will display 'fr-FR' -->

    <p>{{ 1234.56 | number }}</p>
    <!-- will display '1 234,56' -->
  `
})
export class CustomLocaleComponent {
  constructor(@Inject(LOCALE_ID) public locale: string) {}
}
```

All the pipes that handle internationalization can also take a locale as their last parameter. You can then change it dynamically if needed:

```
@Component({
  selector: 'ns-locale',
  template: `
    <p>The locale is {{ locale }}</p>
    <!-- will display 'en-US' -->

    <p>{{ 1234.56 | number: '1.0-3':'fr-FR' }}</p>
    <!-- will display '1 234,56' -->
  `
})
class DefaultLocaleOverriddenComponent {
  constructor(@Inject(LOCALE_ID) public locale: string) {}
}
```

If you want to create an application that uses only one locale, but different from `'en-US'`, then setting the locale as explained above is all you need to do. But often, this is not enough, and you want to really internationalize your application.

24.2. Default currency

The `currency` pipe allows you to specify which currency you want to use by providing an ISO string like `USD`, 'EUR'... If you don't provide one, it uses `USD` by default. So it is a bit cumbersome to have to specify the currency every time, if your application only uses `EUR` for example.

Angular 9 introduced the possibility of configuring the default currency globally using the token `DEFAULT_CURRENCY_CODE`.

```
{ provide: DEFAULT_CURRENCY_CODE, useValue: 'EUR' },
```

And you can then use `currency` without specifying `EUR` in a component. You can also get the currency via dependency injection of course:

```
@Component({
  selector: 'ns-currency',
  template: `
    <p>The currency is {{ currency }}</p>
    <!-- will display 'EUR' -->

    <p>{{ 1234.56 | currency }}</p>
    <!-- will display '1 234,56 €' -->
  `
})
class DefaultCurrencyOverriddenComponent {
  constructor(@Inject(DEFAULT_CURRENCY_CODE) public currency: string) {}
}
```

24.3. Translating text

If you have used AngularJS 1.x before, and have internationalized your AngularJS application, you probably know that there is nothing built-in to display translated text based on the preferred language of the user.

One of the popular libraries to achieve that with AngularJS is angular-translate. The strategy it uses is fairly common: you use a directive or a pipe to translate a key (for example `'home.welcome'`). This key identifies a message, and you provide the translations for all the languages you want to support (for example: 'Welcome' and 'Bienvenue'). At runtime, the directive or pipe uses the preferred language to get the appropriate translation, and updates the DOM with the translated message. You can change the preferred language at runtime, and all the messages on the page are immediately translated to the new language.

Internationalization is now provided by Angular directly, although it was hardly usable before version 4.0, and was completely rewritten in version 9.0. No need for an external dependency anymore. And it uses basically the same strategy based on keys, but with a big difference: it happens at compile-time.

With the compilation time strategy, you prepare one version of your application per locale. When you build your app, Angular parses all the HTML templates of your components, and transforms them to JavaScript code that, basically, analyzes the changes in the model and modifies the DOM accordingly. The translation happens at the end of this compilation phase. That has important consequences:

- you can't change the locale and the translated messages at runtime. The whole application needs to be reloaded and restarted to do that;
- once started, the application is faster, since it doesn't have to dynamically translate the keys again and again;
- if you use the AOT compiler (and you should, at least in production), you must build and serve as many applications as locales that you want to support.

Starting with version 9.0, the Angular team introduced a new `@angular/localize` package. If you want to use i18n in your application, or even if you don't but one of your dependencies does, you need to import `@angular/localize/init` into your application. If you use the CLI, you can just run `ng add @angular/localize` and the CLI adds it for you in the proper place.

This new package introduces a `$localize` global function, that is used under the hood for the localization, and that we'll be able to use in the future to translate messages in your code.

24.4. Process and tooling

In the remaining parts of this chapter, we will assume that you use Angular CLI to build your application. The tools are actually available and usable outside of Angular CLI. But since they're well integrated and simple to use in Angular CLI, and since it's the recommended way to build your applications anyway, we will use that.

That said, how do we proceed? You already know how to create components and write their templates. Will you have to rewrite everything to internationalize them? Thankfully, no. The process is the following one:

1. you mark the parts of the templates that need to be translated using the `i18n` attribute;
2. You run a tool to extract all those marked parts into a file, for example `messages.xlf`. Two file formats, both xml-based and industry-standard, are supported;
3. You ask a competent translator to create a translated version of this file, for example `messages.fr.xlf`
4. You build your application by providing the locale ID (`'fr'` for example) and the file containing the translations (`messages.fr.xlf`). The angular compiler and the CLI replace all the i18n-marked parts of the templates with the translations found in the file, and configures your application to use the provided locale ID.

Let's examine each of those steps in more details.

24.4.1. Marking text with i18n and extracting

Let's start with an example template:

```
<h1>Welcome to Ponyracer</h1>
<p>Welcome to Ponyracer {{ user.firstName }} {{ user.lastName }}!</p>
<img src="/img/pony.gif" alt="running pony" title="Ponies are cool, aren't they?" />
Let's start playing.
```

There are 5 text snippets that need to be translated in this template. Of course, you could imagine translating everything at once, but in a more realistic example, that would expose a lot of HTML boilerplate to the translators, and you don't want them to translate everything again when the HTML structure changes. So you should translate the 5 snippets separately.

One of them, the body of the `h1` element, is purely static text. One of them is a text containing two interpolated expressions. Two are attributes of an HTML element. The last one is static text that is not embedded in any element.

Here's how you would mark them. Let's start with the first, simplest one:

```
<h1 i18n>Welcome to Ponyracer</h1>
```

Now let's extract our very first messages file, using the `extract-i18n` command provided by Angular CLI:

```
ng extract-i18n --output-path src/locale/
```

This will create the file `messages.xlf` in the `src/locale` directory. Here's what it contains:

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en-US" datatype="plaintext" original="ng2.template">
    <body>
      <trans-unit id="7627914200888412251" datatype="html">
        <source>Welcome to Ponyracer</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">2</context>
        </context-group>
      </trans-unit>
    </body>
  </file>
</xliff>
```

As you can see, it generates a `trans-unit` containing, as the source, our static text. The role of the French translator will be to provide a `messages.fr.xlf` file looking like the following:

```
<?xml version="1.0" encoding="UTF-8" ?>
<xliff version="1.2" xmlns="urn:oasis:names:tc:xliff:document:1.2">
  <file source-language="en" datatype="plaintext" original="ng2.template" target-
language="fr">
    <body>
      <trans-unit id="7627914200888412251" datatype="html">
        <source>Welcome to Ponyracer</source>
        <target>Bienvenue dans Ponyracer</target>
      </trans-unit>
    </body>
  </file>
</xliff>
```

This is easy enough, because the source message is easy to understand. You don't need too much context to know what it is about, and how to translate it. But this way of doing things has a big disadvantage. If you change the source code of the template and introduce meaningless white spaces for example, or a dot at the end of the title, here's what happens when extracting the file again:

```
<h1 i18n>
  Welcome to Ponyracer.
</h1>
```

```
      <trans-unit id="6888363845978110110" datatype="html">
        <source> Welcome to Ponyracer.
</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">6,7</context>
        </context-group>
      </trans-unit>
```

Not only does the source change, which is expected, but the id also does. That really makes maintaining the translated messages files more difficult than it should. Fortunately, there's a better way. You can provide a unique ID by yourself:

```
<h1 i18n="@@home.title">Welcome to Ponyracer</h1>
```

Which generates:

```
<trans-unit id="home.title" datatype="html">
  <source>Welcome to Ponyracer</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.html</context>
    <context context-type="linenumber">10</context>
  </context-group>
</trans-unit>
```

And in fact, in order to provide more context to your translators, you can provide a meaning and a description in addition to the message ID:

```
<h1 i18n="welcome title|the title of the home page@@home.fullTitle">Welcome to
Ponyracer</h1>
```

```
<trans-unit id="home.fullTitle" datatype="html">
  <source>Welcome to Ponyracer</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.html</context>
    <context context-type="linenumber">13</context>
  </context-group>
  <note priority="1" from="description">the title of the home page</note>
  <note priority="1" from="meaning">welcome title</note>
</trans-unit>
```

Let's move to the second snippet now:

```
<p i18n="@@home.welcome">Welcome to Ponyracer {{ user.firstName }} {{ user.lastName
}}!</p>
```

Here is what it generates:

```
<trans-unit id="home.welcome" datatype="html">
  <source>Welcome to Ponyracer <x id="INTERPOLATION" equiv-text="{{ user.firstName
}}"/> <x id="INTERPOLATION_1" equiv-text="{{ user.lastName }}"/>!</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.html</context>
    <context context-type="linenumber">16</context>
  </context-group>
</trans-unit>
```

As you can see, this format has several interesting features:

- there is no way a translator could mess up the content of the angular expressions, since they are clearly indicated in the message;

- it's clear that these two interpolations are for the developers but it might also be a good idea to explain that in the description of the message for the translators;
- if, in some language, the last name should come before the first name, the translator is free to reorder the interpolations;
- if the developer chooses to rename the attributes of the component or of the user, nothing will have to be re-translated.

Let's proceed with the two attributes in the `img` element. The syntax to translate attributes is the following:

```
<img
  src="/img/pony.gif"
  alt="running pony"
  i18n-alt="@@home.ponyImage.alt"
  title="Ponies are cool, aren't they?"
  i18n-title="@@home.ponyImage.title"
/>
```

That generates the following translation units:

```
<trans-unit id="home.ponyImage.alt" datatype="html">
  <source>running pony</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.html</context>
    <context context-type="linenumber">21</context>
  </context-group>
</trans-unit>
<trans-unit id="home.ponyImage.title" datatype="html">
  <source>Ponies are cool, aren&amp;apos;t they?</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.html</context>
    <context context-type="linenumber">23</context>
  </context-group>
</trans-unit>
```

Finally, how to translate the last snippet? There is no element where we could place an `i18n` attribute. There is a way to solve the problem: use an `ng-container` element, which won't be rendered in the DOM at runtime:

```
<ng-container i18n="@@home.startMessage">Let's start playing.</ng-container>
```

24.4.2. Translating, building and deploying the application

Now that you generated a complete `messages.xlf` file, someone needs to translate it.



A common mistake is to just replace the original source text (in English in our case) with its translation. That won't work. The translation must be written inside the `<target>` element of each translation unit. The `<source>` element should be kept untouched: it provides the original message that must be translated. For example:

```
<trans-unit id="home.welcome" datatype="html">
  <source>Welcome to Ponyracer <x id="INTERPOLATION" equiv-text="{{ user.firstName
}}"/> <x id="INTERPOLATION_1" equiv-text="{{ user.lastName }}"/>!</source>
  <target>Bienvenue dans Ponyracer <x id="INTERPOLATION" equiv-text="{{ user.firstName
}}"/> <x id="INTERPOLATION_1" equiv-text="{{ user.lastName }}"/> !</target>
</trans-unit>
```

To run or build the application in French, you need to specify the locale, and the location of the messages file, to `ng serve` or `ng build` in `angular.json` by using the `i18n` property:

In versions before 9, to run or build the application in French, you had to pass various options to `ng serve` and `ng build`.

Since version 9, you must specify the locales you want to support, and their associated messages file, in the `angular.json` configuration file by using the `i18n` property:

```
"prefix": "ns",
"i18n": {
  "locales": {
    "fr": "src/locale/messages.fr.xlf"
  }
},
```

You can add as many locales as you want, each one linked to its translations.

You can then run:

```
ng build --prod --localize
```

and the CLI builds the app, and generates as many versions as there are locales defined. In our case, it generates one version for the default locale `en-US`, in the directory `dist/i18n/en-US` (if your project is named `i18n`), and another one for the `fr` locale in the directory `dist/i18n/fr`.

If you want to run `ng serve`, you have to specify one locale, as you can only serve one version.

The easiest way to do so is to define a build `configuration` for this locale in `angular.json`, just like the `production` configuration, but much simpler:

```
"fr": {
  "localize": ["fr"]
}
```

and add a serve configuration:

```
"fr": {
  "browserTarget": "i18n:build:fr"
}
```

you can then run:

```
ng serve --configuration=fr
```

or build for a specific locale with:

```
ng build --configuration=production,fr
```

The AOT compiler will locate all the i18n-marked snippets in the templates, find the corresponding translations in the XLF file, and transform the snippets in the template using the translations. It will then proceed as usual to generate JavaScript code from the templates and bundle your application.

If you want to support English, French and Spanish, for example, you'll have to build only once, and then the localization tool will generate 3 versions (once for each locale) of the application (this is very fast). Then, you need to deploy the 3 built applications to your production web server. You will also need to decide which application to serve to which user. This can be done at server-side, by detecting the preferred locale from the request header and serving the appropriate `index.html` page. Or by getting the preferred locale of the authenticated user from the database and serving the appropriate index.html page. You could also do it at client-side, by serving your three applications on three different URLs (`ponyracer.com`, `ponyracer.fr` and `ponyracer.es`, or `ponyracer.com/en`, `ponyracer.com/fr` and `ponyracer.com/es`), and by redirecting from `ponyracer.com` to the correct URL based on the browser locale.

24.5. Translating messages in the code



This section explains an undocumented feature of Angular which may change in the future versions. Use with caution!

Sometimes, the text you need to translate is not in the templates, but is in the TypeScript code itself. For example, the three states of a pony race PENDING, RUNNING and FINISHED should be translated somehow. Since Angular 9.0, you can use `$localize` to do so, but this is still experimental and undocumented. `$localize` is a tag function that can be applied to a template string (check the ECMAScript 2015 chapter if you need a refresher).

```
status = $localize`PENDING`;
```

Then, when you build your application with the CLI, the `$localize` calls are replaced with their translations!

You can also define an ID with the syntax we have in templates, and have dynamic parts in the string:

```
greetings = $localize`:@@home.greetings:Welcome ${this.user.firstName}!`;
```

`ng extract-i18n` supports the message extraction from `$localize` calls in the code since CLI v10.1, and it generates:

```
<trans-unit id="home.greetings" datatype="html">
  <source>Welcome <x id="PH" equiv-text="this.user.firstName"/>!</source>
  <context-group purpose="location">
    <context context-type="sourcefile">src/app/app.component.ts</context>
    <context context-type="linenumber">17</context>
  </context-group>
</trans-unit>
```

It can then be translated as usual (with `PH` the placeholder for the dynamic part):

```
<trans-unit id="home.greetings" datatype="html">
  <source>Welcome <x id="PH" equiv-text="{{ this.user.firstName }}"/>!</source>
  <target>Bonjour <x id="PH" equiv-text="{{ this.user.firstName }}"/> !</target>
</trans-unit>
```

24.6. Pluralization

Sometimes, the message you want to display depends on the number of elements in a collection, or on a count of elements.

For example, let's say our home page displayed the number of planned races for the day. You could simply show *"Number of planned race(s): 4"*. But a friendlier message would be *"No race is planned"* if there is none, or *"Only one race is planned"* if there is just one, or *"N races are planned"* in the other cases.

Angular actually has a special template syntax to do that. It's hard to read, except maybe for LISP programmers, but it does the job and is fairly easy to understand with the following example. Suppose our component has a property `racesPlanned`, containing the number of races that are planned for today. You can display it as:

```
<p>
  Hello, {racesPlanned, plural, =0 {no race is planned} =1 {only one race is planned}
other
  {{{ racesPlanned }} races are planned}}.
</p>
```

To internationalize this message, you would just use the i18n attribute as usual:

```
<p i18n="@@home.racesPlanned">
  Hello, {racesPlanned, plural, =0 {no race is planned} =1 {only one race is planned}
other
  {{{ racesPlanned }} races are planned}}.
</p>
```

Extracting this generates two translation units, one for the message itself, and one for the expression bundled in the message:

```
      <trans-unit id="home.racesPlanned" datatype="html">
        <source> Hello, <x id="ICU" equiv-text="{racesPlanned, plural, =0 {no race is
planned} =1 {only one race is planned} other
  {{{ racesPlanned }} races are planned}}"/>.
</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">32,34</context>
        </context-group>
      </trans-unit>
      <trans-unit id="5232470321856793506" datatype="html">
        <source>{VAR_PLURAL, plural, =0 {no race is planned} =1 {only one race is
planned} other {<x id="INTERPOLATION"/> races are planned}}</source>
        <context-group purpose="location">
          <context context-type="sourcefile">src/app/app.component.html</context>
          <context context-type="linenumber">32,33</context>
        </context-group>
      </trans-unit>
```

Unfortunately, the second translation unit has an auto-generated ID, and the pluralization syntax must be understood and respected by the translator. It's possible to translate those two translation units, though, and the translation works as expected:

```
      <trans-unit id="home.racesPlanned" datatype="html">
        <source>
  Hello, <x id="ICU" equiv-text="{racesPlanned, plural, =0 {...} =1 {...} other
{...}}"/>..
</source>
        <target>Bonjour, <x id="ICU" equiv-text="{racesPlanned, plural, =0 {...} =1
{...} other {...}}"/>.</target>
      </trans-unit>
      <trans-unit id="5232470321856793506" datatype="html">
        <source>{VAR_PLURAL, plural, =0 {no race is planned} =1 {only one race is
planned} other {<x id="INTERPOLATION" equiv-text="{{ racesPlanned }}"/> races are
planned} }</source>
        <target>{VAR_PLURAL, plural, =0 {aucune course n'est planifiée} =1 {seule une
course est planifiée} other {<x id="INTERPOLATION" equiv-text="{{ racesPlanned }}"/>
courses sont planifiées} }</target>
      </trans-unit>
```

24.7. Best practices

These best practices, acquired in years of development of i18ned applications, are not necessarily related to Angular, but to i18n in general.

Always specify an explicit unique ID for your messages. If you choose a meaningful ID, you often don't need to specify a meaning and a description for your message, because the ID is sufficient. Prefixing the IDs with the name of the component where they are used (like I did in all the examples with the `home.` prefix) allows you to know where they are used, and to find them in the code easily. Relying on auto-generated IDs doesn't allow you to have different translations for two identical messages in your source language. It also makes it very hard to figure out what needs to be changed between two releases of your application.

Even if the translators are not always developers, store your message files in your version control system. This makes sure a branch can have its own additions, which can be merged to the main branch when ready. It makes it easy to spot differences between branches and releases.

Duplication isn't necessarily bad. You might think that two pages sharing a label "Save" or "OK" should use the same key, but maybe you will want to label them "OK, I'll do it" and "OK, I accept" later. Or maybe they need to be differentiated in some foreign language. It's even more important to use two separate keys for words that are identical in English, but not in other languages, like "free", which can mean "free as in beer" or "free as in speech". Other languages use different words for these two concepts.

Don't confuse languages with countries. Don't use country flags to represent languages. Some languages (like English) are spoken in several countries. And some countries use several languages (like Belgium, which uses French, Dutch and German).

Avoid concatenation to translate text with parameters. For example, to translate *"Hello, my name is X and I'm Y years old"*, don't use a first key for *"Hello, my name is "*, a second key for *" and I'm "* and a third key for *" years old"*. Use a single key, containing interpolated expressions.

You should now be ready to conquer the world with your shiny i18ned Angular application.

Chapter 25. Performances



Be careful with premature optimization. Always measure before and after. Beware of the benchmarks you find on the internets: it's pretty easy to make them say what the authors want.

Performances can mean a lot of things: speed, CPU usage (battery consumption), memory pressure… Everything is not important for everybody: you have different needs if you are programming for a mobile website, an e-commerce platform, or a classic CRUD application.

Performances can also be split into different categories, that, once more, won't all matter to you: first load, reload, and runtime performances.

First load is when you open an application for the first time. Reload is when you come back to that application. Runtime performances is what happens when the application is running. Some of the following recommendations are very generic, and could be applied to any framework. We wrote them down because we think it's worth knowing. And because when you talk about performances, the framework is sometimes the bottleneck, but really (really) often not.

25.1. First load

When you load a modern Web application in your browser, a few things happen. First, the `index.html` is loaded and parsed by the browser. Then the JS scripts and other assets referenced are fetched. When one of the assets is received, the browser parses it, and executes it if it is a JS file.

25.1.1. Asset sizes

So the first tip is very obvious: be careful with your asset sizes!

The assets loading phase depends on how many assets you want to load. A lot will be slow. Big ones will be slow. Especially if the network is not that good, which happens more often than you think: you might test your application on a fiber optic connection, but some of your actual users might be in the middle of nowhere, using slow 3G. Here is what you can do.

25.1.2. Bundle your application

When you write your Angular application, you have imports all over the place, and your code is split across hundreds of files. But you don't want your users to load hundreds of files! So before shipping your application, you want to make a "bundle": group all the JavaScript files into one file.

Webpack's job is to take all your JavaScript files (and CSS, and template HTML files) and build bundles. It's not an easy tool to master, but the Angular CLI does a pretty good job at hiding its complexity. If you don't use the CLI, you can build your application with Webpack, or you can pick another tool that may produce even better results (like Rollup for example). But be warned that this requires quite a lot of expertise (and work) to not to mess things up, just to save a few extra kilobytes. I would recommend staying with the CLI. The team working on it is doing a very good job keeping up with the latest Angular, TypeScript and Webpack releases.

More than that, they built some tools to decrease the bundling size. For example, they wrote a plugin that goes through the generated JavaScript, and adds specific comments to help Terser remove dead code.

25.1.3. Tree-shaking

Webpack (or other tools you use) starts from the entry point of your application (the `main.ts` file that the CLI generated for you, and which you probably never touched), and then resolves all the imports tree, and outputs the bundle. This is cool because the bundle will only contains the files from your codebase and your third party libraries that have been imported. The rest is not embedded. So even if you have a dependency in your `package.json` that you don't use anymore (so you don't import it anymore), it will not end up in the bundle.

It's even a bit smarter than that. If you have a file `models` exporting two classes, let's say `PonyModel` and `RaceModel`, and then only import `PonyModel` into the rest of the application, but never `RaceModel`, then Webpack only puts `PonyModel` in the final bundle, and drops `RaceModel`. This process is called **tree-shaking**. And every framework and library in the JavaScript ecosystem is fighting hard to be tree-shakable! In theory, it means that your final bundle contains only what is really needed! But in practice, Webpack (and others) are a bit conservative, and can't figure out some stuff. For example, if you have a class `Pony` with two methods `eat` and `run`, but you only use `run`, the code of the `eat` method will be in the final bundle. So it's not perfect, but it does a good job.

A few techniques can be used in Angular specifically to have a better tree-shaking. First, don't import modules that you don't use. Sometimes you try out a library offering a wonderful component, and you add the NgModule of this library to the imports of your NgModule. Then you don't use it anymore, but maybe forget about the module import, and don't remove it… Bad news: this module and the third party library will be in the final bundle (for now, maybe it will be better in the future). So only import and use what you really need.

Another trick is to use `providedIn` for your services. If you declare a service in the providers of your NgModule, it will always end up in the bundle whether you actually use it or not, simply because it's imported and referenced in the module. Whereas if you don't register in the providers of your NgModule, but use `providedIn: 'root'` instead, then if you never use this service, it will not end up in the bundle.

25.1.4. Minification and dead code elimination

When your bundle has been built, the code is usually minified and dead code will be eliminated. That means all variables, methods names, class names… are renamed to use a one or two characters name through the entire codebase. This is a bit scary and sounds like it could break things, but Terser has been doing a great job. Terser will also eliminate dead code that it can find. It does its best, and I was saying above, the CLI team built a tool that prepares the code with special comments on unneeded code, so Terser can remove it safely.

25.1.5. Other assets

While the above sections were about JS specifically, your application also contains other assets, like styles, images, fonts… You should have the same concerns about them, and do your best to keep them at a reasonable size. Applying all kinds of crazy techniques to optimize your JS bundle sizes,

but loading several MBs of images, wouldn't have a big impact on your page loading time and your bandwidth! As this is not really the scope of this ebook, I won't dig into this topic, but let me point out a great online resource by Addy Osmani about image optimization: Essential Image Optimization.

25.1.6. Compression

All the modern browsers accept a compressed version of an asset when they ask the server for it. That means you can serve a compressed version to your users, and the browser will unzip it before parsing it. This is a must-do because it will save you tons of bandwidth and loading time!

Every server on the market gives the option of activating the compression of assets. Generally the first user to request an asset will pay the cost of the compression on the fly, and then the following ones will receive the compressed asset directly.

The most common compression algorithm used is GZIP, but some others like Brotli are also popular.

25.1.7. Lazy-loading

Sometimes, despite doing your best to keep your JS bundle small, you end up with a big file because your app has grown to several dozens of components, using various third party libraries. And not only will this big bundle increase the time needed to fetch the JavaScript, it will also increase the time needed to parse it and execute it.

One common solution to this problem is to use lazy-loading. It means that instead of having a big bundle of JavaScript, you split your application into several parts and tell Webpack to bundle it in several bundles.

The good news is Angular (its router, and its module system, in particular) makes this task relatively easy to achieve. The other good news is that the CLI knows how to read your router configuration to build several bundles automatically. You can read our chapter about the router if you want to learn more.

Lazy-loading can vastly improve the loading time, as you can make the first bundle really small, with only what's needed to display the home page, and let Angular load the rest on demand when your user navigates to another part. You can also use prefetching strategies to tell Angular to start loading the other bundles when it's idle.

Note that lazy-loading adds complexity to your application (and a few traps with dependency injection), so I would advise to go this way only if it really makes sense.

25.1.8. Ahead of Time compilation

In development mode, when you open the application in your browser, it will receive the JavaScript code resulting from the TypeScript compilation, and the HTML templates of the components. These templates are then compiled by Angular to JavaScript directly in your browser.

This is not optimal in production for two reasons mainly:

- every user pays the cost of this template compilation on every reload;

- the Angular compiler must be shipped to your users (and it's big).

This process is called Just in Time compilation. But there is another type of compilation: Ahead of Time compilation. With this mode, you compile your templates at build time, and ship the resulting JavaScript with the rest of the application to your users. It means that the templates are already compiled when your users open the application, and that we don't need to ship the Angular compiler anymore.

So the parsing and starting time of the application will be way better. And, on paper, not shipping the compiler should lead to smaller bundles, and faster load times. But in fact, the generated JavaScript is generally far bigger than the uncompiled HTML templates. So the bundles tend to be bigger after an AoT compilation. The Angular team has been working hard on this, with big improvements in Angular 4 and Angular 6 (with its experimental Ivy project). If the bundles are still too big and slow your loading time, consider lazy-loading as explained above.

25.1.9. Server side rendering

I'd like to start by saying that this technique is for 0.0001% of you. Server side rendering (or universal rendering) is the technique that consists of pre-rendering the application on the server before serving it to the users. With this, when a user asks for `/dashboard`, she will receive a pre-rendered version of the dashboard, instead of receiving `index.html` and then letting the router do its job after Angular has finished to start.

It can lead to vast improvements in perceived startup time. Angular offers a package `@angular/universal` that allows you to run the application not in a browser but on a server (usually a Node.js instance). You can then pre-render the pages and serve them to your users. The page will display very fast and then Angular will start its job and run as usual.

It's also a big win if you want your web site to be crawlable by search engines which don't execute JavaScript, since you can serve them pre-rendered pages, instead of a blank page.

It's also a way to display previews of your website on social networks like Twitter or Facebook. These sites will try to screenshot the shared URL, but since they don't execute JavaScript, they won't see anything of your dynamically generated page, unless you serve them a page generated on the server. So if you want to be sure that the preview is perfect, like if you are running a news site, or an e-commerce site, you need to add server-side rendering.

The bad news is that it's not as easy as adding the `@angular/universal` package. You application needs to follow some best practices (no direct DOM manipulation for example, as the server won't have a real DOM to manipulate). Then you need to setup your server and think about the strategy you want to adopt. Do you want to pre-render all pages or just a few? Do you want to pre-render the whole page, with the data fetching and authorization check it will need, or just some critical parts of the page? Do you want to pre-render them on build, or to pre-render them on demand and cache them? Do you want to do this for all the possible profiles and languages or just some? All these questions depend on the type of application you are building, and the effort can vary greatly depending on your goal.

So, again, I would advise you to use server side rendering only if it is critical for your application, and not based on the hype…

25.2. Reload

Once your user has opened the application once, it's possible to speed the following reloading.

25.2.1. Caching

You should always cache the assets of your application (images, styles, JS bundles...). This is done by configuring your server and leveraging the `Cache-Control` and `ETag` headers. All the servers on the market allow you to do so, or you can use a CDN for this purpose too. If you do so, the next time your users open the application, the browser won't have to send a request to fetch them because it will have them already!

But a cache is always tricky: you need to have a way to tell the browser "hey, I deployed a new version in production, please fetch the new assets!".

The easiest way to do this is to have a different name for the asset you updated. That means instead of deploying an asset named `main.js`, you deploy `main.xxxx.js` where `xxxx` is a unique identifier. This technique is called cache busting. And, again, the CLI is there for you: in production mode, it will name all your assets with a unique hash, derived from the content of the file. It also automatically updates the sources of the scripts in `index.html` to reflect the unique names, the sources of the images, the sources of the stylesheets, etc.

If you use the CLI, you can safely deploy a new version and cache everything, except the `index.html` (as this will contain the links to the fresh assets deployed)!

25.2.2. Service Worker

If you want to go a step further, you can use service workers.

Service Workers are an API that most modern browsers support, and to simplify, they act like a proxy in the browser. You can register a service worker in your application and every GET request will then go through it, allowing you to decide if you really want to fetch the requested resource, or if you want to serve it from cache. You can then cache everything, even your `index.html`, which guaranties the fastest startup time (no request to the server).

You may be wondering how a new version can be deployed if everything is cached, but you're covered: the service worker will serve from cache and then check if a new version is available. It can then force the refresh, or ask the user if he/she wants it immediately or later.

It even allows you to go offline, as everything is cached!

Angular offers a dedicated package called `@angular/service-worker`. It's a small package, but filled with cool features. Did you know that if you add it to your Angular CLI application, and turn a flag on (`"serviceWorker": true` in `angular.json`), the CLI will automatically generate all the necessary stuff to cache your static assets by default? And it will only download what has changed when you deploy a new version, allowing blazing fast application start!

But it can even go further, allowing you to cache external resources (like fonts, icons from a CDN...), route redirection and even dynamic content caching (like calls to your API), with different strategies possible (always fetch for fresh data, or always serve from cache for speed...). The

package also offers a module called `ServiceWorkerModule` that you can use in your application to react to push events and notifications!

This is quite easy to setup, and a quick win for your reload start time. It's also one of the steps needed to build a Progressive Web App, and to score a perfect 100% on Lighthouse, so you should check it out.

25.3. Profiling

Now that we have talked about first load and reload, we can start talking about runtime performances. But if you run into a performance issue, before trying any of the following tips, you should start by measuring and profiling the application.

Browsers nowadays offer nice developer tools, especially Chrome, which allows you to record your application, and analyze its behavior with quite some detail. You can even simulate some conditions, like using a slower processor, or using a 3G network. You can also dive into the call hierarchy, and see how much time each function call is consuming.

But Angular also offers a precious tool: `ng.profiler`. It's not very well-known, but it can be handy as it allows you to measure how long a change detection run in the current page took.

You can then try to apply one of the tips which we'll look at, and measure again to see if there is any improvement.

In your `main.ts` file, replace the application bootstrapping code with the following:

```
platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .then(moduleRef => {
    const applicationRef = moduleRef.injector.get(ApplicationRef);
    const componentRef = applicationRef.components[0];
    // allows to run `ng.profiler.timeChangeDetection();`
    enableDebugTools(componentRef);
  })
  .catch(err => console.log(err));
```

Then go to the page you want to profile, open your browser console, and execute the following instruction:

```
> ng.profiler.timeChangeDetection()
ran 489 change detection cycles
1.02 ms per check
```

You can see how many change detection cycles it ran (it should be at least 5 cycles or during at least 500ms), and the time per cycle. This is a super useful metric, as many of the tricks we are going to show you directly act on the change detection system. You'll be able to try them, run the profiler again, and compare the results.

You can also record the CPU profile during these checks to analyze them with `ng.profiler.timeChangeDetection({ record: true })`.

The Angular team recommends having a time per check below 3ms, to leave enough time for the application logic, the UI updates and the browser's rendering pipeline to fit within a 16ms frame (assuming a 60 FPS target frame rate).

Let's discover these tips!

25.4. Runtime performances

Angular's magic relies on its change detection mechanism: the framework automatically detects changes in the state of the application and updates the DOM accordingly. So, as a general rule of thumb, you'll want to help Angular and limit the change detection triggering and the amount of DOM to update/create/delete.

To be honest, most applications will be fine, even under heavy load. Read the chapter about Angular and its magic if you want a recap on why Angular is more performant than AngularJS.

But some of us will have to recode Excel in the browser for their enterprise, or will have a component with a tree displaying 10,000 customers, or another unreasonable thing to do in a browser. These things are tricky, whatever framework you use. They tend to update a lot of DOM, and have to check a lot of components. A few of the following tricks can help. And a few of these tricks are **really** mandatory, like the first one.

25.4.1. enableProdMode

When you are in development mode (by default), Angular will run the change detection twice every time there is a change. This is a security to make sure you are not doing strange things, like updating data without following the one-way data flow. If you break the rules, Angular will warn you about it in development, by throwing an exception that will force you to fix your code. But if you are not careful, you will deploy the application in this mode, and change detection will still run twice, slowing your application.

To go in production mode, you need to call a function provided by Angular called `enableProdMode`. This method will disable the double check, and also make the generated DOM "lighter" (less attributes on the elements, attributes that are added to debug the application).

As usual the CLI has you covered, and the call to `enableProdMode` is already present in the generated application, wrapped in an environment check: if you build with the `production` environment, your app will be in production mode.

25.4.2. `trackBy` in `*ngFor`

This is a simple tip that can really speed things up on `*ngFor`: add a `trackBy`. To understand why, let me explain how modern JS frameworks (at least all major ones) handle collections. When you have a collection of 3 ponies and want to display them in a list, you'll write something like:

```
<ul>
  <li *ngFor="let pony of ponies">{{ pony.name }}</li>
</ul>
```

When you add a new pony, Angular will add a DOM node in the proper position. If you update the name of one of the ponies, Angular will change just the text content of the right `li`.

How does it do that? By keeping track of which DOM node references which object reference. Angular will have an internal representation looking like:

```
node li 1 -> pony #e435 // { id: 3, color: blue }
node li 2 -> pony #8fa4 // { id: 4, color: red }
```

It works great, and if you change an object for another one, Angular will destroy the node and build another one.

```
node li 1 (recreated) -> pony #c1ea // { id: 1, color: green }
node li 2 -> pony #8fa4 // { id: 4, color: red }
```

If the whole collection is updated with new objects, the complete DOM list will be destroyed and recreated. Which is fine, except when you just refresh a list with almost the same content: in that case, Angular destroys the complete node list and recreates it, even if there is no need to. For example, when you fetch the same results from the server, you will have the same content, but different references as your collection will have been recreated.

The solution for this use-case is to help Angular track the objects, not by their references, but by something that you know will identify the object, typically an ID.

For this, we use `trackBy`, which expects a method:

```
<ul>
  <li *ngFor="let pony of ponies trackBy: ponyById">{{ pony.name }}</li>
</ul>
```

with the method defined in the component:

```
ponyById(index: number, pony: PonyModel): number {
  return pony.id;
}
```

As you can see, this method receives the current index and the current entity, allowing you to be creative (or simply track by index, but that's not recommended).

With this `trackBy`, Angular will only recreate a DOM node if the id of the pony changes. On a very big list which doesn't change much, it can save a ton of DOM deletions/creations. Anyway, it's quite

cheap to implement and doesn't have cons, so don't hesitate to use it. It's also a requirement if you want to use animations. If a DOM element's style is supposed to be animated (by transitioning smoothly from the previous value to the new one), and the list of ponies is replaced by a new one when refreshed, then `trackBy` is a must: without it, the animation will never happen, because the style of the element never changes. Instead, it's the element itself which is being replaced by Angular.

25.4.3. Change detection strategies

When we explained how Angular detects the changes in your application, we showed the tree of components and said that Angular starts by checking the root component, then its children, then its grand-children, until all components are checked. Then all the necessary DOM updates are applied in one batch.

But you may be wondering if it is a very good idea to check **every** component on **every** change. And you're right, that's often not really necessary.

Angular offers another change detection strategy: it's called `OnPush` and it can be defined on any component.

With this strategy, the template of the component will only be checked in 2 cases:

- one of the inputs of the component changed (to be more accurate, when the **reference** of one of the inputs changes);
- an event handler of the component was triggered.

This can be very convenient when the template of a component only depends on its inputs, and can give a serious boost to your application if you display a lot of components on screen! But once again, be very cautious before applying this optimization: if the preconditions end up not being respected, you will lose your hair wondering why the component (or any of its descendants) isn't always repainting itself after a change.

Let's take a small example to demonstrate.

Imagine that we have 3 components. A very simple `ImageComponent`:

```
@Component({
  selector: 'ns-img',
  template: `
    <p>{{ check() }}</p>
    <img [src]="src" />
  `
})
export class ImageComponent {
  @Input() src: string;

  check(): void {
    console.log('image component view checked');
  }
}
```

used in a PonyComponent:

```
@Component({
  selector: 'ns-pony',
  template: `
    <p>{{ check() }}</p>
    <ns-img [src]="getPonyImageUrl()"></ns-img>
  `
})
export class PonyComponent {
  @Input() ponyModel: PonyModel;

  check(): void {
    console.log('pony component view checked');
  }

  getPonyImageUrl(): string {
    return `images/pony-${this.ponyModel.color}-running.gif`;
  }
}
```

used itself in a RaceComponent:

```
@Component({
  selector: 'ns-race',
  template: `
    <h2>Race</h2>
    <p>{{ check() }}</p>
    <div *ngFor="let pony of ponies">
      <ns-pony [ponyModel]="pony"></ns-pony>
    </div>
    <button (click)="changeColor()">Change color</button>
  `
})
export class RaceComponent {
  ponies: Array<PonyModel> = [
    { id: 1, color: 'green' },
    { id: 2, color: 'orange' }
  ];
  colors: Array<string> = ['green', 'orange', 'blue'];

  check(): void {
    console.log('race component view checked');
  }

  changeColor(): void {
    this.ponies[0].color = this.randomColor();
  }

}
```

The `RaceComponent` displays two ponies, and the user can change the color of the first one by clicking on the `Change color` button.

With the current default change detection strategy, every time that we have a change in the application, all 3 components are checked.

We added a `check()` method in each component, called in each template: it allows us to track if the component is checked or not. And indeed in our example, we can see in our console:

```
pony component view checked
image component view checked
pony component view checked
image component view checked
race component view checked
```

(we can see that twice actually, because we are in development mode - see the section about `enableProdMode` above).

OnPush

But in this case, it's a waste of time: we know that if the pony doesn't change, the template of the `PonyComponent` doesn't need to be checked. Same thing for the `ImageComponent`: if the `src` input is the same, there is no need to recompute the image URL. So let's switch these components to `OnPush`, by adding a `changeDetection` attribute in their `@Component` decorator:

```
@Component({
  selector: 'ns-img',
  template: `
    <p>{{ check() }}</p>
    <img [src]="src" />
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ImageComponent {
  @Input() src: string;

  check(): void {
    console.log('image component view checked');
  }
}
```

```
@Component({
  selector: 'ns-pony',
  template: `
    <p>{{ check() }}</p>
    <ns-img [src]="getPonyImageUrl()"></ns-img>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PonyComponent {
  @Input() ponyModel: PonyModel;

  check(): void {
    console.log('pony component view checked');
  }

  getPonyImageUrl(): string {
    return `images/pony-${this.ponyModel.color}-running.gif`;
  }
}
```

When we click to change the color, we will only see in the console:

```
race component view checked
```

Which is awesome, because it means that we don't check the components that we don't need to check \o/.

OnPush and the mutability trap

But… there is a slight problem: the pony's color doesn't change any more!

I picked this example on purpose: even if `OnPush` is really powerful, it can be tricky and optimizing existing components is not only about adding a few `OnPush` here and there.

Why doesn't it work in our case?

Take a closer look at our `RaceComponent`, and its `changeColor` method:

```
changeColor(): void {
  this.ponies[0].color = this.randomColor();
}
```

This method **mutates** the pony in the `ponies` collection, and this pony is the input of our `PonyComponent`. Now that we shifted our component to be `OnPush`, Angular will only run the change detection if the **reference** of the `pony` input changes. And when you mutate an object, it's still the same object, so the reference doesn't change, and Angular thinks there is no need to run the change detection…

So, is this change detection strategy completely useless? Not really, but it does require you to be more careful.

The simple way to fix our issue is to not mutate our pony in `changeColor`, but to create a new object:

```
changeColor(): void {
  const pony = this.ponies[0];
  // create a new pony with the old attributes and the new color
  this.ponies[0] = { ...pony, color: this.randomColor() };
}
```

Once you've done that, the application is faster **and** correct. If the user clicks on the button, the `changeColor` method creates a new pony object with the old attributes and the new color. As this is a new object, Angular will run the change detection in the `PonyComponent` (an input changed), and then the `src` input of the `ImageComponent` will also change, and the image will display the correct color. And, of course, if another event triggers the change detection in `RaceComponent`, the children component will not be checked (if their inputs did not change).

As you can see, you can quickly fall into a trap when migrating a component to an `OnPush` strategy, so be careful (unit tests are your friend).

One way to avoid this would be to use a library that enforces immutability. Immutable.js (by Facebook) is such a library. I've never used it professionally, so I can't say if it's a good fit in an Angular application or not, but you do see it mentioned often on the internets.

There is a last topic we need to talk about: observables.

OnPush, Observables and the async pipe

Let's say we now have only one component, our well-known `PonyComponent`. It subscribes to an observable from a `ColorService` that returns a new color every second. We obviously expect the image displayed to change every second. The developer of this component thought that an `OnPush` change detection strategy couldn't hurt. What do you think?

```
@Component({
  selector: 'ns-pony',
  template: `
    <p>New color every 1s</p>
    <img [src]="'pony-' + color + '.gif'" [alt]="color" />
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PonyComponent implements OnInit, OnDestroy {
  color = 'green';
  subscription: Subscription;

  constructor(private colorService: ColorService) {}

  ngOnInit(): void {
    this.subscription = this.colorService.get().subscribe(color => (this.color =
color));
  }

  ngOnDestroy(): void {
    this.subscription.unsubscribe();
  }
}
```

Sadly, this doesn't work. With the `OnPush` strategy, Angular only refreshes the template if one of the inputs changed (here, there is no input), or if an event was triggered (there is none either). So the `color` field is updated every second, but the template is never refreshed…

This can be fixed by using the pipe called `async`. We talked about it in the Pipes chapter but maybe it didn't click back then.

The `async` pipe can be used to subscribe to a Promise or an Observable. Let's use it in our `PonyComponent`:

```
@Component({
  selector: 'ns-observable-on-push-with-async',
  template: `<img *ngIf="color | async as c" [src]="'pony-' + c + '.gif'" [alt]="c"
/>`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PonyComponent {
  color: Observable<string>;

  constructor(colorService: ColorService) {
    this.color = colorService.get();
  }
}
```

Now our component is working! The `async` pipe will trigger the change detection when a new value is received. And you can see that we store the result of `async` to use it with `as`. It also frees us from subscribing to the observable in the component, and having to remember to unsubscribe when the component is destroyed: `async` does it for us.

Note that `async` can lead to several HTTP requests if used several times in a template, and that you can use the "smart/dumb" component pattern to make it easier to use OnPush. Check out our chapter about Advanced Observables to learn more about that.

25.4.4. ChangeDetectorRef

There are a few last tricks regarding change detection that I want to show you. They are for more advanced use cases but, you never know, it could be handy one day.

Let's take a hypothetical use case: you have an observable that emits data very very frequently. In my example, it's a clock that emits every 10 milliseconds:

```
@Component({
  selector: 'ns-clock',
  template: `
    <h2>Clock</h2>
    <p>{{ getTime() }}</p>
    <button (click)="start()">Start</button>
  `
})
export class ClockComponent implements OnDestroy {
  time = 0;
  timeSubscription: Subscription;

  start(): void {
    this.timeSubscription = interval(10)
      .pipe(
        take(1001), // 0, 1, ..., 1000
        map(time => time * 10)
      )
      .subscribe(time => (this.time = time));
  }

  getTime(): number {
    return this.time;
  }

  ngOnDestroy(): void {
    this.timeSubscription.unsubscribe();
  }
}
```

The component uses the `Default` change detection strategy, so every time the observable emits a new value, the change detection is triggered, the template is refreshed and the clock value displayed is updated.

But, do we really need a hundred updates per second? Our eyes can't see that fast, and it's putting pressure on our browser for nothing. And remember that not only this component will be checked a hundred times per second, but the whole application too!

Maybe in that case it would be enough to refresh the time displayed every second for example. To do so, you can completely opt out from the automatic change detection in your component, and handle things yourself, by injecting into your component a `ChangeDetectorRef`. This class offers a few methods:

- `detach()`
- `detectChanges()`
- `markForCheck()`
- `reattach()`

The first two work together: you can indicate to Angular to not care about the component with `detach` and then manually call `detectChanges` when you want the change detection to run:

```
@Component({
  selector: 'ns-clock',
  template: `
    <h2>Clock</h2>
    <p>{{ getTime() }}</p>
    <button (click)="start()">Start</button>
  `
})
export class ClockComponent implements OnDestroy {
  time: number;
  timeSubscription: Subscription;

  constructor(private ref: ChangeDetectorRef) {}

  start(): void {
    this.ref.detach();
    this.timeSubscription = interval(10)
      .pipe(
        take(1001), // 0, 1, ..., 1000
        map(time => time * 10)
      )
      .subscribe(time => {
        this.time = time;
        // manually trigger the change detection every second
        if (this.time % 1000 === 0) {
          this.ref.detectChanges();
        }
      });
  }

  getTime(): number {
    return this.time;
  }

  ngOnDestroy(): void {
    this.timeSubscription.unsubscribe();
  }
}
```

As you can see, we slightly changed the component to inject `ChangeDetectorRef`, detach the component from the change detection, and then manually run `detectChanges()` to trigger it when we need it (every second in our case). The `time` field is still updated a hundred times per second, but now the clock displayed to our users is only updated every second!

Note that this only triggers a change detection on that component (and its children) every time we run `detectChanges()`.

But there is a way to go one step further, and completely handle it manually, by updating the DOM yourself (and not triggering a complete change detection):

```
@Component({
  selector: 'ns-clock',
  template: `
    <h2>Clock</h2>
    <p #clock></p>
    <button (click)="start()">Start</button>
  `
})
export class ClockComponent implements OnDestroy {
  time: number;
  timeSubscription: Subscription;
  @ViewChild('clock') clock: ElementRef<HTMLParagraphElement>;

  constructor(private ref: ChangeDetectorRef) {}

  start(): void {
    this.ref.detach();
    this.timeSubscription = interval(10)
      .pipe(
        take(1001), // 0, 1, ..., 1000
        map(time => time * 10)
      )
      .subscribe(time => {
        this.time = time;
        if (this.time % 1000 === 0) {
          this.clock.nativeElement.textContent = `${time}`;
        }
      });
  }

  ngOnDestroy(): void {
    this.timeSubscription.unsubscribe();
  }
}
```

Here we grab a reference to the element we need to update, and then we update the DOM manually when needed, without triggering a change detection.

Another way to do this is possible: you can completely run the code outside of Zone.js, the library that triggers the change detection. To do so, you can inject `NgZone`, and then use its `runOutsideAngular` method to execute code outside of its scope:

```
constructor(private zone: NgZone) {}

start(): void {
  this.zone.runOutsideAngular(() => {
    this.timeSubscription = interval(10)
      .pipe(
        take(1001), // 0, 1, ..., 1000
        map(time => time * 10)
      )
      .subscribe(time => {
        this.time = time;
        if (this.time % 1000 === 0) {
          this.clock.nativeElement.textContent = `${time}`;
        }
      });
  });
}
```

This produces the same results, but here the rest of the component would still be checked automatically by Angular. `runOutsideAngular` is more suited to use cases where you want only specific portions of code to run out of the watch of Zone.js/Angular.

As I was saying, this example is a bit advanced, but `ChangeDetectorRef` can be handy for some use cases. Imagine that the example of changing the color of a pony every second doesn't use an observable, but a simple `setInterval`.

```
@Component({
  selector: 'ns-pony',
  template: `<img [src]="getPonyImageUrl()" [alt]="ponyModel.color" />`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PonyComponent implements OnInit, OnDestroy {
  @Input() ponyModel: PonyModel;
  private intervalId: number;

  ngOnInit(): void {
    this.intervalId = window.setInterval(() => {
      this.ponyModel.color = this.randomColor();
    }, 1000);
  }

  ngOnDestroy(): void {
    window.clearInterval(this.intervalId);
  }
```

No visual update… And in that case, we can't use the `async` pipe as we did with an observable…

But we can use the `markForCheck` method of `ChangeDetectorRef` to manually trigger the change

detection in an `OnPush` component:

```
@Component({
  selector: 'ns-pony',
  template: `<img [src]="getPonyImageUrl()" [alt]="ponyModel.color" />`,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class PonyComponent implements OnInit, OnDestroy {
  @Input() ponyModel: PonyModel;
  private intervalId: number;

  constructor(private ref: ChangeDetectorRef) {}

  ngOnInit(): void {
    this.intervalId = window.setInterval(() => {
      this.ponyModel.color = this.randomColor();
      this.ref.markForCheck();
    }, 1000);
  }

  ngOnDestroy(): void {
    window.clearInterval(this.intervalId);
  }
```

And it works again!

25.4.5. Pure pipes

As you know, you can build your own pipes to format and display your data. For example, to display the full name of a user, you can either write a method in your component:

```
@Component({
  selector: 'ns-menu',
  template: `
    <p>{{ userName() }}</p>
    <p>...</p>
    <p>{{ userName() }}</p>
  `
})
export class MenuComponent {
  user: UserModel = {
    id: 1001,
    firstName: 'Jane',
    lastName: 'Doe',
    title: 'Miss'
  };

  userName(): string {
    return `${this.user.title} ${this.user.firstName} ${this.user.lastName}`;
  }
}
```

or write a custom pipe to encapsulate this logic:

```
@Component({
  selector: 'ns-menu',
  template: `
    <p>{{ user | displayName }}</p>
    <p>...</p>
    <p>{{ user | displayName }}</p>
  `
})
export class MenuComponent {
  user: UserModel = {
    id: 1001,
    firstName: 'Jane',
    lastName: 'Doe',
    title: 'Miss'
  };
}
```

```
@Pipe({
  name: 'displayName'
})
export class DisplayNamePipe implements PipeTransform {
  transform(user: UserModel): string {
    return `${user.title} ${user.firstName} ${user.lastName}`;
  }
}
```

This takes a little bit more work, but writing a pipe allows you to reuse it in other components.

What you may not know is that using a pipe is also more performant. By default, a pipe is "pure". In computer science, we call "pure" a function that has no side effect, and whose result only depends on its entries. A pure pipe is pretty much the same: the result of its `transform` method only depends on arguments. Knowing that, Angular applies a nice optimization: the `transform` method is only called if the reference of the value it transforms changes or if one of the other arguments changes (yes, a bit like the `OnPush` strategy for components).

It means that whereas a method of a component is called on every change detection, a pure pipe will only be executed when needed, and only once in a template if it is used with the same input value and arguments (as in my example).

By default, a custom pipe is pure, so that's great! But sometimes it's not a right fit.

In my example, if we mutate the user to set its `firstName` to a different value, the pipe never refreshes... It's pretty much the same issue that we had with the `OnPush` strategy: the reference of the value doesn't change, so the pipe does not run again.

Here you have two solutions:

- carefully use the pipe with immutable objects (do not mutate the user - create a new user with the new `firstName`);
- mark the pipe as "impure", and Angular will run it every time. You lose a tiny bit in performance, but you are sure that the displayed value is refreshed.

To mark a pipe as impure, just add `pure: false` in its decorator:

```
@Pipe({
  name: 'displayName',
  pure: false
})
export class DisplayNameImpurePipe implements PipeTransform {
  transform(user: UserModel): string {
    return `${user.title} ${user.firstName} ${user.lastName}`;
  }
}
```

To sum up:

- a pure pipe is not called as often as a method in a component
- but it doesn't run again if the input value is mutated, so use carefully.

25.4.6. Split your template wisely

Based on what we learned, here is a trick that doesn't use a specific Angular API, but can be easily understood.

Let's say you have a component displaying a huge list of results, and an input allowing you to update this list. As you don't want to update the list on every key pressed, you are debouncing what the user types, and then updating the list. Something like:

```
@Component({
  selector: 'ns-results',
  template: `
    <input [formControl]="search" />
    <h1>{{ resultsTitle() }}</h1>
    <div *ngFor="let result of results">{{ result }}</div>
  `
})
export class ResultsComponent implements OnInit {
  search = new FormControl('');
  results: Array<string> = [];

  constructor(private searchService: SearchService) {}

  ngOnInit(): void {
    this.search.valueChanges
      .pipe(
        debounceTime(500),
        switchMap(query => this.searchService.updateResults(query))
      )
      .subscribe(results => (this.results = results));
  }

  resultsTitle(): string {
    return `${this.results.length} results`;
  }
}
```

You may think that the change detection is not very often called, as you update the list only when the user has stopped typing. But in fact the change detection is called on every event in the template (so here on every key pressed). You can check it out by adding a simple `console.log` in `resultTitle`, and see it called in the developer console on every key pressed.

To avoid detecting change on the list elements even if not needed (as the results will not change on every new value, but only after some time), the idea is to split your view into two parts, and to introduce a sub-component to display the results. This component can be switched to `OnPush` and the change detection will only update it when really needed, and not on every key press.

```
@Component({
  selector: 'ns-results',
  template: `
    <input [formControl]="search" />
    <ns-results-list [results]="results"></ns-results-list>
  `
})
export class ResultsComponent implements OnInit {
  search = new FormControl('');
  results: Array<string> = [];

  constructor(private searchService: SearchService) {}

  ngOnInit(): void {
    this.search.valueChanges
      .pipe(
        debounceTime(500),
        switchMap(query => this.searchService.updateResults(query))
      )
      .subscribe(results => (this.results = results));
  }
}
```

With the sub-component looking like:

```
@Component({
  selector: 'ns-results-list',
  template: `
    <h1>{{ resultsTitle() }}</h1>
    <div *ngFor="let result of results">{{ result }}</div>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class ResultsListComponent {
  @Input() results: Array<string> = [];

  resultsTitle(): string {
    return `${this.results.length} results`;
  }
}
```

This is a simple pattern to use, often referred to as the smart/dumb component pattern: a smart component deals with data loading, event handling, etc., and simply passes the data to display as input to a second, dumb component. The only responsibility of the dumb component is to display the data, and to emit events to its parent smart component using outputs. This dumb component is the one with the large template, containing many expressions. But since its state only changes when its smart parent passes a new input, it can use OnPush and thus saves a lot of expression evaluations.

25.4.7. Attribute decorator

When using an `@Input()` in a component, Angular assumes that the value passed as input can change, and does what it takes to detect the change and pass the new value to the component. Sometimes, it's not really necessary, as you may want to only pass a value once to initialize the component and never change it. In this very specific case, you can use the `@Attribute()` decorator instead of the `@Input()` one.

Let's consider a button component, to which you want to pass a type to set its aspect (something like `primary`, `success`, `warning`, `danger`...).

Using an input, it would look like:

```
import { Component, Input } from '@angular/core';

@Component({
  selector: 'ns-button',
  template: `
    <button type="button" class="btn btn-{{ btnType }}">
      <ng-content></ng-content>
    </button>
  `
})
export class ButtonComponent {
  @Input() btnType: 'primary' | 'success';
}
```

that you can use with:

```
<ns-button btnType="primary">Hello!</ns-button>
<ns-button btnType="success">Success</ns-button>
```

Since the input is a simple string that never changes, you can switch to use an attribute:

```
import { Attribute, Component } from '@angular/core';

@Component({
  selector: 'ns-button',
  template: `
    <button type="button" class="btn btn-{{ btnType }}">
      <ng-content></ng-content>
    </button>
  `
})
export class ButtonComponent {
  constructor(@Attribute('btnType') public btnType: string) {}
}
```

This produces a "bind-once" like effect, avoiding having Angular do unnecessary work. But keep in mind this only works with non-dynamic, string inputs.

25.4.8. Conclusion

This chapter hopefully taught you some techniques which can help solve performance problems. But remember the golden rules of performance optimization:

- don’t
- don’t… yet
- profile before optimizing.

As a famous computer scientist said:

> premature optimization is the root of all evil.
>
> — Donald Knuth

So strive to make the code as simple and correct and readable as possible, and only start thinking about profiling, then optimizing, if you have a proven performance problem.



Try our exercise Performance 🦄! You will optimize our application and be able to measure every step of progress!

Chapter 26. Going to production

So now you've built an application, and you are seriously thinking about showing it to the world. Let's have a look about what you need to do to go to production!

26.1. Polyfills and differential loading

Angular supports recent browsers, but you may have to add a few JavaScript polyfills if you want to target older ones.

A polyfill is a JavaScript file that you add to your application to "emulate" a feature if the browser doesn't support it.

If you use Angular CLI, this is fairly easy to do: go to the `polyfill.ts` file, read the comments, and uncomment the proper lines.

If you don't use Angular CLI, you can follow the official documentation.

Since version 8.0, the CLI will automatically generate the appropriate bundles for older browsers if you need to. This feature is called "differential loading". All you have to do is to edit the `.browserslistrc` configuration file in your CLI project, and configure the browsers you want to target. If you choose to target older browsers (like IE9 for example), the CLI will generate two types of bundles:

- bundles for the modern browsers (`es2015`)
- bundles for the older browsers (`es5`)

All these bundles are included in the `index.html` file. As adding all these bundles makes the application heavier, you might be wondering if this a good idea. But there is a twist: the CLI adds the scripts for older browsers in `index.html` with a `nomodule` attribute. This attribute indicates to modern browsers (that supports ECMAScript modules) to ignore these scripts, so they are not even fetched by modern browsers!

26.2. Environments and configurations

If you use Angular CLI, you may have noticed the files named `environment.ts` and `environment.prod.ts`.

These files usually contain an object called `environment` with a few properties (by default only one called `production`, `false` in `environment.ts` and `true` in `environment.prod.ts`).

But you may also have noticed that `environment.ts` is the only one imported into the application! This is a bit weird at first but a really useful feature of the CLI.

When serving (with `ng serve`) or building (with `ng build`) your application, the CLI (Webpack, to be more accurate) will use `environment.ts`.

But you can also serve or build your application with a specified configuration. By default the CLI

has another configuration named `production`.

So you can also run `ng build --configuration=production`. The difference between these configurations can be found in the `angular.json` files:

```
"configurations": {
  "production": {
    "fileReplacements": [
      {
        "replace": "src/environments/environment.ts",
        "with": "src/environments/environment.prod.ts"
      }
    ],
    "optimization": true,
    "outputHashing": "all",
    "sourceMap": false,
    "namedChunks": false,
    "extractLicenses": true,
    "vendorChunk": false,
    "buildOptimizer": true,
    "budgets": [
      {
        "type": "initial",
        "maximumWarning": "500kb",
        "maximumError": "1mb"
      },
      {
        "type": "anyComponentStyle",
        "maximumWarning": "2kb",
        "maximumError": "4kb"
      }
    ]
  },
```

As you can see, there is a `production` configuration with a bunch of properties. We'll come back to the other ones, but the one interesting right now is the `fileReplacements` property. You can see that the `environment.ts` file is replaced by `environment.prod.ts`: this is how Webpack knows which file to use.

This means that you can define as many configurations as you want. For example you could add a `preprod` configuration with a dedicated `environment.preprod.ts` file.

It also means that you can replace as many files as you want in your application. You can imagine doing crazy things like replacing `pony.component.ts` with a different version (I don't see why you would do that though ^^).

An environment file can contain whatever you want. But as its name indicates, it's supposed to contain code that is specific to a given environment (development, production, pre-production, etc.). For example, you may have a different API location in development than in the live version.

As the `production` environment is often used, the CLI offers an alias `--prod`, instead of `--configuration=production`.

Since the CLI version 9.0, it is now possible to specify several configurations at once:

```
ng build --configuration=production,preprod
```

The command then uses the `production` configuration, merged with the `preprod` configuration. The `preprod` configuration can re-declare a property of the `production` configuration, to overwrite it.

Note that this replacement mechanism is also available for assets and styles in the CLI, giving you the possibility to theme your applications differently by just using configurations.

26.3. fullTemplateTypeCheck and strictTemplates

When you compile your application in AoT (`aot: true`, the default value since Angular 9.0), the templates are checked by the Angular compiler.

By default, only a light check is run. To go further, you can use the `fullTemplateTypeCheck` option, introduced with Angular 5.0.

```
"angularCompilerOptions": {
  "fullTemplateTypeCheck": true,
}
```

With this option, you are able to catch errors like:

```
<input [(ngModel)]="user.password" required #loginCtrl="ngModel">
<!-- typo in `hasError()` method -->
<div *ngIf="loginCtrl.hasEror('required')">Password required</div>
```

You can even go one step further since Angular 9.0, with the `strictTemplates` option:

```
"angularCompilerOptions": {
  "strictTemplates": true
}
```

With this option, the compiler checks that the input values, DOM events used, references in templates, etc. are all of the correct type. For example, trying to feed a number into an input that expects a boolean, results in a compilation error.

26.4. enableProdMode

A very important thing to do is to always call `enableProdMode()` in your production version. This little function call deactivates the double change detection check we talked about in the

Performances chapter.

It also deactivates a bunch of things that Angular does to facilitate the debugging of the application. That's why you can't analyze a production version of an application with Augury for example.

The cool thing is that, once again, the CLI has got you covered: the code in `main.ts` already wraps the call to `enableProdMode` in a check on `environment.production`. If you understood the previous section, that means `enableProdMode` will be called in production mode but in not in development mode.

26.5. Package your application

I slightly spoiled the next step in the previous sections. If you want to package your application for production using the CLI, you simply have to run:

```
ng build --prod
```

That will create a `dist` folder containing the result of the build. The `prod` flag uses the file replacement we just talked about, but also adds a bunch of other options. Some of them are really interesting.

It also activates tree-shaking and dead code elimination thanks to `optimization: true`. To make this process even more efficient, the Angular CLI team has written a tool named "build optimizer", which is activated by the `buildOptimizer: true` option.

To further reduce the volume of JavaScript code generated, the third party libraries are not bundled in a separate file (`vendorChunk: false`) - the licences are removed (`extractLicences: true`).

You typically don't want to debug in production. And you probably don't want to provide the non-minified source code to any visitor of your application either. So `sourceMap: false` disables the time-consuming generation of the source maps.

A last interesting option is `outputHashing`: it tells Webpack that the generated files should not be called `main.js` but `main.xxxxxx.js` where `xxxxxx` is a cryptographic hash of the content of the file. This is done to be sure that you can cache these files without worrying about it: see the section about "cache busting" in the Performances chapter just above.

As you can see, the CLI does plenty of nice optimizations for you. If you choose not to use the CLI, you'll have to figure out a way to do the same things by using Webpack or another tool.

26.6. Server configuration

The last thing to do is to take the result of the packaging step (in the `dist` folder) and to deploy it on your favorite server. That can be a static server like Apache or Nginx for example.



Do not use `ng serve` in production, even with the `--prod` option. It's only a development tool, and is neither optimized nor secured for production.

You still have a few things to do though. The precise way to do these things depends on your web server.

First you'll want to make sure you are serving all your assets compressed (probably using gzip). Then you want the assets to be cached for a long time. Don't worry about potential cache issues, as we generated the assets with a hash in their names.

The last step is less obvious and often forgotten (been there, done that…): you need to configure your server in a way that ensures that each route will serve `index.html`.

Think about it: you deployed your application on `https://ng-ponyracer.ninja-squad.com`, tweeted it to the world, and people are starting to visit it. Your server will serve the `index.html` file to them and everything is fine. They are navigating in the application, maybe going to the list of races at `https://ng-ponyracer.ninja-squad.com/races`.

But what happens if someone hits *F5*/*Cmd*+*R*? The next request to the server will be for `/races`, and if you did not plan for it, your server will return a `404`…

So you have to make sure that, one way or another, all requests to routes of your application will serve `index.html`. The Angular application will restart from scratch, the router will analyze the URL, and it will navigate to the proper route immediately.

26.7. Conclusion

I think we covered the important parts of going to production. As you can see this is fairly easy if you use Angular CLI, so I strongly encourage you to do so.

Chapter 27. This is the end

Thanks for reading!

There are some other chapters that will be added in the following releases on more advanced stuff and some other goodies. They all need a little more polish, but I'm sure you'll enjoy them. And of course, we'll keep up with the framework releases, so you won't miss the new shiny features that will come out. All these future updates of the book will be available for free, of course!

If you liked what you read, tell your friends about it!

And if you don't already own it, you should know that there is also a *pro* package of this ebook. This package gives access to a whole set of exercises to build a real application, step by step, starting from scratch. For each step we provide a full unit tests suite covering 100% of your code, detailed instructions (which are not a basic copy-paste, but will push you to understand what you are doing), and a solution if you need (which might be the most beautiful one, or at least one consistent with the latest best practices) A home-brewed tool analyzes your code and computes a score for each exercise, and your progression is visible on a dashboard. If you're looking for actual code samples, always up-to-date, which might save you hours of work, our Pro Pack is waiting for you! You can even try the first exercises for free. And as you are already the proud owner of this ebook, we want to thank you for your historic support with a generous discount that you can grab here!

We have tried to give you all the keys, but Web Development looks an awful lot like:



How to draw a horse. Credit to Van Oktop.

So we also provide training, mainly in France and Europe, but all over the world really. We can also do some consulting work to help your team, or work with you to help you build your product. Just shoot us an email at hello@ninja-squad.com and we'll discuss it!

Overall, I would love hearing from you and find out what you liked, loved and hated in this ebook - whether you are writing to signal a small typo, a big mistake, or just to tell us that this book helped you find your dream job (well, you never know…).

I can't finish without thanking a few people. My girlfriend, first, who has been an incredible support, even when I was rewriting something for the tenth time, in a dreadful mood on a Sunday. My colleagues, for their tireless work and feedback, their kindness for encouraging me and giving me the time to do this crazy thing. And my friends and family, for the little words that kept me

going.

And you, for buying this and reading it to the last sentence.

Stay tuned.

Appendix A: Changelog

Here are all the major changes since the first version. It should help you to see what changed since your last read!

By buying this ebook, you'll get all the following updates for free. Go to https://books.ninja-squad.com/claim to obtain the latest version of this ebook.

Current versions:

- Angular: `11.2.3`
- Angular CLI: `11.2.1`

A.1. v11.2.0 - 2021-02-12

A.2. v11.1.0 - 2021-01-21

A.3. v11.0.0 - 2020-11-12

Internationalization

- `ng xi18n` has been renamed `ng extract-i18n` in CLI v11 (2020-11-12)

A.4. v10.2.0 - 2020-10-22

Internationalization

- `xi18N` now extracts messages from the `$localize` calls in TypeScript code (2020-09-10)

A.5. v10.1.0 - 2020-09-03

Testing your app

- `async` has been deprecated and renamed `waitForAsync` (2020-09-01)

Internationalization

- Import the global variants of the locale data. It's simpler, supports all formatting options, and doesn't trigger an optimization bailout warning when building the app with the CLI. (2020-07-01)

A.6. v10.0.0 - 2020-06-25

Global

- Bump to ng `10.0.0` (2020-06-25)

The wonderful world of Web Components

- Use `customElements.define` instead of the deprecated `document.registerElement`. (2020-06-17)

Reactive Programming

- Pass an object as argument to the `Observable.subscribe()` method when an error or a completion must be handled, instead of 2 or 3 functions, because passing several functions will be deprecated in RxJS 7. (2020-06-05)

A.7. v9.1.0 - 2020-03-26

Global

- Bump to ng `9.1.0` (2020-03-26)

From zero to something

- Bump to cli `9.1.0` (2020-03-26)

A.8. v9.0.0 - 2020-02-07

Global

- Bump to ng `9.0.0` (2020-02-07)
- Bump to ng `9.0.0-next.5` (2020-02-06)

A gentle introduction to ECMAScript 2015+

- Add a section about tagged template strings. (2019-08-02)

Diving into TypeScript

- Showcase interface usage for modeling entities (2019-08-10)
- Improve the `enum` section with examples of how to use union types (2019-08-10)

Advanced TypeScript

- Introduce a new chapter about advanced TypeScript patterns, like `keyof`, mapped types, type guards, and other things! (2019-08-10)

From zero to something

- Bump to cli `9.0.1` (2020-02-07)
- Bump to cli `9.0.0-next.3` (2020-02-06)
- Bump to cli `8.3.2` (2019-08-30)
- Bump to cli `8.3.0` (2019-08-22)

Testing your app

- Use `TestBed.inject` instead of the deprecated `TestBed.get` in ng `9.0.0` (2020-02-06)

Internationalization

- Explains how to configure the default currency code (2020-02-07)
- Introduce `@angular/localize` usage in ng `9.0.0` (2020-02-07)

Going to production

- Mention the multiple configurations support introduced in CLI v9.0 (2020-02-07)
- Explain the `fullTemplateTypeCheck` and `strictTemplates` options (2020-02-07)

A.9. v8.2.0 - 2019-08-01

Global

- Bump to ng `8.2.0` (2019-08-01)

From zero to something

- Bump to cli `8.2.0` (2019-08-01)

Testing your app

- Use a more strictly typed `createSpyObj` syntax. (2019-07-31)

A.10. v8.1.0 - 2019-07-02

Global

- Bump to ng `8.1.0` (2019-07-02)

The wonderful world of Web Components

- Mention more recent alternatives to Polymer, remove the dead HTML import spec and mention Angular Elements (2019-06-01)

From zero to something

- Bump to cli `8.1.0` (2019-07-02)

A.11. v8.0.0 - 2019-05-29

Global

- Bump to ng `8.0.0` (2019-05-29)

A gentle introduction to ECMAScript 2015+

- How to use `async/await` with promises (2019-05-19)

From zero to something

- Bump to cli `8.0.0` (2019-05-29)
- Bump cli to `7.3.0` (2019-02-28)

Testing your app

- Showcase the awesome `ngx-speculoos` library for cleaner unit tests (2019-05-20)

Forms

- Showcase the awesome `ngx-valdemort` library for better validation error messages (2019-05-19)

Router

- Use `import` for lazy-loading routes as introduced by ng `8.0.0` (2019-05-20)

Angular compiler

- Update the AoT explanation and generated code for Angular 8.0.0 (Ivy) (2019-05-20)

Advanced components and directives

- Add and explain the `static` flag for `ViewChild` and `ContentChild` introduced by Angular `8.0.0` (2019-05-27)

Going to production

- Differential loading using `browserslist` as introduced by the cli `8.0.0`. (2019-05-20)

A.12. v7.2.0 - 2019-01-09

Global

- Bump to ng `7.2.0` (2019-01-07)
- Bump to ng `7.2.0-rc.0` (2019-01-03)
- Bump to ng `7.2.0-beta.2` (2018-12-14)

From zero to something

- Bump to cli `7.2.0` (2019-01-09)
- Bump to cli `7.2.0-rc.0` (2019-01-07)
- Bump to cli `7.2.0-beta.2` (2019-01-07)

A.13. v7.1.0 - 2018-11-27

Global

- Bump to ng `7.1.0` (2018-11-22)
- Bump to ng `7.1.0-rc.0` (2018-11-20)

- Bump to ng `7.0.2` (2018-11-05)

From zero to something

- Bump to cli `7.1.0` (2018-11-27)
- Bump to cli `7.0.4` (2018-11-05)

Router

- Use `UrlTree` in `CanActivate` guard, as introduced by 7.1 (2018-11-22)

A.14. v7.0.0 - 2018-10-25

Global

- Bump to ng `7.0.0` (2018-10-18)
- Bump to ng `7.0.0-rc.1` (2018-10-18)
- Bump to ng `7.0.0-rc.0` (2018-10-18)
- Bump to ng `7.0.0-beta.6` (2018-10-18)
- Bump to ng `7.0.0-beta.4` (2018-10-18)
- Bump to ng `7.0.0-beta.0` (2018-10-18)

From zero to something

- Bump to cli `7.0.2` (2018-10-24)
- Bump to cli `7.0.1` (2018-10-18)
- Bump to cli `6.2.1` (2018-09-07)
- Bump to cli `6.2.0-rc.0` (2018-09-07)

Performances

- Adds a performances chapter! (2018-08-30)

Going to production

- Adds a new chapter about Going to production! (2018-10-25)

A.15. v6.1.0 - 2018-07-26

Global

- Bump to ng `6.1.0` (2018-07-26)
- Bump to ng `6.1.0-rx.0` (2018-07-26)
- Bump to ng `6.1.0-beta.1` (2018-07-26)
- Bump to ng `6.0.7` (2018-07-06)

From zero to something

- Bump to cli `6.1.0` (2018-07-26)
- Bump to cli `6.0.8` (2018-07-06)
- Bump cli to `6.0.7` (2018-05-30)

Pipes

- Add the `keyvalue` pipe introduced in Angular 6.1 (2018-07-26)
- Show usage of formatting functions available since Angular 6.0 (2018-06-15)

Styling components and encapsulation

- New `ShadowDom` encapsulation option with Shadow DOM v1 support (the old and soon deprecated `Native` option uses Shadow DOM v0) (2018-07-26)

Send and receive data with Http

- HTTP tests now use `verify` every time (2018-07-06)

Router

- Adds the `Scroll` event and `scrollPositionRestoration` option introduced in 6.1 (2018-07-26)

Advanced observables

- Use `shareReplay` instead of `publishReplay` and `refCount` (2018-07-20)

Internationalization

- Update for CLI 6.0 and use a dedicated configuration (2018-05-09)

A.16. v6.0.0 - 2018-05-04

Global

- Bump to ng `6.0.0` (2018-05-04)
- Bump to ng `6.0.0-rc.4` (2018-04-13)
- Bump to ng `6.0.0-rc0` (2018-04-05)
- Bump to ng `6.0.0-beta.7` (2018-04-05)
- Bump to ng `6.0.0-beta.6` (2018-04-05)
- Bump to ng `6.0.0-beta.1` (2018-04-05)

The wonderful world of Web Components

- Replace customelements.io by webcomponents.org (2018-01-19)

From zero to something

- Bump to cli `6.0.0` (2018-05-04)
- The chapter now uses Angular CLI from the start! (2018-03-19)

Dependency Injection

- Use `providedIn` to register services, as recommended for Angular 6.0 (2018-04-15)
- Updates the dependency injection via token section with a better example (2018-03-19)

Reactive Programming

- We now use the pipeable operators introduced in RxJS 5.5 (2018-01-28)

Services

- Use `providedIn` to register the service, as recommended for Angular 6.0 (2018-04-15)

Testing your app

- Simplify service unit tests now that they use `providedIn` from ng 6.0 (2018-04-15)

Advanced components and directives

- Angular 6.0+ allows to type ElementRef<T> (2018-04-05)

Advanced observables

- We now use the imports introduced in RxJS 6.0 (`import { Observable, of } from 'rxjs'`) (2018-04-05)
- We now use the pipeable operators introduced with RxJS 5.5 (2018-01-28)

A.17. v5.2.0 - 2018-01-10

Global

- Bump to ng `5.2.0` (2018-01-10)
- Bump to ng `5.1.0` (2017-12-07)

Building components and directives

- Better lifecycle explanation (2017-12-13)

Forms

- Reintroduce the `min` and `max` validators from version 4.2.0, even if they are not available as directives. (2017-12-13)

Send and receive data with Http

- Remove remaining mentions to the deprecated `HttpModule` and `Http` (2017-12-08)

A.18. v5.0.0 - 2017-11-02

Global

- Bump to ng `5.0.0` (2017-11-02)
- Bump to ng `5.0.0-rc.5` (2017-11-02)
- Bump to ng `5.0.0-rc.3` (2017-11-02)
- Bump to ng `5.0.0-rc.2` (2017-11-02)
- Bump to ng `5.0.0-rc.0` (2017-11-02)
- Bump to ng `5.0.0-beta.6` (2017-11-02)
- Bump to ng `5.0.0-beta.5` (2017-11-02)
- Bump to ng `5.0.0-beta.4` (2017-11-02)
- Bump to ng `5.0.0-beta.1` (2017-11-02)
- Bump to ng `4.4.1` (2017-09-16)

Pipes

- Use the new i18n pipes introduced in ng 5.0.0 (2017-11-02)

Forms

- Add a section on the `updateOn: 'blur'` option for controls and groups introduced in 5.0 (2017-11-02)
- Remove the section about combining template-based and code-based approaches (2017-09-01)

Send and receive data with Http

- Use object literals for headers and params for the new http client, introduced in 5.0.0 (2017-11-02)

Router

- Adds ng 5.0 `ChildActivationStart`/`ChildActivationEnd` to the router events (2017-11-02)

Internationalization

- Remove deprecated i18n comment with ng 5.0.0 (2017-11-02)
- Show how to load the locale data as required in ng 5.0.0 and uses the new i18n pipes (2017-11-02)
- Placeholders now displays the interpolation in translation files to help translators (2017-11-02)

A.19. v4.3.0 - 2017-07-16

Global

- Bump to ng `4.3.0` (2017-07-16)
- Bump to ng `4.2.3` (2017-06-17)

Forms

- Remove min/max validators mention, as they have been removed temporarily in ng 4.2.3 (2017-06-17)

Send and receive data with Http

- Updates the chapter to use the new `HttpClientModule` introduced in ng 4.3.0. (2017-07-16)

Router

- List the new router events introduced in 4.3.0 (2017-07-16)

Advanced components and directives

- Add a section about `HostBinding` (2017-06-29)
- Add a section about `HostListener` (2017-06-29)
- New chapter on advanced components, with `ViewChild`, `ContentChild` and `ng-content`! (2017-06-29)

A.20. v4.2.0 - 2017-06-09

Global

- Bump to ng `4.2.0` (2017-06-09)
- Bump to ng `4.1.0` (2017-04-28)

Forms

- Introduce the `min` and `max` validators from version 4.2.0 (2017-06-09)

Router

- New chapter on advanced router usage: protected routes with guards, nested routes, resolvers and lazy-loading! (2017-04-28)

Angular compiler

- Adds a chapter about the Angular compiler and the differences between JiT and AoT. (2017-05-02)

A.21. v4.0.0 - 2017-03-24

Global

- 🎉 Bump to stable release `4.0.0` 🎉 (2017-03-24)

- Bump to `4.0.0-rc.6` (2017-03-23)
- Bump to `4.0.0-rc.5` (2017-03-23)
- Bump to `4.0.0-rc.4` (2017-03-23)
- Bump to `4.0.0-rc.3` (2017-03-23)
- Bump to `4.0.0-rc.1` (2017-03-23)
- Bump to `4.0.0-beta.8` (2017-03-23)
- Bump to ng `4.0.0-beta.7` and TS 2.1+ is now required (2017-03-23)
- Bump to `4.0.0-beta.5` (2017-03-23)
- Bump to `4.0.0-beta.0` (2017-03-23)
- Each chapter now has a link to the corresponding exercise of our Pro Pack Chapters are slightly re-ordered to match the exercises order. (2017-03-22)

The templating syntax

- Use `as`, introduced in 4.0.0, instead of `let` for variables in templates (2017-03-23)
- The `template` tag is now deprecated in favor of `ng-template` in 4.0 (2017-03-23)
- Introduces the else syntax from version 4.0.0 (2017-03-23)

Dependency Injection

- Fix the Babel 6 config for dependency injection without TypeScript (2017-02-17)

Pipes

- Introduce the `as` syntax to store a `NgIf` or `NgFor` result, which can be useful with some pipes like `slice` or `async`. (2017-03-23)
- Adds `titlecase` pipe introduced in 4.0.0 (2017-03-23)

Services

- New `Meta` service in 4.0.0 to get/set meta tags (2017-03-23)

Testing your app

- `overrideTemplate` has been added in 4.0.0 (2017-03-23)

Forms

- Introduce the `email` validator from version 4.0.0 (2017-03-23)

Send and receive data with Http

- Use `params` instead of the deprecated `search` in 4.0.0 (2017-03-23)

Router

- Use `paramMap` introduced in 4.0 instead of `params` (2017-03-23)

Advanced observables

- Shows the `as` syntax introduced in 4.0.0 as an alternative for the mulitple async pipe subscriptions problem (2017-03-23)

Internationalization

- Add a new chapter on internationalization (i18n) (2017-03-23)

A.22. v2.4.4 - 2017-01-25

Global

- Bump to `2.4.4` (2017-01-25)
- The big rename: "Angular 2" is now known as "Angular" (2017-01-13)
- Bump to `2.4.0` (2016-12-21)

Forms

- Fix the `NgModel` explanation (2017-01-09)
- `Validators.compose()` is no longer necessary, we can apply several validators by just passing an array. (2016-12-01)

A.23. v2.2.0 - 2016-11-18

Global

- Bump to `2.2.0` (2016-11-18)
- Bump to `2.1.0` (2016-10-17)
- Remove typings and use `npm install @types/⋯` (2016-10-17)
- Use `const` instead of `let` and TypeScript type inference whenever possible (2016-10-01)
- Bump to `2.0.1` (2016-09-24)

Testing your app

- Use `TestBed.get` instead of `inject` in tests (2016-09-30)

Forms

- Add an async validator example (2016-11-18)
- Remove the useless (2.2+) `.control` in templates like `username.control.hasError('required')`. (2016-11-18)

Router

- `routerLinkActive` can be exported (2.2+). (2016-11-18)
- We don't need to unsubscribe from the router params in the `ngOnDestroy` method. (2016-10-07)

Advanced observables

- New chapter on Advanced Observables! (2016-11-03)

A.24. v2.0.0 - 2016-09-15

Global

- 🎉 Bump to stable release `2.0.0` 🎉 (2016-09-15)
- Bump to `rc.7` (2016-09-14)
- Bump to `rc.6` (2016-09-05)

From zero to something

- Update the SystemJS config for `rc.6` and bump the RxJS version (2016-09-05)

Pipes

- Remove the section about the replace pipe, removed in rc.6 (2016-09-05)

A.25. v2.0.0-rc.5 - 2016-08-25

Global

- Bump to `rc.5` (2016-08-23)
- Bump to `rc.4` (2016-07-08)
- Bump to `rc.3` (2016-06-28)
- Bump to `rc.2` (2016-06-16)
- Bump to `rc.1` (2016-06-08)
- Code examples now follow the official style guide (2016-06-08)

From zero to something

- Small introduction to NgModule when you start your app from scratch (2016-08-12)

The templating syntax

- Replace the deprecated `ngSwitchWhen` with `ngSwitchCase` (2016-06-16)

Dependency Injection

- Introduce modules and their role in DI. Changed the example to use a custom service instead of Http. (2016-08-15)
- Remove deprecated `provide()` method and use `{provide: ⋯}` instead (2016-06-09)

Pipes

- Date pipe is now fixed in `rc.2`, no more problem with Intl API (2016-06-16)

Styling components and encapsulation

- New chapter on styling components and the different encapsulation strategies! (2016-06-08)

Services

- Add the service to the module's providers (2016-08-21)

Testing your app

- Tests now use the TestBed API instead of the deprecated TestComponentBuilder one. (2016-08-15)
- Angular 2 does not provide Jasmine wrappers and custom matchers for unit tests in `rc.4` anymore (2016-07-08)

Forms

- Forms now use the new form API (FormsModule and ReactiveFormsModule). (2016-08-22)
- Warn about forms module being rewritten (and deprecated) (2016-06-16)

Send and receive data with Http

- Add the HttpModule import (2016-08-21)
- `http.post()` now autodetects the body type, removing the need of using `JSON.stringify` and setting the `ContentType` (2016-06-16)

Router

- Introduce RouterModule (2016-08-21)
- Update the router to the API v3! (2016-07-08)
- Warn about router module being rewritten (and deprecated) (2016-06-16)

Changelog

- Mention free updates and web page for obtaining latest version (2016-07-25)

A.26. v2.0.0-rc.0 - 2016-05-06

Global

- Bump to `rc.0`. All packages have changed! (2016-05-03)
- Bump to `beta.17` (2016-05-03)
- Bump to `beta.15` (2016-04-16)
- Bump to `beta.14` (2016-04-11)
- Bump to `beta.11` (2016-03-19)
- Bump to `beta.9` (2016-03-11)

- Bump to `beta.8` (2016-03-10)
- Bump to `beta.7` (2016-03-04)
- Display the Angular 2 version used in the intro and in the chapter "Zero to something". (2016-03-04)
- Bump to `beta.6` (`beta.4` and `beta.5` were broken) (2016-03-04)
- Bump to `beta.3` (2016-03-04)
- Bump to `beta.2` (2016-03-04)

Diving into TypeScript

- Use `typings` instead of `tsd`. (2016-03-04)

The templating syntax

- `*ngFor` now also exports a `first` variable (2016-04-16)

Dependency Injection

- Better explanation of hierarchical injectors (2016-03-04)

Pipes

- A `replace` pipe has been introduced (2016-04-16)

Reactive Programming

- Observables are not scheduled for ES7 anymore (2016-03-04)

Building components and directives

- Explain how to remove the compilation warning when using `@Input` and a setter at the same time (2016-03-04)
- Add an explanation on `isFirstChange` for `ngOnChanges` (2016-03-04)

Testing your app

- `injectAsync` is now deprecated and replaced by `async` (2016-05-03)
- Add an example on how to test an event emitter (2016-03-04)

Forms

- A pattern validator has been introduced to make sure that the input matches a regexp (2016-04-16)
- Add a mnemonic tip to rememeber the `[()]` syntax: the banana box! (2016-03-04)
- Examples use `module.id` to have a relative `templateUrl` (2016-03-04)
- Fix error `ng-no-form` → `ngNoForm` (2016-03-04)
- Fix errors `(ngModel)` → `(ngModelChange)`, `is-old-enough` → `isOldEnough` (2016-03-04)

Send and receive data with Http

- Use `JSON.stringify` before sending data with a POST (2016-03-04)
- Add a mention to `JSONP_PROVIDERS` (2016-03-04)

Router

- Introduce the new router (previous one is deprecated), and how to use parameters in URLs! (2016-05-06)
- `RouterOutlet` inserts the template of the component just after itself and not inside itself (2016-03-04)

Zones and the Angular magic

- New chapter! Let's talk about how Angular 2 works under the hood! First part is about how AngularJS 1.x used to work, and then we'll see how Angular 2 differs, and uses a new concept called zones. (2016-05-03)

A.27. v2.0.0-alpha.47 - 2016-01-15

Global

- First public release of the ebook! (2016-01-15)